האוניברסיטה הפתוחה

מערכות הפעלה

מדריך למידה

ג'קי הרץ | דוד שריאל

עדכון המהדורה: אריה לויטן

האוניברסיטה הפתוחה
THE OPEN UNIVERSITY OF ISRAEL
الجامعة المفتوحة
Yair
Kurtzman

File #0006182 belongs to Yair Kurtzman- do not distribute

תנאי שימוש בקובץ הדיגיטלי:

1. הקובץ הוא לשימושך **האישי** בלבד. פרטים מזהים שלך מוטבעים בקובץ בצורה גלויה ובצורה סמויה.

2. השימוש בקובץ הוא אך ורק למטרות לימוד, עיון ומחקר אישי.

3. העתקה או שימוש בתכנים נבחרים מותרת בהיקף העומד בכללי השימוש ההוגן, המפורטים בסעיף 19 לחוק זכות יוצרים 2007. במקרה של שימוש כאמור חלה חובה לציין את מקור הפרסום.

4. הנך רשאי/ת להדפיס דפים מחומר הלימוד לצורכי לימוד, מחקר ועיון אישיים. אין להפיץ או למכור תדפיסים כלשהם מתוך חומר הלימוד.

האוניברסיטה הפתוחה

מערכות הפעלה

מדריך למידה

ג'קי הרץ, דוד שריאל

מהדורה פנימית
לא להפצה ולא למכירה
מק"ט 20594-5090

האוניברסיטה הפתוחה
THE OPEN UNIVERSITY OF ISRAEL
الجامعة المفتوحة
Yair
Kurtzman

File #0006182 belongs to Yair Kurtzman- do not distribute

צוות הפיתוח

כותבים : ד״ר ג׳קי הרץ, דוד שריאל
עדכון המהדורה : אריה לויטן
אחראי אקדמי : פרופ׳ לאוניד ברנבוים
מילון מונחים : אלכסנדר קוגן
עריכה : ד״ר חוה ניומן
עריכת העדכון : דפי בר אילן
סדר ועימוד : גילה ישראלי
אסתר טל

© תשפ״ה – 2024. כל הזכויות שמורות לאוניברסיטה הפתוחה.
ההוצאה לאור של האוניברסיטה הפתוחה, הקריה ע״ש דורותי דה רוטשילד, דרך האוניברסיטה 1, ת״ד 808, רעננה 4353701.
Academic Development and Publishing, The Open University of Israel, The Dorothy de Rothschild Campus, 1 University Road, P.O.Box 808, Raanana 4353701.
Printed in Israel.
אין לשכפל, להעתיק, לצלם, להקליט, לתרגם, לאחסן במאגר מידע, לשדר או לקלוט בכל דרך או בכל אמצעי אלקטרוני, אופטי, מכני או אחר כל חלק שהוא מהחומר שבספר זה. שימוש מסחרי בחומר הכלול בספר זה אסור בהחלט, אלא ברשות מפורשת ובכתב ממדור זכויות יוצרים של האוניברסיטה הפתוחה.

תוכן העניינים

פרק 1 – מבוא		9
1.1	מהי מערכת ההפעלה	12
1.1.1	מערכת ההפעלה כמכונה מדומה או כמכונה מורחבת	12
1.1.2	מערכת ההפעלה כמנהל משאבי המערכת	13
1.2	היסטוריה של מערכות הפעלה	13
1.3	סקירה של חומרה	14
1.4	גן החיות של מערכות ההפעלה	14
1.5	מושגים בסיסיים	15
1.6	קריאות מערכת	16
1.7	המבנה של מערכות הפעלה	18
1.8	סיכום	20
1.9	שאלות חזרה	20
1.10	רשימת מושגים	21
פרק 2 – תהליכים ותהליכונים		25
2.0	חזרה על מבנה המעבד [1.3.1 בספר הקורס]	27
2.1	מבוא לתהליכים	31
2.1.1	מודל התהליכים	32
2.1.2	יצירת תהליכים	32
2.1.3	סיום תהליכים	33
2.1.4	היררכיה של תהליכים	33
2.1.5	מכונת מצבים	33
2.1.6	יישום תהליכים	37
2.1.7	מודל לריבוי תוכניות	37
2.2	תהליכונים	39
2.2.1	שימוש בתהליכונים	39
2.2.2	מודל של תהליכונים	39
2.2.3	תהליכוני POSIX	42
2.2.4	מימוש תהליכונים במרחב משתמש	42
2.2.5	מימוש תהליכונים בגרעין המערכת	43
2.2.6	הפעלת מתזמן	44
2.2.7	אלתורים: ריבוי חוטי ביצוע מקוד שנועד לחוט ביצוע אחד	45
2.4	תקשורת בין תהליכים	46
2.4.1	מרוץ קריטי	46
2.4.2	קטעים קריטיים (critical sections)	48
2.4.3	מניעה הדדית הכוללת המתנה פעילה	49
2.4.4	הרדמה והערה	54
2.4.5	סמפורים	56
2.4.6	מנעולים	58
2.4.7	מבני פיקוח (monitors)	60
2.4.8	העברת הודעות (message passing)	63

האוניברסיטה הפתוחה
THE OPEN UNIVERSITY OF ISRAEL
الجامعة المفتوحة
Yair
Kurtzman

File #0006182 belongs to Yair Kurtzman- do not distribute

2.4.9	מחסומים (barriers)	65
2.5	תזמון תהליכים	66
2.5.1	מבוא לתזמון תהליכים	66
2.5.2	תזמון במערכות אצווה	68
2.5.3	תזמון במערכות אינטראקטיביות	70
2.5.4	תזמון במערכות זמן אמת	75
2.5.5	מדיניות (policy) ומנגנונים (mechanisms)	76
2.5.6	תזמון תהליכונים	76
2.6	בעיות IPC קלאסיות	77
2.6.1	בעיית הפילוסופים הסועדים	77
2.6.2	בעיית הקוראים-כותבים	78
2.7	סיכום	81
2.8	בעיות פתורות	81
2.9	שאלות חזרה	86
2.10	תרגילי תכנות לתרגול עצמי	87
2.11	רשימת מושגים	87
	פרק 3 – ניהול זיכרון	91
3.0	זיכרון (סקירת חומרה) [1.3.2 בספר הקורס]	93
3.1	ניהול זיכרון ישיר	95
3.2	מרחב זיכרון	96
3.2.1	ניידות תוכניות במרחב זיכרון	96
3.2.2	החלפה בזיכרון	98
3.2.3	ניהול זיכרון פנוי	99
3.3	זיכרון מדומה	102
3.3.1	דפדוף (paging)	103
3.3.2	טבלת הדפים (page table)	103
3.3.3	כיצד להגביר מהירות פענוח כתובות	107
3.3.4	טבלת דפים למרחבי זיכרון גדולים	113
3.4	אלגוריתמים להחלפת דפים	116
3.4.1	אלגוריתם אופטימלי להחלפת הדפים	116
3.4.2	אלגוריתם Not Recently Used – NRU	117
3.4.3	תור : FIFO	118
3.4.4	ההזדמנות השנייה (second chance)	118
3.4.5	אלגוריתם השעון (clock page replacement)	119
3.4.6	אלגוריתם Least Recently Used – LRU	119
3.4.7	סימולציה של LRU בתוכנה	120
3.4.8	אלגוריתם המסתמך על קבוצת עבודה של תהליך	121
3.4.9	אלגוריתם WSClock	123
3.4.10	רשימה של האלגוריתמים להחלפת דפים	124
3.5	שיקולים בתכנון של מערכות דפדוף בזיכרון	125
3.5.1	מדיניות החלפה מקומית לעומת מדיניות החלפה גלובלית	126
3.5.2	קביעת רמת ריבוי של מקבילות	128
3.5.3	שדים מדפדפים ו-cleaning policy	128
3.5.4	שיקולים בקביעת גודלו של דף	129

3.5.5	מרחבי כתובות נפרדים להוראות ול-data	130
3.5.6	דפים לשימוש משותף	130
3.5.7	ספריות לשימוש משותף	131
3.5.8	מיפוי קבצים לזיכרון	131
3.6	שיקולי יישום (מימוש)	132
3.6.1	מעורבותה של מערכת ההפעלה בתהליך הדפדוף	132
3.6.2	סיכום דרך הטיפול בשגיאות דף	133
3.6.3	שחזור הוראות מכונה לאחר פסיקת דף	134
3.6.4	נעילת דפים בזיכרון	134
3.6.5	אחסון בדיסק	135
3.6.6	הפרדה בין מדיניות למנגנונים	135
3.7	חלוקת הזיכרון לקטעים	136
3.7.1	יישום חלוקת הזיכרון לקטעים	137
3.8	סיכום	138
3.9	בעיות פתורות	139
3.10	שאלות לחזרה	141
3.11	תרגילי תכנות לתרגול עצמי	142
3.12	רשימת מושגים	143
	פרק 4 – מערכת קבצים	151
4.1	קבצים	153
4.1.1	שמות לקבצים	154
4.1.2	המבנה הפנימי של קובץ	154
4.1.3	טיפוסי קבצים	155
4.1.4	גישה לקבצים	156
4.1.5	מאפיינים של קבצים	157
4.1.6	פעולות על קבצים	158
4.1.7	דוגמה לשימוש בקריאות מערכת לעבודה עם מערכת קבצים	159
4.2	מדריכים (directories)	159
4.2.1-4.2.2	ספריות	159
4.2.3	שמות נתיבים	159
4.2.4	פעולות על מדריכים	160
4.3	יישום מערכת הקבצים	161
4.3.1	תסדיר מערכת הקבצים	161
4.3.2	יישום קבצים	162
4.3.3	יישום מדריכים	165
4.3.4	שיתוף קבצים בין משתמשים שונים	166
4.3.5	מערכת קבצים בצורת יומן (log-structured)	167
4.3.6	מערכת קבצים מבוססת פלאש	167
4.4	ניהול ואופטימיזציה של מערכת הקבצים	169
4.4.1	ניהול שטח הדיסק	169
4.4.2	גיבוי מערכת הקבצים	170
4.4.3	עקביות מערכת הקבצים (consistency)	171
4.4.4	ביצועים של מערכת הקבצים	172
4.6	סיכום	173

האוניברסיטה הפתוחה
THE OPEN UNIVERSITY OF ISRAEL
الجامعة المفتوحة
Yair Kurtzman

File #0006182 belongs to Yair Kurtzman- do not distribute

4.7 בעיות פתורות 174
4.8 שאלות לחזרה 176
4.9 רשימת מושגים 176

פרק 5 – קלט/פלט 179
5.1 חומרה של התקני קלט/פלט 181
5.1.1 התקני קלט/פלט 181
5.1.2 בקרים (device controllers) 182
5.1.3 שיטות מיעון של התקני קלט/פלט 182
5.1.4 גישה ישירה לזיכרון 183
5.1.5 חזרה נוספת על פסיקות חומרה 183
5.2 עקרונות למימוש קלט/פלט בתוכנה 184
5.2.1 מטרות תוכנת הקלט/פלט 184
5.2.2 קלט/פלט באמצעות המעבד הראשי 185
5.2.3 קלט/פלט מונחה אירועים 185
5.2.4 קלט/פלט באמצעות DMA 186
5.3 תוכנת קלט/פלט – חלוקה לשכבות 186
5.3.1 מנהל הפסיקות (interrupt handler) 186
5.3.2 מתאמי התקנים (device drivers) 187
5.3.3 תוכנת קלט/פלט המשוחררת מאילוצי חומרה 187
5.3.4 תוכנת קלט/פלט ברמת תהליכי המשתמשים 188
5.4 דיסקים 188
5.4.1 חומרת הדיסק ותזמון זרוע הדיסק 189
5.4.2 כונני מצב מוצק (Solid State Drive – SSD) 191
5.4.3 מערך דיסקים (RAID) 192
5.5 סיכום 195
5.6 בעיות פתורות 196
5.7 שאלות לחזרה 196
5.8 רשימת מושגים 197

פרק 6 – קיפאון 199
6.1 משאבים 201
6.1.1 משאבים שאינם ניתנים לחילוץ לעומת משאבים הניתנים לחילוץ 202
6.1.2 הקצאת משאבים 203
6.2 קיפאון 203
6.2.1 תנאים לקיפאון 204
6.2.2 מודל לקיפאון 204
6.3 שיטת בת היענה 206
6.4 גילוי קיפאון והיחלצות ממנו 206
6.4.1 גילוי קיפאון כאשר יש רק משאב יחיד מכל סוג 207
6.4.2 גילוי קיפאון כאשר המשאבים מחולקים למחלקות 208
6.4.3 היחלצות מקיפאון 210
6.5 התחמקות מקיפאון 212
6.5.1 מסלולים להקצאה בטוחה של משאבים (resource trajectories) 212
6.5.2 מצבים בטוחים לעומת מצבים מסוכנים 213
6.5.3 אלגוריתם הבנקאים בעבור משאב יחיד 214

6.5.4	אלגוריתם הבנקאים בעבור מחלקות של משאבים	214
6.6	מניעת קיפאון	215
6.6.1	תקיפת תנאי המניעה ההדדית	215
6.6.2	תקיפת תנאי ההחזקה וההמתנה	216
6.6.3	חילוץ כפוי	216
6.6.4	מניעת המתנה מעגלית	217
6.7	נושאים נוספים הקשורים לקיפאון	217
6.7.1	נעילה בשני שלבים (two-phase locking)	217
6.7.2-6.7.3	קיפאון שאינו קשור במשאבים מקובלים	218
6.7.4	הרעבה (starvation)	218
6.8	סיכום	219
6.9	בעיות פתורות	220
6.10	שאלות לחזרה	222
6.11	רשימת מושגים	223
	פרק 7 – וירטואליזציה וענן	225
7.1	היסטוריה	233
7.2	דרישות חומרה לווירטואליזציה	233
7.3	hypervisor מטיפוס ראשון ושני	236
7.4	היבטים למימוש וירטואליזציה יעילה	237
7.5	מיקרו-קרנל של hypervisor	242
7.6	וירטואליזציה של זיכרון	244
7.7	וירטואליזציה של I/O	246
7.8	מכונות וירטואליות על מחשבים מרובי מעבדים	249
7.9	ענן	250
7.10	וירטואליזציה ברמת מערכת ההפעלה	251
7.11	תיאור מקרה : VMware	253
	רשימת מקורות	254
	פרק 8 – מערכות מרובות מעבדים	255
8.1	ריבוי מעבדים	257
8.1.1	חומרת ריבוי ליבות	258
8.1.2	סוגי מערכות הפעלה לריבוי מעבדים	259
8.1.3	סנכרון עם ריבוי מעבדים	261
8.1.4	תזמון במערכות ריבוי מעבדים	263
8.5	סיכום	265
8.6	פרקים שאינם מופיעים בספר הלימוד	265
8.6.1	סיכום	265
8.6.2	שאלות לחזרה	265
8.6.3	רשימת מושגים	266
	נספח: מעט על בטיחות	267
9.2	מנגנוני הגנה	269
9.2.1	הגנה על ידי קביעת תחומים (protection domains)	269
9.2.2	רשימות גישה (ACL)	270
9.2.3	רשימות יכולות (C-lists)	270

האוניברסיטה הפתוחה THE OPEN UNIVERSITY OF ISRAEL الجامعة المفتوحة
Yair Kurtzman

File #0006182 belongs to Yair Kurtzman- do not distribute

פרק 1

מבוא

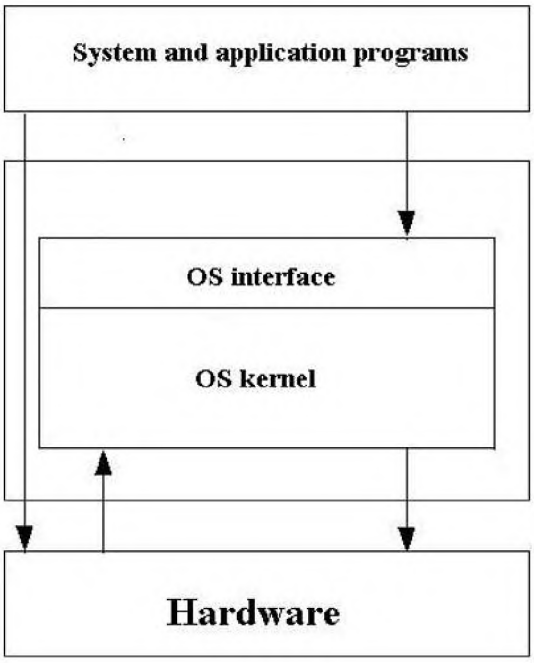
האוניברסיטה הפתוחה
THE OPEN UNIVERSITY OF ISRAEL
الجامعة المفتوحة
Yair
Kurtzman

File #0006182 belongs to Yair Kurtzman- do not distribute

1 מבוא
1.1 מהי מערכת ההפעלה
1.2 היסטוריה של מערכות הפעלה
1.3 סקירה של חומרה
1.4 גן החיות של מערכות ההפעלה
1.5 מושגים בסיסיים
1.6 קריאות מערכת
1.7 המבנה של מערכות הפעלה

מערכת ההפעלה היא תוכנה שמנהלת את חומרת המחשב. מערכות הפעלה גם מספקות בסיס להרצת אפליקציות שונות ומשמשות תווך בין משתמשי המחשב לבין החומרה. עיינו באיור 1-1 בספר הלימוד, הממחיש זאת. ההבדל העיקרי בין מערכת ההפעלה לתוכניות המשתמש הוא ביכולתה לרוץ במצב מיוחס של מעבד ולבצע כל פעולה הנתמכת על ידי חומרת המחשב, כאשר שאר התוכניות רצות רק במצב משתמש ויכולות לבצע באופן ישיר רק חלק מפקודות החומרה.

להבדיל מן התמונה הקונספטואלית שבאיור 1-1 בספר, איור 1 שלהלן הוא תרשים פונקציונלי של מערכת מחשב:



איור 1

מערכת ההפעלה משמשת תווך, אך אין הכוונה שכל פעולה שתוכניות אפליקציה או תוכניות שירות מבצעות תעבור דרך מערכת ההפעלה. התוכניות יכולות לפנות למערכת ההפעלה ולבקש ממנה שירותים דרך ממשק מוגדר, ומערכת ההפעלה תפעל לטובת

האוניברסיטה הפתוחה
THE OPEN UNIVERSITY OF ISRAEL
الجامعة المفتوحة
Yair
Kurtzman

File #0006182 belongs to Yair Kurtzman- do not distribute

מבקש השירות או המשתמש[1] (כך נכנה אותו בהמשך הדיון). באיור 1, חיצים יורדים ממשבצת התוכניות למשבצת של ממשק מערכת ההפעלה וממשיכים לגרעין מערכת ההפעלה ולחומרה. אך הוראות מכונה של תוכניות יכולות לרוץ ישירות על חומרת המחשב, והחץ שיורד ממשבצת התוכניות למשבצת החומרה מדגים זאת. במקרים מסוימים, הוראות של תוכניות אינן יכולות להיות מבוצעות על ידי החומרה. למשל, כאשר התוכניות מבקשות לבצע הוראות מיוחסות הדורשות קיום הרשאות מיוחדות. במקרה כזה, החומרה מבצעת פסיקה ומעבירה שליטה למערכת ההפעלה, שנדרשת לטפל בסוגיה. החץ ממשבצת החומרה לגרעין מערכת ההפעלה מדגים זאת.

שאלה 1

מדוע חייבת להיות אפשרות שהוראות מחשב של תוכניות משתמש יתבצעו בלי תיווך של מערכת ההפעלה? רמז: חשבו מה היה קורה אילו כל הוראה של תוכנית משתמש הייתה מצריכה פנייה לממשק משתמש ותגובת גרעין של מערכת ההפעלה.

ביצוע הוראות מיוחסות גורם למעבד לעבור למצב המכונה ***מצב מיוחס*** **(*kernel mode* או *supervisor mode*)**. הוראות שאינן הוראות מיוחסות מתבצעות במצב המכונה ***מצב משתמש*** **(*user mode*)**. במילים אחרות, מעבד מספק תמיכה כדי שמערכת ההפעלה תוכל להגן על משאבי המערכת. ההגנה מיועדת לחסום את אפשרות ההתערבות של המשתמשים בפעולתה של מערכת ההפעלה עצמה. בפועל, ההבחנה בין מצב משתמש למצב מיוחס נעשית על ידי מידור של קבוצת פקודות מכונה מיוחסות, שאינן עומדות לעולם לרשות משתמש רגיל.

1.1 מהי מערכת ההפעלה

בפרק זה מוצגים שני תפקידים של מערכת ההפעלה:

- מערכת ההפעלה כ**מכונה מדומה** או כ**מכונה מורחבת**.
- מערכת ההפעלה כ**מנהל משאבי המערכת**.

1.1.1 מערכת ההפעלה כמכונה מדומה או כמכונה מורחבת

זהו תפקידה של מערכת ההפעלה, מנקודת מבטו של משתמש "מלמעלה למטה". מכונה מדומה היא שכבת תוכנה שנמצאת מעל החומרה ומתחת ליישומי תוכנה אחרים, ושמספקת למשתמש ממשק המאפשר שימוש בחומרה. בדרך כלל, שכבת תוכנה זו מספקת למשתמש ממשק לחומרה שהוא נוח וקל לתכנות, ובכך מסתירה מהמשתמש

---

[1] בנקודה זו כדאי שנברר לעצמנו, למי אנו מתכוונים כאשר אנו מדברים על **המשתמש** (user) בהקשר של קורס זה. בהקשר של מערכות הפעלה, המונח יתייחס בדרך כלל למתכנת או מתכנתת יישומים. מתכנתים משתמשים ביישומיהם בשירותים שמספקת מערכת ההפעלה. יש להבדיל מובן זה מהמובן הנפוץ יותר של המונח, שמתייחס לאדם המשתמש ביישום מוכן כלשהו (כמו מסד נתונים, סייר האינטרנט, משחק מחשב).

את מורכבות הפעולה האמיתית של החומרה. מתוקף ההפשטה, היא פועלת כמכונה מורחבת.

המכונה המדומה מגבילה במידה רבה את יכולת הגישה של המשתמש להתקנים הפיזיים במערכת, והשימוש בהתקנים הללו מתבצע באמצעות ממשק משתמש שמערכת ההפעלה מעמידה לרשות המשתמש. למשל, כדי לקרוא קובץ במערכת UNIX, אין צורך לדעת דבר על מקום אחסונו הפיזי של הקובץ או על אופן אחסונו. המשתמש צריך רק לדעת את שם הקובץ ולהכיר את הפונקציות open ו-read, המסופקות על ידי מערכת הקבצים של מערכת ההפעלה.

בסעיף 1.7.5 בספר הלימוד נראה דוגמה אחרת למכונה מדומה – שם מדובר בשכבת תוכנה שמספקת לשכבות שמעליה כמה העתקים של החומרה, בעלי סוג זהה של משאבים אשר כמותם קטנה או שווה לכמות החומרה האמיתית. המכונה המדומה המתוארת בסעיף 1.7.5 בספר הלימוד מספקת ממשק של חומרה גולמית ואיננה מציעה שירותים של מכונה מורחבת.

1.1.2 מערכת ההפעלה כמנהל משאבי המערכת

זהו תפקידה של מערכת הפעלה בהסתכלות "מלמטה למעלה". מחשב מורכב מרכיבים רבים ושונים. מנהל משאבים הוא תוכנה שמנהלת את הקצאת המשאבים האלה לתהליכים שרצים במערכת.

1.2 היסטוריה של מערכות הפעלה

סעיף זה במדריך הלמידה הוא סקירה היסטורית קצרה של התפתחות המחשבים ומערכות ההפעלה. בסעיף זה תפגשו את המושגים החשובים האלה:

- ***מערכת אצווה*** **(*batch system*)** – מערכת שיכולה לבצע באופן רצוף סדרה של משימות שהוכנה מראש. במערכת כזו כל משימה מתבצעת מתחילתה ועד סופה וזאת בלי לשתף את המעבד בתהליכים אחרים, כפי שנעשה במערכות עם timesharing.
- ***ריבוי תוכניות*** **(*multiprogramming*)** – שיטה שבה כמה תהליכים יכולים להימצא בזיכרון הראשי בו-זמנית. שיטה זו מייעלת את ניצול המעבד על ידי הקצאתו לתהליך שממתין לו בזמן שתהליך שהתבצע קודם ממתין לאירוע כלשהו (למשל, קבלת קלט מהדיסק).
- ***חלוקת זמן*** **(*timesharing*)** – שיטה שבה המערכת מאפשרת לכמה משתמשים או תהליכים אינטראקטיביים לעבוד בו-זמנית, ומבטיחה לכולם תגובה מהירה, בהתאם לקצב הפקודות שמתקבלות מהם. בשיטה זו כל משתמש או תהליך מקבל את הקצאת המעבד לפרק זמן מוגבל כדי לבצע עבודות, וכשהזמן מסתיים ההקצאה עוברת לאחר. העבודות שמהן נלקח המעבד ימשיכו לרוץ כאשר יקבלו את המעבד בפעם הבאה.

האוניברסיטה הפתוחה THE OPEN UNIVERSITY OF ISRAEL الجامعة المفتوحة
Yair Kurtzman

File #0006182 belongs to Yair Kurtzman- do not distribute

אם ברצונכם להרחיב את ידיעותיכם, תוכלו לקרוא על ההתפתחות ההיסטורית בספר הלימוד.

1.3 סקירה של חומרה

בקריאה ראשונה אפשר לדלג על הסעיף הזה. בהמשך מדריך הלמידה, נפנה על פי הצורך לפרקים הרלוונטיים.

1.4 גן החיות של מערכות ההפעלה

במהלך השנים התפתחו סוגים רבים של מערכות הפעלה. לפניכם רשימה תמציתית של הסוגים הללו:

- ***מערכות הפעלה למחשב מרכזי*** (***mainframe operating systems***) – מחשב מרכזי הוא מחשב בעל עוצמות אחסון וחישוב גדולות, המשמש ארגונים גדולים כגון מוסדות ממשלתיים, חברות מסחריות גדולות, אוניברסיטאות ובנקים. מחשב כזה מבצע עיבוד נתונים רחב היקף כדי להפעיל מגוון יישומים, כדוגמת מרשם תושבים, תזמון טיסות וביצוע פעולות כספיות, ובתוך כך הוא משרת משתמשים רבים. מערכות הפעלה למחשבים מרכזיים תוכננו לנצל בצורה יעילה את משאבי המחשב המרכזי ולספק אמינות ואבטחת מידע גבוהות. מספר בלתי מבוטל של רעיונות שיושמו במערכות הפעלה של מחשבים מרכזיים שימשו השראה לתכנון מערכות הפעלה מן הסוגים שנראה בהמשך.
- ***מערכות הפעלה לשרתים*** (***server operating systems***) – שרת מחשבים מתאפיין בהפעלת מספר גדול של תוכניות ותהליכי מחשב במקביל; כל תוכנית או תהליך משרתים בדרך כלל כמה משתמשי קצה במחשבים המחוברים לשרת, אשר ניגשים אל השרת באמצעות רשת מחשבים. מערכות הפעלה לשרתים שמות דגש על זמינות השירותים שהשרת מספק ועל השרידות של שרתים. למשל, בשרתי אינטרנט כל נפילה או תקלה בתהליך או ביישום על גבי השרת פירושה השבתת שירות, וזו עלולה לגרום לניתוק ולאי-הצגה של אתרי האינטרנט המאוחסנים על השרת. לכן שרתי אינטרנט מסופקים עם רכיבי חומרה עודפים שמערכת ההפעלה יכולה להשתמש בהם במקביל, ואם אחד הרכיבים יוצא מכלל פעולה, המערכת תעבור לעבוד במתכונת חירום עם הרכיב העודף. אפשר גם להחליף on the fly חלקים שאינם תקינים.
- ***מערכות הפעלה מקביליות*** (***multiprocessor operating systems***) – מחשב מקבילי הוא מחשב המצויד ביותר ממעבד אחד. המעבדים במחשב מקבילי חולקים בדרך כלל זיכרון ושעון, והתקשורת נעשית לרוב דרך הזיכרון המשותף. ארכיטקטורה כזו מכונה ***tightly coupled processors***. ניתן להבחין בין שני סוגי ארכיטקטורות של מחשבים מקביליים: האחת עם מעבדים סימטריים והאחרת עם מעבדים

אסימטריים. מערכות הפעלה אשר רצות על גבי מעבדים סימטריים משתמשות בטכניקה המכונה ***Symmetric Multi Processing*** (***SMP***), כאשר מערכת ההפעלה יכולה לרוץ על כל אחד מהמעבדים. תהליכים יכולים להיות מתוזמנים גם על כל אחד מהמעבדים. מחשבים ביתיים בימינו, אשר מצוידים ביותר ממעבד אחד, עושים לרוב שימוש ב-SMP. במערכות בעלות ארכיטקטורה אסימטרית מערכת ההפעלה רצה על מעבד אחד בלבד, הנקרא master. שאר המעבדים מכונים slaves, ומערכת ההפעלה מקצה להם משימות. מערכות מסוג זה נפוצות במחשבים גדולים יותר.

- ***מערכות הפעלה מבוזרות*** (***distributed operating systems***) – במערכות אלו החישוב מבוזר בין כמה מעבדים, ולכל אחד מהמעבדים יש זיכרון ושעון משלו. מעבדים אלה מכונים loosely coupled processors. המעבדים מתקשרים ביניהם דרך קווי תקשורת מסוגים שונים, כמו אפיקים (buses) מהירים או בתקשורת על גבי רשת לוקאלית, שהיא איטית יותר. מערכות הפעלה מבוזרות שמות דגש על שיתוף משאבים, אמינות וביזור עומסים, ומאפשרות חישוב מהיר יותר.
- ***מערכות הפעלה למחשבים ביתיים*** (***personal computer operating systems***) – במערכות מחשב מסוג זה, תפקידו של המחשב הוא לשרת את המשתמש היחיד. לכן מערכת הפעלה כזו מושתתת על עקרונות של נוחות למשתמש וזמן תגובה מהיר. בשנים האחרונות מתעצמת המגמה של ריבוי מעבדים במחשבים הביתיים, ולכן הגבול בין מערכות הפעלה מקבילות למערכות הפעלה למחשבים ביתיים הולך ומיטשטש.
- ***מערכות הפעלה זמן-אמת*** (***real-time operating systems***) – במערכת זמן אמת, הדגש הוא על זמן הביצועים. ניתן להבחין בין שני סוגים של מערכות זמן אמת: מערכות שמציבות אילוצים נוקשים על זמני ביצוע (hard real time) ומערכות שהאילוצים בהן גמישים יותר (soft real time). למערכות מן הסוג הראשון ניתן לשייך למשל מערכות במטוסים ובמכוניות. לסוג השני של מערכות אפשר לשייך מערכות מולטימדיה שמספקות לדוגמה streaming audio, ובהן אפשר לסבול מידה מסוימת של אי-עמידה בזמנים.
- ***מערכות הפעלה למערכות משובצות*** (***embedded operating systems***) – מערכות הפעלה אלו מיועדות לשימוש במכשירי טלפון ניידים, במכשירי DVD וב-smartcards. מערכות משובצות מציבות דרישות מחמירות מאוד על שימוש במשאבי מערכת.

1.5 מושגים בסיסיים

במהלך הקורס נעמיק בנושאים הנידונים בפרק זה, אך כדאי להכיר היכרות ראשונית את המושגים ואת המשמעויות כעת:

- ***תהליך*** (***process***) – תוכנית ריצה (executable) בביצוע. כל תהליך מתאפיין במרחב זיכרון משלו – טווח כתובות RAM מ-0 ועד הגבול העליון שהתהליך יכול להשתמש

האוניברסיטה הפתוחה THE OPEN UNIVERSITY OF ISRAEL الجامعة المفتوحة
Yair Kurtzman

File #0006182 belongs to Yair Kurtzman- do not distribute

בו. תהליך כולל את התוכנית עצמה (code), את הנתונים שהיא יוצרת ומשתמשת בהם (data), מצביע למחסנית, מונה פקודות ואוגרים אחרים שמכילים מידע על הסטטוס הנוכחי של הביצוע. כמו כן, תהליך מכיל מידע על הקבצים שהוא משתמש בהם ונתונים רבים אחרים.

- ***מצב קיפאון*** **(*deadlock*)** – קבוצה של תהליכים נמצאת במצב קיפאון אם כל אחד מהתהליכים אינו יכול להתקדם אלא לאחר שיקרה אירוע אשר יכול להיגרם אך ורק על ידי תהליך אחר בקבוצה. תופעת הקיפאון בצורתה הפשוטה ביותר מתרחשת כאשר קיימים שני תהליכים, שכל אחד מהם מחזיק ברשותו משאב בלעדי הנחוץ לתהליך האחר. במצב זה אף אחד משני התהליכים אינו יכול להתקדם, והם נמצאים בקיפאון.
- ***ניהול זיכרון*** **(*memory management*)** – אנו נלמד שיטות ניהול זיכרון המאפשרות לשכן בזיכרון תהליכים אחדים בו-זמנית, ולעתים אף להריץ תוכניות שמשתמשות בכמות זיכרון גדולה יותר מהזיכרון הפיזי.
- ***קלט/פלט*** **(*input/output*)** – התקן קלט/פלט הוא רכיב חומרה אשר נועד לאחסון ו/או להעברת מידע. דוגמאות להתקני קלט/פלט הן: דיסק קשיח, כונן דיסקט, כרטיס רשת. ישנם התקני קלט/פלט אשר משמשים כהתקני קלט בלבד (עכבר, מקלדת) או כהתקני פלט בלבד (מדפסת, מסך). אחד התפקידים החשובים ביותר של מערכת הפעלה הוא ניהול של התקני קלט/פלט. בשל מספרם הגדול של התקני הקלט/פלט והשונות הרבה במהירות העברת הנתונים ובממשקים שלהם, יש חשיבות רבה ליצירת פתרון המספק גישה עקבית ואחידה לספקטרום רחב מאוד של התקנים כאלה.
- ***קבצים*** **(*files*)** – קובץ הוא יחידת מידע לוגית לאחסון במדיה חיצונית כגון דיסק, תקליטור, כרטיס זיכרון. מכיוון שקבצים הם מבנים לוגיים, מימושם נעשה בתוכנה ולא בחומרה. מערכת ניהול הקבצים (שהיא חלק ממערכת ההפעלה) אחראית למימוש הקבצים.
- ***מעטפת*** **(*shell*)** – תוכנית המהווה ממשק אינטראקטיבי למערכת ההפעלה. מעטפת היא מעבד הפקודות שמקבל פקודות מן המשתמש ומבצע אותן על ידי שימוש בשירותי מערכת ההפעלה. אף על פי שכל מערכת הפעלה מסופקת בדרך כלל עם תוכנת מעבד פקודות, התוכנה אינה חלק מובנה של מערכת ההפעלה אלא יישום שרץ במצב משתמש, ובעבור אותה מערכת הפעלה ניתן לכתוב מעבדי פקודות שונים.

1.6 קריאות מערכת

בתחילת הפרק ראינו כי ביצוע של הוראות מיוחסות מתאפשר רק במצב מיוחס, ולמשתמש רגיל לא ניתנת אפשרות כזאת. לרשותו של המשתמש עומד ממשק של ***קריאות מערכת*** **(*system call*)** שדרכו הוא יכול לבקש ממערכת ההפעלה שירותים אשר דורשים הרשאות מיוחדות (ראו באיור 1 במדריך הלמידה את שכבת ה-OS interface). השירותים שמערכת ההפעלה מספקת יכולים להתבקש על ידי תוכנית הכתובה בשפה

עילית בצורה של פונקציות.[2] כל קריאת מערכת מוציאה את השליטה מידי תוכנית המשתמש ומעבירה אותה זמנית למערכת ההפעלה (ראו איור 1-17 בספר הלימוד). הצלחת הפעולה איננה ודאית, ולכן בעבור כל קריאת מערכת, מערכת ההפעלה מחזירה אינדיקציה על תוצאת הביצוע. זו הסיבה לכך שלאחר כל שימוש בפונקציות שמכילות קריאת מערכת כלשהי, חייבת להתבצע בדיקה של ערך החזרה של הפונקציה. אפשר לומר שממשק משתמש המסופק על ידי מערכת ההפעלה באמצעות קריאות מערכת הוא ביטוי של תפקידה של מערכת הפעלה כמכונה מדומה שדנו בה בסעיף 1.1.

כדי להבין כיצד השליטה עוברת למערכת ההפעלה בעת הקריאה לפונקציות אשר מכילות קריאת מערכת, מוצגת לפניכם דוגמה של תוכנית C המדפיסה את ההודעה "hello world" על המסך תוך שימוש בפונקציית write אשר מכילה קריאה לשירות של מערכת ההפעלה. הקריאה נועדה לבצע פעולת פלט למסך, ובהמשך מוצגת תוכנית בשפת-סף (assembly language) אשר מבצעת את אותה המשימה.

```
#include <unistd.h>
#include <string.h>
#deine STDOUT 1

int main(){
        char *msg[] = "Hello, world\n\0";
         if (write(STDOUT, msg, strlen(msg)) < 0)
                        exit(1);                        // write to screen  failed
         return(0);                                     // success
}
```

```
 1:          .data                   # Data section
 2:
 3: msg:   .asciz "Hello, world.\n"      # The string to print.
 4:       len = . - msg - 1          # The length of the string.
 5:
 5:       .text                           # Code section.
 6:       .global _start
 7:
 8: _start:                               # Entry point.
10:       pushl          $len     # Arg 3 to write: length of string.
11:       pushl          $msg     # Arg 2: pointer to string.
12:       pushl          $1       # Arg 1: file descriptor.
13:       movl           $1, %eax         # Write.
14:       call    do_syscall
15:       addl           $12, %esp        # Clean stack.
16:
```

[2] קריאת מערכת איננה פונקציה של שפת תכנות עילית. פונקציות של שפת תכנות עילית הן עטיפה לקריאות מערכת.

האוניברסיטה הפתוחה THE OPEN UNIVERSITY OF ISRAEL الجامعة المفتوحة | Yair Kurtzman

File #0006182 belongs to Yair Kurtzman- do not distribute


```
17:     pushl       $0              # Exit status.
18:     movl        $1, %eax        # Exit.
19:     call    do_syscall
20:
21: do_syscall:
22:     int     $0x80           # Call kernel.
23:     ret
```

בשורות 10-12 מתבצעת העברה של שלושה פרמטרים המכילים אינדיקציה של יעד הפלט (STDOUT), מצביע להודעה הנפלטת ואת מספר התווים שבהודעה. בשורה 13 מועבר מספר השירות המסופק על ידי מערכת ההפעלה (במקרה שלנו – write). בשורה 22 מתבצעת הוראת TRAP שמעבירה את המעבד למצב מיוחס וגורמת למעבר השליטה למערכת ההפעלה. מערכת ההפעלה תבדוק את הפרמטרים שהועברו בשורות 10-13, ואם הכול כשורה – היא תספק למשתמש את השירות המבוקש, דהיינו תבצע הדפסה של הודעה למסך.

אם אתם רוצים להכיר טוב יותר את סוגי קריאות המערכת הקיימות, תוכלו לעיין בפרקים 1.6.1-1.6.5. סעיפים אלו כוללים בעיקר חומר טכני, והם חומר רשות בקורס זה.

1.7 המבנה של מערכות הפעלה

סעיף זה עוסק במבנה של מערכות הפעלה מנקודת מבט של עיצוב. האפשרויות המוצגות כאן נושאות אופי מופשט במידה מסוימת:

- ***מערכות מונוליתיות*** **(*monolithic systems*)** – מערכת מונוליתית היא למעשה אוסף גדול של פונקציות שונות שפועלות כיחידה אחת במצב מיוחס. ראו איור 1-24 בספר הלימוד.
- ***מערכות מרובדות*** **(*layered systems*)** – אלו הן מערכות שמרכיביהן מאורגנים ברבדים אשר נמצאים ביחס היררכי. לכל רובד מוגדר ממשק שדרכו הרבדים הסמוכים בהיררכיה יכולים לתקשר ביניהם. ראו איור 1-25 בספר הלימוד.
- ***מכונה מדומה*** **(*virtual machine*)** – בסעיף 1.1 ראינו שאחד התפקידים של מערכת הפעלה הוא אספקה של שירותים של המכונה המורחבת ושל המכונה המדומה. ניתן להפריד את שתי הקונספציות הללו ולראות את מערכת ההפעלה כשכבת תוכנה שמספקת לשכבות שמעליה כמה העתקים של החומרה עם כמות משאבים שווה או פחותה מזו שבחומרה הפיזית בפועל. במובן מסוים זו אינה מערכת הפעלה רגילה, שכן על כל מכונה מדומה יכולה לרוץ מערכת הפעלה חדשה, ואולי גם אותה מערכת שתשכפל את עצמה. בספר הלימוד מתוארת מערכת ההפעלה VM/370 בהקשר למכונה מדומה. במהלך הקורס נשתמש במכונה מדומה של חברת vmware, שלא ניתן לכנותה מערכת הפעלה מכיוון שהיא רצה על גבי מערכת הפעלה אחרת.

- ***מערכות exokernel*** (חומר רשות) – במערכות אלה השירותים המסורתיים כגון מערכת קבצים, ניהול זיכרון, תזמון תהליכים ותקשורת הוצאו אל מחוץ לגרעין של מערכת ההפעלה. ההחלטה הזאת נובעת מכך שמערכת ההפעלה איננה כופה על המשתמש את צורת השימוש במשאבים. מערכת ההפעלה דואגת רק להגנה על המשאבים ולהקצאתם. המערכת מקצה משאבים למערכות הספריות ותוכניות המשתמש, שיכולות לעשות שימוש במשאבים אלו בצורה ייחודית רק להן. כל תוכנית יכולה להשתמש בספריות מוכנות המממשות אבסטרקציות כגון קבצים וזיכרון מדומה או, לחלופין, להשתמש במשאבים בצורה ייחודית באמצעות ספריות ייעודיות. כך באופן תיאורטי, מערכת exokernel יכולה לספק ספריות שמאפשרות להריץ עליה את מערכות ההפעלה Windows או Linux.
- **מערכות גרעין מינימליסטי (micro-kernel)** – במערכת מרובדת לא כל פעולות המערכת חייבות להתבצע במצב מיוחס, ולכן ניתן להוציא את השירותים האלה מגרעין המערכת ולבצע אותם במצב משתמש. הגישה הזאת מאפשרת להקטין את גודל גרעין המערכת ולשפר את יציבות המערכת, מכיוון שתקלות במצב המשתמש אינן גורמות לתקלות כלליות במערכת.
- **מערכות שרת-לקוח (client-server systems)** – מערכות שרת-לקוח מבוססות על הרעיון של **גרעין מינימליסטי (micro-kernel)** – גרעין מערכת ההפעלה מתפקד כדוור המריץ בקשות של לקוחות (תהליכים) אל שרתים עצמאיים (תהליכים גם הם), המפוזרים במערכת, כגון שרתי מערכת הקבצים, מנהל הזיכרון, תוכנית התקשורת ותוכנית המסך (חלונות). כאשר מתקבלות תשובות מהשרתים, הגרעין מחזיר את התשובה לתהליך המבקש. בשיטה זו התוכנה מורכבת מיחידות עצמאיות, שכל אחת מהן ממלאת תפקיד מוגדר. עם כל יחידה כזו ניתן להידבר רק באמצעות מספר מוגבל של הודעות, דרך ערוץ תקשורת פנימי או חיצוני כלשהו. שיטה זו מצמצמת מאוד את התלות בחומרה.

שאלה 2

מה היתרון של מערכת שמרכיביה מאורגנים ב**רבדים** (קליפות) לעומת מערכת בעלת מבנה מונוליתי?

שאלה 3

מה הופך את המכונה המדומה המתוארת בסעיף 1.7.3 למערכת הפעלה?

שאלה 4

מהם היתרונות של מערכות שרת-לקוח?

- לחלוקה שבאיור 1-27 בספר הלימוד יש השלכות על מידת הקושי בהרכבת הרכיבים ליחידה פונקציונלית.
- מערכות שרת-לקוח ניתנות לביזור בקלות. דרכי התקשורת בין מרכיבי המערכת יכולות להיות אפיקים מהירים או רשתות מחשבים.

האוניברסיטה הפתוחה
THE OPEN UNIVERSITY OF ISRAEL
الجامعة المفتوحة
Yair
Kurtzman

File #0006182 belongs to Yair Kurtzman- do not distribute

- המודל שרת-לקוח ניתן להרחבה – מערכת הקבצים יכולה גם היא, בתורה, להיות בנויה מיחידות-משנה: לדוגמה, יחידת-משנה העוסקת בחומרה (בדיסק למשל), יחידת-משנה המטפלת במבנה הלוגי של מערכת הקבצים ויחידה נוספת המספקת גיבוי מתמיד של הקבצים.

שאלה 5

לאור היתרונות המובהקים של מודל זה, מדוע הוא איננו מיושם תמיד?

סעיפים שאינם מופיעים בספר הלימוד	
1.8	סיכום
1.9	שאלות חזרה
1.10	רשימת מושגים

1.8 סיכום

נמנה כמה מהנושאים המרכזיים שנלמדו בפרק זה:

- שני התפקידים של מערכות ההפעלה
- קריאות מערכת
- המושג "מכונה מדומה"
- סוגים שונים של מבנה ותפיסה לגבי מערכת ההפעלה
- מושגים יסודיים: קבצים, תהליכים, ניהול זיכרון ועוד

פרק זה הוא קל יחסית ללימוד, אם כי המושגים המופיעים בו אינם פשוטים. מומלץ כי תנצלו את הזמן עתה, כל עוד לא נכנסתם בעובי הקורה, ללימוד החלקים הטכניים הקשורים לשימוש במערכת ההפעלה Linux ולהכרה של סביבת תכנות.

1.9 שאלות חזרה

ענו על השאלות (כל התשובות נמצאות בחומר הלימוד, בפרק עצמו או במדריך זה).

1. האם תפקידה של מערכת ההפעלה כמכונה מורחבת מקבל ביטוי במערכת ההפעלה VM/370?
2. האם ניתן להשיג הגנה על משאבי מערכת ההפעלה ללא תמיכה של מעבד בשני מצבים לפחות: מצב משתמש ומצב מיוחס?
3. אמתו או הפריכו את הטענה: קריאות מערכת אינן גורמות למעבר למצב מיוחס.
4. אמתו או הפריכו את הטענה: פונקציות של שפת תכנות עילית מבוצעות תמיד במצב מיוחס.

1.10 רשימת מושגים

להלן רשימת המושגים המופיעים בפרק זה (על פי סדר הופעתם בפרק):

מכונה מדומה	virtual machine	זהו תפקידה של מערכת ההפעלה מנקודת מבטו של המשתמש (המתכנת) – "מלמעלה למטה". מכונה מדומה היא שכבת תוכנה שנמצאת מעל החומרה ומתחת ליישומי תוכנה אחרים, ומספקת למשתמש ממשק שמאפשר שימוש בחומרה. באופן כללי, המונח שקול למונח "מכונה מורחבת", אך המונח "מכונה מדומה" מופיע בפרק 1 של ספר הלימוד גם בהקשר נוסף – מערכת ההפעלה VM/370 של IBM. במימוש ספציפי זה, מדובר בשכבת תוכנה שמספקת לשכבות שמעליה כמה העתקים מדויקים של החומרה. כלומר, שכבה זו אינה בגדר מכונה מורחבת.
מכונה מורחבת	extended machine	שכבת תוכנה המספקת למשתמש ממשק לחומרה שהוא נוח וקל לתכנות, ובכך מסתירה מהמשתמש את מורכבות הפעולה האמיתית של החומרה. מכונה מורכבת מספקת למשתמש מנגנונים נוחים לשימוש שאינם ממומשים בחומרה, כמו ארגון מידע בקבצים ושליטה נוחה ומופשטת בהתקני קלט/פלט.
מנהל משאבים	resource manager	תפקידה של מערכת הפעלה בהסתכלות "מלמטה למעלה". מחשב מורכב מרכיבים רבים ושונים; מנהל משאבים הוא תוכנה שמנהלת את הקצאת המשאבים האלה לתהליכים שרצים במערכת.
משתמש	user	בהקשר של מערכות הפעלה, המונח יתייחס בדרך כלל למתכנת יישומים. מתכנת משתמש ביישומיו בשירותים שמספקת מערכת ההפעלה. יש להבדיל מובן זה מהמובן הנפוץ יותר של המונח, המציין אדם שמשתמש ביישום מוכן כלשהו (כמו מסד נתונים, סייר האינטרנט או משחק מחשב).
מערכת אצווה	batch system	מערכת שיכולה לבצע באופן רצוף סדרה של משימות שהוכנה מראש. במערכת כזו כל משימה מתבצעת מתחילתה ועד סופה בלי לשתף את המעבד בתהליכים אחרים, כפי שנעשה במערכות בעלות timesharing.
ריבוי תוכניות	multiprogramming	שיטה שבה תהליכים אחדים יכולים להימצא בזיכרון הראשי בו-זמנית. שיטה זו מייעלת את ניצול המעבד על ידי הקצאת המעבד לתהליך שממתין לו, בזמן שתהליך שהתבצע קודם ממתין לאירוע כלשהו (למשל, לקבלת קלט מהדיסק).
חלוקת זמן	timesharing	שיטה שבה המערכת מאפשרת לכמה משתמשים אינטראקטיביים לעבוד בו-זמנית, כל אחד מהמסוף שלו, ומבטיחה לכולם תגובה מהירה לפקודות שמתקבלות מהמסופים. בשיטה זו כל משתמש מקבל את הקצאת המעבד לטובת ביצוע העבודות שלו לפרק זמן קצר בלבד, שבסיומו ההקצאה עוברת למשתמש אחר. העבודות של המשתמש שממנו נלקח המעבד ממשיכות לרוץ כאשר המשתמש מקבל את המעבד בפעם הבאה.
גרעין	kernel	החלק המרכזי והקריטי ביותר של מערכת הפעלה, האחראי לביצוע התפקידים העיקריים שלה. לרוב, הגרעין יכיל

האוניברסיטה הפתוחה
THE OPEN UNIVERSITY OF ISRAEL
الجامعة المفتوحة
Yair Kurtzman

File #0006182 belongs to Yair Kurtzman- do not distribute

		מרכיבים שאחראים על ניהול פסיקות, תזמון תהליכים, ניהול זיכרון, ניהול שטחי אחסון משניים ועוד. התצורה המדויקת של הגרעין תלויה במבנה (architecture) של כל מערכת הפעלה. הגרעין נמצא באופן קבוע בזיכרון הראשי של המחשב.
מצב מיוחס (ראשוני)	kernel mode	המצב הפריווילגי של המעבד שבו ניתן לבצע כל פעולה אפשרית ללא הגבלות. חלקים של מערכת ההפעלה שמחייבים שימוש בפעולות "רגישות", כמו למשל כתיבה להתקני I/O, רצים במצב זה. לעומת זאת, תוכניות רגילות רצות תמיד רק במצב משתמש.
מצב משתמש	user mode	המצב המוגבל של המעבד, שבו לא ניתן לבצע פעולות "רגישות" כגון כתיבה להתקני I/O.
קריאת מערכת	system call	קריאת מערכת היא הדרך שבה תהליך משתמש מבקש ממערכת ההפעלה שירות כלשהו (כמו יצירת תהליך חדש, פתיחת קובץ, שליחת מידע ברשת). קריאת מערכת מתבצעת על ידי השמת הפרמטרים הדרושים במקומות מוגדרים (כמו רגיסטרים) ואז ביצוע פקודת TRAP אשר מעבירה את השליטה למערכת ההפעלה. קריאת מערכת בשפה עילית כמו, למשל, read() בשפת C, היא פרוצדורת מעטפת אשר נראית מנקודת מבטו של המשתמש כפרוצדורת ספרייה רגילה, ותפקידה העיקרי הוא לשים את הפרמטרים של הקריאה ברגיסטרים ולבצע TRAP.
תהליך	process	תוכנית ריצה (executable) בביצוע. תהליך מכיל את התוכנית עצמה (code), את הנתונים שהיא יוצרת ומשתמשת בהם (data), מצביע למחסנית, מונה פקודות ואוגרים אחרים שמכילים מידע על הסטטוס הנוכחי של הביצוע. כמו כן, תהליך מכיל מידע על הקבצים שנמצאים בשימושו ונתונים רבים אחרים.
מעטפת, מעבד פקודות סדרתי	shell, command interpreter	תוכנית המהווה ממשק אינטראקטיבי למערכת ההפעלה. מעבד הפקודות מקבל פקודות שניתנות על ידי המשתמש ומבצע אותן על ידי שימוש בשירותי מערכת ההפעלה. אף שכל מערכת הפעלה מסופקת בדרך כלל עם תוכנת מעבד פקודות, התוכנה אינה חלק מובנה של מערכת ההפעלה אלא יישום שרץ במצב משתמש, ובעבור אותה מערכת הפעלה ניתן לכתוב מעבדי פקודות שונים.
ניידות התוכנה	software portability	תכונה המאפשרת ריצת תוכנה על מערכות הפעלה וארכיטקטורות מחשב שונות בלי שתידרש עבודה רבה כדי לגרום לתוכנה שרצה על פלטפורמה אחת לרוץ על פלטפורמה אחרת. אפשר להבחין בין שני סוגי ניידות עיקריים: ניידות הקוד וניידות בינארית. הסוג הראשון של ניידות מתייחס למצב שבו ניתן ליצור תוכנית ריצה על ידי קומפילציה של הקוד על כל פלטפורמה בלי לבצע שינויים בקוד המקורי. הסוג השני של ניידות מתייחס לאפשרות לקחת קבצים בינאריים אשר הוכנו על פלטפורמה אחת ולהשתמש בהם (כפי שהם) על פלטפורמה אחרת.
תכנות מונחה עצמים	OOP: Object-Oriented Programming	גישה תכנותית המציגה את העולם כאוסף של עצמים, שבו לכל עצם יש תפקיד מוגדר. עצם (אובייקט) מוגדר על ידי אוסף של מאפיינים ושיטות (methods) שהעצם יודע לבצע.
גרעין מינימלי	minimal kernel	המונח מתייחס למערכות הפעלה שבהן כל השירותים של המערכת אינם חלק מהגרעין, אלא ממומשים כתהליכים

		נפרדים שכל אחד מהם אחראי על מתן שירות מסוים. כל תהליך כזה (תהליך שרת) יכול לקבל בקשות לקוח, לטפל בהן ולשלוח ללקוח את התוצאות. במקרה זה תפקידו של הגרעין הוא לספק מנגנון התקשרות (העברת בקשות ותוצאות) בין תהליכי לקוח לבין תהליכי שרת.

האוניברסיטה הפתוחה
THE OPEN UNIVERSITY OF ISRAEL
الجامعة المفتوحة
Yair
Kurtzman

File #0006182 belongs to Yair Kurtzman- do not distribute

פרק 2

תהליכים ותהליכונים

האוניברסיטה הפתוחה
THE OPEN UNIVERSITY OF ISRAEL
الجامعة المفتوحة
Yair
Kurtzman

File #0006182 belongs to Yair Kurtzman- do not distribute

2.0 חזרה על מבנה המעבד [1.3.1 בספר הקורס]
2.1 מבוא לתהליכים
2.1.1 מודל התהליכים
2.1.2 יצירת תהליכים
2.1.3 סיום תהליכים
2.1.4 היררכיה של תהליכים
2.1.5 מכונת מצבים
2.1.6 יישום תהליכים
2.1.7 מודל לריבוי תוכניות

אחת המשימות העיקריות של כל מערכת הפעלה היא ניהול ***תהליכים***. תהליכים נמצאים בכל עת במצב נתון, כגון *מצב חסום*, *מצב מוכן לריצה* או *מצב ריצה*. מערכת ההפעלה דואגת לעקוב אחר המצבים האלה ואוכפת את מדיניות הגישה של התהליכים הרצים בה למשאבי המערכת. לצורך כך היא מתחזקת מבני נתונים המשמשים לניהול התהליכים.

אין ספק כי התהליך הוא המושג המרכזי בנושא מערכות ההפעלה. ייעודה של מערכת הפעלה הוא לאפשר את קיומם הבטוח ואת ביצועם המהיר של תהליכים שונים. כמו במערכות אנושיות, השאיפה היא שמספר גדול של תהליכים במערכת נתונה יפעלו באופן מיטבי וינצלו לשם כך את מרב המשאבים. בתוך כך, קורה לעתים קרובות שתהליכים נקלעים לתחרות על שליטה במשאבים, ופעולתם משתבשת בשל כך. הפתרון הוא שתהליכים אחרים ישליטו סדר במאבק זה. בין התהליכים נוכל למצוא את הבירוקרטיה המקומית – תהליכי המערכת. ישנם גם הפושעים. אלה מנסים לגנוב מידע או להשתלט בצורה בלתי חוקית על משאבי המערכת; כנגדם פועלים תהליכי ביטחון מקומיים. זוהי רק השוואה חלקית אפשרית בין העולם המלאכותי הקיים במערכת ההפעלה לבין ארגון אנושי. השוואה זו מעניקה רובד מעניין נוסף לנושאי הפרק.

טננבאום בחר להתמודד כבר בתחילת ספרו עם נושא שיתוף הפעולה בין תהליכים, שהוא אחד הנושאים המסובכים ביותר להבנה בקורס כולו. על כן, אם אתם מתקשים להבינו בקריאה ראשונה – אל ייאוש! הנושא הוא טכני בעיקרו, אך עם זאת הוא בעל חשיבות עצומה. קראו שוב ושוב את הדוגמאות המובאות במדריך ובספר, עד שתהיו בטוחים כי אתם מבינים את מהות הבעיה ואת דרך פתרונה.

2.0 חזרה על מבנה המעבד [1.3.1 בספר הקורס]

בטרם נמשיך לעסוק במודל התהליכים, נחזור בקצרה על מושגים אחדים מהקורס "ארגון המחשב ותכנותו" ונרענן את ידיעותינו בנוגע לאופן הפעילות של המעבד המרכזי (בהמשך נכנה אותו "המעבד" או ה-CPU). בספר הלימוד, החזרה מופיעה בסעיף 1.3.1. במדריך הלמידה נרחיב בנושא כאן.

האוניברסיטה הפתוחה THE OPEN UNIVERSITY OF ISRAEL الجامعة المفتوحة
Yair Kurtzman

File #0006182 belongs to Yair Kurtzman- do not distribute

לצורכי הקורס שלנו, נתמקד רק בביצוע של הוראות מכונה על ידי המעבד, בטיפול ב**פסיקות חומרה** ו**באוגרים** הכלליים.

אוגרים הם סוג של זיכרון (קטן אך מהיר) המשמש לצמצום מספר הפניות לזיכרון הראשי ולצורך בקרה על פעולות המעבד. כמה מסוגי אוגרי הזיכרון הם ***data registers*** ו-***address registers***. לקבוצה של אוגרי הבקרה שייכים בין השאר: ***program counter*** ***(PC)***, ***instruction register*** ***(IR)***, ***program status word*** ***(PSW)***.

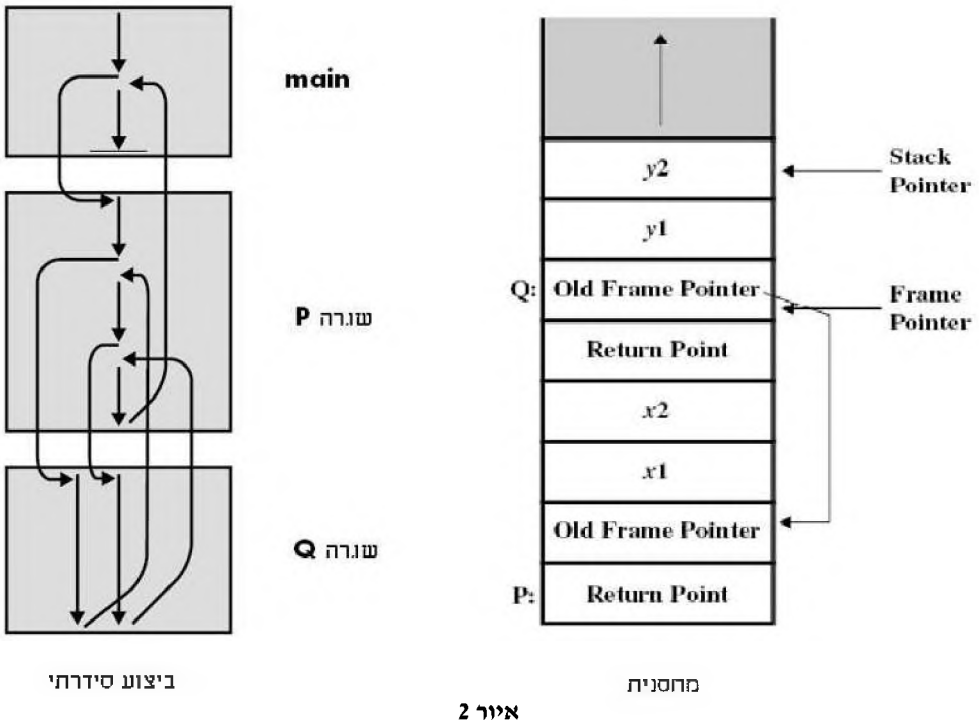
אוגרי data יכולים לשמש מתכנתים למטרות שונות. כך, בשפת C אפשר להציע למהדר לשמור משתנה באוגר (במקום בזיכרון הרגיל) על ידי המילה השמורה *register* בזמן הצהרת המשתנה. אוגרי address שומרים כתובות של הוראות מכונה או של data. עם זאת, הן אוגרי data והן אוגרי address יכולים לשמש גם לתפקידים אחרים. *stack pointer* שומר כתובות של מחסנית, ואוגרים מיוחדים של data משמשים לביצוע של פעולות בנקודה צפה.

קבוצה חשובה ומיוחדת של אוגרים משמשת לבקרה על פעולות המעבד. רובם המוחלט של אוגרי הבקרה אינו "נראה" למשתמש או לפחות אינו ניתן לעדכון על ידי המשתמש. כך *program counter* מחזיק את הכתובת של הוראת המכונה הבאה אשר אמורה להתבצע במעבד. *instruction register* מחזיק את ההוראה (OpCode) הנוכחית שמתבצעת. קיימת חשיבות מכרעת לאוגר (או לקבוצת האוגרים – תלוי במעבד הספציפי) המחזיק את המצב הנוכחי של המעבד – *Program Status Word*. ב-*PSW* מוחזקת בין היתר הסיבית המציינת אם המעבד נמצא במצב מיוחס או במצב משתמש.[3] אוגרי בקרה נוספים משמשים לאחזקה של ***condition codes***, שבהם המעבד מאחסן את המידע על התרחשויות חריגות. למשל, בעקבות חלוקה ב-0 ידליק המעבד דגל מיוחד. כך, במערכת ההפעלה UNIX, חלוקה ב-0 תוביל לסיום התהליך שגרם לכך. בהמשך הקורס נלמד כיצד אפשר להורות למערכת ההפעלה להתנהג אחרת.

התוכניות שהמעבד מריץ מבוצעות על ידי סדרה של ***instruction cycles*** המורכבים מפעולות שליפה של הוראות מכונה וביצוען על ידי המעבד. ההוראות נשלפות מהזיכרון הראשי או מהחוצצים המרכיבים את ההיררכיה של הזיכרון שעליה נלמד בפרק 3. הכתובת של ההוראה הבאה שאמורה להתבצע מוחזקת ב-program counter. לאחר ביצוע ההוראה, המעבד מעדכן את ה-program counter ומקדם אותו כדי שיצביע לכתובת של ההוראה הבאה. בשלב הבא המעבד שולף את ההוראה ומאחסן אותה ב-instruction register. בשלב שלאחריו, המעבד מפענח את ההוראה (לפי קוד ההוראה) ומבצע אותה.

כעת נזכיר כיצד מתבצעת הקריאה לשגרות. קריאות לפונקציות שאינן מקבלות ארגומנטים ואינן מחזירות ערכים אינן שונות לחלוטין מביצוע סדרתי של הוראות מכונה. באיור 2 משמאל: שגרה P נקראת מ-main ו-P קוראת פעמיים ל-Q.

[3] במעבדי Pentium משמשות למטרה זו שתי סיביות המאפשרות ארבע רמות אבטחה שונות.



איור 2

המצב שונה כאשר צריך להעביר לפונקציה פרמטרים. אם הפרמטרים יועברו באמצעות אוגרים מסוימים, צריך לוודא בכל קריאה לשגרה שהתוכניות המשתמשות בשגרה לא יעשו שימוש באוגרים הללו. דבר זה יכול להקשות מאוד את תהליך פיתוח התוכנה. בנוסף, צריך להתמודד עם הקושי הכרוך בהעברת מספר משתנה של ארגומנטים באמצעות אוגרים שמספרם קבוע. אפשרות אחרת היא להשתמש בזיכרון הראשי, אך הפתרון הזה איטי יותר. אותם השיקולים בדיוק רלוונטיים גם להחזרת ערכים משגרות.

כדי לשלב בין מהירות העברת הפרמטרים לבין הגמישות, העברת פרמטרים והחזרת ערכים מתבצעות באמצעות מחסנית. באיור 2 מימין אפשר לראות שלשגרות P ו-Q מוקצות על המחסנית מסגרות (***stack frames***). המסגרות מכילות פרמטרים וערכי חזרה (ב-C הן מכילות גם משתנים מקומיים שמוגדרים בפונקציה). האוגר ***stack pointer*** מחזיק את הכתובת שמעל המסגרת העליונה במחסנית. המסגרת של השגרה Q מחזיקה את כתובת ה-stack pointer הישן (מעל המסגרת של השגרה P). כך, לאחר סיום השגרה Q, תופיע בראש המחסנית המסגרת של השגרה P.

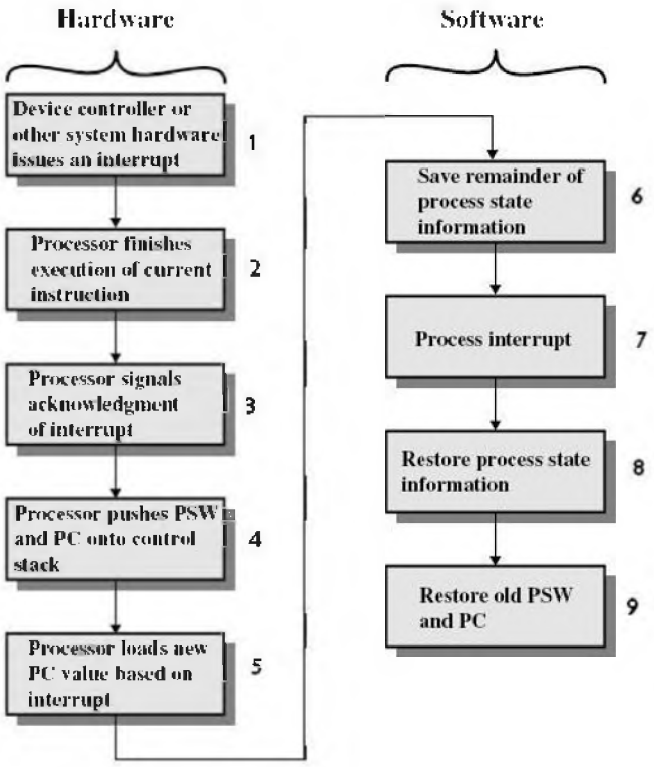
כעת ניזכר כיצד משתלבות בתמונה פסיקות החומרה. פסיקות חומרה נועדו בעיקר להגדיל את ניצולת ה-CPU. בקורס "מבוא למדעי המחשב" כבר נתקלתם בדוגמה כזו – ביצוע של I/O. תוכנית שמבקשת לבצע I/O – קריאה או כתיבה מההתקן – מושהית על ידי מערכת ההפעלה עד לסיום פעולת ה-I/O. בזמן ההשהיה תתבצע תוכנית אחרת ב-CPU. כאשר ההתקן שעליו התבצעה פעולת I/O יסיים את משימתו, יישלח איתות בקשה לפסיקה ל-CPU. עם קבלת האיתות, המעבד משהה מיד את המשך הביצוע של התוכנית שהוא מבצע כעת, ומפעיל שגרה מיוחדת הנקראת ***שגרה לטיפול בפסיקות*** (***interrupt handler routine***). השגרה לטיפול בפסיקות היא בדרך כלל חלק ממערכת

האוניברסיטה הפתוחה
THE OPEN UNIVERSITY OF ISRAEL
الجامعة المفتوحة
Yair
Kurtzman

File #0006182 belongs to Yair Kurtzman- do not distribute

ההפעלה. לאחר סיום השגרה לטיפול בפסיקות, ה-CPU יכול לחזור לביצוע התוכנית המקורית שביצע בטרם קיבל את הפסיקה. עם זאת, ייתכן שהעניינים יתנהלו לפי תסריט אחר. השגרה לטיפול בפסיקות יכולה להורות להפעיל מתזמן תהליכים שיכול "לתת" את ה-CPU לתוכנית אחרת (למשל, לאותה התוכנית שביקשה ביצוע I/O).

המעבר לביצוע של השגרה לטיפול בפסיקות יוצר תקורה, מכיוון שצריך לבצע ***שמירת תוכן*** ***(context)*** של התהליך כדי לאפשר שחזור של ביצוע התוכנית מהנקודה שבה הופסק. שמירת התוכן מתבצעת על ידי החומרה, ורשימת התכנים הנשמרים תלויה במעבד (שלב 6 באיור הבא). שמירת ה-PSW וה-PC מתבצעת על ידי המעבד עצמו (שלב 4) אשר קורא לשגרה לטיפול בפסיקות (בשלב 5). בצורה סכמטית, אפשר לחשוב על התהליך כך:



איור 3

עקב ריבוי הארכיטקטורות השונות של המעבד, ייתכנו גם אפשרויות אחרות. כאן לא נרחיב בנושא זה.

הנושא האחרון שעלינו לעסוק בו במסגרת החזרה הוא הטיפול ב***ריבוי פסיקות***.

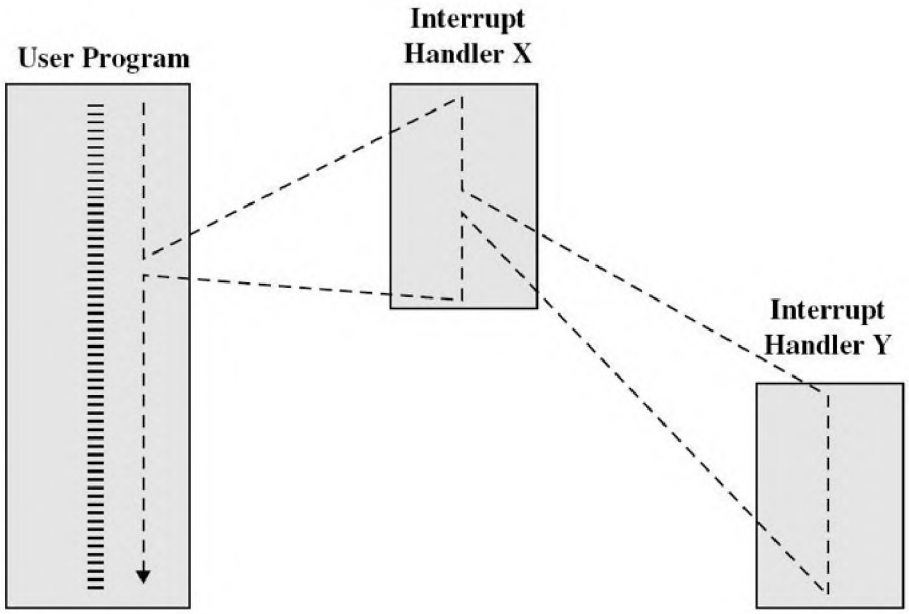
שאלה 1

מה קורה כאשר בעת הטיפול בפסיקה, מגיע איתות חדש לבקשת פסיקה?

אפשר לנקוט אחת משתי גישות. בגישה הראשונה תיחסם קבלת איתותים בזמן שמתבצע טיפול בפסיקה. עם סיום הטיפול בפסיקה, המעבד יבטל את החסימה ויבדוק אם הגיעו בקשות לפסיקות חדשות בזמן שטיפל בפסיקה הקודמת. לפשטותה של הגישה יש חיסרון משמעותי – היא אינה מתמודדת היטב עם פסיקות המחייבות טיפול מהיר

מאוד. למשל, אם תגיע פסיקה המבקשת קריאה מדויקת של שעון, לא סביר לצפות שהיא תמתין לסיומה של הפסיקה הקודמת.

לכן, ברובם המכריע של המעבדים יש אפשרות לקבוע סדר קדימויות של טיפול בפסיקות. בגישה זו (ראו איור למטה) פסיקות בעלות עדיפות גבוהה יכולות לגרום להשהיית הטיפול בפסיקות בעלות עדיפות נמוכה. התוכן של הפסיקה המושהית X, שהיא בעלת עדיפות נמוכה, נשמר, ומתבצע טיפול בפסיקה בעלת העדיפות הגבוהה Y. לאחר הטיפול ב-Y משוחזר התוכן של X. בגישה הזאת יתבצע טיפול בפסיקה רק כאשר לא ממתינות פסיקות בעלות עדיפות גבוהה יותר. לביצוע התוכנית עצמה תהיה העדיפות הנמוכה ביותר.



איור 4

2.1 מבוא לתהליכים

בסעיף זה מוצגת ההבחנה החשובה בין ***מקביליות אמיתית***, אשר בה שני תהליכים לפחות מתקדמים בו-זמנית במעבדים מקביליים, לבין ***מקביליות מדומה***, שהיא אשליה הנובעת מחילוף תהליכים מהיר על מעבד אחד.

מחשבים מודרניים מכילים מספר גדול של משאבים כגון מעבדים, זיכרון, התקני קלט/פלט, דיסקים. מערכת ההפעלה מספקת ייצוג מופשט של משאבי מערכת המחשב, דבר שמקל את כתיבת הקוד והופך את תהליך הכתיבה לבלתי תלוי בחומרה. מטרה נוספת המושגת באמצעות הפשטה היא האפשרות להריץ במחשב מספר גדול של אפליקציות ולאפשר שיתוף של המשאבים ביניהן.

תהליך הוא ישות מופשטת שמערכת הפעלה מספקת לייצוג של תוכנית בריצה. תהליכים משקפים את המצב הנוכחי של ריצת התוכנית ואת המשאבים השייכים לה. חלק

האוניברסיטה הפתוחה
THE OPEN UNIVERSITY OF ISRAEL
الجامعة المفتوحة
Yair
Kurtzman

File #0006182 belongs to Yair Kurtzman- do not distribute

מהמשאבים של המערכת יכולים להיות משותפים לכמה תהליכים. בפרט, שיתוף של המעבד המרכזי מאפשר ליצור אשליה של ריצה בו-זמנית של כמה אפליקציות במחשב עם מעבד מרכזי אחד. אשליה זו מושגת על ידי חילוף מהיר של תהליכים שרצים על המעבד המרכזי. לחלופין, אם קיימים במערכת כמה מעבדים, ניתן להקצות אותם למספר זהה של תהליכים בו-זמנית.

2.1.1 מודל התהליכים

נפתח בכמה מילים על המושג "מודל". הואיל והמציאות עצמה מורכבת מדי לניתוח, אנו מצמצמים את הדיון התיאורטי למסגרת של הנחות יסוד פשוטות. למסגרת זו אנו קוראים מודל. כאשר להנחות יסוד אלו יש ניסוח פורמלי, נוכל להסיק מהן מסקנות נוספות באמצעות שיקולים לוגיים וכלים מתמטיים. ככל שההנחות משקפות את המציאות טוב יותר, כך גובר גם תוקף המסקנות.

ובכן, כל התוכנות שרצות במחשב, וביניהן לעתים גם מערכת ההפעלה, מאורגנות בצורת תהליכים. תהליכים משקפים את המצב הנוכחי של ריצת התוכנית ואת המשאבים השייכים לה. קונספטואלית, כל תהליך רץ על מעבד וירטואלי, כאשר באופן מעשי המעבד הפיזי (או כמה מעבדים, אם ישנם) עומד לרשות התהליך לזמן קצוב בכל פעם. בתום הזמן הזה התהליך הפעיל מתחלף, והמעבד עובר לרשותו של תהליך אחר. כך נוצרת מקביליות מדומה שמאפשרת ***ריבוי תהליכים*** (***multiprogramming***) במספר מצומצם של מעבדים.

ההבחנה בין ***תוכנית*** (***program***) לבין ***תהליך*** (***process***) חיונית להבנת המודל. משל אפיית העוגה מבהיר היטב נקודה זו. במשל זה המרשם לאפיית העוגה הוא הקוד של התוכנית, האופה הוא המעבד, מרכיבי העוגה הם הקלט של התוכנית והעוגה עצמה היא הפלט. התהליך של אפיית העוגה מורכב מקריאת המתכון, מהכנת המרכיבים, מפיקוח על תוצאת הביניים ולבסוף – מהגשת התוצאה הסופית. בחלק מהמתכונים אפשר לקטוע את רציפות תהליך האפייה לצורך ביצוע של משימות אחרות. למשל, ניתן להפסיק ללוש בצק ולענות לשיחת טלפון. בתום שיחת הטלפון אפשר להמשיך בלישה והדבר לא יפגע בתוצאה הסופית. האופה צריך רק לזכור באיזה שלב של ביצוע המתכון היה כדי להמשיך מאותו השלב.

שאלה 2

מהו ההבדל בין תוכנית לבין תהליך? אילו נתונים צריך להחזיק כדי לאפשר את קטיעת הביצוע של תהליך כך שהתוצאה הסופית לא תושפע מכך?

2.1.2 יצירת תהליכים

במערכת הפעלה רב-תכליתית יש כמה מקרים שבהם אפשר או צריך ליצור תהליכים חדשים:

1. בעת האתחול של מערכת ההפעלה נוצרים תהליכים שמטרתם לבצע משימות בעבור מערכת ההפעלה. לדוגמה, תהליכי רקע (daemons) כגון שרת Web או spooler של מדפסת.

2. תהליכים קיימים יכולים ליצור תהליכים חדשים. המטרה יכולה להיות שיפור המודולריות של תוכניות או ניצול של ריבוי מעבדים לשיפור המקביליות.
3. עבודות אצווה – מערכת ההפעלה מקבלת את ההוראות דרך ה-batch control stream ויוצרת לפיהן תהליכים חדשים. ניתן לראות צורה זו של יצירת תהליכים במחשבי mainframe, כאשר משתמשים רבים מבקשים לבצע עבודות אצווה כגון מפתוח (indexing) של בסיסי נתונים.

2.1.3 סיום תהליכים

כפי שאמרנו, תהליכים הם ישויות במודל התהליכים של מערכת הפעלה, ולכן הם עצמם מהווים משאבים של המערכת ודינם ככל משאב אחר בעולם החומרי. כמות המשאבים במערכת מוגבלת, ולכן כאשר תוכנית מסתיימת, גם התהליך המתאים צריך להסתיים. אנו נבדיל בין כמה סוגים של סיום תהליכים:

1. סיום רגיל – תהליך שמסתיים בעקבות סיום של תוכנית ומפנה את המשאבים שתפס.
2. סיום בעקבות שגיאה – סיום מתוכנן של תהליך בעקבות שגיאה במהלך ריצת התוכנית. למשל, תוכנית שמגלה שלא קיבלה מספר נכון של פרמטרים או תוכנת דוא"ל שנכשלה במשלוח הודעה, למשל בעקבות נפילת רשת.
3. סיום בעקבות שגיאה פטאלית – תהליך שמסתיים בעקבות שגיאה שלא נצפתה מראש ותרחיש בעייתי שקוד התוכנית לא סיפק לו מענה. סיום מן הסוג הזה גורם למה שמכונה קריסה (crash) של התוכנית.
4. סיום בעקבות הריגת תהליך – תהליך שמסתיים בעקבות בקשה מפורשת ממערכת ההפעלה לסיים את ריצתו. מערכת הפעלה שמקבלת בקשה כזאת בוחנת את הרשאות המבקש (המבקש יכול להיות תהליך אחר כלשהו, ובפרט – תהליך של מערכת ההפעלה עצמה), ואם קיימות הרשאות מתאימות, היא מסיימת את ריצתו של התהליך כמבוקש.

2.1.4 היררכיה של תהליכים

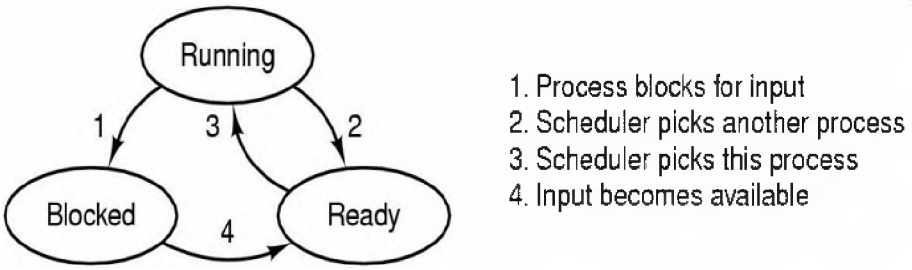
מודל התהליכים מתאפיין בהגדרה של ***מבנה היררכי*** בין תהליכי מערכת ההפעלה והגדרה של כללי שינוי המעמד בהיררכיה.

2.1.5 מכונת מצבים

סעיף זה מתאר את אחד המרכיבים החשובים ביותר של מודל תהליכים – ***מכונת מצבים***. המכונה מתארת את תקופות חייו של התהליך. אפשר לומר שמכונת מצבים היא לב לבו של מודל התהליכים ושככל שהמודל יהיה משוכלל יותר, כך ישרת את מטרתו טוב יותר. בספר מתואר מודל פשוט שבו שלושה מצבים:

האוניברסיטה הפתוחה
THE OPEN UNIVERSITY OF ISRAEL
الجامعة المفتوحة
Yair
Kurtzman

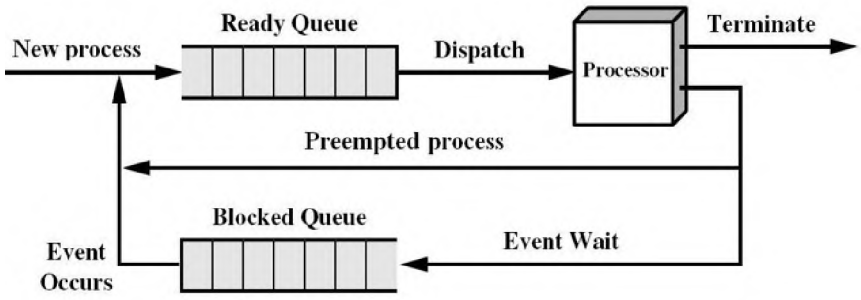
File #0006182 belongs to Yair Kurtzman- do not distribute



איור 5

- המעבר ממצב **רץ** למצב **חסום** מתבצע כאשר התהליך אינו יכול להתקדם עוד. תהליך במצב חסום יחכה לאירוע חיצוני שיגרום לו להשתחרר מחסימה. לדוגמה, תהליך שמבקש לקרוא קובץ מדיסק יכול להיחסם על ידי מערכת ההפעלה עד אשר תתבצע קריאת הקובץ. עם קריאת הקובץ, ישלח בקר הדיסק בקשה לפסיקה אשר תביא לשחרור התהליך ממצב חסום.
- המעבר ממצב **חסום** למצב **מוכן** יתבצע כאשר מתרחש האירוע **החיצוני** אשר בגללו התהליך מחכה במצב חסום. נחזור שוב לדוגמה של קובץ בדיסק: בקר הדיסק שולח בקשה לפסיקה, הפסיקה מטופלת על ידי מערכת ההפעלה שגורמת לשינוי של מצב התהליך והפעלת מתזמן המערכת.
- המעברים ממצב **רץ** למצב **מוכן** ולהפך מתרחשים כתוצאה מהחלטת מתזמן המערכת להפקיע את ה-CPU מתהליך רץ ולהקצותו לאחד התהליכים הממתין לו במצב מוכן.

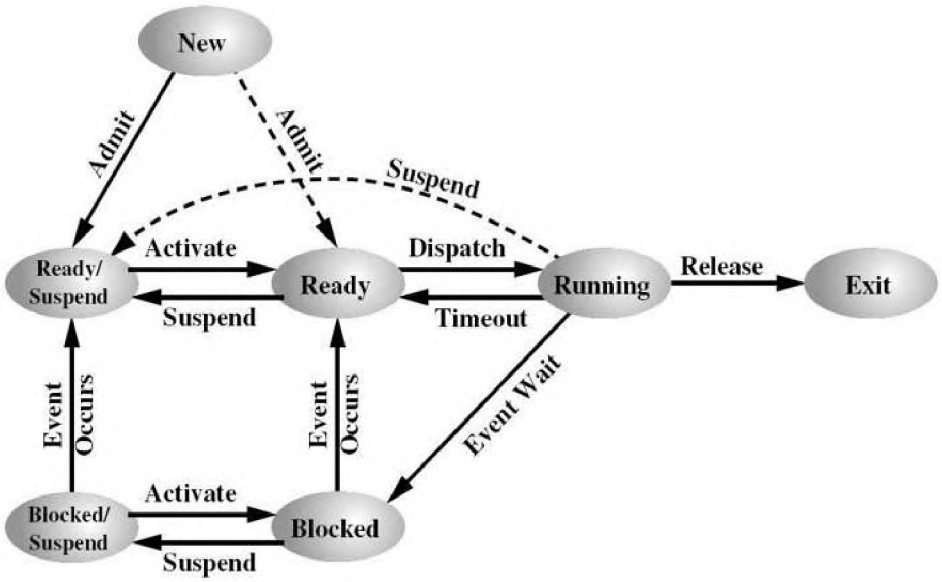
באופן מופשט, נחשוב על המימוש הזה של מודל מכונת המצבים:



איור 6

התהליכים במצב **מוכן** ו**חסום** מוחזקים בתורים נפרדים, כאשר כל תור יכול להיות מנוהל באמצעות מדיניות שנבחרה לכך. מובן שבמציאות המצב מורכב בהרבה.

כדי לתאר את רמת ההפשטה ואת רמת הקירוב של מכונת שלושת המצבים, דמיינו שהיינו מתארים את חייהם של בני אדם באמצעות המצבים **ישן**, **אוכל** או **ממתין לארוחה**. מובן שמכונות מצבים במערכות הפעלה הן שונות זו מזו, והשוני הזה נגזר מן המאפיינים השונים של מערכות ההפעלה. לדוגמה, **מכונת המצבים המשוכללת** באחת הגרסאות של מערכת ההפעלה UNIX נראית כך:



איור 7

מכונת המצבים המשוכללת נותנת ביטוי לעובדה שתהליכים נוצרים ומסתיימים.

שאלה 3

מהם השיקולים לטובת הוספת המצבים **חדש** ו**סיום**?

הוספת המצב **חדש** נגזרת משיקולי תזמון תהליכים. כפי שנראה בהמשך, הסיווג לקבוצות נעשה לפי טווחים שונים. תהליכים חדשים יכולים להיכלל בקבוצה של התהליכים אשר חייבים להיות מתוזמנים בטווח הקצר או בקבוצה של התהליכים שניתן להשהות את תזמונם עד למועד מאוחר יותר. בעקבות החלטת מערכת ההפעלה, התהליכים יכולים להיכנס למצב **מוכן** (בזיכרון הראשי, כלומר מוכן להרצה) או **מוכן בדיסק**.

הוספת מצב **סיום** נובעת מן הצורך במידע על התהליך שסיים לרוץ. ברוב מערכות ההפעלה תהליך שמסיים לרוץ משחרר את כל משאבי המערכת. אך מערכת ההפעלה שומרת רשומת מידע במבני נתונים של גרעין על התהליך שזה עתה סיים לרוץ. רשומת המידע הזאת כוללת נתונים על זמני ריצת התהליך במצב מיוחס ובמצב משתמש, קודים המתארים סטטוס סיום תהליך וכו'. המידע נדרש בעיקר לצורך ניהול נתונים סטטיסטיים שונים במערכת ההפעלה. שימוש חשוב נוסף הוא לצורך קבלת החלטות המסתמכות על אופן הסיום של התוכניות. למשל, בעקבות סיומה של התוכנית שנכשלה בהעתקת קובץ עקב שגיאה בדיסק, יש טעם להריץ תוכנית לבדיקה ולשחזור של מערכת קבצים. בכל אופן, בסיום השימוש במידע מתוך רשומת המידע אפשר וצריך שמערכת ההפעלה תמחק את הרשומה ממבני הנתונים של גרעין המערכת. איסוף נתונים סטטיסטיים יכול להיעשות אחרי העתקת זמני הריצה של תהליך ליומן הסטטיסטיקות, ובמקרה של העתקת קובץ – לאחר קריאת קוד השגיאה שהתרחשה.

האוניברסיטה הפתוחה
THE OPEN UNIVERSITY OF ISRAEL
الجامعة المفتوحة
Yair
Kurtzman

File #0006182 belongs to Yair Kurtzman- do not distribute

נתאר שוב בקצרה גם את המצבים הנותרים:

- ***רץ*** – מצב שבו נמצא תהליך שרץ כרגע במעבד הראשי.
- ***מוכן*** – מצב שבו נמצאים תהליכים אשר נמצאים בזיכרון הראשי וממתינים למעבד.
- ***מוכן בדיסק*** – מצב שבו נמצאים תהליכים אשר יהיו מוכנים לריצה עם טעינתם לזיכרון הראשי, אך כרגע הם נמצאים בזיכרון המשני (דיסק).
- ***חסום*** – מצב שבו נמצאים תהליכים אשר נמצאים בזיכרון הראשי וממתינים לאירוע כלשהו.
- ***חסום בדיסק*** – מצב שבו נמצאים תהליכים אשר ממתינים לאירוע כלשהו ונמצאים בזיכרון המשני.

כמו במקרה של מכונת שלושת המצבים, ניתן לחשוב שבאופן עקרוני מכונת המצבים המשוכללת ממומשת באמצעות תורים (הסתכלו באיור 5 במדריך למידה זה ושרטטו איור המתאים למכונת המצבים המשוכללת).

כעת נתאר את המעברים של המכונה המשוכללת:

- המעבר ממצב ***חסום*** למצב ***חסום בדיסק*** יכול להתרחש כדי לפנות יותר זיכרון ראשי. בעקבות המעבר, קטעי הזיכרון הראשי (או מה שמכונה גם ***מפת הזיכרון***) של התהליך המועבר יירשמו בדיסק בקובץ מיוחד שנקרא ***קובץ החלפה*** (***swap file***). שטחי הזיכרון הראשי שיתפנו יועמדו לטובת תהליכים מוכנים או לטובת התהליך הרץ.
- המעבר ממצב ***חסום בדיסק*** למצב ***מוכן בדיסק*** יכול להתרחש עם קרות האירוע אשר לו ציפה התהליך החסום.
- המעבר ממצב ***מוכן בדיסק*** למצב ***מוכן*** יכול להתרחש כאשר אין תהליכים במצב ***מוכן*** או כאשר עדיפותו של תהליך שהיה במצב ***מוכן בדיסק*** עולה עד כדי כך שהיא גוברת על הצורך בהעתקת מפת הזיכרון מהדיסק לזיכרון הראשי.
- המעבר ממצב ***מוכן*** למצב ***מוכן בדיסק*** יכול להתרחש כאשר עדיפותו של תהליך שהיה במצב ***מוכן*** יורדת עד כדי כך שהיא גוברת על הצורך בהעתקת מפת הזיכרון מהדיסק לזיכרון הראשי.
- המעבר ממצב ***חסום בדיסק*** למצב ***חסום*** יכול להתרחש בעקבות פינוי של מקום בזיכרון הראשי, אם עדיפותו של התהליך החסום בדיסק גוברת על עדיפותו של כל אחד מהתהליכים הנמצאים במצב ***מוכן בדיסק***.
- המעבר ממצב ***רץ*** למצב ***מוכן בדיסק*** יכול להתרחש רק במקרים נדירים, וזאת כאשר הפקעת מעבד מתהליך רץ מתרחשת בעקבות קבלת אירוע על ידי תהליך בעל עדיפות גבוהה מאוד ובהינתן הצורך המיידי בפינוי הזיכרון. בתנאים הללו מתבצעת העתקה של שטח הזיכרון הראשי לדיסק והתהליך שהיה במצב ***רץ*** עובר ישירות למצב ***מוכן בדיסק***.

- באיור של מכונת המצבים המשוכללת לא הוצגו המעברים מכל מצב (פרט למצב ***חדש***) למצב ***סיום***, כדי לא להעמיס על האיור יתר על המידה. אם המעבר למצב ***סיום*** אינו מתבצע ממצב ***רץ***, הוא עדיין יכול להתרחש, למשל כפי שראינו בסעיף 2.1.3, כאשר מערכת ההפעלה גורמת לסיום ריצת תהליך על ידי הריגתו. במקרה כזה, האינדיקציה המתאימה תישמר ברשומת המידע שבגרעין המערכת.

2.1.6 יישום תהליכים

בסעיף זה מתוארים באופן כללי וללא התייחסות למערכת הפעלה ספציפית המבנים הדרושים למימוש מודל התהליכים. מערכת ההפעלה מנהלת טבלה המכונה ***טבלת תהליכים*** וכל רשומה בטבלה מכילה מידע המתייחס לתהליך באופן חד-חד-ערכי. הרשומה יכולה להכיל מידע ישיר או מצביעים למבנים נוספים, בהתאם למימוש זה או אחר, אך באופן כללי המידע השמור ברשומה (או שניתן להגיע אליו מתוך הרשומה) משקף במלואו את מצב התהליך ואת המשאבים המוקצים לו. באמצעות המידע הזה ניתן לשחזר תהליך שהמעבר הופקע ממנו, וכדי שהשחזור יהיה אפשרי – מערכת ההפעלה צריכה לדאוג לעדכון עקבי של מבני הנתונים המתייחסים לתהליך.

כל מערכת הפעלה מנהלת מבני נתונים רבים כדי לאפשר החלפת תהליכים וכדי לאפשר קיום מעברים בין מצב משתמש למצב מיוחס. כאן צריכה להתעורר השאלה: כיצד מתבצע הקוד של מערכת ההפעלה? האם מערכת ההפעלה היא אוסף של תהליכים? או אולי מערכת ההפעלה היא חלק מתהליכי משתמש?

השאלות הללו מגדירות גישות שונות ולכל אחת מהן יש אחיזה במציאות. בפרק הראשון בספר למדנו על מערכות הפעלה במבנה של שרת-לקוח – מערכות אלו ממומשות כאוסף של תהליכים. מערכות הפעלה במחשבים ביתיים רצות במרחב הכתובות של תהליכי המשתמש.

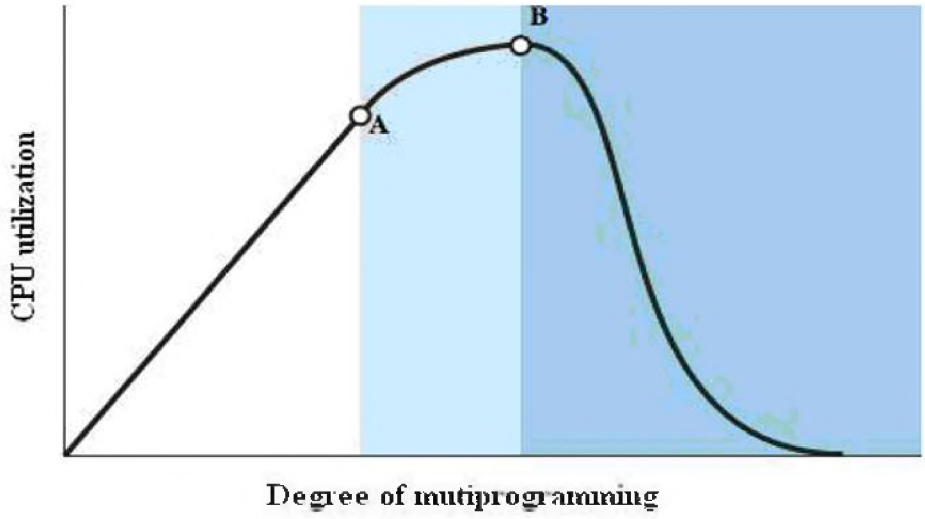
2.1.7 מודל לריבוי תוכניות

העלאת רמת המקביליות של המערכת מביאה לניצול טוב יותר של משאבים. נראה כיצד הגדלה של מספר התהליכים יכולה להשפיע על ניצולת ה-CPU. לשם כך נבנה מודל הסתברותי פשוט. נניח שתהליך מבלה חלק $0 < P < 1$ מזמנו בפעולות קלט/פלט. כאשר n תהליכים נמצאים בזיכרון בבת אחת, ההסתברות לכך שכל התהליכים מבצעים פעולות קלט/פלט היא p^n (בהנחה שהתהליכים אינם תלויים זה בזה). בתנאים אלה, ההסתברות לכך שהמעבד פעיל היא $1-p^n$ וזאת גם הנוסחה לחישוב הניצולת של המעבד. איור 2-6 בספר הוא גרף המציג את הניצולת כפונקציה של מספר התהליכים במערכת, אשר נקרא הדרגה של ריבוי התוכניות.

המודל המתואר באיור הוא כמובן כללי מאוד. הוא גם מניח הנחה לא מציאותית: במערכת שבה מספר התהליכים גדול ממספר המעבדים, התהליכים שמוכנים לריצה יהיו תלויים זה בזה ולו מפני שניתן לבחור את זמן הקצאת המעבדים לתהליכים השונים. התמונה האמיתית, שניתן לראותה בצורה אמפירית או למדל אותה באמצעות כלים מתמטיים קצת יותר מתוחכמים כמו תורת התורים, מוצגת באיור שלהלן:

האוניברסיטה הפתוחה
THE OPEN UNIVERSITY OF ISRAEL
الجامعة المفتوحة
Yair
Kurtzman

File #0006182 belongs to Yair Kurtzman- do not distribute



איור 8

כפי שניתן לראות מן האיור, עד לנקודה A הניצולת גדלה בצורה ליניארית. בנקודה זו, כאשר העומס עולה ניכרת האטה בגידול הניצולת עם הגדלת רמת המקביליות (העקומה בין הנקודות A ו-B). החל משלב מסוים, הגדלת רמת המקביליות לא רק שלא תגרום לגידול הניצולת, אלא יתרחש תהליך הפוך (החל מנקודה B). המערכת תעבוד תחת עומס כבד, וככל שהעומס יגבר, כך תפחת יעילותה של המערכת. בסופו של דבר, אם ממשיכים להגדיל את דרישות המקביליות, המערכת תקרוס. הנקודה B בגרף היא נקודה שבה המערכת חייבת להוציא תהליכים מהזיכרון הראשי אל הדיסק. הצורה המדויקת של העקומה תלויה במספר גדול של גורמים, אך התבנית הכללית הכוללת עלייה חדה, האטה ולאחר מכן ירידה אקספוננציאלית היא תבנית אופיינית למערכות רבות. אפשר להביא לכך דוגמה ממערכות תנועה. אם מספר מכוניות מסוים ייסעו בכביש נתון – התנועה תזרום כסדרה. הגדלת מספר המכוניות תביא לניצולת גבוהה של הכביש, אך לאחר כמות מסוימת של מכוניות, האטה משמעותית בתנועה תהיה בלתי נמנעת. אם מספר המכוניות ימשיך לגדול, ייווצר פקק. כל מי שנסע בנתיבי איילון בשעות שונות של הבוקר, חווה על בשרו את השינויים בניצולת הכביש כתוצאה מהגדלת הדרישה למקביליות בשימוש של משאב משותף.

2.2 תהליכונים
2.2.1 שימוש בתהליכונים
2.2.2 מודל של תהליכונים
2.2.3 תהליכוני POSIX
2.2.4 מימוש תהליכונים במרחב משתמש
2.2.5 מימוש תהליכונים בגרעין המערכת
2.2.6-2.2.7 הפעלת מתזמן
2.2.8 תהליכונים "קופצים"
2.2.9 אלתורים: ריבוי חוטי ביצוע מקוד שנועד לחוט ביצוע אחד

2.2 תהליכונים

ניתן לומר שלכל תהליך יש ***חוט בקרה*** אחד, אך קורה שאנו צריכים יותר מחוט בקרה יחיד במרחב הכתובות של כל התהליך. למשל, חוטי בקרה נפרדים בעבור פרוצדורות שרצות במקביל (ומכאן השימוש במונח threads). אנחנו בחרנו להשתמש במונח ***תהליכונים*** כדי להדגיש את העובדה כי מודל זה מאפשר להריץ – בתוך תהליך אחד – כמה תת-תהליכים, *תהליכונים*, הרצים במקביל.

שאלה 4

בניגוד לתהליכים, כל התהליכונים של תהליך יחיד חולקים את אותו מרחב כתובות. מתי עובדה זו יכולה להיות שימושית?

2.2.1 שימוש בתהליכונים

בסעיף זה מופיעות דוגמאות אחדות של שימוש בתהליכונים. חשוב להבין שגם אלמלא היה עומד לרשותנו מודל לתהליכונים, היה אפשר לתכנת שרת שביצועיו היו יעילים באותה מידה. במקום להשתמש בקריאות מערכת חוסמות, היה אפשר להשתמש בקריאות מערכת שאינן חוסמות את התהליך הקורא. למשל, כאשר השרת מקבל בקשה להבאת קובץ אפשר לבצע בקשה לקריאה מן הדיסק, לשמור את סטטוס הבקשה ולעבור לטיפול בבקשות אחרות. לכל הבקשות שמגיעות לשרת תוחזק שורה בטבלה שבה יישמר מהו המצב של כל אחת מן הבקשות. המידע שבטבלה ישמש לשחזור של ביצוע הבקשות בדיוק מן המקום שבו הושהה הביצוע עקב הצורך לטפל באירועים חיצוניים או עקב המתנה לנתונים. קונספציית התכנות הזאת, כאשר יש כמה חוטים של ביצוע וכל חוט ביצוע יכול להימצא במצב מסוים או לעבור ממצב למצב עקב כמה אירועים שמוגדרים מראש, נקראת ***מכונת מצבים סופית***. כפי שניתן לראות מן הדוגמה של שרת, מימושה של מכונת מצבים סופית הוא מימוש של תהליכונים (חוטי ביצוע) למקרה היחיד של שרת. קיומם של תהליכונים במערכת ההפעלה חוסך מלאכה רבה הנגזרת ממימוש של מכונת מצבים במקרה זה ובמקרה של תכנות אפליקציות רבות דומות. נסיים את הדיון בשאלה למחשבה:

שאלה 5

האם ייתכן שבאפליקציות מסוימות, מימוש באמצעות מכונת מצבים סופית יהיה יעיל יותר ממימושן באמצעות תהליכונים?

2.2.2 מודל של תהליכונים

המושג "תהליך" מאגד שני היבטים שונים בתכלית. האחד הוא היבט הבקרה של ביצוע סדרתי של תוכניות, והאחר הוא ניהול המשאבים של מערכת. במערכת הפעלה התומכת בתהליכונים, מבצעים הפרדה בין שני ההיבטים הללו כדי לאפשר מקביליות בתוך התהליך עצמו. התהליך מוגדר כישות שמאגדת את קבוצת המשאבים, אולם התהליכונים מוגדרים כישויות הניתנות לתזמון (ביצוע על ידי המעבד). מערכות הפעלה

האוניברסיטה הפתוחה
THE OPEN UNIVERSITY OF ISRAEL
الجامعة المفتوحة
Yair
Kurtzman

File #0006182 belongs to Yair Kurtzman- do not distribute

אשר תומכות במקביליות בתוך תהליכים נקראות ***מערכות התומכות בריבוי תהליכונים*** (***multithreading operating systems***). כל אחד מהתהליכונים שייך לאחד מתהליכי המערכת. מערכות הפעלה שבהן לכל תהליך יכולה להיות רק ישות אחת הניתנת לתזמון נקראות ***מערכות הפעלה שאינן תומכות בריבוי תהליכונים*** (***single threaded operating systems***). במילים אחרות, מודל התהליכונים הוא הרחבה של מודל התהליכים.

דוגמאות:

- MSDOS היא מערכת הפעלה single processed ו-single threaded.
- יש גרסאות של UNIX התומכות ב-multithreading (Solaris, System V) ויש גרסאות שהמאפיין הזה אינו קיים בהן (4BSD). LINUX היא מערכת התומכת בתהליכונים.
- Windows (10, XP, Vista), Mach, OS/2 הן דוגמאות נוספות למערכות התומכות בתהליכונים.

במערכות הפעלה שתומכות בתהליכונים יש לכל תהליך במערכת מרחב זיכרון מדומה משלו, וכל תהליך כזה מחזיק במשאבים שהוקצו לו על ידי מערכת ההפעלה. בכל תהליך יכולים להימצא כמה תהליכונים, ולכל אחד מהם יש המאפיינים האלה:

- מצב התהליכון (running, ready, blocked)
- מצב הרגיסטרים של המעבד
- מחסנית (stack) נפרדת בעבור כל תהליכון
- אפשרות גישה לזיכרון ולמשאבים המשותפים בתוך התהליך

מצב התהליכון, מצב הרגיסטרים והמחסנית יוצרים ביחד את המידע שישמש לשחזור חוט הבקרה של התהליכון. חשוב לעמוד על הבדל מהותי בין תהליכים לתהליכונים. תהליכונים שרצים במרחב כתובות של תהליך מארח משתפים גישה למרחב הזיכרון שלו. כך, למשל, אפשר ליצור תהליכונים בתוך תהליך אחד, ואז הם יכולים לגשת לכל המשאבים שהוקצו לתהליך. שינוי של משתנה גלובלי על ידי תהליכון אחד בתהליך המארח יגרום לכך שכל שאר התהליכונים של אותו התהליך יראו את השינוי. דוגמה אחרת: אם אחד התהליכונים פותח קובץ עם הרשאות קריאה, שאר התהליכונים של התהליך המארח יכולים גם הם לקרוא מאותו הקובץ.

שאלה 6

מודל התהליכונים הוא הרחבה של מודל התהליכים. מהם היתרונות שמספק המודל המורחב?

אפשר לזהות כמה יתרונות:

- יצירת תהליך חדש מצריכה משאבים רבים יחסית לדרישות של תהליכון. למשל, יצירה של תהליכון חדש נוסף איננה דורשת יצירה של מפת זיכרון. לכן זמן היצירה של תהליכונים קטן בהרבה מזמן יצירתם של תהליכים חדשים.

- בשל אותם שיקולים, זמן ההשמדה של תהליכונים, בתנאי שאין מדובר בתהליכון האחרון בתהליך המארח, קטן בהרבה מזמן ההשמדה של תהליכים.
- החלפת תהליכים ותהליכונים מצריכה שמירת תוכן לשם שחזור עתידי. ברור כי החלפת תהליכונים תהיה מהירה מהחלפת תהליכים. אגב, זאת הסיבה שתהליכונים מכונים לעתים ***lightweight processes***.
- תקשורת בין תהליכים דורשת מעורבות של גרעין מערכת ההפעלה. לעומת זאת, תקשורת בין תהליכונים בתוך תהליך מארח יכולה להתבצע ללא התערבותו של גרעין מערכת ההפעלה, מכיוון שתהליכונים אלה משתפים זיכרון, קבצים וכו׳.

מודל התהליכונים מאפשר לשנות את הגישה התכנותית שהייתה מוכרת לנו. בשלבי עיצוב התוכנה קיום התהליכונים מאפשר חלוקת תוכנית למודולים שונים, כאשר כל מודול כזה מבצע משימה ייחודית שלו במקביל למודולים אחרים. תקשורת בין המודולים השונים במודל תהליכונים, שהיא יעילה יותר מתקשורת בין תהליכים, הופכת את הגישה למושכת מאוד. למשל, תוכנת שרת קבצים יכולה ליצור תהליכון לכל בקשה שמגיעה אל השרת. שימוש בתהליכונים יכול לשפר את זמן התגובה של שרת הקבצים, מכיוון שכמות הבקשות שמגיעות לשרת הקבצים בדרך כלל גדולה מאוד, ולכן החיסכון בזמן של יצירת תהליכונים יכול להשפיע על טיב השירות. עקב השימוש בנתונים משותפים, רמת התיאום הנדרשת בין המודולים השונים של שרת קבצים יכולה להיות תובענית מאוד, ולפיכך שימוש בתהליכונים יכול לשפר את הביצועים הודות לתקשורת יעילה.

בדוגמה של שרת הקבצים, השימוש בתהליכונים התאים לאופי של הבעיה. באופן כללי כדאי לשקול את השימוש בתהליכונים במקרה של אפליקציות שנותנות מענה למספר רב של אירועים לא סינכרוניים. לכל מאורע לא סינכרוני אפשר ליצור תהליכון שייתן מענה מתאים. באופן עוד יותר כללי, השימוש בתהליכונים יכול להתאים לתכנות אפליקציות שיכולות להיות או צריכות להיות ממומשות כאוסף של מודולים בעלי גישה לנתונים (משאבים) משותפים, כאשר כל אחד מהמודולים הללו נועד לבצע משימה מסוימת וקיים צורך בתיאום יעיל בין המודולים.
מימוש של מודל התהליכונים מעלה שאלות רבות.

שאלה 7

נתבונן בתהליך הכולל כמה תהליכונים. אחד התהליכונים קורא קריאת מערכת דמוית הפונקציה fork של UNIX. כזכור, הקריאה ל-fork גורמת ליצירת תהליך חדש עם תכונות דומות לאלה של התהליך היוצר. נשאלת השאלה: האם מערכת ההפעלה צריכה ליצור תהליכונים בתוך התהליך החדש? אם המערכת יוצרת תהליכונים בתהליך החדש, האם הם צריכים להיות שכפול של התהליכונים שהיו בתהליך-האב? אם התהליכונים החדשים הם שכפול של התהליכונים של תהליך-האב, אזי אם אחד התהליכונים בתהליך-האב היה במצב חסום, האם גם התהליכון המשוכפל יהיה באותו מצב? במקרה כזה נשאלת השאלה: איזה מן התהליכונים החדשים מגיב על האירוע שלו המתין במצב חסום? (כי כעת יש שני תהליכונים המחכים כך!)

האוניברסיטה הפתוחה
THE OPEN UNIVERSITY OF ISRAEL
الجامعة المفتوحة
Yair
Kurtzman

File #0006182 belongs to Yair Kurtzman- do not distribute

שאלות אלה מציגות קשיים במישור התכנוני של מערכות הפעלה. קשיים אחרים מתעוררים בשלב התכנות של אפליקציות שמשתמשות במודל תהליכונים.

נניח שתהליך מסוים מכיל כמה תהליכונים המבצעים משימה משותפת שדורשת הקצאת זיכרון. מכיוון שלא ידוע מראש איזה מהתהליכונים יהיה מתוזמן, כל אחד מהם מבצע בדיקה אם יש מספיק זיכרון לביצוע המשימה. ייתכן שהתהליכון שביצע בדיקה יוחלף בדיוק לפני הקצאת הזיכרון וירוץ תהליכון אחר שיבצע גם הוא את הבדיקה. במקרה כזה יכול להיות ששני התהליכונים יגרמו להקצאת זיכרון, ואז הזיכרון יוקצה פעמיים.

ברור שמקור הבעיה שתיארנו הוא חוסר תיאום בין התהליכונים. בהמשך הפרק נראה שהבעיה שתיארנו כרגע שייכת לקבוצה של ***בעיות מרוץ***, ונקדיש לפתרון בעיות כאלה תשומת לב רבה.

2.2.3 תהליכוני POSIX

סעיף זה כולל חומר רשות.

2.2.4 מימוש תהליכונים במרחב משתמש

מודל התהליכונים יכול להיות ממומש ברמת המשתמש או בגרעין המערכת. לכל אחת מן האפשרויות הללו יש יתרונות וחסרונות. כמו תמיד, הרצון להשיג בו-זמנית את היתרונות של שתי השיטות ולמזער את חסרונותיהן הביא להתפתחות השיטות המעורבות (ראו סעיף 2.2.4 בספר).

בשיטה הראשונה, ניהול התהליכונים מתבצע במרחב של התהליך המארח, וגרעין המערכת כלל אינו מודע לקיום התהליכונים. מימוש אפשרי של השיטה הוא באמצעות ספריית פונקציות שמספקת ממשק ליצירה, להשמדה ולתזמון של תהליכונים, ומספקת מנגנוני תקשורת ביניהם. באחריותה של הספרייה לדאוג לניהולה של ***טבלת התהליכונים***, המכילה מאפיינים של כל התהליכונים הנמצאים בתהליך המארח. תהליך יכול, בכל שלב של ריצתו, לקרוא לפונקציה של הספרייה שנועדה לייצור תהליכונים. בעקבות הקריאה לפונקציה זו תתבצע יצירת מבני הנתונים של התהליכון החדש ורישומו של התהליכון החדש בטבלת התהליכונים המנוהלת על ידי הספרייה. אם נוצרו כמה תהליכונים בתוך התהליך המארח, הספרייה תהיה אחראית לתזמונם. הספרייה תבצע שמירת תוכן של תהליכונים ממתינים ועדכון מבני נתונים מתאימים. צריך לזכור שכל הפעולות שמבצעת הספרייה מתבצעות במצב משתמש, וגרעין המערכת איננו מודע כלל וכלל לקיום התהליכונים. מכאן נובעים כמה יתרונות, אבל גם חסרונות המאפיינים את השיטה.

נסכם את היתרונות של מימוש תהליכונים בספרייה ברמת המשתמש:

- החלפת תהליכונים ברמת המשתמש איננה דורשת מעבר למצב ראשוני, כפי שנדרש בהחלפת תהליכים. להבדיל מהחלפת תהליכים שדורשת את הפעלת המתזמן של

מערכת ההפעלה, החלפת תהליכונים מתבצעת על ידי המתזמן שנמצא במרחב הזיכרון של התהליך המארח.

- שימוש תהליכונים בספרייה ברמת המשתמש מאפשר את התאמת אלגוריתם התזמון של ספריית התהליכונים לצרכים של כל יישום בנפרד. ספריית תהליכונים ברמת המשתמש יכולה אף לספק אפשרות לשנות לגמרי את אלגוריתם תזמון התהליכונים.
- חבילת תוכנה שממממשת תהליכונים ברמת המשתמש יכולה לרוץ ללא שום תמיכה מצד גרעין המערכת, ולכן הדבר נותן אפשרות לממש אפליקציות שיש בהן מספר גדול של תהליכונים גם במערכות הפעלה שאינן תומכות בתהליכונים.

נסכם את החסרונות של מימוש תהליכונים בספרייה ברמת המשתמש:

- קריאות מערכת רבות במערכת ההפעלה יכולות לגרום לכך שהתהליך שביצע קריאה עובר למצב חסום. אם אחד התהליכונים בתהליך המארח מבצע קריאה שגורמת לחסימת התהליך, אזי בשיטה שממממשת תהליכונים ברמת המשתמש, מערכת ההפעלה תשהה את התהליך ואיתו יושהו גם כל התהליכונים שלו. זאת אף שייתכן שבתהליך המארח היו תהליכונים שהיו מוכנים לריצה. מכאן שכאשר מימוש התהליכונים נעשה ברמת המשתמש, אפליקציות מרובות תהליכונים לא יוכלו לנצל את היתרונות שנותן ריבוי התהליכים, ובפרט ריבוי של מעבדים. גרעין המערכת יכול להקצות מעבד לתהליך המארח, אך אינו מסוגל להקצות מעבדים לתהליכונים נפרדים מכיוון שהגרעין כלל אינו מודע לקיומם. יתרה מזו, הניסיון להריץ אפליקציות מרובות תהליכונים שממומשים ברמת המשתמש במערכת הפעלה שאיננה תומכת בתהליכונים יכול להביא לתוצאות חמורות יותר מירידה ברמת המקביליות.
- חיסרון נוסף של יישום תהליכונים ברמת המשתמש הוא שבדרך כלל הספריות הממשמשות את התהליכונים ברמת המשתמש משאירות את האופציה להפקיע מעבד מתהליכון אחד בידי התהליכונים עצמם. הדבר, כמובן, פוגע ברמה התכנונית של השימוש בתהליכונים כחוטים נפרדים של ביצוע.

לסיכום, מימוש תהליכונים ברמת המשתמש מעורר בעיות רבות. המטרידות שביניהן קשורות לקריאות של פונקציות המערכת שגורמות למעבר של התהליך המארח למצב חסום, בעוד שקיימים בתוכו תהליכונים שמוכנים לריצה. ספר הלימוד מציע דרכים אחדות להתגבר על חלק מהבעיות במימוש התהליכונים ברמת המשתמש.

2.2.5 מימוש תהליכונים בגרעין המערכת

כאשר נוקטים גישה זו בצורתה הטהורה, כל מלאכת ניהול התהליכונים מתבצעת על ידי גרעין המערכת. ברמת האפליקציה אין ולו פיסת קוד אחת המיועדת לניהול התהליכונים. האפליקציה משתמשת רק בקריאות מערכת המפעילות את מנגנון ניהול התהליכים, וזה מתבצע בתוך גרעין המערכת.

האוניברסיטה הפתוחה
THE OPEN UNIVERSITY OF ISRAEL
الجامعة المفتوحة
Yair
Kurtzman

File #0006182 belongs to Yair Kurtzman- do not distribute

אפליקציות יכולות להיות מתוכנתות באמצעות תהליכונים כאשר כל התהליכונים מאוגדים תחת תהליך אחד. גרעין המערכת אחראי לתחזק את תוכן תהליכי המערכת ואת תוכן התהליכונים אשר רצים תחת התהליכים הללו. גרעין המערכת גם מבצע את התזמון הנדרש.
כפי שניתן לראות, גישה זו מתגברת בקלות על הבעיות העיקריות של הגישה הקודמת: הגרעין יכול לתזמן את התהליכונים השייכים לאותו התהליך על מעבדים שונים, וגם אם אחד מהם עובר למצב חסום, אין מניעה לתזמן תהליכון אחר השייך לתהליך המארח. יתרון נוסף של הגישה הזאת הוא שפרוצדורות של גרעין המערכת עצמו יכולות להיות ממומשות בעזרת תהליכונים!

כמובן, החיסרון המשמעותי של מימוש התהליכונים בגרעין המערכת הוא בזבוז הזמן הכרוך במעבר ממצב משתמש למצב ראשוני. אם נשווה בין זמן יצירת תהליכון ברמת המשתמש לבין זמן יצירת תהליכון ברמת גרעין המערכת, נוכל לראות במקרה השני תוספת של סדר גודל בזמן הדרוש.

שאלה 8

א. אחת האפשרויות להקטנת הבזבוז הזה היא על ידי ביצוע החלפת תהליכים באופן פחות תדיר. האם גישה זו פותרת את הבעיה?

ב. מנו חסרונות נוספים של מימוש התהליכונים בגרעין המערכת.

גישות מעורבות אומצו כדי לנצל יתרונות של הגישות שראינו בסעיפים הקודמים ולמזער את השפעת החסרונות. קיימים מימושים רבים של הגישות המעורבות, אך לכולן מכנה משותף אחד: גרעין מערכת ההפעלה מספק תהליכונים ברמת הגרעין, כאשר תהליכונים ברמת המשתמש שמסופקים על ידי ספריית תהליכונים כלשהי רצים בלא שהגרעין מודע לקיומם.

2.2.6 הפעלת מתזמן

אפשר לסווג את הגישות השונות למימוש תהליכונים בהתאם ליחס תהליכון/תהליך. כך בגישה הישנה שבה לכל תהליך יש חוט ביצוע אחד בלבד, היחס הוא 1:1. היחס 1:M מאפיין את הגישה שראינו בסעיף 2.2.5, כאשר התהליכונים ממומשים בגרעין של מערכת ההפעלה. מקרה מעניין של היחס M:1 מתאפיין באפשרות של נדידת תהליכונים מתהליך מארח אחד לתהליך אחר. נדידת תהליכונים יכולה להיות שימושית במערכות מבוזרות או במערכות מרובות מעבדים לאיזון עומסים. האפשרות של נדידת תהליכונים בשילוב עם אפשרות קיומם של כמה תהליכונים בתוך תהליך מארח מבוטאת על ידי היחס M:N.

דוגמה למימוש היחס M:N היא מערכת ההפעלה NetBSD. הרעיון שעומד מאחורי המימוש מכונה שיטת הפעלת מתזמן והוא הוצע על ידי Anderson et al. (במאמר בשם "*Scheduler Activations: Effective Kernel Support for User Level Management of Parallelism*").

בשיטה זו, גרעין המערכת יכול להקצות לכל תהליך כמה מעבדים מדומים. מספר המעבדים יכול להשתנות בהתאם לבקשת תהליך או לפי שיקולה של מערכת ההפעלה, שיכולה להחליט על הקצאת מספר גדול יותר (או קטן יותר) של מעבדים מדומים, לפי עדיפותו של התהליך. גרעין של מערכת ההפעלה מספק מנגנון הקרוי ***upcall***, אשר דרכו גרעין מערכת ההפעלה יכול להודיע לתהליך על שינוי במצבם של המעבדים המדומים שלו.

בהינתן המעבדים המדומים ומנגנון ה-upcall, אפשר להשתמש בספרייה שמספקת תהליכונים ברמת המשתמש בצורה זו:

- התהליך יכול להקצות לתהליכונים הנמצאים במצב מוכן לריצה מעבדים מדומים.
- אם אחד התהליכונים, תוך כדי ריצתו במעבד המדומה שהוקצה לו, מבצע קריאת מערכת שדורשת המתנה (למשל פסיקת I/O), אזי גרעין המערכת יפעיל את מנגנון ה-upcall ויעביר לתהליך המארח מידע על כך. התהליך המארח יוכל להשתמש בשירותי ספריית התהליכונים כדי להעביר את התהליכון שלא יכול להמשיך לרוץ למצב מושהה, ולהקצות את המעבד המדומה לתהליכון אחר שיימצא מוכן לריצה. כאשר יקרה האירוע שלו ממתין התהליכון המושהה, גרעין המערכת יודיע לתהליך המארח דרך אותו המנגנון של upcall על קרות האירוע. התהליך המארח יוכל אז לקבל החלטת תזמון, לשחזר את תוכן התהליך המושהה ולהקצות לו מעבד מדומה, או לעשות זאת מאוחר יותר.

בצורה כזאת גרעין מערכת ההפעלה כביכול אינו מודע לקיום תהליכונים ברמת המשתמש, ואינו מתערב בהחלטות התזמון שלהם כל עוד הריצה של תהליכונים איננה קשורה למשאבים של מערכת ההפעלה. למשל, אם תהליכון צריך להמתין לתהליכון אחר בתוך התהליך המארח, הדבר לא יגרום למעבר לגרעין מערכת ההפעלה.

2.2.7 אלתורים: ריבוי חוטי ביצוע מקוד שנועד לחוט ביצוע אחד

החומר בסעיף זה טכני לחלוטין ואינו דורש הסברים נוספים. אתם מוזמנים לעיין בספר הלימוד.

2.4 תקשורת בין תהליכים
2.4.1 מרוץ קריטי
2.4.2 קטעים קריטיים (critical sections)
2.4.3 מניעה הדדית הכוללת המתנה פעילה
2.4.4 הרדמה והערה
2.4.5 סמפורים
2.4.6 מנעולים
2.4.7 מבני פיקוח (monitors)
2.4.8 העברת הודעות (message passing)
2.4.9 מחסומים (barriers)

האוניברסיטה הפתוחה THE OPEN UNIVERSITY OF ISRAEL الجامعة المفتوحة
Yair Kurtzman

File #0006182 belongs to Yair Kurtzman- do not distribute

2.4 תקשורת בין תהליכים

שם הסעיף עלול להטעות. שם מדויק יותר לנושא הדיון הוא: **שיתוף פעולה בין תהליכים**. התקשורת עצמה היא פועל יוצא מהצורך בשיתוף הפעולה. מתברר כי שיתוף פעולה זה יוצר לעתים בעיות לא פשוטות. הקושי העיקרי הוא, כפי שנראה בהמשך הסעיף, בכך שהבעיות אשר מתעוררות אינן ניתנות לשחזור מדויק. הסיבה לכך היא שכדי לשחזר במדויק מצב בעייתי צריכים להתקיים כל התנאים שהובילו לבעיה. תזמון של תהליכים ומצבם, מצב המשאבים – אלה הם מקצת הגורמים שאמורים להיות בדיוק באותם התנאים כדי לשחזר את הסיטואציה הבעייתית. בסעיף זה נלמד לזהות את אופן התרחשותן של בעיות הקשורות לשיתוף פעולה בין תהליכים, ונכיר פתרונות גנריים להתמודדות עמן.

לשיתוף פעולה בין תהליכים יש שני היבטים עיקריים:

- תהליכים צריכים להחליף מידע.
- תהליכים השייכים לקבוצה חייבים לשתף פעולה ולפעול כל אחד תוך התחשבות בשאר התהליכים, הואיל והתהליכים עלולים לבצע פעילויות אשר יכולות להשפיע על תקינות התוצאה של תהליכים אחרים.

סיגנלים, משרשרים (pipes), שקעים (sockets) ואזורי זיכרון משותפים, כל אלה הם דוגמאות לכלים המאפשרים תקשורת למטרת העברת מידע. לא נעסוק בפרק זה בנושא של תקשורת בין תהליכים, אבל לחלק מן הכלים הללו יוקדשו תרגולים מעשיים במהלך הקורס. פרק זה עוסק בהרחבה במנגנונים שמאפשרים שיתוף פעולה בין תהליכים – זהו הפן השני של הצורך בתקשורת בין תהליכים.

באופן עקרוני, הנושאים שקשורים בשיתוף פעולה בין תהליכים תקפים גם לגבי תהליכונים. לכן, אף שבדיון נעשה שימוש במילה תהליכים, הדברים נכונים גם לגבי תהליכונים.

2.4.1 מרוץ קריטי

נראה דוגמה להמחשת הצורך בשיתוף פעולה. נניח ששני תהליכונים אשר רצים במרחב כתובות משותף מנסים לעדכן רשימה משורשרת. התהליכונים משתמשים בשגרות של ספרייה שמספקת ממשק לניהול רשימות משורשרות. נתבונן בפסידוקוד[4] של השגרה deleteFirst() שמספקת הספרייה:

```
deleteFirst(){
        //TmpNodePtr is a local variable, ListHeadPtr is global
        Node* TmpNodePtr=ListHeadPtr;
        ListHeadPtr=TmpNodePtr->next;
        delete(TmpNodePtr);
}
```

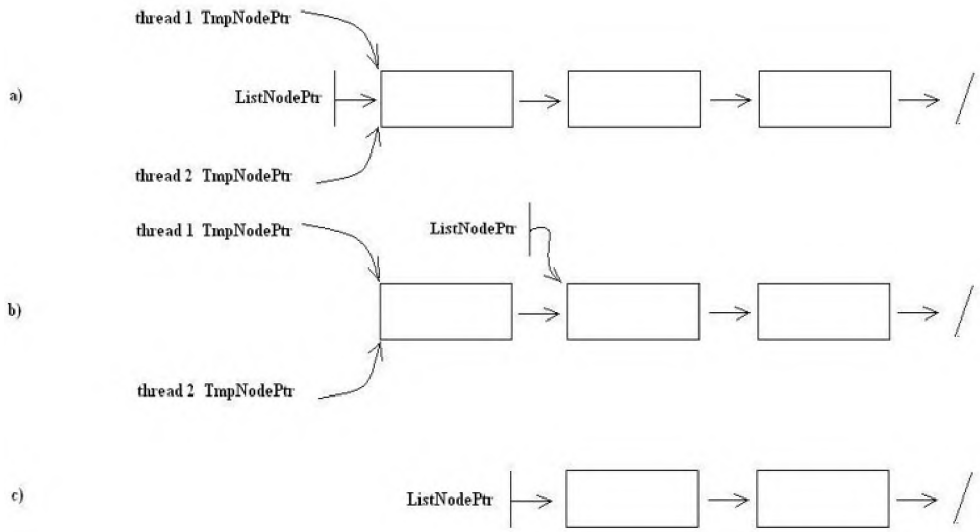
[4] פסידוקוד משמש כשפת מחשב מלאכותית שדומה מאוד לשפה אמיתית. בהמשך נשתמש בפסידוקוד דמוי שפת C, אך לא תמיד נדאג לתאר ערכים מוחזרים של פונקציות, טיפוסים של משתנים וכו'.

ונתבונן בקוד שמבצעים התהליכים:

תהליכון 1	תהליכון 2
... deleteFirst(); ...	... deleteFirst(); ...

התוצאה הרצויה היא ששני התהליכונים ימחקו מראש הרשימה שני איברים ראשונים. הפסידוקוד הזה אינו מטפל במקרה של תור ריק. לצורך הדיון נניח שבתור קיימים לפחות כמה איברים. נתאר תרחיש שמדגים כיצד, ללא שיתוף פעולה בין התהליכונים, התוצאה של הרצת התהליכונים יכולה להיות שונה מן המצופה.

נניח שבתור יש ארבעה עצמים. תהליכון 1 מגיע ל-deleteFirst() ומבצע את השורה הראשונה. לאחר ביצוע השורה, מתרחשת החלפת תהליכונים ותהליכון 2 מגיע לפונקציה deleteFirst() שלו ומבצע גם הוא שורה ראשונה. ראו באיור שלהלן את חלק a.



איור 9

ניזכר כי לכל תהליכון יש מחסנית נפרדת, ולכן עותקים נפרדים של deleteFirst() "רצים במקביל". מכיוון שהמשתנה TmpNodePtr הוא משתנה מקומי, עותקים נפרדים שלו יופיעו בכל מחסנית. לעומת זאת, המשתנה ListNodePtr הוא משתנה גלובלי של הספרייה, ולכן הוא משותף לשני התהליכונים.

נניח שהתהליכונים מתחלפים לאחר הביצוע של כל שורה. באיור (חלק b) ניתן לראות מה מתרחש בתור לאחר שהתהליכונים ביצעו את השורה השנייה. בחלק האחרון, c, ניתן לראות כי לאחר שהתהליכונים סיימו לבצע כל אחד את העותק שלו של deleteFirst(), רק איבר אחד נמחק מראש הרשימה. התוצאה הרצויה הייתה צריכה להיות מחיקת 2 איברים ראשונים.

האוניברסיטה הפתוחה
THE OPEN UNIVERSITY OF ISRAEL
الجامعة المفتوحة
Yair
Kurtzman

File #0006182 belongs to Yair Kurtzman- do not distribute

ברור שבלעדי ההחלפה התדירה של התהליכונים, הדברים היו יכולים להיראות אחרת. אך במציאות ההסתברות לתרחישים מן הסוג שתיארנו היא ניכרת. דוגמה זו ממחישה את הצורך לזהות[5] מצבים שבהם התוצאה הסופית תלויה בתזמונם של תהליכונים (או תהליכים) ולטפל בהם. מצבים כאלה מכונים ***מצבי מרוץ***. בסעיפים הבאים נלמד מנגנונים שעוזרים להתמודד עם מצבי מרוץ.

שאלה 9

האם שינוי השורה השנייה של deleteFirst ל ;ListHeadPtr=ListHeadPtr->next היה פותר את בעיית המרוץ המתוארת לעיל?

2.4.2 קטעים קריטיים (critical sections)

בסעיף הקודם אמרנו שלולא ההחלפה התדירה של תהליכונים, היה אפשר למנוע את היווצרות הבעיה. הבעיה הייתה נמנעת אילו שני התהליכונים לא היו ניגשים לעדכון הרשימה המשורשרת בו-זמנית. למעשה, קטע הקוד שבו התבצע עדכון של הרשימה המשורשרת הפך להיות, הוא עצמו, מושא למרוץ בין התהליכונים. דבר זה לא היה קורה לו התהליכונים היו משתפים פעולה. סוג של שיתוף פעולה הקרוי ***מניעה הדדית***, ואשר מסוגל למנוע מתהליכון לבצע עדכון של הרשימה המשורשרת כל עוד התהליכון השני לא סיים לעדכנה, היה פותר את בעיית המרוץ. כל קטע קוד עם משאב משותף לשני תהליכים או יותר ואשר מתבצע בו עדכון יכול להביא למצב מרוץ. קטעי קוד כאלה מכונים "קטעים קריטיים".

כדי להבטיח פעולה תקינה של תוכנית הכוללת קטע קריטי, שיתוף פעולה מסוג המניעה ההדדית אינו מספיק, אם כי הוא תנאי הכרחי לפתרון סביר של בעיית מרוץ. ארבעה תנאים דרושים להשגת פתרון סביר:

- שני תהליכים לא ישהו בו-זמנית בתוך קטע קריטי.
- הפתרון לא יסתמך על השערות הקשורות למהירות או למספר המעבדים.
- תהליך מחוץ לקטע קריטי לא ימנע מתהליך אחר להיכנס לקטע קריטי.
- תהליך לא ימתין לנצח כדי להיכנס לקטע קריטי.

התנאים הללו כמובן נכונים גם לגבי תהליכונים. הערה חשובה נוספת היא שזמן השהייה בקטע קריטי הוא סופי.[6]

---

[5] ספרייה להקצאת זיכרון malloc מנהלת שטחי זיכרון באמצעות רשימות משורשרות. עם התחלת השימוש בתהליכונים היה צורך בשכתוב של הספרייה כדי שתתמוך בריבוי של תהליכונים.

[6] מערכת ההפעלה אינה יכולה לבדוק שזמן השהייה של תהליך בקטע קריטי הוא סופי. הנחה על סופיות זמן השהייה בקטע קריטי קשורה לשאלה עמוקה הידועה בשם "בעיית העצירה". לפרטים אפשר לפנות ל-*Introduction to the Theory of Computation* מאת Sipser.

2.4.3 מניעה הדדית הכוללת המתנה פעילה

בסעיף זה נבחן פתרונות שונים להשגת מניעה הדדית.

מניעת הפסיקות

מניעת הפסיקות לפני כניסת תהליך לקטע קריטי היא הדרך הפשוטה ביותר להשיג מניעה הדדית. הפסיקות מופעלות שוב רק לאחר היציאה מהקטע הקריטי. מניעת פסיקות גורמת לכך שהמעבד אינו עובר מתהליך לתהליך, ולכן אין חשש שתהליך אחר יימצא בקטע קריטי.

לשיטה יש שלושה חסרונות מהותיים:

- לא סביר לתת לתהליך משתמש אפשרות למנוע פסיקות. תהליכי משתמש יכולים למנוע תהליכים אחרים מלקבל זמן מעבד עקב שגיאות תכנות – למשל כאשר מתכנת שוכח לאפשר פסיקות מחדש. קוד זדוני של תהליך יכול לגרום לכך בכוונה, ולכן מתן אפשרות כזאת לתהליך משתמש היא פרצת בטיחות.
- חסימת הפסיקות לפרק זמן ארוך יכולה להביא לתופעות לא נעימות. ירידה ברמת המקביליות של מערכת ההפעלה יכולה לגרום למשל הצפה של שידור בכרטיס תקשורת או תגובה איטית של מקלדת עכבר.
- חסימה של פסיקות תקפה לגבי מעבד אחד בלבד. פתרון בסביבה שבה כמה מעבדים נידון בפרק 8.

לעומת זאת, בגרעין המערכת חייבים לעתים למנוע פסיקות למשך הביצוע של כמה הוראות ובהמשך נראה דוגמה לכך.

לסיכום, מניעת פסיקות היא שיטה שימושית בגרעין המערכת, אולם לא סביר להשתמש בה בעבור תהליכי משתמש.

משתני נעילה (lock variables)

כניסיון שני, נתבונן בפתרון תכנותי. נניח שיש לנו משתנה משותף lock מאותחל 0. הפתרון המוצע בעבור שני תהליכים הוא:

תהליך 1	תהליך 2

While(true){
… // non critical section
while (lock == 1)
; // loop
lock = 1;
CriticalSection();
lock = 0;
…// non critical section
}
	While(true){
	… // non critical section
	while (lock == 1)
	; // loop
	lock = 1;
	CriticalSection();
	lock = 0;
	…// non critical section
	}

האוניברסיטה הפתוחה
THE OPEN UNIVERSITY OF ISRAEL
الجامعة المفتوحة
Yair
Kurtzman

File #0006182 belongs to Yair Kurtzman- do not distribute

כאשר תהליך רוצה להיכנס לקטע קריטי, הוא בודק תחילה את משתנה המנעול. אם המנעול הוא 0, התהליך משנה אותו ל-1 ונכנס לקטע הקריטי. אם ערך המנעול הוא 1, התהליך ימתין עד אשר ערך המנעול ישתנה ל-0.
פתרון זה מכיל בכל זאת גורם לבעיית מרוץ. נניח שתהליך 1 קורא את ערך המנעול ורואה שערכו הוא 0. בטרם ינעל את המנעול, מתבצעת החלפת תהליכים. תהליך 2 רואה שערך המנעול הוא 0 ונכנס לקטע הקריטי. כעת אם שוב תתבצע החלפת תהליכים, גם תהליך 1 וגם תהליך 2 יהיו במרוץ.

שאלה 10

הפתרון שהצגנו לעיל אינו פתרון סביר מכיוון שלא מתקיים התנאי של מניעה הדדית.
האם קיימים תרחישים שבהם פתרון זה מפר תנאים נוספים שדרשנו מפתרון סביר?

פתרון התור

הבעיה עם השיטה הקודמת הייתה שהבדיקה והעדכון של משתנה הנעילה הופכים לקטע קריטי בפני עצמו, שנוצר בו מרוץ בין שני התהליכים. לכן הפתרון שלהלן אינו מבצע שינוי בערך של משתנה משותף:

תהליך 0	תהליך 1

While(true){
… // non critical section
while (turn != 0)
; // loop
CriticalSection_0();
turn = 1;
…// non critical section
}
	While(true){
	… // non critical section
	while (turn != 1)
	; // loop
	CriticalSection_1();
	turn = 0;
	…// non critical section
	}

זוהי דוגמה מצוינת לפתרון שמקיים תנאי של מניעה הדדית: שני תהליכים לעולם לא יהיו ביחד בקטע הקריטי. אך פתרון זה איננו פתרון סביר מכיוון שהוא מפר את אחד מארבעת התנאים לקיומו של פתרון סביר.

שאלה 11

איזה תנאי מופר על ידי פתרון התור?

המשתנה המשותף turn מאותחל ל-0 בהתחלה וערכו משקף את מספר התהליך שתורו להיכנס לקטע קריטי. בתחילה תהליך 0 בודק את turn ונכנס לקטע קריטי. תהליך 1 שבודק את ערך ה-turn ומקבל שהוא 0, נכנס ללולאה שדוגמת ללא הפסק אם ערך ה-turn השתנה ל-1.

תהליך 1 שדוגם את המשתנה turn נמצא במצב המכונה ***המתנה פעילה*** (***busy waiting***).
המתנה פעילה היא מצב שבו תהליך מקבל את המעבד ומנצל את מלוא הזמן המוקצב

לו, אך איננו מתקדם כלל. ברוב המקרים, המתנה זו נעשית ביוזמת התהליך, כדי לחכות לאירוע מסוים. אלא שבמקרה שאירוע זה איננו מתרחש, המתנה זו עלולה להימשך זמן רב (ואז נוצרת תופעת ***ההרעבה*** – ***starvation***). לכן שימוש בהמתנה פעילה אינו רצוי במקרים שבהם לא ניתן לחזות מראש את משך הזמן שבו התהליך צפוי להימצא בהמתנה פעילה.

נחזור לבדוק מה קורה כאשר תהליך 0 עוזב את הקטע הקריטי. הוא משנה את turn ל-1 כדי לאפשר לתהליך 1 להיכנס לקטע הקריטי. נשים לב, שתהליך 0 אינו יכול להיכנס לקטע הקריטי שלו כל עוד תהליך 1 לא אִפשֵר זאת על ידי שינוי ערכו של המשתנה turn. פירוש הדבר הוא שאם קיים הבדל בזמן הביצוע של הקטעים הלא הקריטיים של התהליכים, תיתכן הפרה של תנאי 3 של הפתרון הסביר לעיל, כי התהליך שנמצא מחוץ לקטע קריטי יכול למנוע מתהליך אחר להיכנס לקטע הקריטי שלו.

הפתרונות של דקר ושל פטרסון

ב-1965 הציע מתמטיקאי דני בשם Dekker פתרון סביר לבעיית המרוץ הקריטי. הפתרון שילב במובן מסוים את פתרון התור ופתרון באמצעות משתנה נעילה. הפתרון דורש שני משתני נעילה ומשתנה אחד של התור. משתנים אלו משותפים לשני תהליכים:

```
boolean lock[2];
int turn;
lock[0] = lock[1] = FALSE:
turn = 1;
```

תהליך 0	תהליך 1

while (TRUE){
…// non critical section
lock[0] = TRUE;
while(lock[1]) == TRUE){
if (turn == 1){
lock[0] = FALSE;
while(turn == 1)
; // loop
lock[0] = TRUE;
}
}
CriticalSection_0();
turn = 1;
lock[0] = FALSE;
…// non critical section
}
	while (TRUE){
	…// non critical section
	lock[1] = TRUE;
	while(lock[0]) == TRUE){
	if (turn == 0){
	lock[1] = FALSE;
	while(turn == 0)
	; // loop
	lock[1] = TRUE;
	}
	}
	CriticalSection_1();
	turn = 0;
	lock[1] = FALSE;
	…// non critical section
	}

האוניברסיטה הפתוחה
THE OPEN UNIVERSITY OF ISRAEL
الجامعة المفتوحة
Yair
Kurtzman

File #0006182 belongs to Yair Kurtzman- do not distribute

ב-1981 הציע Peterson פתרון אלגנטי ופשוט יותר. הפתרון דורש שלושה משתנים משותפים כמו הפתרון של Dekker, אך הוא פשוט יותר להבנה. כדי להיות עקביים לגבי הדרך שבה הצגנו פתרונות קודמים, נציג את פתרון פטרסון בצורה שונה במקצת מזו שמוצגת בספר:

```
boolean interested[2];
int turn;
interested [0] = interested [1] = FALSE:
turn = 1;
```

תהליך 0	תהליך 1

while (TRUE){
…// non critical section
interested [0] = TRUE;
turn = 0;
while(interested [1] == TRUE &&
turn == 0){
; // loop
}
CriticalSection_0();
interested [0] = FALSE;
…// non critical section
}
	while (TRUE){
	…// non critical section
	interested [1] = TRUE;
	turn = 1;
	while(interested [0] == TRUE &&
	turn == 1){
	; // loop
	}
	CriticalSection_1();
	interested [1] = FALSE;
	…// non critical section
	}

תהליכים מצהירים על רצונם להיכנס לקטע קריטי על ידי שינוי תא במערך interested. לכן ברור שאם רק אחד משני תהליכים רוצה להיכנס לקטע קריטי, התהליך נכנס אליו כי התא של מערך interested המתאים לתהליך השני כבוי. כך שתנאי 3 לפתרון סביר מתקיים.

נבדוק עתה אם מתקיים התנאי הראשון של המניעה ההדדית. נניח ששני תהליכים עדכנו את תאיהם במערך interested. לאחר ששני התהליכים "הצהירו על רצונם" להיכנס לקטע קריטי, פתרון פטרסון מבטיח שהראשון שייכנס לקטע קריטי יהיה התהליך שיעדכן ראשון את המשתנה turn. כך פתרון פטרסון מקיים תנאי 1. תהליך שמעוניין להיכנס לקטע קריטי נכנס אליו, אלא אם כן כבר יש תהליך אחר שנמצא בקטע קריטי ולכן גם תנאי 4 של הפתרון הסביר מתקיים. ברור שתנאי 2 גם הוא מתקיים. לכן פתרון פטרסון מהווה פתרון סביר לבעיות מרוץ.

שאלה 12

פתרון פטרסון בעבור שני תהליכים מבטיח **הגינות** בכניסה של תהליכים לקטע קריטי. הביטוי לכך הוא שלאחר עדכון של מערך interested על ידי שני תהליכים, הפתרון מבטיח שהתהליך הראשון שייכנס לקטע קריטי יהיה התהליך שיעדכן ראשון את המשתנה turn. מדוע הטענה נכונה? הבהירו זאת לעצמכם על ידי התבוננות בכל התרחישים האפשריים.

האם אפשר לטעון להגינות של הפתרון של דקר?

TSL

ניזכר בכך שאחת הבעיות של השימוש בביטול פסיקות לפתרון של בעיות מרוץ הייתה שביטול פסיקות היה תקף רק לגבי מעבד אחד. כעת נראה מה ניתן לעשות כאשר יש כמה מעבדים.

במחשבים בעלי כמה מעבדים, המעבד חייב להציע כמה הוראות שחייבות להיות *אטומיות*. פירוש הביטוי *הוראה אטומית* הוא שההוראות מתבצעות כצעד אחד שלא ניתן להפרידו לשלבים. בדרך כלל הוראות כאלה מבוצעות במחזור פקודה (instruction cycle) אחד. דוגמה להוראה כזאת היא ההוראה Test and Set Lock (TSL).

ספר הלימוד מדגים את ההוראה בשפת סף מדומה. למען העקביות, אנחנו נציג את הוראת TSL כשגרה אטומית של פסידוקוד.

```
int TSL(int lock){
        if (lock == 0){
                lock = 1;
                return 0;
        }
        return lock;
}
```

lock היא מילה בזיכרון הראשי שמשמשת לנעילה. הוראת TSL בוחנת את תוכנה של lock. אם התוכן הוא 0, ההוראה משנה את תוכן המילה בזיכרון ל-1 ומחזירה ערך ישן, כלומר 0. אם תוכן מילת הנעילה שונה מ-0, TSL מחזירה את ערכה.

בארכיטקטורה שיש בה כמה מעבדים שמשתפים גישה לזיכרון ראשי, ביצוע של הוראת TSL נועל את ערוץ הזיכרון (memory bus) למשך זמן הביצוע של ההוראה, וזה מאפשר קיום של פתרון לבעיות מרוץ במחשבים כאלה.

הפתרון באמצעות הוראת TSL משתמש במילת הזיכרון המשותפת lock ונראה כך:

תהליך 1	תהליך 2

while(TRUE){
… // non critical section
while(TSL(lock) != 0)
; // loop
CriticalSection_1();
lock = 0;
… // non critical section
}
	while(TRUE){
	… // non critical section
	while(TSL(lock) != 0)
	; // loop
	CriticalSection_2();
	lock = 0;
	… // non critical section
	}

האוניברסיטה הפתוחה THE OPEN UNIVERSITY OF ISRAEL الجامعة المفتوحة
Yair Kurtzman

File #0006182 belongs to Yair Kurtzman- do not distribute

אם רק תהליך אחד רוצה להיכנס לקטע קריטי, ההוראה הראשונה בודקת את ערך מילת הזיכרון lock ומעדכנת אותו בצורה אטומית. אם שני תהליכים מעוניינים להיכנס לקטע קריטי, אז הראשון שייכנס יהיה זה שקרא ראשון ל-TSL. לאחר סיום של קטע קריטי תהליך פשוט מציב ב-lock ערך 0 ובכך "משחרר" את המנעול.

ברור שמתקיימים ארבעת התנאים של פתרון סביר כאשר מדובר על שני תהליכים. אבל הפתרון יכול לגרום להרעבה במקרה של כמה תהליכים. פתרון TSL אינו מסדיר את אופן הכניסה של התהליכים שרצו להיכנס לקטע קריטי ולא יכלו לעשות זאת, מכיוון שהקטע היה תפוס. התהליכים הללו מטופלים על ידי המתזמן כתהליכים מן השורה והמתזמן אינו יודע על "רצונם" להיכנס לקטע קריטי. לכן בהחלט יכול לקרות מצב שבו תהליך ימצא את עצמו מורעב – כלומר לא יצליח להיכנס לקטע קריטי במשך פרק זמן ארוך. לכן הדבר הופך את פתרון ה-TSL לפתרון בלתי סביר במקרה של כמה תהליכים.

2.4.4 הרדמה והערה

החיסרון המובהק של הפתרונות שראינו בסעיף הקודם הוא בזבוז של זמן עיבוד. לא זו בלבד שההמתנה הפעילה מבזבזת זמן מעבד, אלא שיכולה להיות לה תוצאה בלתי צפויה, כמתואר בדוגמה שלהלן. נניח שקיים במחשב תהליך בעדיפות נמוכה L שנכנס לקטע קריטי. בהיותו בקטע קריטי, נוצר במערכת תהליך בעל עדיפות גבוהה H. מתזמן של מערכת ההפעלה אינו מאפשר לתהליך בעל עדיפות נמוכה לרוץ על CPU כל עוד קיים תהליך בעל עדיפות גבוהה יותר. H מתחיל לרוץ ומנסה גם הוא להיכנס לקטע קריטי, אך אינו מצליח מכיוון שתהליך L נועל אותו. אבל L לעולם לא יצא מהקטע הקריטי מכיוון שמתזמן מערכת ההפעלה לא ייתן לו לרוץ על CPU. במצב זה היפוך עדיפויות לתקופה קצרה היה עוזר לצאת ממצב שנראה מצב ***קיפאון*** – כאשר שני תהליכים ממתינים זה לזה בגלל אילוצים כלשהם. בעיית זיהוי מצבים שמצריכים היפוך עדיפויות ידועה גם בשם priority inversion problem.

ראינו גם שהפתרון שלמדנו, TSL,[7] אינו סביר במקרה של כמה תהליכים. לכן נרצה לקבל פתרון העונה על התנאים האלה:

- הוא מקיים את ארבעת התנאים של הפתרון הסביר לכמה תהליכים
- הוא אינו גורם להמתנה פעילה
- הוא מסדיר את אופן הטיפול בתהליכים שרצו להיכנס לקטע קריטי ולא יכלו לעשות זאת מכיוון שהקטע הקריטי היה תפוס
- הוא אינו מצריך זיהוי מצבים הדורשים היפוך עדיפויות

לשם כך ישמשו אותנו שני פרימיטיבים (primitives), קריאות מערכת בשם **SLEEP** ו-**WAKEUP**. בכל הפתרונות המוצעים בהמשך, אנו מניחים כי תהליך יכול להשהות את עצמו על ידי קריאת המערכת SLEEP ולהתעורר כאשר תהליך אחר מבצע קריאת מערכת WAKEUP. כדי לדעת איזה תהליך להעיר, קריאת מערכת WAKEUP מקבלת כפרמטר את המזהה של התהליך שאותו צריך להעיר.

---

[7] בסוף הפרק הבאנו הרחבה של פתרון פטרסון לכמה תהליכים שאינו סובל מבעיית הרעבה במקרה של כמה תהליכים, אם כי הוא מתאפיין בהמתנה פעילה ואינו יכול להימנע ממצבי קיפאון.

תהליכים שמבצעים קריאת מערכת SLEEP עוברים למצב חסום. כך אנו מקבלים המתנה רדומה שאיננה מעסיקה את המעבד. מערכת ההפעלה יכולה לנהל תור של תהליכים רדומים וכך להסדיר את אופן כניסתם של תהליכים שרוצים להיכנס לקטע קריטי, להבדיל מפתרון TSL למשל.

בעיית היצרן-צרכן

עתה נראה כיצד ניתן להתמודד עם מצבי מרוץ בעבור שני תהליכים המשתמשים במאגר עם קיבולת מוגבלת באמצעות הפרימיטיבים SLEEP ו-WAKEUP. נתבונן בדוגמה שלהלן. נתונים שני תהליכים. אחד התהליכים קרוי **יצרן** והוא מכניס פריטים למאגר. התהליך השני קרוי **צרכן**, והוא מוציא את הפריטים מן המאגר. מכיוון שגודל המאגר מוגבל, הבעיה מתעוררת כאשר היצרן רוצה להכניס פריט למאגר מלא. הפתרון בעבור היצרן הוא להרדים את עצמו, ולהתעורר רק כאשר הצרכן הוציא לפחות פריט אחד. מה קורה כאשר המאגר ריק? אם הצרכן רוצה להוציא פריט אולם מוצא כי המאגר ריק, הוא מרדים את עצמו עד אשר היצרן יכניס פריט לחוצץ ויעירו.

כדי לעקוב אחר מספר הפריטים שבמאגר, נזדקק למונה count. נניח כי החוצץ יכול להכיל N פריטים. היצרן בודק אם המונה שווה ל-N. אם זה אכן המצב, הוא הולך לישון, ואם לא – הוא מאחסן פריט ומגדיל את המונה בהתאם. היצרן גם דואג לקרוא ל-WAKEUP כדי להעיר את הצרכן כאשר הוא מאחסן את הפריט הראשון, כי במקרה שהמאגר היה ריק והיצרן אחסן פריט חדש, הוא צריך לגרום לצרכן להתעורר כדי לאפשר מחדש עבודה במקביל.

הצרכן מצידו הולך לישון כאשר הוא מגלה שהמונה הוא 0. אחרת הוא מפנה פריטים ודואג להעיר את היצרן אם נוצר מקום פנוי במאגר. עיינו בקוד לשני התהליכים באיור 2-27 בספר הלימוד.

אנחנו רואים שהפרימיטיבים SLEEP ו-WAKEUP שמשמשים לסנכרון בין התהליכים של יצרן וצרכן אכן מונעים את בעיית ההמתנה הפעילה. אך מתברר כי הפתרון של בעיית יצרן-צרכן באיור 2-27 עדיין סובל בכל זאת מבעיית מרוץ.

מצב מרוץ יכול לקרות מכיוון שהגישה למשתנה של המונה אינה מבוקרת ולכן עקב המרוץ שנוצר על המשתנה, ייתכן שאחת מקריאות ה-WAKEUP תאבד. קראו כיצד זה קורה בספר הלימוד.

הדוגמה מלמדת גם כי אנו זקוקים לפעולה פרימיטיבית המסוגלת לא רק להורות לתהליך אחר להתעורר אלא גם לברר אם הדבר אכן קרה.

האוניברסיטה הפתוחה
THE OPEN UNIVERSITY OF ISRAEL
الجامعة المفتوحة
Yair
Kurtzman

File #0006182 belongs to Yair Kurtzman- do not distribute

2.4.5 סמפורים

כאן אנו מגיעים לפתרון כללי ואלגנטי מאוד לבעיית הקטע הקריטי. בשנת 1965 הציע Dijkstra להשתמש במשתנה מיוחד, s, שישמש כמונה של מספר קריאות ל-WAKEUP שעתידות להתבצע וגם יבדוק אם תהליך אחד התעורר.

Dijkstra הציע שתי פעולות, up ו-down, המהוות הכללה של SLEEP ו-WAKEUP במובן שתיארנו בפסקה הקודמת. הפעולות up ו-down פועלות על המשתנה המיוחד שמסוגל למנות את מספר ה-WAKEUPS כדלקמן:

```
down(s){
        if (s==0) place the process on the semaphore queue and move it to the block
        state;
        if (s > 0) s--;
}
```

```
up(s){
        if (there is at least one process sleeping in the semaphore queue){
                wake him up;
                return;
        }
        s++;
}
```

המשתנה המיוחד s מכונה "סמפור" (semaphore). למעשה, סמפור איננו משתנה רגיל, אלא מנגנון מיוחד המסופק על ידי מערכת הפעלה או שפת תכנות. המנגנון משתמש בתור המנהל את התהליכים הממתינים על הסמפור במצב רדום. כך ניתן לדעת אם תהליך שהיה אמור להתעורר אכן עשה זאת.

שימו לב: אם מאתחלים את הסמפור לערך חיובי, הסמפור לא יקבל לעולם ערך שלילי. ערך אתחול הסמפור משמש לקביעת מספר התהליכים שיכולים להיות ערים במקביל באותו הסמפור.

הפעולות up ו-down חייבות להיות אטומיות, שאם לא כן עלולים להיווצר מצבי מרוץ בעת עדכון התור של תהליכים הממתינים על הסמפור.

פתרון של בעיית היצרן-צרכן באמצעות סמפורים

נתבונן תחילה בשימוש הפשוט ביותר בסמפור, ונראה כיצד הוא פותר את בעיית המניעה ההדדית. נאתחל סמפור S בעל ערך 1. הפרוטוקול[8] לפתרון הבעיה יהיה:

[8] פרוטוקול מגדיר כללי התנהגות של תהליכים, והוא משול לכללי משחק או לכללי טקס שכל המשתתפים במשחק או בטקס חייבים להישמע להם כדי שהמשחק או הטקס יתנהלו כהלכה.

תהליך 0	תהליך 1

while (TRUE){
…// non critical section;
down(S);
critical_section_0();
up(S);
…// non critical section;
}
	while (TRUE){
	…// non critical section;
	down(S);
	critical_section_1();
	up(S);
	…// non critical section;
	}

ודאו שהפרוטוקול דלעיל אכן מקיים את ארבעת התנאים של הפתרון הסביר. הפרוטוקול הזה מדגים כיצד, בעזרת תמיכה של מערכת ההפעלה (או שפת תכנות), מקבלים פתרון טוב שאיננו סובל מהמתנה פעילה.

שאלה 13

האם הפתרון דלעיל מתאים למצב של כמה תהליכים?

סמפורים שהערך שלהם יכול להיות רק 0 או 1 נקראים ***סמפורים בינאריים***, והם מספקים מניעה הדדית למספר שרירותי של תהליכים. סמפורים שהערך שלהם יכול להיות גדול מ-1 מכונים לעתים ***סמפורים כלליים***. כאן ראוי לציין שהמימוש של סמפורים אינו מתאים תמיד להגדרה שראינו לעיל. אפשר גם לממש סמפור שיקבל ערכים שליליים או יבצע תוספת לערכו ברווחים שונים מ-1.

נאמר כמה מילים על מימוש של סמפורים. סמפורים יכולים להיות מסופקים כקריאות מערכת של מערכת ההפעלה. לחלופין, במערכת הפעלה שאינה תומכת בסמפורים, שפת תכנות יכולה להציע אותם בתנאי שמערכת ההפעלה מציעה שירותים לתקשורת בין תהליכים שהם שווי עוצמה לסמפורים. אם מערכת ההפעלה מספקת את השירותים up ו-down, אזי האטומיות שלהם מובטחת על ידי גרעין מערכת ההפעלה שיכול להשתמש בשיטת מניעת פסיקות או TSL כדי לעדכן את הסמפור. שימו לב שמניעה הדדית לעדכון של סמפור באמצעות מניעת פסיקות או TSL שונה בתכלית ממניעה הדדית לשימוש בפתרונות לבעיות מרוץ כלליות. זאת מכיוון שהעדכון של סמפור דורש הוראות מעטות בלבד ומתבצע במהירות.

נראה עתה פתרון לבעיית היצרן-צרכן באמצעות שלושה סמפורים. שניים מהם משמשים לסנכרון, והשלישי משמש להגנה על קטע קריטי.
סמפור בשם full מונה את מספר המקומות במאגר שכבר תפוסים על ידי הפריטים. סמפור empty מונה את מספר המקומות הפנויים במאגר. סמפור בינארי mutex משמש למניעה הדדית כדי שהיצרן והצרכן לא ייכנסו למרוץ בעת העדכון של המאגר ושאר הנתונים המשותפים. סמפור full מאותחל ל-0 וסמפור empty מאותחל ל-N שהוא מספר המקומות הפנויים במאגר.

הסמפור הבינארי שמגן על הקטע הקריטי של שינוי מספר הפריטים מאותחל כמובן ל-1.

האוניברסיטה הפתוחה THE OPEN UNIVERSITY OF ISRAEL الجامعة المفتوحة
Yair Kurtzman

File #0006182 belongs to Yair Kurtzman- do not distribute

עיינו בפתרון של בעיית יצרן-צרכן באמצעות סמפורים באיור 2-28 בספר הלימוד.
נסכם את יתרונות השימוש בסמפור:

- הוא מונע המתנה פעילה.
- אין הגבלה (עקרונית) על מספר התהליכים שהוא יכול לשרת.
- זהו שירות הניתן על ידי מערכת ההפעלה או שפת תכנות, ויישומו אינו מוטל על המשתמש.
- הסמפור מאפשר לפתור בעיות הקשורות לקטע הקריטי ולסנכרון.

2.4.6 מנעולים

מנעול (mutex) הוא משתנה במרחב זיכרון משותף המשמש לניהול הגישה לקטע קריטי, למשל בעבור תהליכונים או בעבור תהליכים שמשתפים אזור בזיכרון.[9] בדומה לסמפור בינארי, יש למנעול שני מצבים: "נעול" ו"לא נעול". בניגוד לסמפור בינארי, השימוש במנעולים יעיל יותר כאשר הוא אפשרי, מכיוון שלא נדרש בעבורו מעבר בין רמת המשתמש לרמת הגרעין.

נראה לדוגמה כיצד אפשר לממש את המנעול באמצעות הוראת TSL בספריית תהליכונים ברמת המשתמש. כדי להיות עקביים, נשכתב בפסידוקוד את מה שרשום בספר בפסידו שפת סף.

```
void MutexLock(mutex){
        while (TSL(mutex) != 0)  // lock mutex if is unlocked
                thread_yield(); // mutex is locked by some thread, schedule another
thread
}
```

```
void MutexUnlock(mutex){
        mutex = 0;
}
```

שאלה 14

מדוע צריך להשתמש בהוראת TSL? תארו תרחיש שבו השימוש בצמד לא אטומי של פעולות יכול לגרום לבעיית מרוץ.

עלייה ברמת המקבילות דורשת פתרונות סנכרון יעילים. המתנה פעילה נותנת פתרון מהיר כשזמן ההמתנה קצר, כיוון שאין צורך בקריאות המערכת, אבל כשהוא מתארך, היא גורמת לבזבוז זמן המעבד. לכן כשתהליכים רבים רצים במקביל, יהיה חסכוני יותר

[9] מערכות הפעלה מספקות אפשרות לשתף אזורי זיכרון בין תהליכים. נראה בהמשך כיצד הדבר אפשרי ב-Unix וב-Windows.

לחסום את התהליך הממתין כשהקטע הקריטי תפוס, ולשחרר אותו כשיתפנה. אך לא תמיד זה ניתן ליישום כי במקרים רבים קשה לחזות את רמת המקבילות.
קיים פתרון מעניין בשם ***futex*** – מנעול מהיר במרחב המשתמש, שמשלב את יתרונותיהן של שתי הגישות: תהליכון מנסה לתפוס את המנעול בעזרת TSL. במקרה של הצלחה, הוא בודק אם המנעול היה חסום בפניו. אם לא, הוא ממשיך לרוץ. אם הייתה חסימה, ספריית **futex** אינה מבצעת המתנה פעילה, אלא מעבירה את התהליכון למצב חסום בעזרת קריאת מערכת. כאשר המנעול יתפנה, גרעין המערכת יוציא את התהליכון ממצב חסימה לפי התור. המנגנון מאפשר למעט במספר קריאות המערכת.

שימוש ישיר במשתני תנאי (condition variables)

בנוסף למנעולים mutexes קיים מנגנון סנכרון שני: משתני תנאי. משתמשים במנעולים כדי לאפשר או לחסום גישה לקטע קריטי, ומשתני תנאי מאפשרים חסימת תהליכון בגלל תנאי כלשהו שלא מתקיים.
כדוגמה פשוטה, ניקח שוב את בעיית היצרנים-צרכנים: תהליכונים יצרנים מכניסים פריטים למאגר וצרכנים מוציאים אותם. אם היצרן מגלה שאין יותר תאים פנויים במאגר, עליו להיחסם עד שתא אחד יהיה זמין. מנעול מאפשר לבצע את הבדיקה בצורה אטומית בלי הפרעות מתהליכונים אחרים, אך לאחר גילוי שהמאגר מלא, היצרן זקוק לדרך להיחסם ולהתעורר מאוחר יותר. הצרכן צריך לנהוג בדרך דומה כשמאגר ריק. זה מה שמשתני תנאי מאפשרים.
הפעולות העיקריות במשתני תנאי הן cond_wait(cv,mutex) ו-cond_signal(cv).
הראשונה חוסמת את התהליכון הקורא ומשחררת את המנעול בצורה אטומית עד שתהליכון אחר מבצע cond_signal. קיימת פעולה נוספת, cond_broadcast, המאפשרת להעיר את כל החסומים.
משתני תנאי מאפשרים להמתנה ולחסימה להתבצע בצורה אטומית. משתני תנאים ומנעולים משמשים כמעט תמיד יחד: תהליכון בודק אם הוא יכול לקבל את מה שצריך, ואם לא, ממתין למשתנה תנאי. בסופו של דבר, תהליכון אחר יאותת לזה והוא יוכל להמשיך.
הקריאה ל cond_wait לא רק חוסמת את התהליכון הקורא, אלא גם פותחת באופן אטומי את המנעול שהוא מחזיק. מסיבה זו, המנעול הוא אחד הפרמטרים.
ראוי גם לציין שלמשתני תנאי (בניגוד לסמפורים) אין "זיכרון". אם אות (signal) נשלח למשתנה תנאי שאף תהליכון לא ממתין עליו, האות אובד. מתכנתים צריכים להיזהר מאובדן אותות.
ראו מימוש מעשי בעזרת הכלים הקיימים בספריית pthreads באיור 2-32 בספר הלימוד. שימו לב לסדר הקריאות לפונקציות הסנכרון.

הערה: במערכת Windows לא קיימים משתני תנאי, אבל קיים כלי המתנה אחר: event, שבניגוד למשתנה תנאי, "זוכר" את המצב.

מנעולים בתהליכוני POSIX

תת-סעיף זה הוא חומר רשות.

האוניברסיטה הפתוחה THE OPEN UNIVERSITY OF ISRAEL الجامعة المفتوحة
Yair Kurtzman

File #0006182 belongs to Yair Kurtzman- do not distribute

2.4.7 מבני פיקוח (monitors)

סמפורים הם כלים רבי עוצמה ורבי תועלת, אך הבעיה בשימוש בהם היא שמספר רב של סמפורים המפוזרים על פני תוכנית הופכים אותה לקשה לניפוי שגיאות. נתבונן למשל בפתרון של יצרן-צרכן שבאיור 2-28 בספר הלימוד, כאשר מוחלפות בו שתי שורות down בקוד של יצרן. נניח שהמאגר מלא לחלוטין. הרצה של הקוד החדש של היצרן תגרום לכך שהיצרן ינעל את הסמפור הבינארי mutex ואז יבצע down על סמפור empty. דבר זה יגרום ליצרן להיכנס למצב חסום עד שהצרכן יפרוק פריט מן המאגר. אבל כשהצרכן מגיע לפרוק פריט, הוא צריך לבקש גישה בלעדית לזמן עדכון המאגר על ידי נעילת mutex, ולכן גם הוא נחסם. מצב שבו שני תהליכים מחכים כל אחד בתורו להתרחשות של אירוע אשר יכול להיגרם מתהליך אחר נקרא מצב **קיפאון**. מה שמקשה מאוד הוא העובדה כי טעויות, אם ישנן כאלה, אינן ניתנות לשחזור מדויק הואיל והן תלויות בסדר של חילופי התהליכים.

מכאן נובעת השאיפה להעביר את הטיפול במניעה הדדית אל מנגנונים משוכללים יותר, שיהיו חסינים מפני שגיאות מקריות כגון זו המתוארת. כך נולד הרעיון להעביר את האחריות ליישום הטכני של פתרון בעיית המניעה ההדדית אל המהדר (הקומפיילר). כל שהמתכנת צריך לעשות במקרה זה הוא להגדיר קטע קוד כקטע קריטי (להעביר את ניהולו למבנה פיקוח), וכך יובטח כי לעולם לא ישהה בקטע זה יותר מתהליך אחד. זו מהותו של הפתרון המוצע בסעיף זה.

המבנה של שפת התכנות שהוצע לטיפול במניעה הדדית קרוי ***מבנה פיקוח*** (***monitor***). מבנה פיקוח הוא ספרייה של שגרות, משתנים ומבני נתונים המוגדרים יחדיו במבנה של שפת תכנות, והיא זאת שמבטיחה שתהליכים הקוראים לשגרות המוגדרות במבנה פיקוח מממשים מניעה הדדית. הקומפיילר מתרגם את הקריאות לפונקציות המוגדרות במבנה פיקוח לקוד שמוגן על ידי פרוטוקול למניעה הדדית. הואיל והקומפיילר הוא זה שמופקד על מימוש המניעה ההדדית, יש פחות סיכוי לטעויות מן הסוג שתיארנו בתחילת הסעיף.

אך אף שהמוניטור מספק דרך נוחה להשגת מניעה הדדית, הוא עדיין אינו יכול להיות שווה-עוצמה לסמפורים כל עוד הוא איננו מספק מנגנון לסנכרון בין תהליכים. בבעיית יצרן-צרכן, למשל, פרט למניעה הדדית בגישה אל המאגר, אנו זקוקים גם לאפשרות לספק בו-זמנית עם המניעה ההדדית אפשרות להכתיב לצדדים את המשך פעילותם – להפסיק לייצר בעת שהמאגר מלא או להתחיל לפרוק פריטים כאשר יש לפחות פריט אחד במאגר.

הפתרון לבעיות סנכרון שמוצע על ידי מבנה פיקוח הוא האפשרות להגדרה של משתנים מיוחדים הנקראים ***משתני תנאי*** (***condition variables***). ניתן לשלוט על המשתנים הללו באמצעות שתי פעולות:

- פעולת signal(c) מעירה אחד מהתהליכים שהיה חסום על ידי מבנה פיקוח על התנאי c.

- פעולת wait(c) גורמת להשהיית תהליך שיוכל להתעורר אחרי ביצוע signal(c) בעתיד, כל עוד התנאי c לא יתקיים. מבנה פיקוח מבטיח שלאחר הקריאה ל-wait, אחד מהתהליכים שהיה חסום בכניסה למבנה פיקוח, יוכל להיכנס אליו.

נעיין בפתרון של בעיית יצרן-צרכן באיור 2-35 בספר הלימוד כדי להבהיר את השימוש ב-condition variables. במבנה פיקוח מוגדרות שגרות insert ו-remove שמעדכנות את המאגר, ולכן צריכות להיות מוגנות על ידי מניעה הדדית. שני המשתנים full ו-empty משמשים לסנכרון בין היצרן לצרכן. המשתנה full מסמל כי "המאגר מלא". המשתנה empty מסמל כי "המאגר ריק". שגרת insert שמופעלת על ידי יצרן מבצעת קריאה ל-wait(full) כאשר מתגלה שאין עוד מקום לאחסון פריטים, והדבר משהה כמובן את תהליך היצרן. שימו לב כי השימוש במבנה פיקוח פוטר את המתכנת מן הצורך לשחרר את הנעילה על קטע קריטי, ולכן היצרן יושהה, אבל זה לא ימנע בעתיד מהיצרן להמשיך לפעול!

שגרת signal (empty) מופעלת על ידי היצרן לאחר שזה יצר פריט והכניסו למחסן אשר היה ריק לפני כן. הפעלת השגרה מעירה את הצרכן שהיה רדום עקב המחסור בפריטים במחסן. שימו לב למקום של הפעלת השגרה signal. היצרן מעיר את תהליך הצרכן כאשר סיים לעדכן את המאגר ונמצא בסיומו של קטע קריטי. הדבר מביא אותנו לדיון על האופנים שבהם אפשר לממש את הטיפול של מבנה הפיקוח בתרחיש מורכב שבו הקריאה ל-signal מעירה תהליך שייתכן מאוד שיקרא אף הוא לשגרות של מבנה פיקוח. תרחיש מסוג זה עלול לסבך מאוד את הטיפול במניעה ההדדית.

כדי להימנע משני תהליכים פעילים בתוך מבנה פיקוח, אפשר לנקוט גישות אלה:

- לתת לתהליך ש-signal(c) העיר אותו לרוץ, תוך כדי השהיה של התהליך שקרא ל-signal.
- לתת לתהליך שקרא ל-signal(c) לסיים את השגרה של מבנה הפיקוח ורק אז להעיר את אחד התהליכים שממתינים על c.
- לדרוש, ברמת התחביר של שפת התכנות, שהקריאה ל-signal(c) תתבצע רק בסוף השגרה של מבנה הפיקוח.

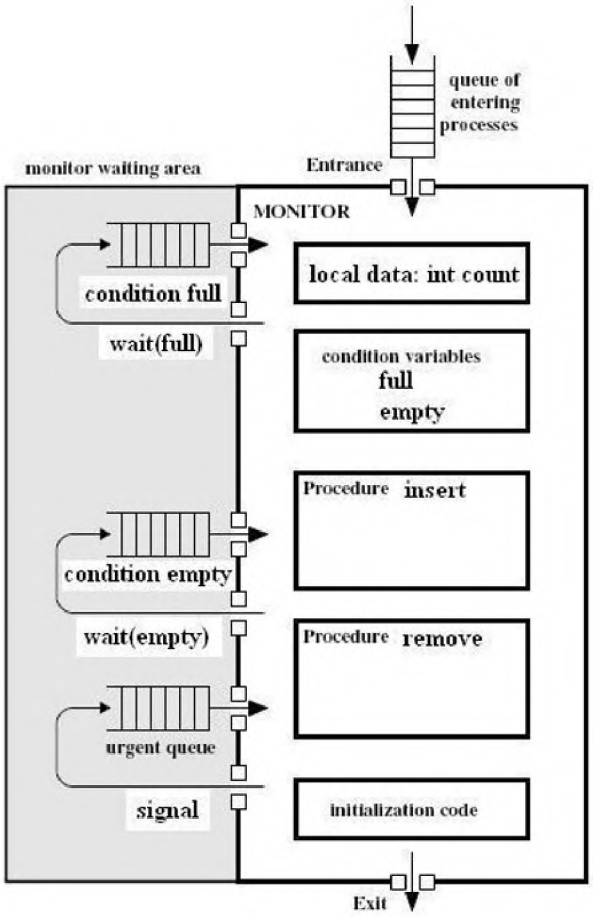
שאלה 15

השוו בין היתרונות והחסרונות של הגישות הללו.

באיור הבא ניתן לראות שעבור כל condition variable קיים תור של תהליכים שממתינים לאחר שביצעו קריאה ל-wait. למעלה נמצא תור של תהליכים שמבקשים לבצע את אחת השגרות של מבנה הפיקוח, ולכן נאלצים להמתין בגלל המניעה ההדדית. התור urgent נועד לתת עדיפות לתהליכים שקיבלו signal על פני תהליכים שעדיין לא נכנסו למבנה פיקוח. הדבר תלוי במימוש ספציפי ואפשר להעביר את התהליכים האלה לתור חיצוני. כדי להמחיש את משמעות המרכיבים של מבנה פיקוח, באיור שלפניכם מופיעים שוב המשתנים של פתרון בעיית היצרן-צרכן.

האוניברסיטה הפתוחה
THE OPEN UNIVERSITY OF ISRAEL
الجامعة المفتوحة
Yair
Kurtzman

File #0006182 belongs to Yair Kurtzman- do not distribute



איור 10

שאלה 16

ראינו פתרון של בעיית יצרן-צרכן באמצעות מבנה פיקוח.

האם הפתרון הזה יפעל גם במקרים האלה:

- יצרן-צרכנים
- יצרנים-צרכן
- יצרנים-צרכנים

יש לציין שלהבדיל מסמפורים שמשמשים כמונים (בינריים או כלליים), condition variables אינם "זוכרים" את מצבם (המושג "משתנה" משמש רק במובן שתנאי יכול להשתנות). אם מבוצע signal(c) כאשר אין שום תהליך שממתין על c, אז הסיגנל אובד. כדי שהדבר לא יקרה, הוראת wait מוכרחה לבוא לפני ה-signal.

מבני פיקוח מקטינים את הסיכוי לשגיאות שקשורות למניעה הדדית, אך שפות תכנות רבות אינן מספקות מבני פיקוח. ספר הלימוד מציין שברמה של מימוש בשפת תכנות, קל יותר לספק סמפורים מאשר לספק מבני פיקוח.

נסכם את הדיון בהבחנה שמבני פיקוח, בדיוק כמו הסמפורים, הם מנגנונים שפועלים בארכיטקטורה של מעבד אחד-זיכרון אחד או כמה מעבדים-זיכרון משותף. למערכות

מבוזרות עם ארכיטקטורה של כמה מעבדים-כמה זיכרונות, אנו נזדקק למנגנון אחר של תקשורת בין תהליכים. בסמפורים או במבנה פיקוח לא ניתן להשתמש למשל במערכות מבוזרות שמופעל בהן ניוד של תהליכים לביזור עומסים.

2.4.8 העברת הודעות (message passing)

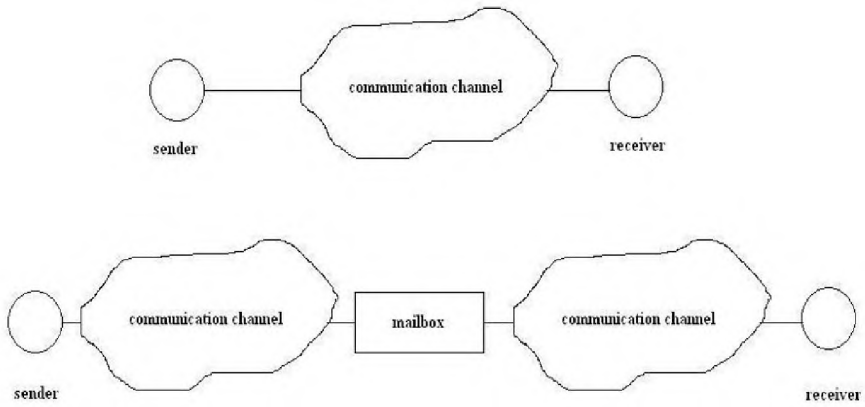
הפתרון המוצע בסעיף זה פועל הן במערכת מבוזרת והן במודל של כמה מעבדים עם זיכרון משותף, ולכן הוא כללי יותר. באופן כללי, השיטה של ***העברת הודעות*** מספקת שני פרימיטיבים : send ו-receive, המשמשים לשליחה ולקבלה של הודעות :

- השגרה send (destination, &message) שולחת את ההודעה message ליעד נתון.
- השגרה receive (source, &message) מקבלת הודעה ממקור נתון.

השליחה של הודעות יכולה להיות שליחה שחוסמת את התהליך השולח (עד שתתקבל על ידי היעד) או כזו שאיננה חוסמת (בדומה לכתיבה חוסמת או לא חוסמת לקובץ). כך גם לגבי קבלת הודעות.

היעדים יכולים להיות ישירים או עקיפים. ***מיעון ישיר*** מחייב להעביר כ-destination מזהה של תהליך. לסוג כזה של העברת הודעות קוראים גם בשם ***שיטת המפגש*** ***(rendezvous)***, זאת מכיוון שתהליכים מקיימים ביניהם מעין שיח בעת המפגש.

מיעון עקיף, לעומת זאת, משתמש בתחנת ביניים שאליה מועברות הודעות וממנה מתבצעת משיכתן. לעתים מכנים את תחנת הביניים ***תיבת דואר (mailbox)***. באיור הבא מוצגים מיעון ישיר (למעלה) ומיעון עקיף (למטה). השתמשנו בתצוגה כללית מאוד כאשר התהליכים מתקשרים דרך ערוץ תקשורת שיכול להיות גם צינור (pipe) של מערכת ההפעלה, אם מדובר על שולח ונמען השוכנים באותו מחשב, וגם ערוץ וירטואלי העובר דרך רשת מחשבים במקרה הכללי, כאשר התהליך המשדר והנמען שוכנים במחשבים שונים. במקרה של מיעון עקיף, תיבת דואר יכולה להיות במחשב של השולח או של הנמען או להיות במחשב שונה לגמרי.



איור 11

שאלה 17

מהם היתרונות והחסרונות של שיטת המפגש ושל תיבת דואר?

האוניברסיטה הפתוחה THE OPEN UNIVERSITY OF ISRAEL الجامعة المفتوحة
Yair Kurtzman

File #0006182 belongs to Yair Kurtzman- do not distribute

מאפיינים של מנגנון העברת ההודעות

נבחן היבטים אחדים הקשורים למנגנון העברת ההודעות. אפשר לחלק היבטים אלו לשתי קטגוריות:

- היבטים הקשורים להעברת הודעות במערכות מבוזרות דרך רשת תקשורת
- היבטים הקשורים להעברת הודעות באותה המכונה

איננו מעוניינים לסקור היבטים העוסקים ברמה של פרוטוקולי תקשורת כגון בקרת שגיאות או מנגנונים לאימות זיהוי השולח. גם הדיון על מימוש יעיל של העברת הודעות באותה המכונה נמצא מחוץ לתחום שלנו. אנחנו נמשיך לדון דיון עקרוני על גישות מימוש של העברת הודעות ונתייחס לתהליך של העברת הודעות כפרוצדורה נתונה שאיננה חושפת את הסיבוכיות המונחת ביסודה.

פתרון בעיית יצרן-צרכן באמצעות העברת הודעות

בטרם נעיין בפתרון של בעיית יצרן-צרכן, נראה כיצד אפשר לפתור בעיה של מניעה הדדית באמצעות העברת הודעות בשיטת מיעון עקיף:

תהליך 0	תהליך 1

message msg;
while(true){
…// non critical section
receive(message_box, msg);
CriticalSection_0();
send(message_box, msg);
… // non critical section
}
	message msg;
	while(true){
	…// non critical section
	receive(message_box, msg);
	CriticalSection_1();
	send(message_box, msg);
	… // non critical section
	}

```
main(){
    message token;
    SetUpMessageBox(message_box); // create mailbox named message_box
    PutMessage(message,_box, token); // send token to message_box
    CreateAndSheduleProcesses(P1, P2);
}
```

פתרון זה משתמש בפרימיטיב receive הגורם לחסימה של התהליך הקורא עד לקבלת ההודעה. כלומר, תהליך שמנסה לקבל הודעה מתיבת דואר ריקה – נחסם. הפתרון מתממש בתיבת דואר שבה מכניסים הודעה שתשמש כאסימון. כל אחד מהתהליכים (p1 ו-p2) ינסה למשוך הודעה מן התיבה. התהליך שמצליח מקבל לידו אסימון שמאפשר את כניסתו לקטע קריטי. מכיוון שמלכתחילה שידרנו לתיבה רק אסימון אחד, משיכת הודעה על ידי תהליך אחד תגרום לחסימתו של האחר וכך תתממש מניעה הדדית.

שאלה 18

כתבו פרוטוקול למניעה הדדית שמשתמש בהעברת הודעות במיעון ישיר.

באיור 2-37 מוצג הפתרון של בעיית היצרן-צרכן באמצעות העברת הודעות בשיטת מיעון עקיף. שימו לב שבעיית הסנכרון בין התהליכים נפתרת בפשטות על ידי הגדרה של תיבת דואר המכילה מראש N הודעות, כאשר N הוא מספר הפריטים שהמאגר יכול להכיל.

שאלה 19

האם הפתרון לבעיית יצרן-צרכן באמצעות העברת הודעות במיעון עקיף, שמוצג באיור 2-37, יפעל בצורה תקינה גם במקרה של בעיות דומות כגון:

- יצרן-צרכנים
- יצרנים-צרכנים
- יצרנים-יצרנים

2.4.9 מחסומים (barriers)

במקרים רבים מתעורר צורך להשהות קבוצה שלמה של תהליכים עד אשר יתקיים תנאי כלשהו. לדוגמה, פעולות על מטריצות גדולות יכולות להיות עתירות חישוב. אם קיימת דרך למקבל את הפעולה ולהריץ את חלקיה על מעבדים שונים, מתעורר צורך לחכות לסיום כל חלקי הפעולה כדי להמשיך לבצע את הפעולה הבאה. זהו הרציונל שעומד מאחורי שימוש במנגנון תקשורת תהליכים שנקרא ***מחסום***.

דוגמה אחרת קשורה לייעול של קוד שמבצעים קומפיילרים על ידי שינוי של סדר הפעולות. לעתים יש צורך שבקטעים מסוימים של הקוד הנתון, הקומפיילר לא יבצע ייעול כזה אבל באשר לשאר התוכנית אנו מעוניינים להשתמש בייעול שהקומפיילר מציע. שינוי סדר הפעולות מביא לידי קריאות ו/או עדכונים של אזורי זיכרון. לכן, כדי לאכוף ביצוע סדרתי של הוראות, אפשר לשים מחסומים על העדכון של אזורי זיכרון.

בגרעין של Linux קיימים מנגנונים לחסימה של קריאות ו/או כתיבות של אזורי זיכרון.

2.5 תזמון תהליכים
2.5.1 מבוא לתזמון תהליכים
2.5.2 תזמון במערכות אצווה
2.5.3 תזמון במערכות אינטראקטיביות
2.5.4 תזמון במערכות זמן אמת
2.5.5 מדיניות (policy) ומנגנונים (mechanisms)
2.5.6 תזמון תהליכונים

האוניברסיטה הפתוחה
THE OPEN UNIVERSITY OF ISRAEL
الجامعة المفتوحة
Yair
Kurtzman

File #0006182 belongs to Yair Kurtzman- do not distribute

2.5 תזמון תהליכים

בסעיף זה נלמד כיצד מערכת ההפעלה מבצעת תזמון של תהליכים. תזמון של תהליכים הוא נושא מורכב, המושפע מאופייה של מערכת ההפעלה ויש לו השלכות מרחיקות לכת. כך, למשל, ראינו בסעיף 2.3 ועוד נראה בסעיף 2.5 כיצד חוסר ודאות בבחירה של תהליכים על ידי המתזמן במערכת הפעלה רב-תכליתית משפיע על המתכנת. בסעיף זה נסקור בקצרה אלגוריתמים לתזמון במערכות אצווה ובמערכות אינטראקטיביות, ונראה מהו התנאי ההכרחי לתזמון משימות במערכות זמן אמת.

2.5.1 מבוא לתזמון תהליכים

כאמור, תזמון התהליכים במערכות הפעלה רב-תכליתיות הוא מורכב מאוד. תזמון תהליכים משפיע על כל מרכיבי מערכת ההפעלה: ניהול זיכרון, מערכת קבצים ועוד. מערכות הפעלה רב-תכליתיות מתמודדות עם מגוון רחב מאוד של משימות. משימות אלו ניתנות לסיווג לפי קטגוריות:

- משימות עתירות חישובים (לדוגמה, חישובים מדעיים)
- משימות עתירות פעילות I/O (לדוגמה, במערכות בנקאיות בעלות מספר גדול של טרנסקציות)
- משימות אינטראקטיביות (לדוגמה, מעבד תמלילים)
- משימות זמן אמת (לדוגמה, בנגני קול ווידאו)

במהלך הסעיף הזה, רובו ככולו, נתמקד באלגוריתמים לתזמון של תהליכים שנמצאים במצב מוכן לריצה. האלגוריתמים שנלמד יהיו מופקדים על המשימה הזאת: בהינתן קבוצה של תהליכים מוכנים לריצה, לבחור מתוכה איזה מן התהליכים יהיה התהליך הבא שמקבל זמן CPU ומתי תהליך שרץ ב-CPU עובר למצב מוכן. במילים אחרות, נתמקד במעברים בין מצב Ready למצב Running. אלגוריתמים אלה נקראים ***אלגוריתמי תזמון לטווח קצר*** (***Short-time Scheduling Algorithms***). בהמשך נראה כיצד האלגוריתמים לתזמון לטווח קצר משתלבים במנגנון התזמון המקבל החלטות תזמון לטווח ארוך. בסעיף 2.1.5 למדנו מהי מכונת מצבים מורחבת, ושם ראינו דוגמאות להחלטות לטווח ארוך. נזכיר דוגמה אחת להחלטת תזמון לטווח ארוך – תהליך חדש שמגיע למערכת יכול להתקבל למצב מוכן לריצה או למצב מוכן לריצה בדיסק.

ספר הלימוד נוטה להתייחס אל מנגנוני התזמון בצורה מופשטת. מערכות התזמון מסווגות כמערכות אצווה, מערכות אינטראקטיביות או מערכות זמן אמת. בהמשך נראה כיצד החלטות תזמון לטווח קצר משתלבות במנגנון התזמון במערכת האינטראקטיבית המופשטת ובמערכת אצווה מופשטת.

קריטריונים להערכה של אלגוריתמי תזמון לזמן קצר (scheduling algorithm goals)

כדי לקבוע מדד לטיב התזמון לטווח קצר, הונהגו הקריטריונים האלה בכל סוגי המערכות:

- ***הגינות*** (*fairness*) – דאגה לכך שכל תהליך יקבל אותו זמן CPU.
- ***אכיפת מדיניות*** (*policy enforcement*) – פעמים רבות קיימת קבוצה של תהליכים שחשוב להריץ אותם לפני תהליכים אחרים. אפשרות לקבוע מדיניות לקבוצות שונות של תהליכים שנמצאים במצב מוכן לריצה יכולה לשמש אחד המדדים.
- ***איזון*** (*balance*) – במערכות בעלות כמה מעבדים, הנושא של איזון עומסים בין המעבדים חשוב אף הוא.

הקריטריונים לתזמון לטווח קצר במערכות לעיבוד באצווה:

- ***קצב העבודה*** (*throughput*) – קצב העבודה מודד את כמות התהליכים שמסיימים ביחידת זמן. אלגוריתמי תזמון שונים יכולים לתזמן את אותה קבוצה של תהליכים, כך שקצב הסיום יהיה שונה בכל מקרה. לכן חשוב לבחור באלגוריתם תזמון בעל קצב עבודה גבוה.
- ***זמן השהייה*** (*turnaround time*) – זמן השהייה לתהליך הוא הזמן שחלף מהרגע שהתקבל במערכת ועד הרגע שבו סיים לרוץ. בעבור אלגוריתם תזמון, מודדים את זמן השהייה הממוצע של תהליכים. אלגוריתם תזמון טוב ממזער את המדד הזה.
- ***ניצולת ה-CPU*** (*CPU utilization*) – אלגוריתם התזמון צריך לדאוג לכך שהמעבד יהיה עסוק רוב הזמן.

הנה קריטריונים אחדים לתזמון לטווח קצר במערכות אינטראקטיביות:

- ***זמן תגובה*** (*response time*) – כולנו מכירים את שעון החול שצץ בעקבות הלחיצה על העכבר. במערכות אינטראקטיביות, זמן התגובה הוא ממוצע הזמנים שחלפו מרגע מתן הפקודה (כגון לחיצת כפתור מקלדת או עכבר) ועד לקבלת התגובה הראשונה. אלגוריתם תזמון צריך לשאוף להביא את זמן התגובה למינימום.
- ***יחסיות*** (*proportionality*) – למשימות אינטראקטיביות שונות קיים רף שונה לקביעת זמן תגובה טוב. ההגדרה של זמן תגובה טוב בהקלדת אות במעבד תמלילים שונה מההגדרה של זמן תגובה טוב להעתקה של קטע טקסט שמורכב ממספר גדול של אותיות. אם בהקלדת אות ניתן לספוג זמן תגובה של עד שנייה אחת, הרי שלא הגיוני שהעתקה של משפט שמורכב מ-50 אותיות תארך 50 שניות.

קריטריונים לתזמון לטווח קצר במערכות זמן אמת:

- ***עמידה בזמנים*** (*meeting deadlines*) – במערכות זמן אמת זהו אחד הקריטריונים החשובים ביותר. דוגמה לכך היא מערכת מחשב במטוס קרב שצריכה לבצע תמרון

האוניברסיטה הפתוחה THE OPEN UNIVERSITY OF ISRAEL الجامعة المفتوحة
Yair Kurtzman

File #0006182 belongs to Yair Kurtzman- do not distribute

נגד יירוט. אם הפעולות בתמרון אינן מתבצעות בסדרה מסוימת ועל פי לוח זמנים מוכתב תוך התחשבות במצב המשתנה של הסביבה האוויריות, התוצאות יכולות להיות קשות.

- ***ניתנות לחיזוי*** **(*predictability*)** – זהו מדד למשימות זמן אמת עם אילוצים פחות נוקשים. למשל, במערכות מולטימדיה חשוב להשיג העברה רציפה של זרם אודיו (audio stream). לצורך כך התזמון מטפל בתבניות רגולריות. דבר זה דורש חיזוי של תבניות עתידיות.

שאלה 20

כמה מן הקריטריונים שמנינו סותרים זה את זה. אלגוריתמי תזמון מיישבים סתירות אלו על ידי מציאת פשרה בין הקריטריונים השונים או על ידי בחירה של קריטריונים אחדים והקרבה של אחרים.

בעבור כל זוג קריטריונים שיש ביניהם סתירה, חשבו כיצד ניתן ליישבה.

2.5.2 תזמון במערכות אצווה

נתבונן כעת בשלושה אלגוריתמים לתזמון לזמן קצר במערכות אצווה.

First Come First Served – FCFS

FCFS, שלעתים מכונה גם ***FIFO*** **(*First In First Out*)**, הוא האלגוריתם הפשוט ביותר. המתזמן מנהל תור של תהליכים מוכנים לריצה. כאשר תהליך שרץ ב-CPU מבצע קריאת מערכת שגורמת לחסימה, המתזמן מקצה את CPU לתהליך הבא בתור. כאשר התהליך חוזר ממצב רדום, הוא מוכנס לסוף התור.

היתרון של האלגוריתם הוא פשטות המימוש שלו. החיסרון של האלגוריתם הוא זמן השהייה הארוך שלו. כך, למשל, ביצוע תהליכים עתירי קלט/פלט יארך זמן רב, אם במערכת קיימים הרבה תהליכים עתירי חישוב.

שאלה 21

הציעו שיפור פשוט של האלגוריתם המתואר שמתגבר על הבעיה עם תהליכים עתירי I/O.

Shortest Job First

אלגוריתם SJF יכול להתגבר על החיסרון של אלגוריתם FSFC בתנאי שזמני הריצה של התהליכים ידועים מראש ובתנאי שמפעילים אותו על קבוצה של תהליכים שנקבעו מראש.

אופן הפעולה: האלגוריתם בוחר לתת זמן CPU לתהליך בעל זמן הריצה הקצר ביותר ומריץ אותו עד לסיום. אלגוריתם תזמון כזה, דהיינו אלגוריתם תזמון שאינו משתמש בהפקעת מעבד, נקרא ***non-preemptive***.

נראה שאופן הפעולה החמדני של האלגוריתם אכן משפר את זמן השהייה. נניח שיש לנו ארבעה תהליכים P1, P2, P3, P4 בעלי זמני החישוב a, b, c, d בהתאמה. אם סדר הרצת התהליכים על ידי אלגוריתם תזמון כלשהו הוא P1, P2, P3, P4, אזי
זמן השהייה של P1 הוא a
זמן השהייה של P2 הוא a+b
זמן השהייה של P3 הוא a+b+c
זמן השהייה של P4 הוא a+b+c+d

ולכן זמן השהייה (הממוצע) יהיה (4a+3b+2c+d)/4. ומכאן ברור שאופן הבחירה של אלגוריתם SJF משיג זמן שהייה מזערי. שימו לב כי הטענה הזאת טומנת בתוכה הנחה נוספת והיא שהזמן המצטבר שהתהליכים P1-P4 בילו בפעולות קלט/פלט היה זהה לכל תהליך.

Shortest Remaining Time Next

השינוי הבא של אלגוריתם SJF מבטיח שאם זמני הריצה של תהליכים הם ידועים מראש וקבוצת התהליכים שהאלגוריתם המשופר פועל עליה היא קבועה, אזי זמן השהייה יהיה מזערי, ללא הנחות על זמני קלט/פלט.

הפתרון הוא פשוט אלגוריתם שמפקיע את המעבד. אלגוריתמים כאלה נקראים ***preemptive***. הפקעת מעבד יכולה להתבצע במקרים האלה:

- אם תהליך שהיה במצב חסום הופך להיות מוכן
- אם מגיע תהליך חדש שמוכן לריצה בזיכרון
- אם תהליך רץ מבצע קריאת מערכת שגורמת להרדמתו

במקרים הללו מתזמן המערכת מבצע בדיקה ובוחר לריצה את התהליך בעל הזמן הקטן ביותר שנותר. אלגוריתם זה ידוע בכינוי ***Shortest Remaining Time Next – SRTN***. כמו במקרה של SJF, לאלגוריתם זה דרושה ידיעה מוקדמת של זמן CPU.

שאלה 22

ודאו כי אתם מבינים מדוע SRTN מביאה למזעור של זמן שהייה בעבור קבוצת תהליכים קבועה. רמז: השתמשו בטיעון דומה לזה שהשתמשנו בו במקרה של SJF.
האם טיעון זה יהיה תקף גם כאשר קבוצת תהליכים איננה קבוצה קבועה ומגיעים תהליכים חדשים?

תזמון בשלוש רמות (סעיף זה איננו מופיע בספר הלימוד)

בסעיף הקודם דיברנו על תזמון לטווח קצר, ואמרנו שנראה כיצד הדבר משתלב במנגנון תזמון שצריך גם לקבל החלטות לטווח ארוך. ובכן, מנגנון תזמון של מערכת אצווה מופשטת מורכב משלוש רמות:

- ***CPU scheduler*** – ברמה זו מתבצע תזמון לטווח קצר. תזמון ברמה זו הוא שגורם למעברים בין מצב Ready למצב Running ולהפך. החלטות תזמון ברמה זו מתבצעות

האוניברסיטה הפתוחה THE OPEN UNIVERSITY OF ISRAEL الجامعة المفتوحة
Yair Kurtzman

File #0006182 belongs to Yair Kurtzman- do not distribute

בכפוף למדיניות המתזמן. המקרים שבדרך כלל תתבצע בהם הפעלת מתזמן CPU הם: תהליך רץ הקורא לפונקציית מערכת שגורמת להרדמתו; תהליך שהיה במצב רדום וחוזר למצב מוכן בזיכרון; תהליך מוכן לריצה המגיע לזיכרון הראשי. בשאר המקרים, הפעלת המתזמן תלויה במדיניות המתזמן.

- ***memory scheduler*** – ברמה זו מתבצעות החלטות תזמון שבעקבותיהן תהליכים עוברים ממצב מוכנות לריצה בזיכרון למצב מוכנות לריצה בדיסק ולהפך, או ממצב רדום בזיכרון למצב רדום בדיסק ולהפך. ראו שוב את איור מכונת המצבים המורחבת במדריך למידה זה. תזמון ברמה זו נקרא גם ***תזמון לטווח בינוני***. החלטות תזמון ברמה זו נועדות לשפר את הניצולת של מרכיבי המערכת. כך, למשל, הבאת חמישה תהליכים לזיכרון, שהזמן המצטבר של ביצוע פעולות הקלט/פלט שלהם הוא חמישית מזמן השהייה שלהם, יכולה להביא לניצולת גבוהה של המעבד ושל התקני הקלט/פלט. מובן שהמידע אשר יכול לשפר את הניצולת של מרכיבי המערכת אינו מצוי בדרך כלל בידי המערכת בתחילת ריצתם של תהליכים. לכן, החלטות תזמון ברמת ביניים מתבצעות בהסתמך על עברם של התהליכים, כך שבסופו של דבר התוצאה מביאה לרוב לשיפור בניצולת.
- מכלול הקריטריונים שמסייעים בהחלטות תזמון ברמת הביניים כולל את המדדים האלה: הזמן שתהליך שהה לאחרונה בדיסק או בזיכרון, זמן ה-CPU שתהליך צרך לאחרונה, גודל התהליך (המקום שהוא תופס בזיכרון) ושיקולים של אכיפת מדיניות הנובעים מחשיבותם של תהליכים לעומת תהליכים אחרים במערכת. אלה הם מקצת הדברים שיכולים להשפיע על ההחלטה להביא תהליך לזיכרון הראשי או להוציא אותו משם. לדוגמה, אם תהליך נמצא בזיכרון זמן רב, אך התהליך הוא גדול, העברתו לדיסק יכולה לארוך יותר זמן מן הזמן הדרוש לו לסיום. לכן אם קיים ידע מוקדם (למשל, מערכת שמנהלת סטטיסטיקות על הריצות הקודמות של תהליכים או אם תהליך הצהיר לפני הריצה כמה זמן חישוב הוא עומד לצרוך), החלטת התזמון יכולה להיות שונה מזאת שהייתה מתקבלת בהיעדר הידע הזה.
- ***admission scheduler*** – ברמה זו מתבצעות החלטות תזמון "גסות". התזמון ברמה זו מכונה לעתים ***תזמון לטווח ארוך***. רמת תזמון לטווח ארוך מאפיינת רק מערכות אצווה. במערכות אצווה, עבודות צריכות להצהיר על כמות משאבי המערכת שדרושה להן. על פי המידע הזה, המערכת בונה תכנון ארוך טווח לניצול משאביה ויכולה להחליט אם לקבל עבודה חדשה. החלטות תזמון אלה אינן מיוצגות באיור מכונת המצבים המורחבת. זאת הסיבה שהמילה שהשתמשנו בה בהקשר לישויות שהמתזמן פועל עליהן לטווח ארוך הייתה "עבודות" ולא "תהליכים". תהליך נוצר רק כאשר המתזמן מחליט לקבל עבודה למערכת; זהו מקור השם admission scheduler.

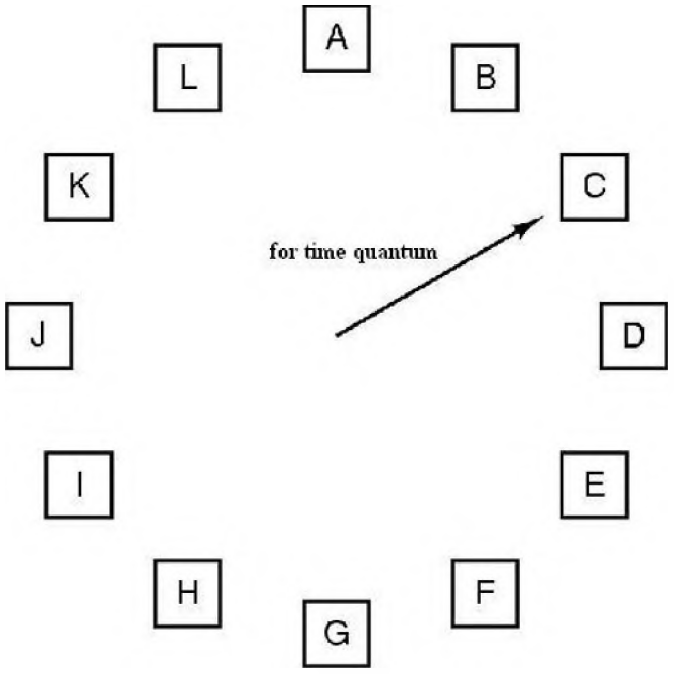
2.5.3 תזמון במערכות אינטראקטיביות

כעת נתבונן באלגוריתמים לתזמון במערכות אינטראקטיביות. להבדיל ממערכות אצווה, מנגנון התזמון של מערכות אינטראקטיביות אינו כולל מתזמן לטווח ארוך, ולכן כל העבודות מתקבלות למערכת ותהליכים חדשים נוצרים בעבורן. האלגוריתמים שנכיר בסעיף זה מתייחסים לתזמון לטווח קצר. את השיקולים לתזמון לטווח בינוני נבחן

כאשר נלמד על מנגנוני תזמון של Unix ו-Windows. חשוב לציין כי באלגוריתמים שנראה כאן אפשר להשתמש גם בתזמון לטווח קצר במנגנוני אצווה.

אלגוריתם *RR – Round-Robin*

הרעיון שעומד ביסודו של אלגוריתם זה הוא פשוט מאוד: מחלקים זמן CPU לאינטרוולים שווים ולכל תהליך שמוכן לריצה מקצים את ה-CPU לפרק זמן קצוב. פרק זמן זה מכונה ***time quantum***. לאחר סיום quantum, מפקיעים את ה-CPU ונותנים אותו לתהליך אחר. התהליכים מנוהלים בתור מעגלי ומכאן שם האלגוריתם. חלוקת זמן CPU לאינטרוולים שווים נקראת ***time slicing***.



איור 12

באיור, המשבצות מסמנות תהליכים והחץ מסמן את הקצאת המעבד בעבור תהליך C לזמן קצוב. בסיום הזמן מתבצעת החלפת תהליכים וה-CPU יוקצה לתהליך D.

תהליכים עשויים שלא לנצל את זמן ה-quantum עד תומו, עקב הקריאה לפונקציית מערכת הגורמת להרדמת תהליך. במקרה כזה, התהליך הרדום מוצא מהתור המעגלי של RR והחץ עובר לתהליך הבא בתור, לפי כיוון השעון. תהליך שהיה רדום וחזר להיות מוכן מוכנס לסוף התור, כלומר במקום שלפני התהליך המבוצע כעת. כפי שרואים, אלגוריתם RR הוא קל למימוש.

מכיוון שהחלפת תהליכים דורשת זמן, קביעת אורך ה-quantum היא נושא חשוב מאוד. קביעת "פרוסת זמן" קצרה גורמת לתקורה גבוהה של הזמן המתבזבז על החלפת תהליכים. עם זאת, קביעת פרוסת זמן ארוכה יכולה להביא לזמן תגובה ארוך, דבר הפוגע אנושות בזמן התגובה של תהליכים אינטראקטיביים. לכן, קביעת ה-quantum

האוניברסיטה הפתוחה THE OPEN UNIVERSITY OF ISRAEL الجامعة المفتوحة
Yair Kurtzman

File #0006182 belongs to Yair Kurtzman- do not distribute

למשך של ה-***CPU burst*** שהוא משך הזמן שלוקח לפעולה האינטראקטיבית להסתיים תהיה כנראה בחירה טובה.

בעבור פעולות אינטראקטיביות שונות, משך הזמן הזה יכול להיות שונה, ולכן מתעורר הצורך בדיפרנציאציה. לכן נשתמש בריבוי תורים, ונרחיב על כך בהמשך.

תזמון עם עדיפויות (קדימויות)

כבר ציינו כי האלגוריתם RR אינו מפריד בין תהליכים שזמני הפעולות האינטראקטיביות שלהם שונים. בשיטה של ריבוי תורים נראה פתרון גנרי שלפיו אפשר לבצע דיפרנציאציה. לעומת זאת, בסעיף הזה נראה כיצד אפשר לפתור את בעיית הדיפרנציאציה בשיטה פשוטה של תעדוף תהליכים.

הרעיון הבסיסי הוא שכל התהליכים המוכנים לריצה בזיכרון מדורגים לפי עדיפויות. תהליך בעל עדיפות גבוהה רץ על ה-CPU כל עוד עדיפותו גבוהה מעדיפותם של שאר התהליכים שמוכנים לריצה. אם התהליך הרץ מבצע קריאת מערכת שגורמת להרדמתו, ה-CPU עובר לתהליך בעל העדיפות הגבוהה ביותר שיימצא מוכן לריצה. החלטת תזמון דומה מתבצעת כאשר תהליך שהיה במצב רדום או בדיסק חוזר להיות במצב מוכן בזיכרון.

מבחינים בין שיטת מתן ***עדיפות קבועה*** ובין שיטת מתן ***עדיפות משתנה*** (דינאמית). מתן עדיפות גבוהה בצורה סטאטית יכול להיות מועיל במקרה של תהליכים שדורשים זמן תגובה מהיר מאוד, ובתנאי שלאחר פעולה אינטראקטיבית קצרה מאוד התהליך עובר למצב רדום. אחרת – תהליך בעל עדיפות גבוהה שאיננה משתנה מקבל את ה-CPU לתקופה בלתי מוגדרת (עד שיופיע במערכת תהליך בעל עדיפות גבוהה יותר). שאר התהליכים מורעבים.

בשיטה השנייה, שינוי עדיפויות מתבצע בצורה ***דינאמית***. לדוגמה, אלגוריתם תזמון יכול להפקיע את ה-CPU כעבור פרק זמן (quantum) מסוים כדי למנוע הרעבה. עדיפותו של תהליך תיקבע לפי החלק היחסי של ה-quantum שנוצל על ידיו. ככל שהחלק המנוצל יהיה גדול יותר, כך עדיפותו של התהליך תהיה קטנה יותר. כמו בדוגמה הקודמת עם עדיפות סטאטית, בהינתן תהליך שמבלה רוב הזמן במצב רדום אך דורש זמן תגובה מהיר כשהוא מתעורר, האלגוריתם עם מתן העדיפות הדינאמית שתיארנו לעיל יספק זמן תגובה טוב יותר לתהליכים הנמצאים זמן רב במצב רדום. יחד עם זאת, אלגוריתם זה אינו גורם להרעבה של תהליכים אחרים כי מובטח שלאחר שעובר ה-quantum מתבצעת הפקעת ה-CPU לטובת תהליך אחר.

הבעיה בשיטת העדיפויות היא שקשה מאוד להתאים אותה לקשת הרחבה של סוגי השירות שתהליכים דורשים. יתרה מזו, התנהגותם של תהליכים יכולה להשתנות במרוצת הזמן כתוצאה של מעבר לוגי מתקופת חיים (קטע קוד) אחת לתקופת חיים בעלת אופי שונה.

לכן הוצעה שיטה גנרית קלה להבנה, וגם קלה למימוש, שמאפשרת ביצוע שינויים עתידיים. שיטה זו משלבת את היתרונות של שיטת מתן עדיפויות ושל שיטת מתן העדיפות המשתנה, ומונעת הרעבה על ידי הקצבת פרקי זמן. זהו הנושא של הסעיף הבא.

ריבוי תורים

ריבוי תורים משמש פתרון לדיפרנציאציה בין קבוצות תהליכים שונות, כאשר כל התהליכים בקבוצה מצפים לאותו סוג שירות. לכל תור משויכת עדיפות, וסוג השירות שהתהליכים הנמצאים בתור מצפים לו נגזר ממנה. לכן כל התהליכים הנמצאים באותו תור הם בעלי עדיפות שווה. תהליכים יכולים לעבור בין התורים כתוצאה מהחלטת המתזמן. כל אחד מהתורים יכול להיות מנוהל לפי מדיניות שונה.

בשלב זה רצוי מאוד לראות כיצד שיטת ריבוי התורים פועלת במערכות Unix ובמערכת Windows, אילו קבוצות של תהליכים בעלי עדיפות זהה מוגדרות על ידי המתזמן, מהי מדיניות הניהול בכל תור וכיצד מתבצע מעבר בין התורים.

Shortest Process Next – SPN

בסעיף קודם ראינו שאלגוריתם SRTN יכול להביא למזעור של זמן השהייה הכולל בתנאים מסוימים. בין היתר, השימוש באלגוריתם SRTN מציב דרישה של ידיעת זמן ריצה צפוי. זה אולי סביר למערכות אצווה, אך דרישה כזאת במערכת אינטראקטיבית איננה סבירה. תהליכים אינטראקטיביים הם ***מונחי אירועים*** ***(event driven)***: דרוש זמן חישוב כדי להגיב על אירוע כלשהו, כגון הקלדת תו במקלדת או קבלת הודעה בערוץ תקשורת. לכן קשה לצפות מראש מה יהיה משך הזמן שיידרש לביצוע פעולה אינטראקטיבית.

אחת הגישות שניתן להשתמש בהן היא ניבוי הזמן הדרוש לפעולה הבאה, בהסתמך על ההיסטוריה של התהליך:

- נסמן ב-T(i) את הזמן שנדרש בפועל לביצוע הפעולה האינטראקטיבית ה-I–ית
- נסמן ב-E(i) מה היה משך הזמן המשוער לביצוע הפעולה האינטראקטיבית ה-I–ית
- ההשערה ההתחלתית על משך הזמן שיידרש לביצוע הפעולה האינטראקטיבית הראשונה צריך להיות נתון, והוא E(1).

הניבוי לזמן הפעולה האינטראקטיבית הבאה מחושב לפי הנוסחה:

$$E(i+1) = (1-a)\,T(i) + a\,E(i)$$

כאשר $0 < a < 1$.

הנוסחה מבצעת שקלול של זמני הפעולה שהיו בעבר. על ידי בחירת ערכו של a אפשר לייחס חשיבות רבה או פחותה להיסטוריה. למשל, על ידי בחירת $a = 0.5$, נקבל את סדרת הניבויים הזאת:

$$E(1)$$
$$E(2) = 0.5\,T(1) + 0.5\,E(1)$$
$$E(3) = 0.5\,T(2) + 0.25\,T(1) + 0.25\,E(1)$$
$$E(4) = 0.5\,T(3) + 0.25\,T(2) + 0.125\,T(1) + 0.125\,E(1)$$

האוניברסיטה הפתוחה THE OPEN UNIVERSITY OF ISRAEL الجامعة المفتوحة
Yair Kurtzman

File #0006182 belongs to Yair Kurtzman- do not distribute

כלומר, חשיבותם של הזמנים בעבר יורדת בצורה מעריכית. לשיטת ניבוי זו קוראים שיטת המיצוע המעריכי (exponential averaging) והיא מבוססת על גיל התהליך (aging).

שיטה זו מנסה לספק זמן תגובה מזערי. להבדיל מהניתוח שעשינו במקרה של SRTN, במקרה זה הניתוח איננו פשוט.

שיטת **SPN** יכולה לגרום להרעבה של תהליכים שהפעולה האינטראקטיבית שלהם ארוכה מממוצע של משך זמן הפעולה האינטראקטיבית במערכת. לכן ריבוי תורים יכול לעזור להתגבר על הבעיה.

שאלה 23

כיצד משפיעה בחירת ערכי a קטנים, במונחים של תהליך בודד שמבצע פעולת חישוב שמשכה חורג מהממוצע המאפיין את התהליך?

תזמון מובטח

גישה שונה בתכלית לתזמון תהליכים היא להבטיח הבטחה לגבי אופי השירות. לדוגמה, בהינתן מספר המשתמשים במערכת, אפשר להבטיח להם שיקבלו כמות שווה של זמן CPU או אפשר לתת הבטחה כזאת ברמה של תהליכים.

אופן הפעולה של המתזמן: מערכת ההפעלה מנהלת בטבלת תהליכים רישום של משך הזמן שבו עמד ה-CPU לרשות כל אחד מהתהליכים במערכת. בהינתן הבטחה לספק חלוקת זמנים שווה, ובהינתן המידע כמה זמן קיבל כל תהליך ביחס לזמן שהובטח לו, אלגוריתם התזמון יבחר את התהליך בעל היחס הנמוך ביותר ויריץ אותו ב-CPU עד שהיחס יעלה על היחס של תהליך אחר במערכת (או עד שהתהליך הרץ יבצע קריאת מערכת שתגרום להרדמתו).

lottery scheduling

הפעלת שיטת התזמון המובטח יכולה לדרוש תקורה לא קטנה. למשל, במקרה של ההבטחה על חלוקה שווה של זמני CPU, צריך לחשב מחדש את היחס לכל תהליכי המערכת כאשר מגיע למערכת תהליך חדש או כאשר אחד מהתהליכים עוזב את המערכת.

שיטת ההגרלה יכולה להפחית את התקורה של חישוב המידות המובטחות ולהביא לתוצאות דומות באופן סטטיסטי.

במקרה של חלוקה שווה של זמני CPU, מתזמן ייתן לכל תהליך חדש שנכנס למערכת כרטיס הגרלה המזכה בנתח כלשהו של זמן CPU. החלטות התזמון מתבצעות בפרקי זמן שמשכם נגזר מכמות הכרטיסים המשתתפים בהגרלה או מקריאת מערכת והגעת תהליך חדש. בעת החלטת תזמון מתבצעת הגרלה שבעקבותיה נבחר תהליך שיקבל את ה-CPU. כך שלאורך זמן השיטה משיגה חלוקה שווה למדי של זמני העיבוד.

השיטה איננה גורמת לתקורה, כי אין צורך לחשב את היחס של שימוש ה-CPU בכל החלטת תזמון. צריך רק לעדכן את משך הזמן של הריצה הבאה.

עוד יתרון של השיטה הוא שקל לתת ביטוי באמצעותה לחשיבותו של תהליך. תהליכים חשובים יכולים לקבל כמה כרטיסי זכייה, ובכך להגדיל באופן סטטיסטי את זמן ה-CPU המוקצה להם.

תהליכים שמשתפים פעולה ביניהם יכולים להעביר כרטיסים זה לזה, והדבר מאפשר פתרונות נוספים לבעיות כמו יצרן-צרכן באמצעות מנגנונים חדשים לתקשורת בין תהליכים.

Fair-Share Scheduling **(חומר רשות)**

ביסוד המושג "תזמון בחלוקה הוגנת" עומדת הבחנה חשובה: במערכת מחשב מרובת תהליכים התהליכים אינם ישויות בלתי תלויות זו בזו. בפועל, אפליקציות רבות מאורגנות כאוסף של תהליכים (או תהליכונים) ושיטות התזמון שלמדנו אינן מביאות זאת בחשבון. שיטות תזמון נבחנות על פי קריטריונים מערכתיים, כמו זמן תגובה מינימלי או זמן שהייה מינימלי. אבל קיימת גם פרספקטיבה נוספת, והיא נקודת המבט של המשתמש, שמוטרד בעיקר מהביצועים של קבוצת התהליכים שמהם מורכבת האפליקציה שלו.

לכן, אסטרטגיה של תזמון בחלוקה הוגנת נועדה להביא בחשבון את השתייכותם של תהליכים לקבוצה מסוימת. ההסבר המובא בספר הוא הסבר כללי שמתייחס למחשבה שמאחורי השיטה ואינו מתייחס לאופן המימוש, אבל קיימות גרסאות של Unix ששיטת החלוקה ההוגנת ממומשת בהן.

2.5.4 תזמון במערכות זמן אמת

אפשר לסווג מערכות זמן אמת לשתי קטגוריות מופשטות: מערכות שמציבות אילוצים נוקשים על זמני ביצוע ומערכות שהאילוצים בהן גמישים יותר. כך, למערכות מן הסוג הראשון ניתן לשייך מערכות במטוסים, במכוניות וכן הלאה. לסוג השני של מערכות אפשר לשייך מערכות מולטימדיה שמספקות לדוגמה streaming audio. במערכות אלו אי-עמידה בזמנים יכולה להיות נסבלת במידה מסוימת.

המשימות במערכות זמן אמת יכולות להיות מחזוריות או לא מחזוריות. ***משימות מחזוריות*** מתבצעות עם תקופת מחזור מסוימת, ואילו לגבי המשימות הלא מחזוריות הדבר איננו תקף. קיום של משימות מחזוריות במערכת זמן אמת מציב קריטריון לאפשרות ביצוע תזמון, שהוא בעצם תנאי הכרחי:

- נסמן ב-C(i) את מספר יחידות זמן החישוב ב-CPU הדרושות למשימה המחזורית i.
- נסמן ב-P(i) את מחזוריות המשימה – משך זמן המחזור של המשימה ה-i–ית. למשל, אם ניקח דוגמה שבה צריכות להתבצע 4 משימות, כל משימה דורשת רבע שנייה וכל אחת מהן צריכה להתבצע פעמיים בשנייה, נקבל את הטבלה הזאת:

האוניברסיטה הפתוחה THE OPEN UNIVERSITY OF ISRAEL الجامعة المفتوحة
Yair Kurtzman

File #0006182 belongs to Yair Kurtzman- do not distribute

משימה	P in seconds	C in seconds
A	1/2	1/4
B	1/2	1/4
C	1/2	1/4
D	1/2	1/4

ואז 1/2*4 <1. הקריטריון לאפשרות לתזמן m משימות בעלות המאפיינים C(i) ו-P(i), כאשר 1 =<i <=m, הוא: C(1)/P(1) + … + C(m)/P(m) < = 1, ולפיכך לא ניתן לתזמן.

שאלה 24

איזו הנחה חבויה בנוסחה של קריטריון התזמון?

תזמון של משימות במערכת זמן אמת יכול להיות סטאטי או דינאמי. תזמון סטאטי קובע תסריט לתזמון של כל המשימות האפשריות ובדרך כלל נמצא במערכות שיש בהן אילוצים נוקשים. תזמון דינאמי אינו מגביל כל כך ומאפשר לתזמן משימות שלא היה קיים לגביהן תכנון מוקדם.

2.5.5 מדיניות (policy) ומנגנונים (mechanisms)

בסעיף הקודם אמרנו כי האלגוריתמים לתזמון שלמדנו מגדירים יעדים בפרספקטיבה מערכתית, וכתוצאה מכך ייתכן שבמקרים רבים משתמש מקבל שירות שוויוני כאשר היה יכול לקבל שירות טוב יותר, והדבר היה מתאפשר אף בלי פגיעה בשאר המשתמשים.

אם מקדישים לכך מחשבה, אפשר להגיע לפשרה בין שיקולים מערכתיים לבין מתן גישה ייחודית. האפשרות הזאת נגזרת מהפרדה של מדיניות ומנגנון, והתפיסה תהיה כי תפקיד מערכת ההפעלה הוא להציע מנגנונים, כאשר דרך השימוש בהם תיקבע ברמת המשתמשים.

כדוגמה פשוטה לכך יכול לשמש אלגוריתם RR לתזמון תהליכים, כאשר מערכת ההפעלה יכולה לאפשר לשנות את הפרמטר המשפיע על מדיניותו. ההפרדה בין מנגנון למדיניות במקרה של RR היא מיידית. המנגנון מאפשר להחליף תהליכים בתום ה-quantum. המדיניות נקבעת לפי אורכה של יחידת ה-quantum.

דוגמה אחרת היא שיטת תזמון בחלוקה הוגנת, כאשר מנגנון התזמון מסופק על ידי מערכת ההפעלה אך המשתמשים יכולים להגדיר מדיניות לגבי קבוצה מסוימת של תהליכים.

2.5.6 תזמון תהליכונים

שיטת תזמון תהליכונים היא נושא שמושפע מן המימוש של תהליכונים: ברמת הגרעין, ברמת המשתמש או בשיטות מעורבות. בסעיף 2.2 דנו בהיבטים הקשורים לתזמון

תהליכונים. לא נעמיק בנושא מעבר להיבטים שהזכרנו. דיון מסוג זה דורש ניתוח מפורט של מימושים ספציפיים.

2.6 בעיות IPC קלאסיות
2.6.1 בעיית הפילוסופים הסועדים
2.6.2 בעיית הקוראים-כותבים

הערה: החומר הרלוונטי בספר הלימוד נמצא במקומות שונים

2.6 בעיות IPC קלאסיות

הבעיות המתוארות בסעיף זה הן בעיות קלאסיות בתחום שיתוף הפעולה בין תהליכים. כאן המקום להזכיר את רעיון ***הפרוטוקול*** (***protocol***). לכל בעיה אנו מגדירים כללי התנהגות של תהליכים, כך שאם התהליכים יתנהגו לפי כללים אלה, נוכל לדעת בוודאות את התוצאה. במילים אחרות, הפרוטוקול משול לכללי משחק או לכללי טקס שכל אחד מהמשתתפים במשחק או בטקס חייב לבצע כדי שהמשחק או הטקס יתנהלו כהלכה. בהמשך, הפרק עוסק בהצגת בעיות תיאום בין תהליכים ובמציאת פרוטוקולים המתאימים לבעיות אלו.

שאלה 25

מהו הפרוטוקול לפתרון בעיית הקטע הקריטי לזוג תהליכים בעזרת סמפור בינארי?

בסעיף 2.4.5 קבענו שלושה כללים לפתרון הבעיה:

- קיים סמפור בינארי S המאותחל למצב S=1.
- תהליך הנכנס לקטע קריטי מבצע DOWN(S).
- תהליך היוצא מקטע קריטי מבצע UP(S).

שלושת הכללים האלה יוצרים את הפרוטוקול המבוקש.

2.6.1 בעיית הפילוסופים הסועדים

זוהי אולי הבעיה המפורסמת ביותר בהקשר זה. פתרון של בעיה זו משמש כמודל שבאמצעותו בוחנים יעילות של מנגנונים חדשים לתקשורת בין תהליכים בסביבה שיש בה היקף מוגבל של משאבים, כמו התקני קלט/פלט.

והנה תיאור הבעיה: פילוסופים אחדים יושבים מסביב לשולחן עגול ולפני כל אחד מהם מונחת צלחת ספגטי. בין כל שתי צלחות מונח מקלון. כדי לאכול, פילוסוף צריך לאחוז בשני מקלונים בו-זמנית. חייו של פילוסוף מתחלקים לתקופות שבהן הוא חושב (ואינו רעב); רעב אך אין ברשותו שני מקלונים; אוכל (וברשותו, כאמור, שני מקלונים). כאשר פילוסוף הופך לרעב, הוא מנסה ליטול לידיו שני מקלונים. אם הוא מצליח לקבל את

האוניברסיטה הפתוחה THE OPEN UNIVERSITY OF ISRAEL الجامعة المفتوحة
Yair Kurtzman

File #0006182 belongs to Yair Kurtzman- do not distribute

המקלונים, הוא אוכל. עם סיום סעודתו, הפילוסוף מניח את המקלונים בחזרה בשני הצדדים של צלחתו וחוזר להרהר בבעיות שברומו של עולם. אנו מבקשים למצוא פרוטוקול המבטיח שאף אחד מהפילוסופים לא יגווע ברעב.

באיור 6-4 מוצג פתרון שלפיו כל פילוסוף נוטל לידו קודם את המקלון השמאלי ואחריו את המקלון הימני. אם המקלון שהוא מנסה לקחת אינו פנוי, הפילוסוף אינו מחזיר את המקלון שבידו (אם יש כזה), אלא ממשיך בעקשנות לנסות להשיג את המקלון. אם כל אחד מן הפילוסופים לקח מקלון שמאלי, אף אחד מהם לא יצליח להשיג את המקלון הימני, ומכיוון שהפילוסופים משחררים את המקלונים רק בסיום הסעודה – כל אחד מהם יחכה לשכן שמימינו. כך יגוועו כולם ברעב. מצב זה נקרא מצב ***קיפאון***, ואנו ננתח אותו בהרחבה בפרק 6.

אילו הפילוסופים היו משחררים את המקלון השמאלי בכל פעם שלא הצליחו להשיג מקלון ימני, המצב היה משתפר פלאים. עם זאת, גם התרחיש הזה אפשרי: פילוסופים לוקחים בזה אחר זה את המקלון השמאלי. כעת הם מנסים – כל אחד בתורו – לקחת את המקלון הימני. הם כמובן נכשלים, ובזה אחר זה משחררים את המקלונים שברשותם. לאחר מכן, הסבב חוזר חלילה. תיאורטית, הסבבים יכולים להימשך לנצח ללא התקדמות של ממש. אך אם הזמנים שבהם הפילוסופים מבקשים לאכול הם מקריים, ההסתברות לכך שהדבר יקרה שוב היא אפסית. לכן, הגרלה מקרית של זמני הארוחות האישיות מהווה פתרון הסתברותי לבעיה.

מצב שבו התהליכים רצים אך אין בריצה זו התקדמות של ממש למשך זמן נקרא מצב ***הרעבה***. תיאורטית, מצב הרעבה יכול להימשך זמן בלתי מוגבל. כפי שנראה בהמשך הקורס, ההבדל בין הרעבה לקיפאון הוא שכל התהליכים שנמצאים במצב קיפאון ממתינים לאירוע מאחד מתהליכי הקבוצה. לעומת זאת, תהליכים במצב הרעבה אינם מקבלים משאב כלשהו (בלי קשר לקבוצה נתונה) לאורך תקופת זמן שיכולה תיאורטית להימשך לנצח, וכתוצאה מכך הם אינם מסוגלים להתקדם במידה משמעותית.

שאלה 26

האם תהליך השוהה בלולאה אינסופית נמצא במצב של הרעבה?
האם תהליך השוהה בלולאה אינסופית נמצא במצב של קיפאון?

איור 2-47 מציג פתרון דטרמיניסטי, שונה מהפתרון ההסתברותי שראינו.

שאלה 27

כתבו את פתרון בעיית הפילוסופים הסועדים באמצעות מבנה פיקוח.

2.6.2 בעיית הקוראים-כותבים

בעיה ידועה נוספת שיכולה לשמש מודל שבאמצעותו בוחנים יעילות של מנגנונים חדשים בעבור בסיסי נתונים היא בעיית הקוראים-כותבים. כמו במקרה הקודם, ערכו הלימודי של המודל אולי גדול יותר מערכו המעשי כמודל.

תיאור הבעיה: כמה תהליכים מעדכנים קובץ משותף שמשמש כבסיס נתונים. הכללים האלה צריכים להתקיים:

- מספר בלתי מוגבל של תהליכים רשאים לקרוא מתוך הקובץ במקביל.
- כאשר מתבצע עדכון של הקובץ, רק התהליך המעדכן רשאי לגשת לקובץ. לא תותר גישה של קוראים או של כותבים אחרים.

ראו את הפתרון המוצג באיור 2-29 וודאו כי אתם מבינים מדוע הפתרון נותן העדפה לקוראים.

כעת נראה פתרון אשר נותן העדפה לכותבים. כאשר יש לפחות תהליך אחד שמעוניין לעדכן את בסיס הנתונים, הפתרון מונע גישה אל הקובץ מכל הקוראים. פתרון זה מתאים במקרים שבהם חשוב שהקוראים יקבלו מבסיס הנתונים מידע מעודכן ככל האפשר.

```
semaphore rsem = 1;
semaphore wsem = 1;
semaphore mutex_rc = 1;
semaphore mutex_wc = 1;
semaphore other_readers = 1;
int rc = 0;
int wc = 0;
```

```
void reader(void){
        while(true){
                down(other_readers);
                down(rsem);
                down(mutex_rc);
        rc++;
                if(rc == 1)
                        down(wsem);
                up(mutex_rc);
                up(rsem);
                up((other_readers);
                read_data_base();
                down(mutex_rc);
                rc--;
                if (rc==0)
                        up(wsem);
                up((mutex_rc);
        }
}
```

```
void writer(){
        while(true){
                down(mutex_wc);
```

האוניברסיטה הפתוחה THE OPEN UNIVERSITY OF ISRAEL الجامعة المفتوحة
Yair Kurtzman

File #0006182 belongs to Yair Kurtzman- do not distribute


```
                wc++;
                if(wc == 1)
                        down(rsem);
                up(mutex_wc);
                down(wsem);
                write_data_base();
                up(wsem);
                down(mutex_wc);
                wc--;
                if(wc == 0)
                        up(rsem);
                up(mutex_wc);
        }
}
```

המשתנים rc ו-wc מחזיקים את מספר הקוראים ואת מספר הכותבים המעוניינים לקרוא מקובץ או לכתוב לקובץ, בהתאמה. הסמפורים הבינאריים mutex_rc ו-mutex_wc משמשים להגנה על הגישה למונים rc ו-wc, בהתאמה. הסמפור rsem מונע מכל הקוראים לגשת לבסיס נתונים כל עוד יש לפחות תהליך כותב אחד. הסמפור wsem נועד לממש מניעה הדדית בעת עדכון של בסיס הנתונים. הסמפור האחרון other_readers נועד למנוע מכל הקוראים, פרט לקורא הנוכחי, לגשת לקובץ כל עוד יש תהליך שמעוניין לעדכן את בסיס הנתונים.

כעת נעיין במקרים האלה:

- כאשר יש רק תהליכים קוראים, אכן הערך של rc מנוהל כהלכה באמצעות מניעה הדדית.
- כאשר יש רק תהליכים כותבים, יש מניעה הדדית על עדכון הערך של wc ועל הגישה לעדכון של בסיס נתונים.
- כאשר קיימים תהליכים קוראים ומגיעים תהליכים כותבים, התהליך הקורא הנוכחי מסוגל להעביר את שאר התהליכים הקוראים להמתנה בסמפור other_readers. בדקו את שאר התרחישים, כתרגיל.
- כאשר קיימים תהליכים כותבים ומגיעים תהליכים קוראים, הם אינם יכולים להיכנס כי הם ימתינו בסמפור rsem. המעדכן האחרון שמסיים את פעולתו מבצע up(rsem) כדי לאפשר לקוראים לקרוא לאחר שאין עוד שום כותב. בדקו את שאר התרחישים, כתרגיל.

סעיפים שאינם מופיעים בספר הלימוד
2.7 סיכום
2.8 בעיות פתורות
2.9 שאלות חזרה
2.10 תרגילי תכנות לתרגול עצמי
2.11 רשימת מושגים

2.7 סיכום

בפרק זה עסקנו בשלושה נושאים:

- בעיית המרוץ הקריטי והדרכים לפתרונה.
- תיאום ושיתוף פעולה בין תהליכים או תהליכונים שונים.
- תזמון תהליכים ותהליכונים.

שני הנושאים הראשונים הם קשים להבנה למי שנתקל בהם לראשונה.
מומלץ לעבור עליהם פעם נוספת.

2.8 בעיות פתורות

שאלה 1

במערכת נתונה יש 8 מעבדים ו-20 דיסקים. המערכת עמוסה במשימות שכל אחת מהן דורשת 4 דיסקים כדי להסתיים. מבין 4 הדיסקים, 3 דרושים לכל משימה במהלך ריצתה, והדיסק הרביעי דרוש רק לפרק זמן קצר, לקראת סיום המשימה. המשימות מן הסוג המתואר מגיעות למערכת כל הזמן.

א. נניח שהדיסקים המוקצים למשימות משוחררים רק עם סיומן. אם לא נתזמן את המשימות כל עוד אין ברשותן 4 דיסקים, מהו מספר המשימות המרבי שיכול לרוץ במערכת במקביל? מהו המספר המרבי (או המספר המזערי) של הדיסקים שיכולים להיות מושבתים כתוצאה ממדיניות התזמון?

ב. הציעו מדיניות טובה יותר שתשפר את הניצולת של הדיסקים. מהו מספר המשימות המרבי שיכול לרוץ במערכת בהינתן המדיניות החדשה? מהו המספר המרבי (המספר המזערי) של הדיסקים שיכולים להיות מושבתים כתוצאה ממדיניות התזמון החדשה?

האוניברסיטה הפתוחה THE OPEN UNIVERSITY OF ISRAEL الجامعة المفتوحة
Yair Kurtzman

File #0006182 belongs to Yair Kurtzman- do not distribute

תשובה

א. מספר המשימות המרבי הוא 20/4 = 5. מכיוון שלכל משימה יכול להיות דיסק אחד מושבת, המספר המרבי של הדיסקים המושבתים הוא 5. המספר המזערי הוא 0 כי במקרה הטוב שום דיסק לא יהיה מושבת ברגע נתון.

ב. כדי לשפר את הניצולת, נקצה לכל משימה 3 דיסקים ואת הדיסק הרביעי נקצה על פי דרישה. מספר המשימות המרבי יגדל להיות $\lfloor 20/3 \rfloor = 6$. המספר המרבי של הדיסקים המושבתים הוא 2. המספר המזערי הוא 0.

שאלה 2

האם תהליך השוהה בלולאה אינסופית נמצא במצב של הרעבה?

תשובה

לא. במקרה של לולאה אינסופית, הגורם לקפיאת התהליך הוא פעולתו של התהליך עצמו, ולא פעולתו של גורם חיצוני.

שאלה 3

סנטה קלאוס ישן בביתו בקוטב הצפוני, ויכול להתעורר רק באחד המקרים האלה:

- כאשר כל תשעת האיילים שלו חוזרים מהחופשה כדי לצאת ולחלק מתנות לילדים
- כאשר לפחות שלושה גמדים נתקלים בבעיה בבניית הצעצועים

כאשר שלושה גמדים שנתקלו בבעיה מעירים את סנטה קלאוס, גמדים אחרים שנתקלו בבעיות נאלצים לחכות עד שהשלושה הראשונים יסיימו לפתור את בעיותיהם. אם סנטה קלאוס מתעורר כדי לפתור בעיות לגמדים, והאייל האחרון מגיע מהחופשה, הגמדים ייאלצו לחכות עד אחרי חלוקת המתנות לילדים מכיוון שהחלוקה דחופה יותר. האייל התשיעי שחוזר מהחופשה (זה עשוי להיות אייל אחר מדי שנה) צריך להעיר את סנטה קלאוס בזמן שהאחרים מחכים ליד המזחלת.

כתבו שגרות לסנטה, לאייל ולגמד. הפתרון יהיה באמצעות סמפורים. הניחו כי הפעולות HelpDwarves ו-GiveGifts מוגדרות מראש.

תשובה

```
Initialization(){
        Int Number_of_deer = 9      // constant
        Int Size_of_dwarves_group = 3       // constant
        Int deer_counter = 0        // number of deer that are ready (between 0 and 9)
        Bool deer_ready = false       // true iff all nine deer are ready
        Int dwarves_counter = 0     // number of waiting dwarves
        Int dwarves_ready = 0       // number of groups (threes) of waiting dwarves
        Sem s_deercount = 1     // protects (ensures mutual exclusion) deer_counter
```

```
        Sem s_deerready = 1        // protects deer_ready
        Sem s_dwarfcount = 1       // protects dwarves_counter
        Sem s_dwarvesready = 1     // protectes dwarves_ready
        Sem s_deerfree = 0         // causes deer to wait until santa comes
        Sem s_dwarffree = 0        // causes dwarves to wait until santa comes
        Sem s_santa = 0            // causes santa to sleep while nobody needs him
}
```

```
Deer(){
        while (1) {
                enjoy_my_vacation();
                bridle_myself_to_sledge();
                wait(s_deercount){
                deer_counter = deer_counter + 1;
                if (deer_counter == Number_of_deer ){
                        wait(s_deerready);
                        deer_ready = true;
                        signal(s_deerready);
                        signal(s_santa);
                }
                signal(s_deercount);
                                wait(s_deerfree);
        }
}
```

```
Dwarf(){
        while (true){
                build_toys();
                wait(s_dwarfcount);
                dwarves_counter = dwarves_counter + 1;
                if (dwarves_counter modulo Size_of_dwarves_group == 0){
                        wait(s_dwarvesready);
                        dwarves_ready = dwarves_ready + 1;
                        signal(s_dwarvesready);
                        signal(s_santa);
                }
                wait(s_dwarfcount);
                signal(s_dwarffree);
        }
}
```

```
Santa(){
        while (true){
                wait(s_santa);
```

האוניברסיטה הפתוחה THE OPEN UNIVERSITY OF ISRAEL الجامعة المفتوحة
Yair Kurtzman

File #0006182 belongs to Yair Kurtzman- do not distribute


```
                wait(s_dearready);
                if (deerready){
                        give_gifts_to_children();
                        deerready = false;
                        for i := 1 to Number_of_deer
                                signal(s_deerfree);
                }
                signal(s_dearready);
                wait(s_dwarvesready);
                if (dwarvesready > 0){
                        help_three_dwarves()
                        dwarvesready = dwarvesready – 1;
                        for i := 1 to Size_of_dwarves_group
                                signal(s_dwarffree);
                }
        }
}
```

שאלה 4

הוכיחו כי קיימת שקילות בין העברת הודעות לבין סמפורים על ידי כך שתראו שניתן לממש את העברת ההודעות באמצעות סמפורים ולהפך.

רמז לכיוון הראשון של ההוכחה: השתמשו בחוצץ משותף המשמש להחזקת ה-mailboxes כאשר כל mailbox הוא מערך המורכב מהודעות.
רמז לכיוון השני של ההוכחה: השתמשו בתהליך עזר לסנכרון.

תשובה

בכיוון ראשון:

```
send (message, dest){
        wait (mbuf)             //wait for message buffer available
        wait (mutex)            //mutual exclusion on message queue
        acquire free buffer slog
        copy message to slot
        link slot to other messages
        signal (dest.sem)               //wake destination process
        signal (mutex)          //release mutual exclusion
}
```

```
Receive(message){
        wait (own.sem)          //wait for message to arrive
```

```
wait (mutex)              //mutual exclusion on message queue
unlink slot from own.queue
        copy buffer slot to message
        add buffer slot to freelist
        signal (mbuf)             // indicate message slot freed
        signal (mutex)  //  release mutual exclusion
}
```

בכיוון השני:

תהליך שמבצע סנכרון מחזיק מונה ורשימה מקושרת של תהליכים שממתינים על סמפור מסוים. תהליך שמבצע wait או signal קורא לפונקציית ספרייה מתאימה. הספרייה דואגת לשלוח הודעה לתהליך הסנכרון ובהודעה נמסרים סוג הפעולה וזהות הסמפור שעליו מתבצעת הפעולה. הספרייה גם דואגת לקבל תשובה מתהליך הסנכרון. כאשר ההודעה מגיעה לתהליך הסנכרון, עליו לבדוק אם קיימת אפשרות לסיים את הפעולה בהצלחה (כי wait לא תמיד מסתיים, ולעתים עלול לגרום להשהיית התהליך המבקש לבצע אותו). אם הפעולה המבוקשת ניתנת לביצוע, אז תהליך הסנכרון שולח חזרה הודעה ריקה. בעקבות קבלת ההודעה, התהליך שביקש לבצע פעולה יימשך. לעומת זאת, אם הפעולה המבוקשת היא wait וערך הסמפור היה 0, אז תהליך הסנכרון יכניס את התהליך הקורא לתור מתאים וכתוצאה מכך התהליך הקורא יישאר חסום. מאוחר יותר, כאשר יישלח signal אל אותו הסמפור, תהליך הסנכרון יבחר את אחד התהליכים הממתינים בתור וישלח הודעה ריקה לתהליך הקורא המתאים.

שאלה 5

נגדיר אלגוריתם תזמון המבוסס על עדיפות דינאמית. מספר גדול מבטא עדיפות גבוהה. כאשר תהליך ממתין ל-CPU, כלומר נמצא במצב ready, עדיפותו משתנה בקצב של α ליחידת זמן. כאשר תהליך רץ, עדיפותו משתנה בקצב של β ליחידת זמן. כל התהליכים מקבלים עדיפות התחלתית 0 בהגיעם למערכת. שינוי הפרמטרים α,β מאפשר לקבל אלגוריתמי תזמון רבים.

א. איזה אלגוריתם מתקבל כאשר $\beta>\alpha>0$?

ב. איזה אלגוריתם מתקבל כאשר $\alpha<\beta<0$?

תשובה

א. כאשר $\beta>\alpha>0$ מתקבל האלגוריתם FCFS.

תהליך שנמצא במערכת במשך זמן t ועדיין לא התחיל לרוץ, עדיפותו תהיה αt. התהליך שייבחר לריצה לא יפסיק לרוץ כיוון ש-$\beta>\alpha$, ומכיוון שכך עדיפותו רק תעלה ככל שימשיך לרוץ.

ב. כאשר $\alpha<\beta<0$, נקבל אלגוריתם חדש הפועל כדלהלן:

- האלגוריתם יבחר לתזמן את התהליכים שנמצאים במערכת במשך הזמן הקצר ביותר. זאת כיוון שעדיפותו של התהליך הנמצא בהמתנה במשך זמן t תהיה $-|\alpha t|$.

האוניברסיטה הפתוחה
THE OPEN UNIVERSITY OF ISRAEL
الجامعة المفتوحة
Yair
Kurtzman

File #0006182 belongs to Yair Kurtzman- do not distribute

- תהליך שהתחיל לרוץ יסיים את ריצתו בתנאי שלא יגיעו תהליכים חדשים למערכת. זאת מכיוון שכאשר התהליך נבחר לריצה, עדיפותו הייתה מרבית, ובזמן הריצה העדיפות תרד בקצב איטי יותר מזו של התהליכים האחרים במערכת.
- תהליך שהתחיל לרוץ ייאלץ לעבור למצב ready עם הגעת תהליך חדש. זאת מכיוון שלתהליך חדש תמיד מוקנית עדיפות גבוהה יותר.

שאלה 6

הרבה מאלגוריתמי התזמון כוללים פרמטרים. Round Robin, למשל, דורש את מנת הזמן הניתנת לכל תהליך בכל סיבוב, ו-Multilevel Feedback Queues כולל את מספר התורים, את מנת הזמן המוקצבת לתהליך מכל תור ואת הקריטריון למעבר בין תורים. על ידי כך אלגוריתמים אלו מגדירים למעשה משפחה של אלגוריתמים. משפחות אלו יכולות להכיל זו את זו: FCFS, למשל, הוא מקרה פרטי של RR עם מנת זמן אינסופית. אילו קשרי הכלה, אם בכלל, קיימים בין האלגוריתמים האלה:

א. Priority ו-SJF
ב. FCFS ו-Multilevel Feedback Queues
ג. Priority ו-FCFS
ד. RR ו-SJF

תשובה

א. SJF שייך למשפחה של אלגוריתמים עם עדיפויות. נגדיר את עדיפותו של תהליך כהופכי של זמן הריצה המשוער שלו.
ב. FCFS שייך ל-MFQ עם מנת זמן אינסופית.
ג. FCFS שייך למשפחה של אלגוריתמים עם עדיפויות, כאשר העדיפויות מוגדרות כך שכל תהליך חדש מקבל את העדיפות של התהליך שקדם לו פחות אפסילון חיובי קטן.
ד. אין קשר הכלה בין SFJ ו-RR. גם אם נקבע פלח זמן אינסופי ל-RR, לא נוכל לעשות דבר עם מדיניות ההכנסה לתור. לכן אין אפשרות למצוא פרמטרים כך ששני האלגוריתמים ייתנו את אותה התשובה על כל הרכב משימות אפשרי.

2.9 שאלות חזרה

אילו מבין הטענות שלהלן הן טענות נכונות? (כל התשובות נמצאות בחומר הלימוד, בסעיף עצמו או במדריך זה.)

1. כדי לפתור את בעיית הקטע הקריטי המשותף לשני תהליכים, די להשתמש במשתנה משותף יחיד.
2. בעזרת סמפורים בינאריים ניתן לפתור את בעיית הקטע הקריטי בעבור שני תהליכים בלבד.

3. קיימים מקרים שבעבורם ניתן להוכיח אופטימליות של שיטת תזמון.
4. כל התהליכים ב-UNIX הם בנים של תהליך יחיד.
5. תהליך אינו יכול לעבור ממצב שבו הוא ממתין ל-CPU למצב שבו הוא חסום בלי לקבל זמן עיבוד.
7. הפתרון של פטרסון מונע קיפאון.
8. כאשר משתמשים בעדיפויות קבועות לתזמון תהליכים, קיימת סכנת הרעבה.
9. הפתרון ההסתברותי לבעיית הפילוסופים הסועדים מונע לחלוטין הרעבה.

2.10 תרגילי תכנות לתרגול עצמי

כתבו תוכנית המאפשרת הזמנות וביטולים בו-זמניים של מקומות בטיסה. ההזמנה נעשית על ידי כמה תהליכים במקביל. במערכת זו תהליך אחד הוא המנהל, תהליך אחד הוא מרכז ההזמנות של הטיסה ואילו התהליכים האחרים הם סוכני הנסיעות. כל תהליך שהוא סוכן נסיעות מבקש מדי פעם מספר מקרי של מקומות במטוס ומקבל תשובה ממרכז ההזמנות. אם הדבר אפשרי, הוא מקבל מספר הזמנה ואת מספרי המושבים שהוקצו להזמנה. אם אין מקומות, הוא מקבל הודעה מתאימה. סוכן נסיעות יכול להודיע מדי פעם גם על ביטול הזמנה ואז המקומות שהוקצו – מתפנים. כל ההזמנות והביטולים נרשמים במקביל גם על ידי המנהל. לאחר זמן מסוים, הטיסה נסגרת וכל התהליכים מסתיימים, אולם לפני סיום כל התהליכים, המנהל עורך ביקורת כדי לבדוק אם כל הפעולות התנהלו כהלכה (כלומר, אם כל מושב מוקצה לכל היותר להזמנה אחת, ואם כל ההזמנות והביטולים נקלטו).

הדרכה:
השתמשו ב-NPTL (native POSIX threads library). לשם כך עיינו ב-man pages של מערכת ה-Knoppix בנושא pthreads.

2.11 רשימת מושגים

להלן רשימת המושגים המופיעים בסעיף זה (על פי סדר הופעתם בסעיף):

תוכנית	program	רצף של פקודות שיכולות להתבצע על ידי המחשב. אנו נתכוון בדרך כלל לקובץ בינארי של פקודות מכונה שניתן להריצו.
תהליך	process	מושג מופשט בתחום מערכות ההפעלה: מעין מחשב וירטואלי או תוכנית בריצה, המכילה את המרכיבים האלה: מצב המעבד, מפת זיכרון, טבלת שגרות איתות, הרשאות, רשימת משאבים זמינים, מצב תזמון וסטטיסטיקות שונות.
טבלת תהליכים	process table	מבנה נתונים פנימי של מערכת ההפעלה שבו בעבור כל תהליך שרץ במערכת, נשמרת רשומה הכוללת מידע על אודות מצבו הנוכחי של התהליך.

האוניברסיטה הפתוחה THE OPEN UNIVERSITY OF ISRAEL الجامعة المفتوحة
Yair Kurtzman

File #0006182 belongs to Yair Kurtzman- do not distribute

תהליכון	thread	מושג מופשט בתחום מערכות ההפעלה: מעין מעבד וירטואלי או מצב מסוים של תוכנית בריצה, המכיל את המרכיבים האלה – מצב המעבד, מצב תזמון וסטטיסטיקות שונות. תהליכונים משמשים להפרדה בין ניהול משאבים לביצוע משימות או ניבים של תוכנית. תהליכונים יכולים להיות ברמת גרעין המערכת וברמת המשתמש. ייתכנו גם מימושים משולבים.
תנאי מרוץ	race condition	מצב שבו כמה תהליכים עובדים על מידע המשותף לכולם, והתוצאה הסופית של פעולתם תלויה בסדר המדויק של הקצאת המעבד לתהליכים אלה.
קטע קריטי	critical section	קטע תוכנית שתוצאות ריצתו תלויות בהחלטות של מנגנון התזמון, כלומר בסדר שבו תהליכים שונים מקבלים הקצאת מעבד. במילים אחרות, זהו קטע שבו יכולים להיווצר תנאי מרוץ (race conditions).
מניעה הדדית	mutual exclusion	תנאי (או הבטחת התנאי) האומר כי אם תהליך אחד משתמש במשאב מסוים, שום תהליך אחר לא יקבל גישה למשאב זה עד שהתהליך המשתמש במשאב ישחרר אותו.
המתנה פעילה	busy waiting	מצב שבו תהליך מבצע בלולאה בדיקת זמינות של משאב כלשהו, עד שהמשאב מתפנה. המתנה פעילה היא בזבוז זמן המעבד, מאחר שתהליך שנמצא במצב זה מנצל את המעבד אך אינו מתקדם בפועל.
פסיקה	interrupt	אות אשר נשלח למעבד על ידי רכיב חומרה (כגון דיסק, שעון, בקר הרשת) או על ידי תהליך, וגורם למעבר שליטה מיידי למערכת ההפעלה (לצורך הטיפול בפסיקה).
משתני נעילה	lock variables	משתנה משותף שערכו קובע אם ניתן להיכנס לקטע קריטי מסוים או לא.
פעולה אטומית	atomic action	פעולה על משאב כלשהו המבטיחה כי שום גורם חיצוני לא יוכל לשנות את מצבו של המשאב מתחילת הפעולה ועד סיומה.
סמפור	semaphore	אובייקט מיוחד של מערכת הפעלה שאפשר לבצע עליו שתי פעולות אטומיות: UP (הגדלת ערך הסמפור) ו-DOWN (הקטנת ערך הסמפור). תהליך שמבצע פעולת DOWN על סמפור שערכו לא חיובי הולך "לישון". בכל פעם שמתבצעת פעולת UP על סמפור שישנים עליו תהליכים, תהליך אחד מבין אלה הישנים "מתעורר". הערה: המימוש הספציפי של מנגנון הסמפורים שונה ממערכת למערכת. יתרה מזו, קריאות המערכת המשמשות לביצוע פעולות על סמפורים אינן מגדירות בהכרח את הפעולות בדיוק בצורה המתוארת לעיל (לדוגמה, ב-UNIX לא מוגדרות פעולות על סמפור בודד, אלא רק פעולות על קבוצות של סמפורים).
סמפור בינארי	binary semaphore	סמפור שערכו יכול להיות 0 או 1 בלבד.

מנעול	mutex	מנעול הוא משתנה במרחב זיכרון משותף (למשל לכמה תהליכונים) המשמש לניהול גישה לקטע קריטי. בדומה לסמפור בינארי, למנעול שני מצבים: "נעול" ו"לא נעול". בניגוד לסמפור בינארי, השימוש במנעולים יעיל יותר במקרים שאפשר להשתמש בהם מכיוון שאין צורך להחליף בין רמת המשתמש לרמת הגרעין.
מבנה פיקוח	monitor	שפות תכנות אחדות מספקות למתכנת אפשרות להגדיר מבנה שמכיל אוסף של שדות ופונקציות, הנקרא מבנה פיקוח (מוניטור). המוניטור מתאפיין בתכונות האלה: א. לא ניתן לגשת לנתונים ששייכים למוניטור, אלא רק באמצעות פונקציות ששייכות לאותו המוניטור. ב. מובטח כי בכל נקודת זמן תהליך אחד לכל היותר יימצא בתוך המוניטור (כלומר, יבצע פונקציה כלשהי ששייכת למוניטור). הערה: מאחר שמוניטור הוא כלי ששפה עילית מספקת לנו, המהדר של השפה הוא זה שאחראי ליצירת קוד שיקיים את שתי התכונות לעיל. לכן המהדר חייב להיעזר במנגנוני מניעה הדדית שקיימים במערכת ההפעלה, כגון סמפורים.
תזמון ללא הפקעה	non-preemptive scheduling	שיטת תזמון שבה מערכת ההפעלה אינה יכולה, ביוזמתה, להפסיק ריצת תהליך כדי להקצות את המעבד לתהליך אחר, כל עוד התהליך המתבצע לא מסתיים או עובר למצב blocked.
תזמון המאפשר הפקעה	preemptive scheduling	שיטת תזמון שבה מערכת ההפעלה יכולה, ביוזמתה, להפסיק ריצת תהליך כדי להקצות את המעבד לתהליך אחר.
מדדי תזמון	goals of scheduling algorithms	שיטה לתזמון מערכות באמצעות אחד המדדים האלה לפחות: הגינות, פיזור עומסים, קצב סיום עבודות, זמן שהייה, ניצולת CPU, זמן תגובה, זמן תגובה מובטח לאירועים מסוימים.
שיטת התור	First Come First Served	שיטת תזמון שבה כל התהליכים המוכנים לריצה מסודרים בתור לפי זמן הגעתם. התהליך הראשון בתור מקבל את המעבד ומפנה אותו לתהליך הבא בסיום ריצתו או כתוצאה ממעבר למצב blocked. עם החזרה למצב ready, התהליך חוזר לסוף התור.
שיטת המשימה הקצרה ביותר	Shortest Job First	שיטת תזמון לתהליכי אצווה שבה זמני הריצה של תהליכים ידועים מראש. התהליכים המוכנים לריצה מסודרים בתור לפי משך זמן הריצה ונבחרים על ידי מתזמן לפי סדר הופעתם בתור.
שיטת זמן הסיום הקצר ביותר	Shortest Remaining Time	גרסה של שיטת המשימה הקצרה ביותר המאפשרת הפקעה של תהליכים. התהליכים המוכנים לריצה מסודרים בתור לפי משך זמן הריצה שנותר להם עד הסיום ונבחרים על ידי מתזמן לפי סדר הופעתם בתור.
שיטת ההחלפה המחזורית	Round Robin	שיטת תזמון שבה כל התהליכים המוכנים לריצה מסודרים בתור מעגלי, וכל תהליך מקבל את המעבד לפרק זמן שאינו עולה על חסם קבוע מסוים (quantum). התהליך הראשון מקבל את המעבד ומפנה אותו לתהליך הבא בתור כאשר הוא עובר למצב blocked או כאשר זמן השהייה שלו במעבד מגיע לערך של ה-quantum.
שיטת העדיפויות	Priority Scheduling	שיטת תזמון שבה כל התהליכים המוכנים לריצה מסודרים בתור לפי עדיפות. נוסחאות לחישוב עדיפות יכולות להיות שונות מאוד זו מזו.

האוניברסיטה הפתוחה
THE OPEN UNIVERSITY OF ISRAEL
الجامعة المفتوحة
Yair
Kurtzman

File #0006182 belongs to Yair Kurtzman- do not distribute

שיטת ריבוי התורים	Multiple Queues	המנגנון המשמש את מדיניות התזמון של מערכת ההפעלה מבוסס בדרך כלל על מספר תורים שלעתים קוראים להם multilevel feedback queues. כל אחד מהתורים יכול להתנהל בשיטת תזמון אחרת. המנגנון ממומש על ידי מערך של תורים, כאשר התור שנמצא במקום 0 מכיל את התהליכים שנמצאים בעדיפות הגבוהה ביותר לריצה, והתור האחרון מכיל את התהליכים שנמצאים בעדיפות הנמוכה ביותר. התורים מכילים אך ורק תהליכים המוכנים לריצה. המנגנון מגדיר חוקים למעבר תהליכים מתור לתור.
שיטת התהליך הקצר ביותר	Shortest Process First	שיטת תזמון לתהליכים אינטראקטיביים שחוזה את זמני הביצוע של התהליכים בין פעולות עוקבות של I/O ומנהלת תור לפי משך הזמן הזה. התהליכים המוכנים לריצה נבחרים על ידי מתזמן לפי סדר הופעתם בתור. הערה: השם של השיטה מבלבל במקצת. לביצוע תהליך אינטראקטיבי במעבד בין שתי פעולות עוקבות של I/O קוראים birst. לפיכך, כינוי מוצלח יותר של השיטה היה יכול להיות Shortest Birst First.
תזמון מובטח	Guaranteed Scheduling	שיטת תזמון שבה לכל משתמש או תהליך מובטחת מנה שווה של זמן חישוב ב-CPU.
שיטת ההגרלה	Lottery Scheduling	שיטת תזמון שבה כל תהליך מקבל כרטיסי הגרלה. כל כרטיס כזה משתתף בהגרלה המקנה למחזיק הכרטיס הזוכה את המשאב המוגרל (או חלק ממנו).
מנגנון	mechanism	אלגוריתם שמבוסס על פרמטרים אשר נקבעים מחוץ לאלגוריתם עצמו ומועברים אליו על ידי גורם חיצוני.
מדיניות	policy	קביעת פרמטרים לאלגוריתם שממומש במנגנון, ועל ידי כך – שליטה באופן פעולתו של האלגוריתם.

פרק 3

ניהול זיכרון

האוניברסיטה הפתוחה
THE OPEN UNIVERSITY OF ISRAEL
الجامعة المفتوحة
Yair
Kurtzman

File #0006182 belongs to Yair Kurtzman- do not distribute

ניהול זיכרון ישיר
1.3.2 זיכרון (סקירת חומרה) [1.3.2 בספר הקורס]
3.1 ניהול זיכרון ישיר

3.0 זיכרון (סקירת חומרה) [1.3.2 בספר הקורס]

לכל הבעיות הקשורות בזיכרון, או ליתר דיוק במחסור בזיכרון, מתבקש פתרון טכני פשוט. ביום שבו נוכל להרכיב בכל מחשב זיכרון ראשי אשר יספיק לכל צורכי התוכניות המורצות בו, חלק גדול מהחומר שבפרק שלפנינו יהפוך למיותר. לכאורה, יום זה הולך וקרב: מחיריהם של רכיבי הזיכרון פוחתים במהירות – דבר המאפשר בנייה של מחשבים בעלי זיכרונות גדולים יותר ויותר. אלא שבמציאות, התמונה מורכבת מעט יותר. עם הגדלת זיכרון המחשבים, גם הדרישות לזיכרון ראשי הולכות וגדלות. לפיכך, בעיית ניהול הזיכרון בתנאי מחסור עומדת בעינה.

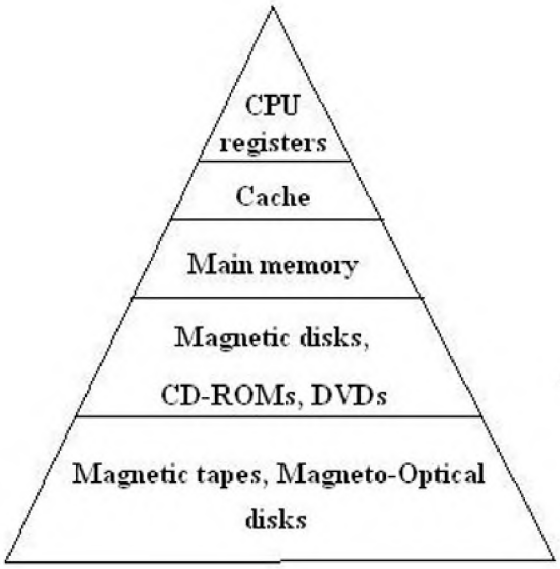
זיכרונות מתאפיינים במחירם, במהירות הגישה שלהם ובקיבולת שלהם. תנאי חליפין (tradeoffs) מתקיימים בין כל שתי אפשרויות. מחירם של זיכרונות בעלי מהירות גישה גבוהה הוא גבוה. זיכרונות בעלי קיבולת גבוהה מתאפיינים במהירות גישה נמוכה, אך יתרונם במחירם הנמוך. כדי לפשר בין המחיר לדרישה להשגת ביצועים נאותים, עושים שימוש בהיררכיה של זיכרונות.

עיינו באיור שלהלן. שלושת הסוגים הראשונים של הזיכרון, הנמצאים בראש הפירמידה, הם זיכרונות שתוכנם אינו נשמר עם הפסקת זרם החשמל אליהם. מביניהם הזיכרון היקר, המהיר ביותר ובעל הקיבולת הקטנה ביותר הוא הרגיסטרים של CPU. הזיכרון הראשי הוא הזיכרון בעל הנפח הגדול ביותר משלושת הסוגים שהזכרנו, אך יחד עם זאת, מהירות הגישה של ה-CPU אליו היא הנמוכה ביותר. ביניהם נמצא זיכרון מטמון שמתאפיין במהירות גישה גדולה מזו של הזיכרון הראשי אך קטנה ממהירות הגישה אל הרגיסטרים של ה-CPU. המחיר והקיבולת של זיכרון המטמון גם הם נמצאים בטווח שבין הזיכרון הראשי לרגיסטרים.[10]

[10] למעשה, התמונה מורכבת בהרבה בהשוואה להצגה המופשטת שהצענו כאן. קיימים גם סוגים שונים ורמות רבות של זיכרון מטמון, אבל בשלב זה אנחנו נתייחס אליו כאל מדרגה בהיררכיה של הזיכרונות, הנמצאת בין הרגיסטרים של ה-CPU לזיכרון הראשי.

האוניברסיטה הפתוחה
THE OPEN UNIVERSITY OF ISRAEL
الجامعة المفتوحة
Yair
Kurtzman

File #0006182 belongs to Yair Kurtzman- do not distribute



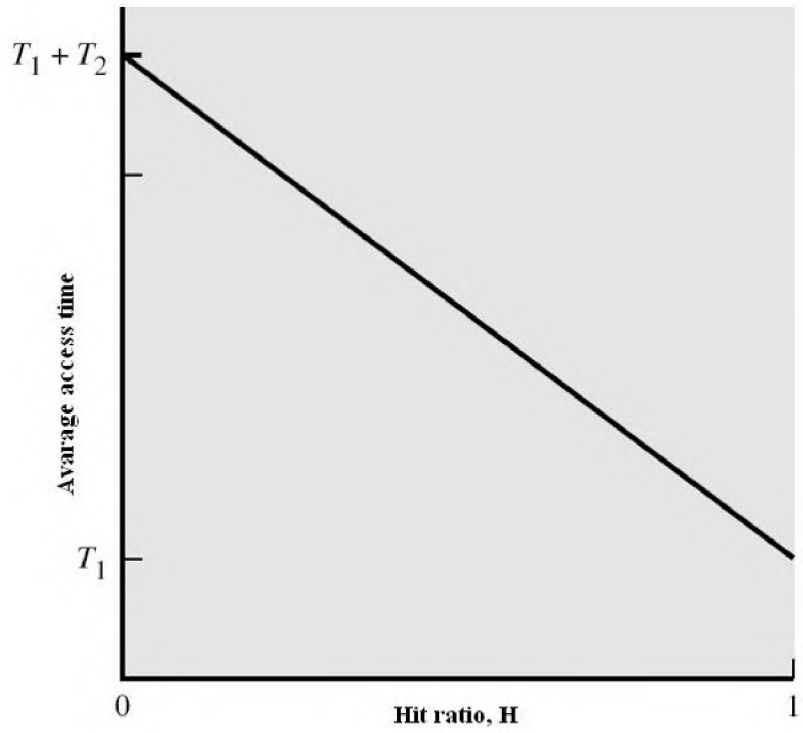
איור 1

במהלך הדיון במכונת מצבים מורחבת, ראינו שתהליכים יכולים להיות במצב מוכן או במצב חסום בדיסק, מה שמאפשר להגביר את רמת המקביליות של התהליכים במערכת באמצעות תזמון תהליכים לטווח רחוק. זיכרון זה מכונה לעתים ***זיכרון משני***. לבסוף, ברמה האחרונה בהיררכיה נמצאים התקני זיכרון בעלי נפחים גדולים מאוד, זולים ואיטיים שמשמשים לאחסון מידע לצורך גיבוי. הזיכרון שנמצא ברמות ההיררכיה הנמוכות שומר תמיד את תוכנו, גם אם זרם החשמל מופסק.

בטרם נמשיך לדון בנושא ניהול הזיכרון, ננסה להבין מדוע באופן עקרוני ניתן באמצעות היררכיה של זיכרונות להשיג תוצאות סבירות מבחינת מהירות הגישה, הנפח והמחיר. לשם כך נעיין בשתי מדרגות סמוכות של היררכיה. נניח כי זמן הגישה לרמה העליונה הוא T1 וכי זמן הגישה לרמה נמוכה יותר הוא T2. נסמן ב-H את הסיכוי להימצאותו של המידע הנדרש ברמה עליונה יותר. לסיכוי זה קוראים גם יחס הצלחה (hit ratio), ולערך 1-H קוראים יחס ההחמצות (miss ratio). אם המידע איננו מצוי ברמה עליונה יותר, נזדקק לזמן T1 כדי לבדוק וכאשר המידע חסר, גם לזמן T2 כדי לקבל את המידע מהרמה הנמוכה יותר. לכן זמן הגישה המשוקלל יהיה:

$$H * T1 + (1-H) * (T1 + T2) = H*T1 + T1 - H*T1 + (1-H) * T2 =$$
$$= T1 + (1-H)*T2$$

כפי שניתן לראות בבירור מן הנוסחה, זמן הגישה המשוקלל במקרה של שתי רמות בהיררכיה תלוי ליניארית ביחס ההצלחה H, וככל שיחס ההצלחה גדל, כך הזמן המשוקלל יהיה קרוב יותר לזמן הגישה של הרמה הראשונה. ניתן לראות זאת באיור שלהלן:



איור 2

הערך של יחס ההצלחה הוא פונקציה של כמה גורמים, כגון גודל הזיכרון ברמה עליונה יותר וארגון המידע. לא נפרט עוד על כך. הדבר החשוב הוא שניתן להחיל את העיקרון הפועל במקרה הפשוט של שתי רמות היררכיה גם על מספר גדול יותר של רמות.

3.1 ניהול זיכרון ישיר (חומר רשות)

בסעיף זה נבחן את שיטות ניהול הזיכרון הישיר. בסעיפים 3.2 ו-3.3 נכיר שיטות ***החלפה*** ו***דפדוף*** שהן שיטות ניהול זיכרון אשר מאחוריהן עומד מודל לניהול זיכרון הנקרא ***מרחב זיכרון***.

השיטה הפשוטה ביותר לניהול זיכרון היא חלוקה של הזיכרון הראשי בין מערכת ההפעלה ותוכנית משתמש יחידה. תוכנית המשתמש חייבת להסתיים סיום וולונטרי או סיום כפוי כדי שיהיה אפשר להביא לזיכרון תוכנית אחרת.

קיימות דרכים אחדות לארגון של הזיכרון הראשי במודל הזיכרון הפשוט הזה. איור 3-1 בספר הלימוד מדגים אותן:

- בחלק א של האיור, מערכת ההפעלה תופסת כתובות נמוכות של מרחב הזיכרון הראשי בעת שתוכנית המשתמש יכולה להתמקם בכתובות העליונות.

האוניברסיטה הפתוחה THE OPEN UNIVERSITY OF ISRAEL الجامعة المفتوحة
Yair Kurtzman

File #0006182 belongs to Yair Kurtzman- do not distribute

- בחלק ב של האיור, מערכת ההפעלה תופסת כתובות עליונות של מרחב הזיכרון הראשי בעת שתוכנית המשתמש שוכנת בכתובות הנותרות.
- בחלק ג של האיור, מערכת ההפעלה תופסת כתובות נמוכות של מרחב הזיכרון הראשי, בכתובות העליונות שוכנת שכבה של מתאמי התקנים האחראית על פעולות קלט/פלט בסיסיות, ותוכנית המשתמש יכולה להתמקם במרחב הנותר. שיטה זו הייתה בשימוש במערכת ההפעלה DOS.

שיטת ניהול הזיכרון הישיר מאפשרת להריץ תוכניות אחדות בו-זמנית. ודאו כי אתם מבינים את ההסבר לאיור 3.2 בספר הלימוד.

3.2 מרחב זיכרון
3.2.1 ניידות תוכניות במרחב זיכרון
3.2.2 החלפה בזיכרון
3.2.3 ניהול זיכרון פנוי

3.2 מרחב זיכרון

3.2.1 ניידות תוכניות במרחב זיכרון

כאשר יש בזיכרון יותר מתוכנית אחת, צריך לשים לב לשתי בעיות מרכזיות:

- בעיה אחת היא כיצד להבטיח שתהליכים השוכנים במרחב כתובות משותף לא יחבלו זה בזה, ובפרט, כיצד להגן על תהליכי מערכת הפעלה החולקים את אותו המרחב.
- הבעיה השנייה קשורה לנושא ה***ניידות***. תוכניות יכולות להיות ממוקמות בכל פעם באזורים פיזיים שונים בזיכרון. בתנאים אלו אין ביכולתם של המהדר ושל תוכנית הקישור, linker, לצפות מראש מה יהיו הכתובת הפיזיות של המודולים השונים של התוכנית. לכן, בעת יצירת תוכניות הרצה (executable), ההנחה היא שמרחב הכתובות של התוכנית מתחיל תמיד בכתובת 0. כך כל שאר הכתובות מחושבות באופן יחסי לנקודת האפס – נקודת ההתחלה. לפיכך יש לדאוג בכל פעם מחדש לתרגום המענים המופיעים בתוכנית המקורית לכתובות הפיזיות של המחיצה שהוקצתה (באופן זמני) לתהליך. הבעיה המרכזית היא שתרגום זה צריך להתבצע בצורה מקוונת (on-fly) בזמן ביצוע התוכנית בלא שיפגע בביצועים.

למרחב הכתובות של תהליך שהכתובות שבו אינן תלויות בכתובות הפיזיות קוראים ***מרחב כתובות וירטואלי*** (***virtual address space***), ואילו לכתובות שבמרחב זה קוראים ***כתובות וירטואליות*** (***virtual addresses***) או ***כתובות לוגיות*** (***logical addresses***).

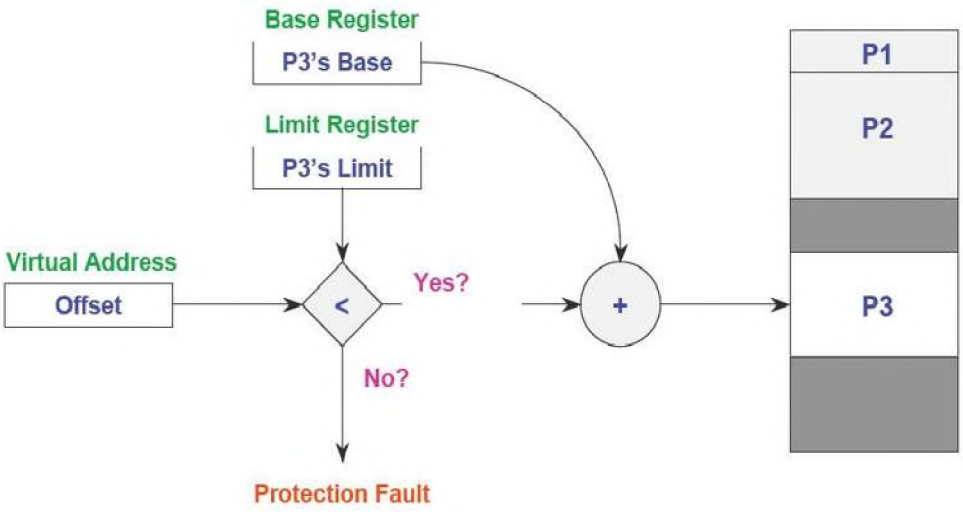
כדי שהתרגום ממרחב הכתובות הווירטואלי של התהליך למרחב הכתובות הפיזי של המחיצה שבה שוכן התהליך לא יפגע בביצועים, תרגום זה חייב להתבצע בחומרה. הפתרון שנציג כעת נותן מענה הן לסוגיית הניידות והן לסוגיית ההגנה.

בחומרה העוסקת בתרגום הכתובות יש שני אוגרים מיוחדים: base ו-limit. באוגר base ייטען המען של המחיצה המוקצית לתהליך, ובאוגר limit ייטען הגודל של המחיצה. שני אוגרים אלו מאפשרים לתרגם את מרחב הכתובות הווירטואלי של תוכנית למרחב הכתובות הפיזי של המחיצה בזיכרון שהוקצתה לריצת התהליך.

חישוב כתובות פיזיות יתבצע בשני שלבים:

- בכל התייחסות לכתובת וירטואלית תתבצע השוואתה אל הערך באוגר limit. אם הכתובת הווירטואלית מתייחסת לאזור שחורג מגבולות המחיצה, הדבר מעיד על ניסיון לגישה בלתי חוקית. במקרה של גישה בלתי חוקית, המעבד יגרום לפסיקת protection fault.
- במקרה של התייחסות לכתובת חוקית, יתווסף לכתובת הווירטואלית הערך של האוגר base, והתוצאה המתקבלת תהיה הכתובת הפיזית.

האיור שלהלן הוא תיאור סכמטי של השלבים האלה:



איור 3

אף שתרגום המענים מתבצע בחומרה, פעולת החיבור שצריך לבצע בכל פנייה לכתובת היא פעולה איטית ביחס לפעולת ההשוואה. לכן הרכיב האחראי לתרגום מענים וירטואליים המכונה ***יחידת תרגום מענים*** (***MMU – Memory Management Unit***) מבצע תחילה את ההשוואה.

האוניברסיטה הפתוחה THE OPEN UNIVERSITY OF ISRAEL الجامعة المفتوحة
Yair Kurtzman

File #0006182 belongs to Yair Kurtzman- do not distribute

3.2.2 החלפה בזיכרון

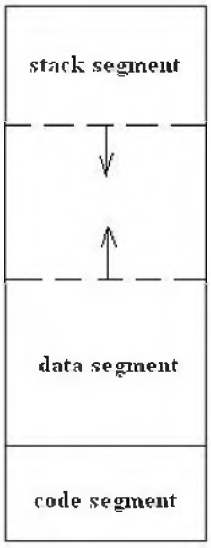
בדרך כלל, סך כל דרישות הזיכרון הראשי של כל התהליכים שרצים במערכת גדול בהרבה מנפח הזיכרון שקיים במערכת. הדרך הפשוטה ביותר להתמודד עם הבעיה היא להחזיק בזיכרון ראשי את התהליך רק כאשר הוא רץ, לשמור אותו בדיסק כאשר הוא עובר למצב חסום ולשחזר אותו כאשר הוא מתוזמן שוב לריצה.

בסעיף זה מתוארת השיטה של חלוקת הזיכרון למחיצות ניידות. מספר המחיצות אינו קבוע מראש, וכך גם גודלן. כאשר תהליך חדש מגיע למערכת, מוקצה לו שטח זיכרון שמסוגל להכיל אותו (ראו איור 4-3 בספר הלימוד). כפי שניתן לראות באיור, תהליך ההקצאה של שטחי זיכרון בגדלים משתנים מתחיל בניצול מרבי של הזיכרון, אך מהר מאוד (3-4d) גורם להופעתם של שטחי זיכרון פנויים (או של חורים בזיכרון).

במרוצת הזמן, כמות החורים בזיכרון תלך ותגדל, ושטחי הזיכרון הפנוי ילכו ויקטנו עד כדי כך ששום תהליך לא יהיה מסוגל עוד להשתמש בהם. לתופעה זו קוראים ***ריסוק חיצוני*** (***external fragmentation***).

כדי להתגבר על בעיית הריסוק החיצוני, אפשר לבצע העתקה של שטחי זיכרון תפוסים במטרה לצופף אותם וכך להביא לשטח זיכרון פנוי רציף. אלא שפעולת ***הציפוף*** (***compaction***) יקרה מאוד בזמן ולכן אינה יכולה להתבצע בכל פעם שנוצר חור בזיכרון. בהמשך נציג אלגוריתמים לניהול שטחי זיכרון שמנסים להקטין את התופעה של ריסוק חיצוני.

במהלך חייהם, תהליכים יכולים לשנות את הדרישות שלהם לזיכרון. למשל, הסידור הטיפוסי של מודל זיכרון של שפת C נראה כך:



איור 4

הסגמנט של המחסנית מכיל משתנים לוקאליים של פונקציות וכתובות חזרה, והוא עשוי להתרחב בעקבות קריאה לפונקציה מקוונת או להתכווץ מחדש בעקבות החזרה מהפונקציה. סגמנט המיועד ל-data מכיל משתנים גלובליים, משתנים סטאטיים ומשתנים שהוקצו בהקצאה דינאמית, ולכן הוא יכול להתרחב בעקבות הקצאה דינאמית, ובמקרים אחרים – להצטמצם לאחר שחרור משתנים דינאמיים.

בשיטת המחיצות הניידות, אם מתגלה כי דרישות הזיכרון של תהליך גדלו והשטח המיועד להתרחבות הצטמצם עד אפס מקום, אפשר יהיה להקצות לתהליך שטח פנוי גדול יותר או להוציאו מן הזיכרון עד אשר יימצא שטח פנוי מספיק גדול.

בטרם נעבור לאלגוריתמים לניהול שטחים פנויים, נאמר מילים אחדות על הגנה וניידות. מנגנון תרגום המענים שמשתמש באוגרים base ו-limit משמש לניידות של תהליכים וכך מתקיימת הגנה מסוימת של מערכת ההפעלה מפני תהליכים אחרים וגם של תהליכים מפני תהליכים אחרים. עם זאת, במודל הזיכרון של שפת C שתיארנו קודם, ניתן לראות כי סגמנט מתרחב אחד עלול לדרוך על סגמנט אחר. במילים אחרות, נדרשת גם הגנה על תהליך מפני עצמו! ההגנה על הסגמנטים היא באחריותו של המעבד הראשי, אשר שומר באוגרים מיוחדים את כתובות ההתחלה של הסגמנטים ואת אורכיהם. לפני שינוי הגבולות של סגמנטים, המעבד בודק אם תיווצר חפיפה. אם תתרחש ההתרחבות, אזי הוא ייצר פסיקת חומרה שבעקבותיה מערכת ההפעלה יכולה להקצות לתהליך שטח זיכרון גדול יותר או להעביר את הטיפול לתוכנית המשתמש (למשל, בהודעה על חוסר אפשרות להקצאה נוספת של זיכרון).

3.2.3 ניהול זיכרון פנוי

נעבור כעת לדיון באלגוריתמים להקצאת שטחי זיכרון בשיטת המחיצות הניידות. בשני הפרקים הבאים נעסוק בשיטת הקצאת שטחי זיכרון פנויים באמצעות מפת סיביות ובשיטת הקצאת שטחים פנויים המנוהלים ברשימה מקושרת.

הקצאת שטחי זיכרון פנויים באמצעות מפת סיביות (bits-map)

בשיטה להקצאת שטחי זיכרון פנויים באמצעות מפת סיביות מחלקים את הזיכרון לקטעים בגודל קבוע (וקטן יחסית). הקצאת הקטעים מנוהלת על ידי מפת סיביות וכל סיבית בה משקפת את מצב ההקצאה של הקטע (למשל, 1 – קטע תפוס, 0 – קטע פנוי), ראו איור 3-6 (a) ו-(b). הקצאה של הזיכרון תתבצע על ידי חיפוש של סדרת 0-ים רציפה במפת הסיביות כך שהקטעים המתאימים יספיקו להכיל את התהליך.

גרעיניות החלוקה, כלומר הגודל של קטעי ההקצאה, היא נושא חשוב. כרגיל, השיקולים לבחירה של גודל הקטע יוצרים קונפליקט. בחירת קטעים גדולים תביא לבזבוז גדול יותר בקטע האחרון, מכיוון שגודלם של תהליכים לא יהיה תמיד כפולה של גודל קטע ההקצאה. מצד אחר, ככל שקטעי ההקצאה יהיו גדולים יותר, כך מפת הסיביות תהיה קטנה יותר (וכך ייחסך מקום בזיכרון שדרוש לשמירתו). בחירת קטעי הקצאה קטנים תקטין את הבזבוז בקטע האחרון של התהליך, אך תגרום לכך שמפת הסיביות תהיה גדולה יותר.

האוניברסיטה הפתוחה
THE OPEN UNIVERSITY OF ISRAEL
الجامعة المفتوحة
Yair
Kurtzman

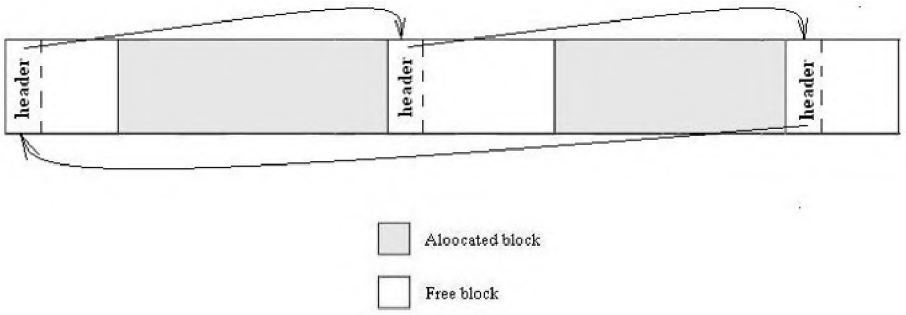
File #0006182 belongs to Yair Kurtzman- do not distribute

ניהול הקצאות של שטחי זיכרון באמצעות מפת סיביות הוא שיטה פשוטה למימוש. התקורה הניהולית (מפת הסיביות) היא קבועה לאחר שנקבע גודל של קטע הקצאה ובהינתן גודל הזיכרון הפיזי. החיסרון של השיטה הוא שהחיפוש במפת הסיביות יכול להיות איטי למדי.

הקצאת שטחי זיכרון פנויים באמצעות רשימות מקושרות (linked lists)

באמצעות הקצאת שטחים נבונה, אפשר להקטין את תופעת הריסוק החיצוני. בטרם נדון בשיטות הקצאה נוספות, נבחן כיצד אפשר לנהל את השטחים הפנויים.

- אפשרות אחת לניהול שטחים פנויים היא להחזיק רשימה מקושרת ממוינת של שטחים פנויים ושל שטחים תפוסים, כאשר המיון הוא לפי הכתובת ההתחלתית של השטחים – ראו איור 3-6 (c) שמתאים למצב ההקצאות המוצג באיור 3-6 (a). כפי שניתן לראות מאיור 3-7, סיום תהליכים מצריך לעתים עדכון של צומת קודם ברשומה, וכדי לייעל את העדכון כדאי להחזיק ברשימה ממוינת דו-מקושרת. במקרה זה, חיפוש שטחים פנויים מצריך מעבר הן על הצמתים של שטחים פנויים והן על הצמתים המתאימים לשטחים תפוסים, וזה דורש זמן. עם זאת, קל לבצע עדכון של רשימה משורשרת לאחר פינוי שטחי זיכרון על ידי תהליכים שהסתיימו.
- אפשרות אחרת היא לנהל רשימות דו-משורשרות נפרדות: אחת לשטחים פנויים ואחרת לשטחים תפוסים. למעשה, ניתן לחסוך את המקום שתופסת רשימה דו-משורשרת של השטחים הפנויים, כאשר ההצבעה על השטח הפנוי הבא מתבצעת מתוך השטח עצמו. הדבר מודגם באיור שלהלן. באמצעות הרשימות הנפרדות, מציאת שטחים פנויים דורשת פחות זמן, אך שחרור זיכרון על ידי תהליכים שהסתיימו דורש עדכון מתואם בשתי הרשימות (זמן כפול).



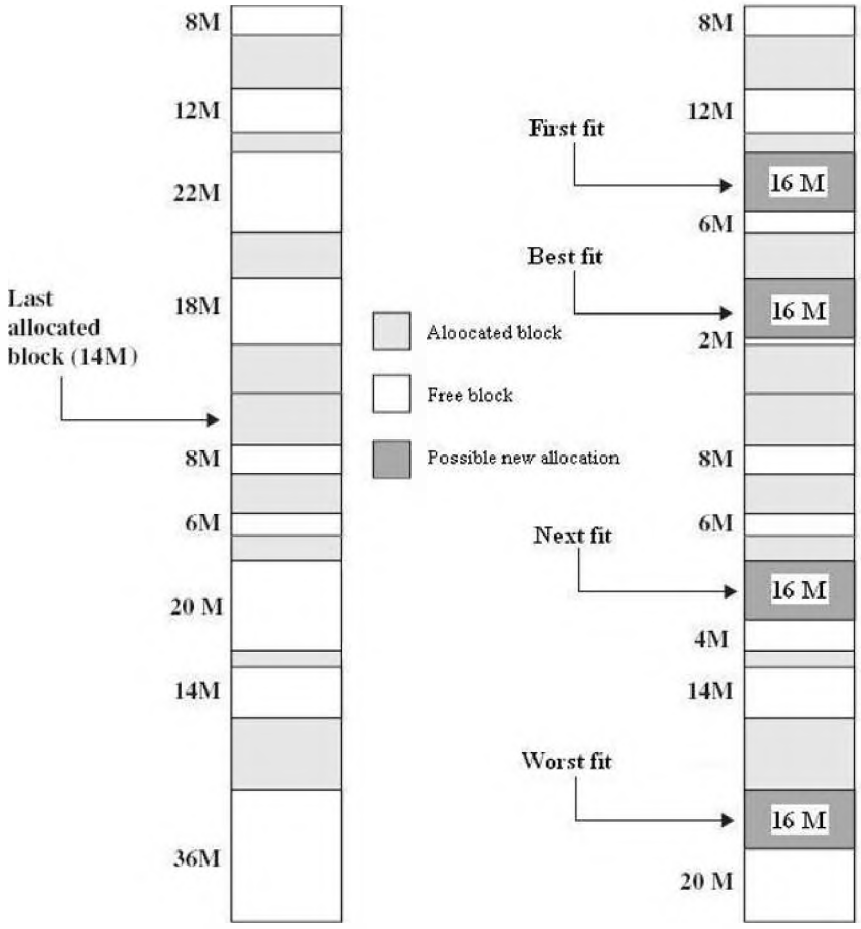
איור 5

הנה שיטות אחדות להקצאה של מחיצות בעבור תהליכים:

- first-fit – זהו האלגוריתם הפשוט ביותר. כדי למצוא שטח זיכרון פנוי, מבצעים סריקה מתחילת רשימת השטחים הפנויים עד אשר מוצאים בה שטח זיכרון גדול דיו היכול להכיל את התהליך. לאחר מכן מתבצע פיצול של השטח שהאלגוריתם מצא – השטח שנדרש בעבור התהליך והעודף: שטח המחיצה הנותר הופך להיות בעצמו שטח פנוי למחיצה חדשה. החיפוש באמצעות אלגוריתם זה הוא מהיר מכיוון שהסריקה נעצרת מיד עם מציאת שטח פנוי.

- next-fit – זהו שיפור של האלגוריתם first-fit. כאן הסריקה איננה מתחילה בכל פעם מתחילת רשימת השטחים הפנויים, אלא נעשית סריקה מעגלית אשר מתחילה ונגמרת במקום שבו נמצא שטח פנוי בפעם הקודמת (או קודם לכן, אם נמצאה מחיצה מתאימה).
- best-fit – אלגוריתם זה מבצע סריקה של כל הרשימה ומוצא את השטח הפנוי הקטן ביותר שמסוגל להכיל את התהליך.
- worst-fit – אלגוריתם זה מבצע סקירה של כל הרשימה ומוצא את השטח הפנוי הגדול ביותר המסוגל להכיל את התהליך.

באיור שלהלן ניתן לראות כיצד תתבצע הקצאה של זיכרון בכל אחת מן השיטות, לאחר שקודם לכן הוקצה שטח בגודל 14 Mb וכעת רוצים להקצות שטח בגודל 16 Mb. ההנחה היא שהרשימה היא דו-משורשרת והיא ממוינת לפי כתובות ההתחלה של שטחי הזיכרון.



איור 6

קשה לקבוע איזו מן השיטות הללו טובה יותר, מכיוון שהדבר תלוי בסידור התהליכים בזיכרון ובסדר ההחלפות המתבצעות בזיכרון. עם זאת, ניתן לטעון כמה טענות כלליות:

האוניברסיטה הפתוחה
THE OPEN UNIVERSITY OF ISRAEL
الجامعة المفتوحة
Yair Kurtzman

File #0006182 belongs to Yair Kurtzman- do not distribute

- אם מנהלים את השטחים באמצעות שתי רשימות נפרדות (אחת לשטחים פנויים והאחרת לשטחים תפוסים), אפשר למיין את השטחים הפנויים לפי גודל (ולא לפי כתובת התחלה, כמקודם) וכך גם האלגוריתמים best-fit ו-worst-fit יהיו מהירים באותה מידה כמו האלגוריתמים first-fit או next-fit.
- האלגוריתם first-fit הוא ככל הנראה הבחירה הטובה מבין השלושה בגלל פשטותו, מהירותו ותבניות הריסוק שהוא יוצר. תבנית הריסוק של first-fit היא של שטחי זיכרון לא פנויים (שהוקצו לתהליכים), הנמצאים בתחילת הזיכרון, ושטח רציף פנוי גדול בסוף הרשימה.
- האלגוריתם next-fit מהיר באותה מידה כמו first-fit, אך תבנית הריסוק שלו אחידה יותר. הוא נוטה לפצל את השטח הפנוי בסוף הזיכרון, ולכן ייתכן שבסופו של דבר אלגוריתם זה ידרוש את הביצוע של פעולת הציפוף (compaction) יותר פעמים.
- למרות שמו, דווקא האלגוריתם best-fit הוא בדרך כלל הגרוע ביותר מבין שלושת האלגוריתמים. הוא נוטה ליצור הרבה שטחים פנויים שקשה לנצלם עקב גודלם הקטן. לכן השימוש בו ידרוש לרוב את הציפוף התכוף מבין שלושת האלגוריתמים.
- האלגוריתם worst-fit יוצר גם הוא תבנית ריסוק אחידה, מכיוון שבכל פעם הוא מנסה לפצל את השטחים הרציפים הגדולים ביותר.

אפשר לשכלל את הרעיון של רשימת שטחים פנויים אחת ולהחזיק כמה רשימות של שטחים פנויים, כאשר כל רשימה מכילה טווח מסוים של גדלים. מציאת שטח פנוי במקרה כזה הופכת להיות מהירה מאוד, ולכן מכנים את השיטה quick-fit. עם זאת, בעת שחרור שטחי זיכרון, יש צורך לבדוק אם ניתן לאחד את השטח שהתפנה עם שטח פנוי אחר. לשם כך צריך לעבור במקרה הגרוע על כל הרשימות המשורשרות. מעבר זה דורש זמן ניכר.

3.3 זיכרון מדומה
3.3.1 דפדוף (paging)
3.3.2 טבלת הדפים (page table)
3.3.3 כיצד להגביר מהירות פענוח כתובות
3.3.4 טבלת דפים למרחבי זיכרון גדולים

3.3 זיכרון מדומה

זיכרון וירטואלי הוא שמה של שיטת ניהול זיכרון המבוססת על העיקרון של אחסון חלקי של תהליך בזיכרון, כאשר רק החלק שחיוני לביצוע ברגע נתון חייב להימצא בזיכרון הראשי, וחלקים אחרים של התהליך יכולים להיות מאוחסנים בדיסק. שיטה זו מאפשרת הרצת תהליכים שגודלם עולה על גודל הזיכרון הפיזי במערכת.

אנחנו נלמד שלוש שיטות למימוש של זיכרון וירטואלי. בסעיף 3.3 נסקור בהרחבה את שיטת הדפדוף, שהיא השיטה הנפוצה ביותר למימוש של זיכרון וירטואלי, ובסעיף 3.7 ניגע בשיטת הסגמנטציה ובשיטה המשלבת את שיטת הדפדוף עם סגמנטציה.

3.3.1 דפדוף (paging)

כאשר דנו במדריך למידה זה בשיטות ניהול זיכרון באמצעות מחיצות, הכרנו את מרחב הזיכרון המדומה ואת מנגנון תרגום המענים המאפשר את מימושו. גם במקרה של ניהול זיכרון באמצעות דפדוף, לכל תהליך יש מרחב זיכרון וירטואלי. התיאור הסכמטי אשר מובא באיור 3-8 בספר הלימוד מדגים כיצד מתבצע תרגום של כתובת וירטואלית לכתובת פיזית. תיאור זה מתאים לכל שיטות ניהול הזיכרון המאפשרות את הימצאותם של תהליכים אחדים בו-זמנית בזיכרון הראשי. להבדיל משיטות ניהול זיכרון באמצעות מחיצות, מרחב הזיכרון הווירטואלי של תהליכים במערכות המנהלות זיכרון באמצעות דפדוף איננו חייב להיטען בצורה רציפה לזיכרון הראשי. כלומר, חלקים ממנו יכולים להימצא בזיכרון הראשי במקומות שונים בעת ששאר החלקים נמצאים בזיכרון משני.

מרחב הכתובות של התהליך מחולק לדפים בגודל קבוע. גם הזיכרון הראשי מחולק למסגרות בגודל הזהה לגודל הדפים של התהליכים (זהו גודל קבוע לכל התהליכים). דפים יכולים להיות ממופים למסגרות פיזיות בזיכרון הראשי או להימצא בזיכרון משני. התבוננו באיור 3-9 בספר הלימוד: דפים שאין מהם חץ המצביע על הימצאותו של דף במסגרת זיכרון, נמצאים בזיכרון משני. המיפוי של הדפים השמור במבנה נתונים נקרא ***טבלת דפים*** (*page table*). שורותיה של טבלת דפים מכילות מידע על הדפים, כגון אם הדף ממופה למסגרת זיכרון, לאיזו מסגרת ממופה הדף וכן הלאה.

אם מתבצעת פנייה לכתובת זיכרון וירטואלית, שהדף המכיל אותה נמצא בזיכרון משני, פנייה זו תגרום לפסיקת מעבד המכונה פסיקת דף (page fault interrupt). פסיקה זו גורמת למערכת ההפעלה לתזמן בקשה להביא את הדף החסר לזיכרון הראשי. עם הגעת הדף, מערכת ההפעלה תעדכן את טבלת הדפים (כי צריך לסמן בטבלה שהדף שזה עתה הגיע נמצא בזיכרון), והתוכנית תמשיך לבצע את ההוראה האחרונה שניתנה לפני פסיקת הדף.

עיינו באיור 3-10 בספר הלימוד. ניתן לראות בו כיצד מתבצע התרגום של כתובת מדומה נתונה לכתובת פיזית בעזרת טבלת דפים שכל שורה בה מכילה:

א. סיבית present/absent לציון הימצאותו של הדף המתאים בזיכרון ראשי.
ב. מספר מסגרת פיזית שאליה ממופה דף מתאים אם הוא נמצא בזיכרון ראשי.

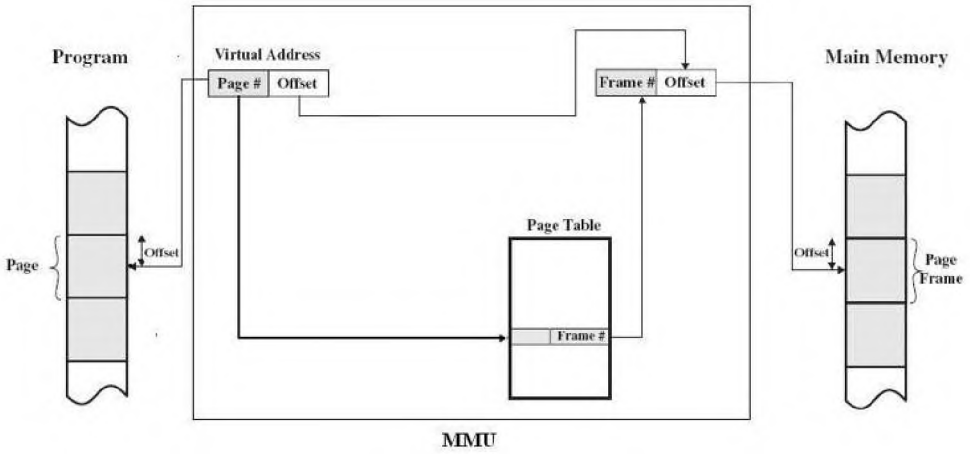
ודאו כי אתם מבינים את הדוגמה המספרית המתייחסת לאיור 3-10 בספר הלימוד.

3.3.2 טבלת הדפים (page table)

באופן כללי, תרגום כתובות וירטואליות לכתובות פיזיות נראה כך:

האוניברסיטה הפתוחה THE OPEN UNIVERSITY OF ISRAEL الجامعة المفتوحة
Yair Kurtzman

File #0006182 belongs to Yair Kurtzman- do not distribute



איור 7

כתובת וירטואלית מחולקת למספר הדף ולהיסט בתוך הדף. כתובת פיזית מורכבת ממספר המסגרת בזיכרון ומערכו של ההיסט בתוך המסגרת עצמה. מספר הדף משמש כאינדקס של שורה בטבלת הדפים. מכיוון שתרגום המענים מתבצע בכל פנייה לזיכרון, התרשים שבאיור מניח שטבלת דפים נמצאת בחומרה במנגנון MMU. בעבר הדבר היה אפשרי, אך בימינו טבלאות הזיכרון הפכו להיות גדולות כל כך שלא יעלה על הדעת להחזיקן ב-MMU.

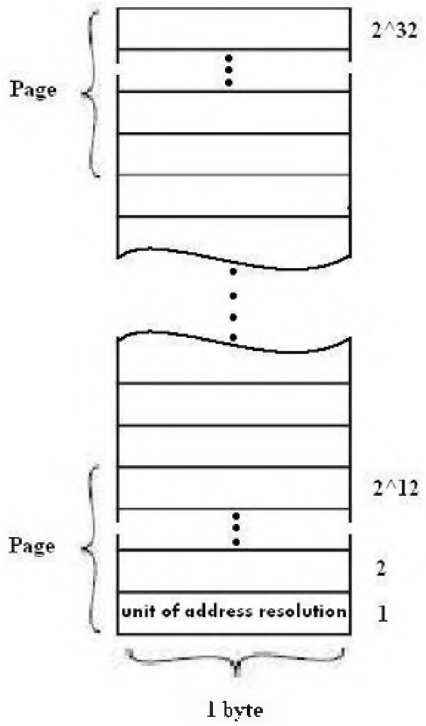
שאלה 1

נתון כי:

- כתובת וירטואלית כוללת 32 bits
- יחידות המען הקטנות ביותר הן בגודל 1 byte
- גודל כל דף הוא 4 kb
- גודל שורה של טבלת דפים הוא 4 bytes

מהו גודל טבלת הדפים?

באיור שלהלן נראה כיצד שלושת הנתונים הראשונים של הבעיה קשורים זה לזה:



איור 8

כתובת בת 32 סיביות יכולה לפנות ל-2^{32} מילות זיכרון במרחב וירטואלי של תהליך. אם גודל של מילת זיכרון הוא בית אחד, אזי גודל המרחב הווירטואלי הוא 2^{32}*1byte=4Gb. מכיוון שגודל הדף הוא 4 kb, אזי כמות הדפים שאליהם מתחלק המרחב הווירטואלי היא:

$4Gb/4kb = 2^{32}bytes / 2^{12}bytes = 2^{20}$ pages

זהו גם מספר השורות של טבלת דפים של תהליך. מכיוון שגודל שורה בטבלת הדפים הוא 4bytes, אזי הגודל של טבלת הדפים כולה יהיה: 2^{20} *4 bytes = 2^{22} bytes = 4 Mb. מכאן שלא ניתן להחזיק את טבלת הדפים בדף זיכרון בגודל 4 kb.

כאן נרצה להוסיף שתי הערות:

- רוב הארכיטקטורות של מעבדים בימינו עובדות עם יחידות של 1 byte, שהן היחידות הקטנות ביותר שהן בעלות מען. המונח "מילת זיכרון" (word) מתייחס ליחידת data "טבעית" שהמעבד מבצע בה פעולות. במעבדי 32 bit זו יחידה של 32 סיביות ובמעבדי 64 bit זו יחידה של 64 סיביות.
- ההערה השנייה מתייחסת למיקומה של טבלת הדפים. הגודל של טבלת הדפים מחייב להחזיק אותה ברמות נמוכות יחסית של היררכיית הזיכרון (כלומר, לא ברגיסטרים אלא בזיכרון הראשי ובזיכרון המשני). מכיוון שביצוע של הוראות מצריך לפחות תרגום אחד של כתובת וירטואלית (אם הוראת מכונה כוללת גם פרמטרים, אז מספר התרגומים יכול להיות גבוה יותר), כדי להשיג ביצועים נאותים עושים שימוש נרחב בטכניקות ה-caching. בסעיף 3.3.3 נראה סוג של זיכרון מטמון

האוניברסיטה הפתוחה
THE OPEN UNIVERSITY OF ISRAEL
الجامعة المفتوحة
Yair
Kurtzman

File #0006182 belongs to Yair Kurtzman- do not distribute

המאפשר לאחסן בו חלק קטן מהמיפויים של טבלת דפים של תהליך. מכיוון שטבען של התוכניות שאנחנו כותבים הוא בחלקו הגדול סדרתי, מתקיים ***עקרון הלוקאליות***, ולכן קיימת סבירות גבוהה שההוראה אשר המעבד צריך לבצע כבר נמצאת באותו דף שבו הייתה ההוראה הקודמת. לכן תרגום המענים יכול להתבצע לפי המידע המצוי בזיכרון המטמון.

המבנה של שורה בטבלת דפים

מלבד המידע המצביע על הדף והמסגרת המתאימים למען מסוים, טבלת הדפים מספקת מידע חיוני נוסף על מצבם של הדפים. להלן תיאור השדות של טבלת דפים גנרית:

- מספר מסגרת הזיכרון (page frame number) – אם הדף נמצא בזיכרון הראשי, השדה מצביע על המקום שאליו ממופה הדף.
- סיבית הנוכחות (present/absent bit) – נותנת אינדיקציה למקום הימצאותו של הדף: בזיכרון הראשי או בזיכרון המשני.
- שדה ההגנה (protection field) – מציין מהן ההרשאות של הדף. כך למשל ניתן לציין כי דפים מסוימים של הקוד מוגנים מפני כתיבה, ואז כל ניסיון לשנות את הקוד יגרום לפסיקת חומרה.
- סיבית ההתייחסות (reference bit) – מציינת שהייתה פנייה לדף. מעת לעת מערכת ההפעלה נאלצת להוציא דפים שלא הייתה אליהם התייחסות בזמן האחרון, כדי להקצות מסגרות זיכרון שהם תפסו לדפים אחרים. בסעיף 3.4 נלמד על אלגוריתמים שעושים שימוש בסיבית זו.
- סיבית השינוי (modified bit) – משמשת לאינדיקציה על שינוי תוכן של דף. אם תוכן הדף שונה, הוצאתו מהזיכרון מצריכה את כתיבתו לדיסק. לכן, אם צריך לפנות דף ששונה מזיכרון, נעשה זאת כאופציה אחרונה. גם סיבית זו נחוצה לאלגוריתמים לפינוי דפים מזיכרון (שמירתן בדיסק בקובץ החלפה).
- סיבית השימוש בזיכרון מטמון (caching disabled/enabled) – שימושית רק בארכיטקטורות שבהן התקני קלט/פלט משתמשים במרחב הכתובות של הזיכרון הראשי (בפרק 5 נדון בהרחבה גם בארכיטקטורות שונות). דפים הממופים לכתובות של התקני קלט/פלט בזיכרון הראשי צריכים לבטל את האפשרות של caching, כדי לא לקבל תוכן ישן שהיה שמור בזיכרון המטמון. אם בדפים אלו תבוטל האפשרות להטמנה, הדבר יגרום לקבלת תכנים עדכניים.

נסכם את הדיון בשתי בעיות שעמדנו עליהן בסעיף זה.

- טבלת הדפים בצורתה הפשוטה היא גדולה מדי. מכיוון שתוכניות אינן משתמשות בכל המרחב של הכתובות, מספיק להחזיק רק חלקים של טבלת דפים המתייחסים לדפים שנמצאים בשימוש. לשם כך הוצע הרעיון של טבלת דפים שכבתית. בסעיף 3.3.4 נראה דרך חלופית להתמודדות עם בעיה זו.
- תרגום המענים דורש פנייה לטבלת דפים. אם טבלת הדפים שוכנת בזיכרון הראשי, אזי ביצוע של הוראות ידרוש פנייה אחת לזיכרון לשם תרגום מען ופנייה נוספת

לביצוע ההוראה עצמה. דבר זה יגרום להקטנת מהירות המעבד למחצית המהירות שהוא מסוגל להגיע אליה. בהקשר זה, יש לטכניקות ההטמנה (caching) חשיבות מכרעת.

3.3.3 כיצד להגביר מהירות פענוח כתובות

Translation Lookaside Buffers – TLB

כזכור, מפאת גודלן, טבלאות הדפדוף מוחזקות ברמות נמוכות יחסית של היררכיית הזיכרון. כדי למזער את זמן החיפוש, חלק קטן מהמיפויים של טבלת הדפים מוחזקים בזיכרון מטמון. אופיין הסדרתי של תוכניות מותיר תקווה שיישמר ***עקרון הלוקאליות*** כדי שבסבירות גבוהה למדי תרגום מענים יתבצע לפי המידע המצוי בזיכרון המטמון. בניתוח בסעיף 1.3.2 הראינו שאם יחס ההצלחה של מציאת מידע בזיכרון מטמון הוא גבוה, אזי זמן התרגום המשוקלל של כתובת יהיה קרוב יותר לזמן הגישה לזיכרון המטמון Translation Lookaside Buffer – TLB, כי גם הוא סוג של זיכרון מטמון.

באיור 3-12 בספר הלימוד מוזכר המידע שבאופן טיפוסי מוחזק ב-TLB. כפי שניתן לראות מן האיור, ב-TLB מוחזקת רק קבוצה קטנה של שורות מטבלת דפים. להבדיל מטבלת דפים שבה הגישה לשורה מתאימה התבצעה לפי מספר הדף אשר שימש כאינדקס של שורה בטבלה, שורות TLB צריכות להכיל שדה עם מספר דף כדי לציין לאיזה דף מתייחס המידע שבשורה. שימו לב כי שורה בטבלת TLB איננה מכילה שדה של התייחסות (referenced bit). הסיבה לכך היא שסיבית התייחסות משמשת לצורך החלטה על הוצאת דף מן הזיכרון הראשי, ואלגוריתמים של החלפת דפים נמצאים בטיפולה של מערכת ההפעלה ולא באחריות החומרה. הנפח של טבלת TLB מוגבל, ולכן שורותיה אינן מכילות מידע על דפים לשימושו של האלגוריתם להחלפת דפים. כל הדפים המוזכרים ב-TLB הם מיוחסים. השדה האחרון – valid – מציין אם השורה של TLB פנויה או שמא היא מאחסנת מידע לגבי דף. מכיוון שהנפח של TLB מוגבל, הרי שרוב הזמן כל השורות בטבלה יהיו תפוסות.

שאלה 2

מה קורה כאשר כל השורות בטבלת ה-TLB מנוצלות?

המדיניות של פינוי תוכן שורות ה-TLB והחלפתו ממומשת באמצעות מעגלים חשמליים. מבין אפשרויות רבות אחרות, אפשר לנקוט במדיניות Least Recently Used – LRU, שלפיה השורה המוחלפת תהיה זו שלא השתמשו בה במשך פרק הזמן הארוך ביותר. בסעיף 3.4 נסביר כיצד ניתן לממש מדיניות LRU בחומרה.

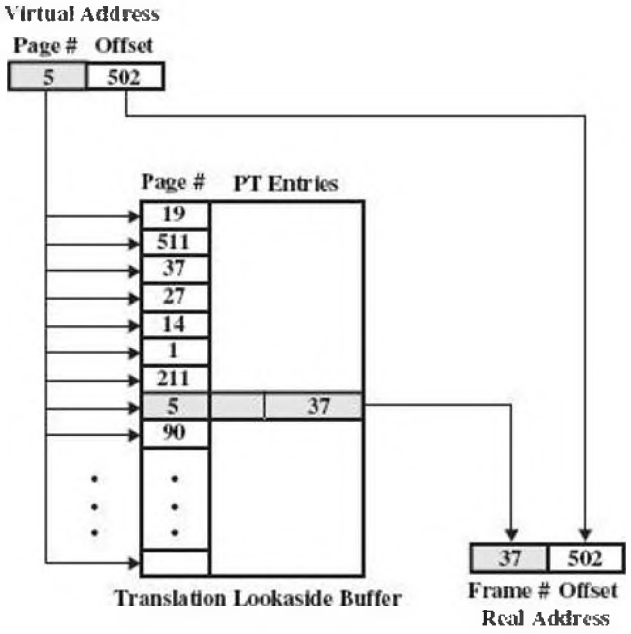
שאלה 3

אמרנו שלא ניתן לפנות לשורה של TLB לפי מספר הדף, כשם שנעשה בפנייה לטבלת דפדוף. עם זאת, מעבר שורה שורה ב-TLB יארך בוודאי זמן רב. כיצד מתמודדים עם בעיית החיפוש?

האוניברסיטה הפתוחה THE OPEN UNIVERSITY OF ISRAEL الجامعة المفتوحة
Yair Kurtzman

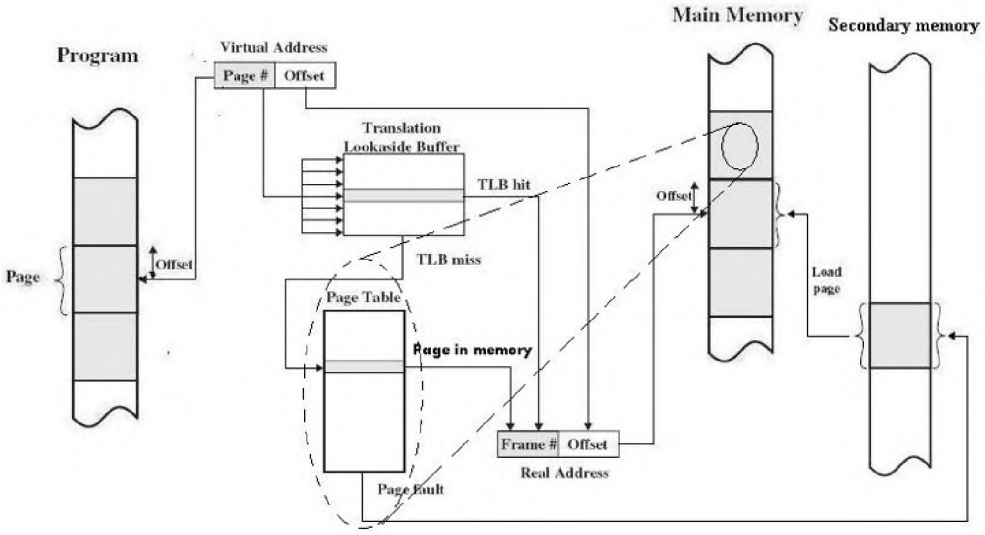
File #0006182 belongs to Yair Kurtzman- do not distribute

כדי שהחיפוש ב-TLB יהיה מהיר, המעגלים החשמליים של TLB מבצעים חיפוש מקבילי בכל השורות גם יחד. עקב כך, TLB מכונה לעתים ***זיכרון אסוציאטיבי***. ניתן לראות זאת באיור שלהלן:



איור 9

כעת נראה תמונה כללית של תרגום כתובות באמצעות TLB:



איור 10

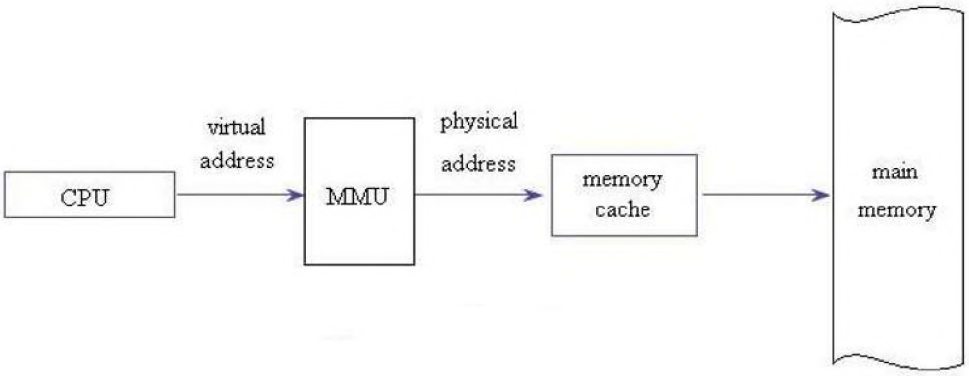
- מנגנון תרגום המענים מבצע בדיקה אם המידע לגבי הדף נמצא ב-TLB. אם התשובה חיובית, מתבצע תרגום מיידי של הכתובת הווירטואלית.
- אם אין ב-TLB מידע מתאים, החיפוש יתבצע בטבלת הדפים, בהנחה שהיא נמצאת בזיכרון הראשי. אם הדף הדרוש נמצא בזיכרון הראשי, MMU מעדכן את טבלת TLB ומתבצע תרגום של הכתובת הווירטואלית.

- אם על פי טבלת הדפים הדף אינו נמצא בזיכרון הראשי, מתרחשת פסיקת דף, ומערכת ההפעלה מביאה את הדף לזיכרון הראשי. לאחר הבאתו, היא מעדכנת את טבלת הדפים. בעקבות עדכונה של טבלת הדפים מתבצע עדכון של TLB, ולבסוף מתבצע התרגום של הכתובת הווירטואלית.

במקרים מסוימים, ייתכן שטבלת הדפים הדרושה לתרגום הכתובת הווירטואלית אינה נמצאת בזיכרון הראשי, ומתרחשת פסיקת דף על הדף של טבלת הדפים. במקרה זה צריך להביא גם את טבלת הדפים מזיכרון משני. לכן, למדיניות של החלפת דפים יש תפקיד מכריע לגבי הביצועים של הזיכרון הווירטואלי.

כדי להשיג ביצועים טובים, חלק מהמעבדים מצוידים במנגנון תרגום מענים בעל TLB caches נפרדים להוראות ול-data שמיוחס בהוראות, וקיימים פרוטוקולים לתיאום בין ה-TLBs. כך, לדוגמה, חשוב ששליפת הוראות תתבצע בקדימות גבוהה יותר מכיוון שהדבר עוזר להעלות את רמת מקביליות הביצוע של הוראות בהמשך. עם זאת, לא צריך לגרום לכך שהוראות "ימתינו" ב-pipelines ל-data.

בנוסף ל-TLB cache שמשמש לתרגום כתובות וירטואליות, מעבדים שומרים באופן זמני מידע מצומצם אך חיוני לתהליך פעיל בזיכרון המטמון במקום בזיכרון הראשי. כאשר מנגנון תרגום המענים מפענח כתובת וירטואלית, הטמנה כזו יכולה לחסוך זמן גישה לזיכרון הראשי. ראו באיור שלהלן:

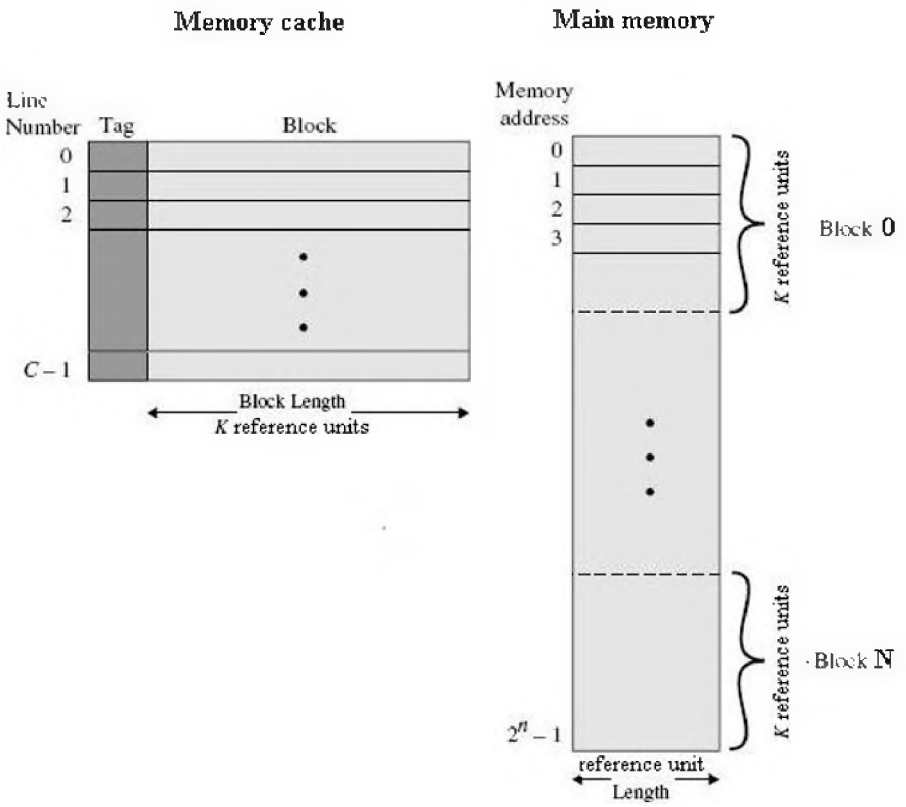


איור 11

האוניברסיטה הפתוחה
THE OPEN UNIVERSITY OF ISRAEL
الجامعة المفتوحة
Yair
Kurtzman

File #0006182 belongs to Yair Kurtzman- do not distribute

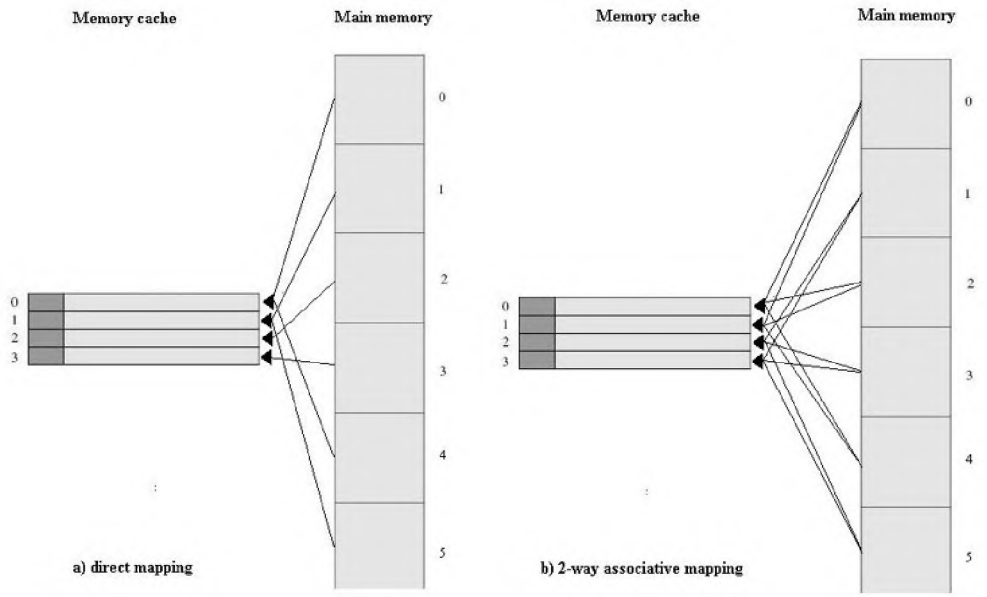
המבנה של זיכרון המטמון (memory cache) נראה כך:



איור 12

זיכרון המטמון מורכב משורות. כל שורה מכילה מזהה (tag) שמצביע על השתייכותה של השורה לבלוק מסוים בזיכרון. בלוקים של זיכרון הם בדרך כלל קטנים מדפים ומכילים מספר מסוים של יחידות ממוענות של זיכרון. מספרן תלוי בארכיטקטורה של המעבד. לצורך הדיון שלנו נסמן אותו ב-K.

מיפוי בלוקים של זיכרון ראשי לשורות של זיכרון מטמון עשוי להתבצע בכמה דרכים. באיור שלהלן נראה שתיים מהן, המכונות מיפוי ישיר (direct mapping) ומיפוי דו-אסוציאטיבי (2-way associative):



איור 13

בשיטת המיפוי הישיר, כל בלוק של זיכרון ראשי יכול להיות ממופה אך ורק לשורה מסוימת של זיכרון מטמון. בשיטת מיפוי דו-אסוציאטיבית, כל בלוק של זיכרון ראשי יכול להיות ממופה לאחת משתי שורות (מסוימות) של זיכרון מטמון. שיטת מיפוי דו-אסוציאטיבית דורשת חיפוש במקביל בשתי שורות של זיכרון מטמון. זיכרון TLB מספק אפשרות של חיפוש במקביל על כל שורותיו, ולכן הוא ***אסוציאטיבי באופן מלא*** (***fully associative***).

כרגיל, התרשים שבאיור 13 מפשט את המציאות. במעבדים מודרניים קיימים סוגים רבים של זיכרונות מטמון. למשל, זיכרון מטמון יכול להיות מאורגן בשתי רמות, level 1 ו-level 2, כלומר נוספת רמה בהיררכיה של הזיכרונות המוצגת באיור 1 במדריך למידה זה. זיכרונות מטמון אלו יכולים להיות inclusive או exclusive. אם הזיכרונות ברמות L1 ו-L2 הם exclusive, אזי המידע שנמצא ברמה האחת אינה נמצא ברמה השנייה. אם זיכרונות הם inclusive, הם יכולים להכיל את אותו המידע.

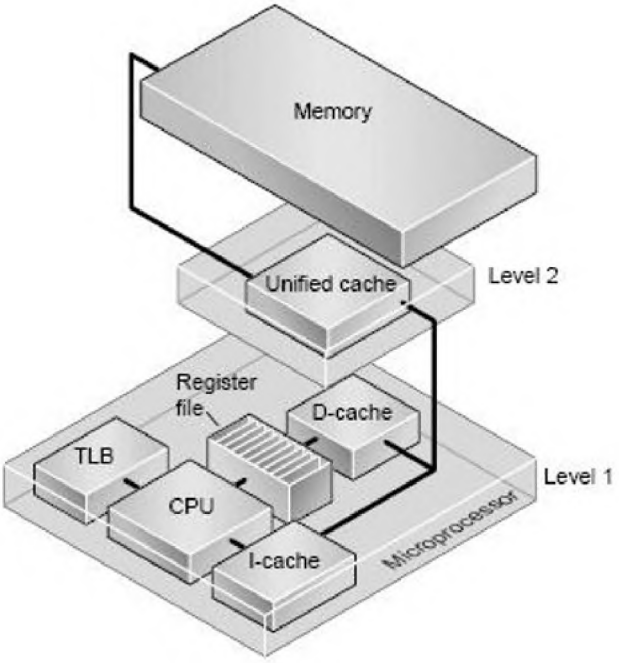
שאלה 4

- ציינו יתרון אחד של השימוש ב-exclusive cache.
- ציינו יתרון אחד של השימוש ב-inclusive cache.

באיור שלהלן מוצג תיאור סכמטי של מבנה המעבד עם 2 רמות של זיכרון מטמון:

האוניברסיטה הפתוחה
THE OPEN UNIVERSITY OF ISRAEL
الجامعة المفتوحة
Yair
Kurtzman

File #0006182 belongs to Yair Kurtzman- do not distribute



איור 14

כאן I-cache ו-D-cache הם memory caches מופרדים להוראות ו-data ברמה הראשונה. ברמה השנייה נמצא זיכרון מטמון שמשמש גם להטמנה לצורך תרגום מענים (למשל, טבלת דפים) וגם להטמנה של הוראות ו-data.

בארכיטקטורות בעלות מעבדים אחדים קיימת גם רמה שלישית (L3 cache) הנמצאת בין המעבדים לזיכרון הראשי.

סעיף זה משקף את מורכבותה של סוגיית תמיכת החומרה בתרגום כתובות של זיכרון וירטואלי, במיוחד לאור העובדה שטכניקות תכנות עכשוויות פוגעות בעקרון הלוקאליות. השימוש בשפות תכנות מונחות עצמים (object oriented) והשימוש בתכנות מונחה אירועים (event-driven programming) גורמים לריבוי מודולים קטנים ולריבוי ההתייחסויות אליהם, השזורות בזרימה סדרתית. שימוש בריבוי תהליכונים גורם לחריגה מתמדת מביצוע הוראות סדרתי. המגמה של התעצמות המורכבות של החומרה נובעת אפוא מן הדרישות האפליקטיביות הגדלות. חלק מארכיטקטורות המעבדים (כגון IA-32, IA-64, Alpha) מספקות תמיכה בדפים בגדלים שונים. כך, אזורים רציפים בזיכרון, כמו קוד של תוכניות, ימופו לדפים גדולים, ואזורים אחרים, כגון מחסניות של תהליכונים, ימופו לדפים קטנים. הגרעין של Linux, שתומך בגדלים שונים של דפים, משפר את הביצועים של אפליקציות הכתובות בשפות תכנות עדכניות ב-10% עד 35%!

3.3.4 טבלת דפים למרחבי זיכרון גדולים

טבלת דפים שכבתית (multilevel page table)

המאפיינים שהוצגו בשאלה 2 בסעיף 3.3.2 הם טיפוסיים למעבדי 32 סיביות בארכיטקטורות כמו Intel XScale, Power PC וכיוצא בהן. בארכיטקטורות שתומכות ב-64 סיביות (כגון IA-64), ממדי הבעיה גדלים בהתאם.

שאלה 5

נתון כי:

- כתובת וירטואלית היא בעלת 64 bits
- היחידות בעלות המען הקטנות ביותר הן בגודל של 1 byte
- גודל דף הוא 8 kb
- גודל שורה של טבלת דפים הוא 4 bytes

מהו גודל טבלת הדפים?

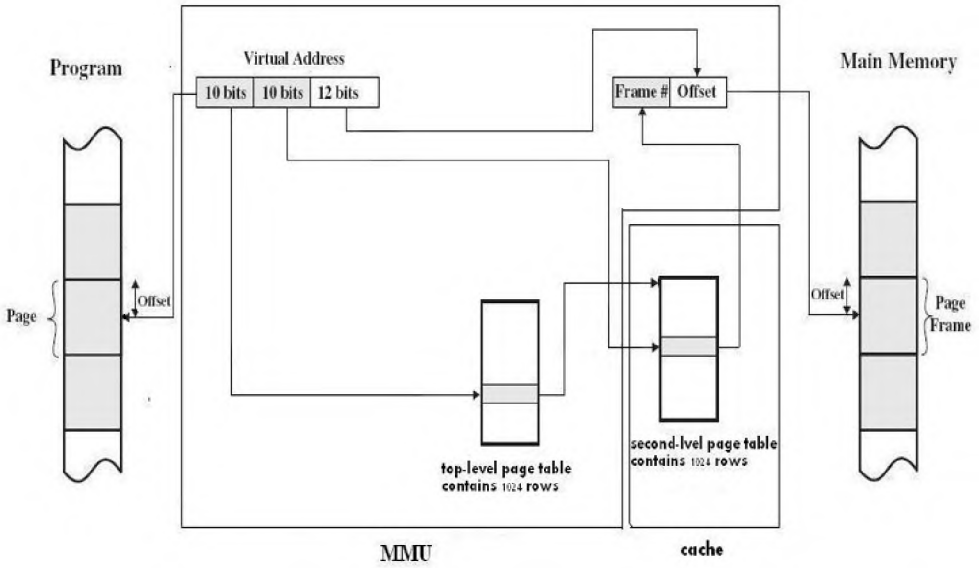
בארכיטקטורה של 64 bits היה צורך בדיסק שלם כדי להחזיק את טבלת הדפים. הפתרון לבעיית כמות הדפים העצומה הוא להשתמש באותו רעיון העומד ביסוד מנגנון הדפדוף עצמו. התהליכים מנצלים בו-זמנית רק חלק זעיר ממרחב הכתובות הווירטואלי. ואם זהו המצב לגבי הדפים עצמם (כלומר, רק חלק זעיר מהם מנוצל בו-זמנית), כך גם טבלת הדפים יכולה להכיל בו-זמנית רק את המידע הנחוץ לתרגום מרחב מענים קטן שנמצא בשימוש. משיגים זאת באמצעות שימוש בטבלאות דפדוף שכבתיות. מספר השכבות נקבע כך שטבלאות דפים של כל רמה ייכנסו לדף בודד. כך, דפים שמכילים טבלאות דפדוף שימושיות יובאו לזיכרון על ידי מנגנון הדפדוף עצמו והשאר יאוחסנו בדיסק.

איור 3-13 בספר הלימוד מדגים כיצד נראית טבלת דפים דו-שכבתית. כל כתובת וירטואלית מחולקת לשלושה שדות: שדה PT1 שערכו משמש כאינדקס של שורה בטבלת הדפים של הרמה הראשונה – 10 סיביות, שדה PT2 שערכו משמש כאינדקס של שורה בטבלת הדפים של הרמה השנייה – 10 סיביות, והיסט – 12 סיביות. מכיוון שהגודל של יחידת מען הוא 1 byte, נובע מכך שגודל דף הוא 2^{12} * 1 byte = 4 kb. שימו לב לכך שכל טבלאות הדפים ברמה השנייה מכילות 2^{10} שורות. אם הגודל של כל שורה הוא 4 bytes, אזי הן תופסות בדיוק דף אחד! לגבי הטבלה ברמה הראשונה המצב יכול להיות שונה. היא יכולה להכיל רק מידע מינימלי הכולל רק את המספר של טבלה ברמה השנייה ומספר סיביות המציינות את מקום הימצאותה של הטבלה הראשית בהיררכיית הזיכרון. כך ניתן לחסוך בנפח של הטבלה הראשית, דבר שמאפשר למקמה באופן קבוע במנגנון תרגום המענים.

באופן כללי, תרגום כתובות וירטואליות לכתובות פיזיות נראה כך:

האוניברסיטה הפתוחה
THE OPEN UNIVERSITY OF ISRAEL
الجامعة المفتوحة
Yair
Kurtzman

File #0006182 belongs to Yair Kurtzman- do not distribute



איור 15

טבלת דפים מהופכת (inverted page table)

בסעיף 3.3.4 הצגנו דרך אחת להתמודדות עם טבלאות זיכרון גדולות. השתמשנו בהבחנה שחלקים גדולים של מרחב הזיכרון הווירטואלי אינם נמצאים בשימוש. כעת נתאר דרך חלופית, אשר משתמשת בהבחנה זו בצורה קיצונית יותר.

השיטה עושה שימוש ב***טבלת דפים מהופכת*** (***inverted page table***) שמכילה מיפוי של מסגרות זיכרון לדפים של תהליכים או, במילים אחרות, כל שורה בטבלת דפים מהופכת מתייחסת למסגרת בזיכרון הפיזי ומכילה מידע לגבי איזה דף ממופה למסגרת ולאיזה תהליך הוא שייך. שימוש בטבלת זיכרון מהופכת חוסך כמות גדולה של זיכרון, ולכן ניתן להחזיק אותה ברמות גבוהות בהיררכיה של הזיכרון.

שאלה 6

נתון כי:

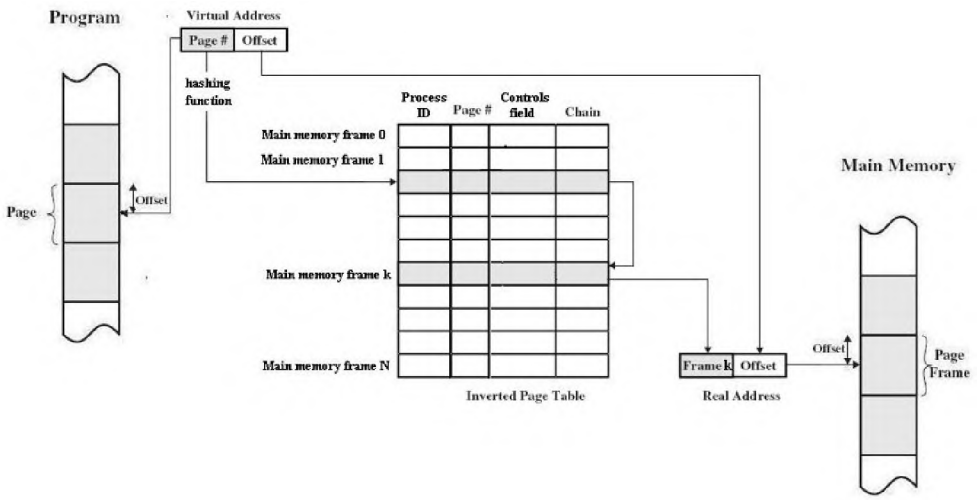
- גודל הזיכרון הפיזי הוא 1 Gb
- גודל דף הוא 8 kb
- גודל שורה של טבלת דפים הוא 4 bytes

מהו גודל טבלת הדפים המהופכת?

גודל הטבלה המהופכת הוא:

1 Gb/8kb * 4 bytes = 2^30bytes/2^13bytes * 4 bytes = 2^19 bytes = 512 kb

הבעיה היא ששורות הטבלה המהופכת מתייחסות למסגרות זיכרון, וזה בדיוק מה שאיננו יודעים. איננו יכולים להשתמש במספר הדף כאינדקס בטבלת דפים מהופכת. כדי לא לבצע מעבר על כל טבלת הדפים המהופכת, אנו משתמשים בחיפוש המבוסס על פונקציית ערבול. באיור שלהלן מוצג תיאור סכמטי של תרגום כתובת וירטואלית לכתובת פיזית:



איור 16

פונקציית הערבול מצביעה על השורה שרישום הדף עשוי להימצא בה. כאשר מעדכנים את הטבלה המהופכת, בודקים אם השורה שעליה הצביעה הפונקציה פנויה או שמא כבר שובץ במסגרת זיכרון זו דף אחר. אם השורה תפוסה, ממשיכים לחפש מקום פנוי על פי השרשור. בזמן תרגום של כתובת וירטואלית, פונים לשורה בטבלת הערבול לפי פונקציית הערבול ובודקים אם מספר הדף ומזהה התהליך הנמצאים בשורה זו תואמים את אלו של הכתובת הווירטואלית המתורגמת. אם לא, ממשיכים לטייל על פי השרשור. אם לא מוצאים ברשימה משורשרת את הזוג (מספר דף, מזהה תהליך) שלנו, סימן שהדף כלל אינו ממופה לזיכרון הראשי.

בשיטה זו אפשר להשתמש גם בטבלת דפים רגילה הנשמרת על הדיסק בלבד. שימוש ב-TLB caching הוא כמובן הכרחי, מכיוון שחישוב של פונקציית ערבול וחיפוש בשרשור דורשים זמן.

ארכיטקטורות 64 סיביות כמו IA-64, Power PC משתמשות בווריאציות שונות של השיטה המתוארת כאן.

האוניברסיטה הפתוחה
THE OPEN UNIVERSITY OF ISRAEL
الجامعة المفتوحة
Yair Kurtzman

File #0006182 belongs to Yair Kurtzman- do not distribute

3.4	אלגוריתמים להחלפת דפים
3.4.1	אלגוריתם אופטימלי להחלפת הדפים
3.4.2	אלגוריתם Not Recently Used – NRU
3.4.3	תור: FIFO
3.4.4	ההזדמנות השנייה (second chance)
3.4.5	אלגוריתם השעון (clock page replacement)
3.4.6	אלגוריתם Least Recently Used – LRU
3.4.7	סימולציה של LRU בתוכנה
3.4.8	אלגוריתם המסתמך על קבוצת עבודה של תהליך
3.4.9	אלגוריתם WSClock
3.4.10	רשימה של האלגוריתמים להחלפת דפים

3.4 אלגוריתמים להחלפת דפים

הסעיפים הבאים סוקרים אלגוריתמים המטפלים בבעיה של החלפת דפים בזיכרון מדומה. ברור כי כאשר מרחב הכתובות האפשריות גדול מהזיכרון הפיזי, מתעוררת מדי פעם הבעיה של דף חסר: תהליך נזקק לדף מסוים שאינו נמצא באותו רגע בזיכרון. אם הזיכרון מלא וכל המסגרות תפוסות, מתעוררת הבעיה איזו מסגרת לפנות בעבור הדף הנדרש.

בספר הלימוד, נושא החלפת הדפים מטופל בשני רבדים. סעיף 3.5 דן בסוגיות של חלוקת מסגרות זיכרון בין תהליכי המערכת, כגון:

- כמה דפים מוקצים לכל תהליך ובין כמה תהליכים מתחלקות המסגרות של הזיכרון הראשי.
- מהן ההשלכות של שינוי כמות הדפים המוקצית ומה קורה אם כמות הדפים המוקצית היא סטטית.
- האם החלפת דפים מתבצעת על בסיס גלובלי (כאשר אלגוריתם ההחלפה מביא בחשבון את כל המסגרות במערכת) או על בסיס לוקאלי, כאשר האלגוריתם להחלפת דפים מתבצע רק על הדפים שמוקצים לתהליך ספציפי.

סעיף זה (3.4) דן ברובד השני: תיאור אלגוריתמים המבצעים את החלפת הדפים מתוך קבוצת דפים נתונה.

3.4.1 אלגוריתם אופטימלי להחלפת הדפים

אי-אפשר ליישם את האלגוריתם הזה, אך הוא יכול לשמש להערכה השוואתית לביצועים של אלגוריתמים אחרים.

תיאור האלגוריתם
האלגוריתם יחליף את הדף שהתייחסות אליו תקרה בעתיד הרחוק ביותר מבין כל הדפים הנמצאים בקבוצת דפים נתונה.

ברור מדוע האלגוריתם איננו ישים. למערכת ההפעלה אין אפשרות לדעת לאילו דפים תיעשה הפנייה הבאה ומתי. אילו היה ישים, הוא היה גורם רק לכמות המינימלית של פסיקות דף.

שאלה 7
מדוע מערכת ההפעלה אינה יכולה לדעת בבירור לאיזה דף תיעשה הפנייה הבאה?

3.4.2 אלגוריתם Not Recently Used – NRU

ראינו ששורה של טבלת דפים טיפוסית מכילה סיבית התייחסות וסיבית שינוי (R ו-M בהתאמה). הסיבית R מציינת אם נעשתה לדף פנייה לאחרונה והסיבית M מציינת דפים שהתוכן שלהם שונה. השמת ערך 1 ל-R ו-M מתבצעת בדרך כלל בחומרה, ורק בעקבות התערבותה של מערכת ההפעלה ניתן "לכבות" אותם.

תיאור האלגוריתם
בכל פנייה לדף מתבצע שינוי ערך של R ל-1. בכל פרק זמן נתון, מערכת ההפעלה "מכבה" את הסיבית. אם התוכן של דף משתנה, גם הערך של הסיבית M משתנה. רק לאחר שמערכת ההפעלה רושמת את הדף לזיכרון המשני, היא "מכבה" את הסיבית.

בעת התרחשותה של פסיקת דף, מערכת ההפעלה בוחרת לפנות דף על ידי הסתכלות בקבוצות הדפים האלה:
קבוצת דפים לפינוי בעדיפות הראשונה: R = 0, M = 0
קבוצת דפים לפינוי בעדיפות השנייה: R = 0, M = 1
קבוצת דפים לפינוי בעדיפות השלישית: R = 1, M = 0
קבוצת דפים לפינוי בעדיפות הרביעית: R = 1, M = 1
בחירת הדף תתבצע באופן אקראי מתוך הקבוצה הלא ריקה בעלת העדיפות הגבוהה ביותר.

האלגוריתם NRU קל יחסית למימוש. ביצועיו אמנם אינם אופטימליים, אולם הם סבירים.

האוניברסיטה הפתוחה
THE OPEN UNIVERSITY OF ISRAEL
الجامعة المفتوحة
Yair Kurtzman

File #0006182 belongs to Yair Kurtzman- do not distribute

שאלה 8

א) מה ההבדל בתקורה בין פינוי דף שבו M=0 לבין פינוי דף שבו M=1?

ב) מדוע האלגוריתם NRU מעדיף לפעמים לפנות דף שבו M=1 במקום דף שבו M=0?

שאלה 9

האלגוריתם האופטימלי אינו מתייחס לסיבית שינוי של דף. הראו שהדבר אינו חשוב כדי לגרום לכמות מינימלית של פסיקות דף.

3.4.3 תור: FIFO

תיאור האלגוריתם

הדפים מוחזקים ברשימה שבראשה נמצא הדף ה"ותיק" ביותר, ובסופה – הדף ה"צעיר" ביותר, זה שהגיע אחרון.

כאשר מתרחשת פסיקת דף, הדף שבראש הרשימה נבחר והדף החדש מתווסף לסוף הרשימה.

שיטה זו פשוטה למימוש, אך חסרונה הוא שגם דפים הנמצאים בשימוש לעתים קרובות יכולים להיות מפונים מן הזיכרון, מכיוון שמדיניות FIFO מתייחסת רק לוותק של הדף במערכת ולא למידת נחיצותו. לכן, בצורתה הטהורה, FIFO איננה בחירה אטרקטיבית כלל וכלל.

3.4.4 ההזדמנות השנייה (second chance)

זהו שכלול של FIFO אשר אינו מפנה דפים לפי ותק, אלא נותן להם הזדמנות שנייה. אם גם בבדיקה חוזרת האלגוריתם לא יגלה שהייתה פנייה, רק אז מתבצע הפינוי. כך מתייחס האלגוריתם גם לוותק וגם למידת נחיצותם של הדפים.

תיאור האלגוריתם

כאשר צריך לבחור דף לפינוי, אלגוריתם ההזדמנות השנייה בוחן את סיבית ההתייחסות של הדף הוותיק ביותר, ואם ערכה הוא 0 – אזי מפנים את הדף. אם ערך הסיבית הוא 1, אזי "מכבים" אותה, מעבירים את הדף לסוף התור וממשיכים בסריקה בתור.

שאלה 10

מה קורה כאשר בכל הדפים R=1?

3.4.5 אלגוריתם השעון (clock page replacement)

העיקרון המנחה באלגוריתם זה זהה לחלוטין לעיקרון המנחה באלגוריתם ההזדמנות השנייה. ההבדל הוא אך ורק במבנה הנתונים. עם זאת, הדבר מביא לשיפור הביצועים של האלגוריתם.

תיאור האלגוריתם

הדפים מוחזקים ברשימה מקושרת מעגלית (ראו איור 3-16) לפי הוותק שלהם. כאשר צריך למצוא דף לפינוי, האלגוריתם בוחן את סיבית ההתייחסות של הדף שעליו מצביע המחוג (הדף הוותיק ביותר), ואם ערכה הוא 0 – אזי מפנים את הדף. אם ערך הסיבית הוא 1, אזי "מכבים" אותה, והמחוג מתקדם לכיוון הדף הפחות "ותיק" במעגל.

שאלה 11

במה מתבטא הייעול לעומת אלגוריתם ההזדמנות השנייה?

אלגוריתם השעון אינו צריך לטפל בהעברת דפים לסוף התור כפי שעושה אלגוריתם ההזדמנות השנייה.

שאלה 12

האם ייתכן שאלגוריתם השעון לא יעצור, כלומר שהמחוג יסתובב לנצח?

שאלה 13

אלגוריתם השעון איננו מתייחס לסיבית השינוי של דפים. שכללו אותו על ידי הוספת התייחסות לסיבית M. רמז: השתמשו בעדיפויות של אלגוריתם NFU.

3.4.6 אלגוריתם Least Recently Used – LRU

אמרנו שהאלגוריתם האופטימלי להחלפת דפים יכול לשמש להערכה השוואתית של ביצועיהם של אלגוריתמים אחרים בהינתן מחרוזת התייחסויות של דפים. כלומר, אם יודעים את הסדר המדויק של פניות עתידיות לדפים, ניתן להשוות אלגוריתם נתון לאלגוריתם האופטימלי, למשל על פי הקריטריון של מספר פסיקות הדף.

שאלה 14

נתון כי:

- זיכרון פיזי מורכב מ-4 מסגרות.
- במערכת יש 4 תהליכים, וכל אחד מהם מכיל 2 דפים.

נסמן את הדפים במספרים 1 עד 8. נניח שסדר הפנייה לדפים הוא:

האוניברסיטה הפתוחה THE OPEN UNIVERSITY OF ISRAEL الجامعة المفتوحة
Yair Kurtzman

File #0006182 belongs to Yair Kurtzman- do not distribute

2, 3, 1, 8, 6, 4, 6, 5, 7, 8, 2, 1, 4, 5, 6

בדקו מהי כמות פסיקות הדפים, אם מדיניות החלפת הדפים משתמשת:

- באלגוריתם FIFO
- באלגוריתם השעון

באופן כללי, כדי להעריך את מידת הקרבה של אלגוריתם נתון לאלגוריתם האופטימלי, לא מספיק להתבונן במחרוזת התייחסויות אחת. אבל אינטואיטיבית אפשר להגיד שביצועיו של אלגוריתם השעון בצורתו הפשוטה רחוקים עדיין מביצועיו של האלגוריתם האופטימלי.

כעת נציג אלגוריתם שביצועיו קרובים לאלו של האלגוריתם האופטימלי.

תיאור האלגוריתם

החזק דפים ממוינים לפי זמן ההתייחסות. החלף את הדף ה"ותיק" ביותר מבחינת התייחסות, כלומר את הדף שהפנייה אליו נעשתה בעבר הרחוק ביותר.

מתברר שידיעת עבר מאפשרת תוצאות טובות מאוד לחיזוי העתיד, כי דפים שלא השתמשו בהם בעבר יישארו, ככל הנראה, ללא התייחסות גם בעתיד. לכן האלגוריתם מכונה Least Recently Used.

החיסרון של האלגוריתם הוא העלות הגבוהה שנובעת מן הצורך להחזיק דפים ממוינים לפי זמן ההתייחסות, כי בכל פנייה לדף צריך לעדכן את המיקום שלו בין שאר הדפים ברשימה. מימוש של LRU בתוכנה דורש אחזקה של מבנה נתונים שיכיל את המיון של דפים לפי זמן הגישה. מימושו בחומרה מצריך לצייד כל מסגרת בתווית שתכלול חתימת זמן ומעגלים לחיפוש אסוציאטיבי. בכל אחד משני המקרים התקורה היא גבוהה מאוד. לאור החשיבות הקריטית של זמן תרגום כתובות, לא מממשים את אלגוריתם LRU בצורה מלאה, כלומר לא נעשה מיון מלא של דפים לפי זמן הגישה. אלגוריתם השעון המשופר שבשאלה 13 יכול לספק ביצועים דומים מאוד לאלה של LRU.

פתרונות בחומרה הם תובעניים פחות מבחינת הזמן, אולם עלותם עדיין גבוהה מאוד. לכן נפוץ יותר לראות פתרונות כאלה לניהול של שורות בזיכרון מטמון. ראו מימוש LRU בחומרה המוצע בספר הלימוד.

3.4.7 סימולציה של LRU בתוכנה

בסעיף 3.4.7 ציינו כי אלגוריתם LRU אינו מיושם בתוכנה בצורתו הטהורה, ואמרנו כי קיימים אלגוריתמים משופרים שפועלים בצורה דומה לאלגוריתם השעון אשר יכולים לספק ביצועים דומים לאלה של LRU (אינם כבדים מדי מבחינה חישובית). בסעיף זה נכיר אלגוריתם נוסף שגם הוא מספק ביצועים דומים מאוד לאלה של LRU.

תיאור אלגוריתם ההזדקנות (aging)
לכל דף מוחזק מונה שגדל ב-1 בסיבית השמאלית ביותר אם הייתה אליו פנייה בפרק זמן מסוים שמוגדר על ידי האלגוריתם. כדי לא להתייחס להיסטוריה ישנה מדי, לפני כל אינקרמנטציה של מונה, עושים לו הזזה ימינה. בעת הצורך בהחלפת דף, יוחלף הדף שהוא בעל המונה הקטן ביותר. ייתכן שיהיה יותר מדף אחד כזה. במקרה זה הבחירה תתבצע באקראי.

ראו את אופן פעולתו של אלגוריתם ההזדקנות באיור 3-17 בספר הלימוד. קיימת גם גרסה של אלגוריתם ההזדקנות שאינה מבצעת הזזה של מונה. היא מכונה NFU – Not Frequently Used. החיסרון העיקרי של גרסה זו הוא שהמונים אינם שוכחים דבר. בגרסה זו של האלגוריתם, ערך המונה לעולם אינו יורד. כך ייתכן שמונה גבוה מייצג מצב היסטורי שבו מספר הפניות לדף היה גדול מאוד. המונה אינו קטֵן גם כאשר אין עוד פניות לדף.

האלגוריתם שונה מאלגוריתם LRU מכיוון שאיננו יודעים את סדר הפניות לדפים בין שני פרקי זמן עוקבים, וגם גודל המונה מוגבל (מספר הסיביות של המונה). לכן ייתכן מצב שבו למונים של שני דפים יהיה ערך זהה והאלגוריתם יצטרך לבחור ביניהם באקראי, בהיעדר אינדיקציה לאיזה מבין הדפים בעלי אותו ערך מונה פנו קודם.

שאלה 15
מדוע אלגוריתם זה יעיל יותר מאלגוריתם LRU, במונחים של תקורת החישוב הנדרשת?

3.4.8 אלגוריתם המסתמך על קבוצת עבודה של תהליך

אלגוריתם להחלפת דפים המסתמך על עיקרון של ***קבוצת עבודה*** נשען על הנחה סבירה שבכל שלב של ביצוע תהליך, התהליך פונה לקבוצה מצומצמת של דפי הזיכרון. קבוצת עבודה של תהליך היא קבוצת דפי זיכרון הנחוצים לריצת התהליך בשלב הנוכחי של הביצוע. אם קבוצת עבודה של תהליך נמצאת בזיכרון, תהליך ירוץ בלי לגרום למספר רב של פסיקות דף.

בכלל, כאשר תהליך חדש מתחיל לרוץ, הוא גורם לכמות מסוימת של פסיקות דף, בפנייה לדפים שמכילים הוראות של תחילת ריצתו ובפנייה לדפי data ו-stack. כעבור פרק זמן הדרוש להבאת דפים ראשוניים הנחוצים לעבודתו, התהליך יכול להמשיך לרוץ עם מספר מצומצם של פסיקות דפים עד לשינוי לפאזה הבאה (למשל, לעבור משלב הקומפילציה של מודול אחד למודול אחר או לבצע שינוי תוכן של תהליכון אחד בתוכן של תהליכון שני). בכניסה לפאזה הבאה שוב מתרחשת כמות גדולה יחסית של פסיקות דף עד שהמצב מתייצב, וכך עד לשינוי הבא. האסטרטגיה המתוארת כאן, של הבאת דפים לפי צורך, נקראת ***demand paging***, כי הדפים מובאים לזיכרון לפי הצורך. לחלופין, אם יודעים מהי קבוצת העבודה, אפשר להביא אותה לזיכרון לפני שהתהליך מתחיל לרוץ. אם מנהלים מעקב אחרי קבוצת עבודה של תהליך במהלך ריצתו, אפשר, גם לאחר החלפת תוכן של התהליך, לדאוג שקבוצת העבודה שלו תימצא בזיכרון בטרם

האוניברסיטה הפתוחה THE OPEN UNIVERSITY OF ISRAEL الجامعة المفتوحة
Yair Kurtzman

File #0006182 belongs to Yair Kurtzman- do not distribute

ימשיך לרוץ. טכניקה זו נקראת ***prepaging*** והיא יכולה לחסוך כמות ניכרת של פסיקות דף.

שאלה 16

האם ניתן לחסוך במספר פסיקות הדף גם כאשר לא יודעים מהי קבוצת העבודה של תהליך בטרם התחיל לרוץ?

לאחר שרכשתם את התובנות הללו, אפשר לנסח את העיקרון שעליו מושתתים אלגוריתמים להחלפת דפים מבוססי קבוצת עבודה. העיקרון פשוט – אפשר לפנות דפים שאינם נמצאים עוד בקבוצת עבודה של תהליך. השאלה היא איך מגדירים את קבוצת העבודה הזאת בצורה פורמלית.

קבוצת עבודה של תהליך בזמן t מוגדרת כקבוצת דפים שנעשתה אליהם פנייה ב-k ההוראות האחרונות ומסומנת ב-W(k,t).[11] ההגדרה הזאת בעייתית משתי סיבות:

- הסיבה העיקרית היא שקשה לנהל מעקב אחר קבוצת דפים ל-k הוראות אחורה לגבי כל תהליך.
- הסיבה הנוספת קשורה לבעיית קביעת הערך של k – קביעת ערך קטן מדי של k אינה משקפת את גודל קבוצת הדפים שנחוצים לתהליך בפאזה ארוכה בחייו. לעומת זאת, קביעת k גדול מדי לא תשקף את המציאות במעברים בין הפאזות. לקביעת k יש חשיבות קטנה יחסית מכיוון שהעלייה של כמות פסיקות דף בתוך פאזה ארוכה לא תהיה חדה מאוד עקב קביעה של k קטן, ואילו קביעת k גדול לא תשפיע על האלגוריתם להחלפת דפים אם יש מספיק מסגרות זיכרון.

כאן נזכיר שבשיטות להקצאת מספר המסגרות נדון בסעיף 3.5, ולכן נדחה את הדיון בשימוש בקבוצת עבודה לצורך הקצאת מסגרות לסעיף זה. כאן נציין רק שאם מקצים לתהליך קבוצת מסגרות שגודלה קטן מגודל קבוצת העבודה שלו, יכולה להתרחש תופעה של ***סחרור*** (***thrashing***). סחרור מתרחש כאשר תהליך גורם לפסיקות דף מיד אחרי ביצוע של מספר קטן של הוראות קוד. במצב זה התהליך כמעט אינו מתקדם, מכיוון שמרבית הזמן מתבזבז על החלפת דפים בין הזיכרון הראשי לזיכרון המשני. התקדמות התהליך מואטת משום שזמן הטיפול בפסיקת דף גדול בהרבה מזמן הביצוע של הוראה שאינה גורמת לפסיקת דף.

אם כך, הבעיה היא כיצד לזהות מהי קבוצת העבודה של תהליך לצורך השימוש באלגוריתם להחלפת דפים. הדרך להתגבר על הקושי הזה אינה לנהל מעקב אחר קבוצת העבודה אלא להשתמש בקירוב (בסעיף הקודם כבר ראינו שקירוב יכול לספק ביצועים טובים במקרה של אלגוריתם הזדקנות שמהווה קירוב ל-LRU). קירוב מקובל הוא להשתמש ב***זמן הווירטואלי הנוכחי*** של תהליך במקום לספור k הוראות אחורה. הזמן הווירטואלי הנוכחי הוא זמן CPU שתהליך השתמש בו בפועל מתחילת ריצתו. לכן, לפי הגדרה אופרטיבית חדשה זו, קבוצת עבודה של תהליך בזמן נתון t תהיה קבוצת הדפים

[11] לפעמים, במקום k ההוראות האחרונות משתמשים ב-k הדפים האחרונים שנעשתה אליהם פנייה.

שאליהם פנו במהלך של τ יחידות הזמן הווירטואליות האחרונות. אנו נסמן אותה ב- $W(\tau, t)$.

ראו באיור 3-19 בספר הלימוד את אופן פעולתו של אלגוריתם לפינוי דפים שמבוסס על קבוצת עבודה. האלגוריתם מבסס את קביעתו על שני נתונים:

- על סיבית ההתייחסות לדף.
- על הזמן הווירטואלי של הפנייה האחרונה לדף.

תיאור האלגוריתם

כאשר צריכים לפנות דף, עוברים על הדפים שנמצאים בזיכרון ובודקים לכל דף:

- אם R=1, אזי עדכן את הזמן הווירטואלי של הדף לזמן הווירטואלי הנוכחי ועדכן את R=0. לדף הייתה התייחסות (כלומר פנו אליו) לאחרונה, ולכן הוא איננו מועמד לפינוי.
- אם R=0, הדף מועמד להיות מפונה.
  1. חשבו את current virtual time – time of last use = age.
  2. אם $age > \tau$, אזי הדף כבר אינו בקבוצת עבודה ולכן הוא מפונה.
  3. אם $age <= \tau$, אזי הדף עדיין נמצא בקבוצת עבודה, אבל ייתכן שיפונה אם בסוף המעבר על כל הדפים יימצא שהוא היה ה"ותיק" ביותר מבין כל הדפים בקבוצת העבודה עם סיבית התייחסות "כבויה". (מובן שפינוי דף כזה שהיה בקבוצת עבודה יתבצע רק אם לא התגלה דף מחוץ לקבוצה.)

אם עברנו על כל הדפים ולא מצאנו דף עם סיבית התייחסות "כבויה", נבחר באקראי דף עם R=1 ונפנה אותו. במידת האפשר, הדף הנבחר יהיה עם סיבית שינוי "כבויה", כדי לחסוך כתיבה לדיסק.

3.4.9 אלגוריתם WSClock

אלגוריתם WSClock הוא גרסה מעגלית של האלגוריתם שהכרנו בסעיף 3.4.8. נוסיף כאן שכלול קטן שמציע כתיבה של דפים בחזרה לדיסק. ראו איור 3-20 בספר הלימוד.

תיאור האלגוריתם

הדפים הנמצאים בזיכרון מנוהלים ברשימה מעגלית. בכל מיפוי של דף לזיכרון נוסף לרשימה איבר חדש. עם פינוי של דף מזיכרון, הדף מוסר מן הרשימה. בעת החלפת דף, מתבצעים הצעדים כפי שמפורט בהמשך. מחוג מצביע על דף אחד מן הדפים ברשימה אשר עליו התבצעה הפעולה האחרונה. כאשר צריך למצוא דף לפינוי, בודקים:

- אם המחוג מצביע על דף ש-R=1, אזי עדכן את הזמן הווירטואלי של הדף לזמן הווירטואלי הנוכחי, עדכן את R=0 וקדם את המחוג לדף הבא. הייתה פנייה לדף לאחרונה ולכן הוא איננו מועמד לפינוי.
- אם R=0, הדף מועמד לפינוי.
  - חשב את current virtual time – time of last use = age

האוניברסיטה הפתוחה THE OPEN UNIVERSITY OF ISRAEL الجامعة المفتوحة
Yair Kurtzman

File #0006182 belongs to Yair Kurtzman- do not distribute

- אם $age > \tau$, אזי הדף כבר אינו בקבוצת עבודה. בדוק אם M=0, ואם כן – פַּנֵּה את הדף. אם לא – תזמן כתיבה של הדף לדיסק וקדם את המחוג לדף הבא (אולי יימצא דף שלא צריך לכתוב לדיסק וניתן יהיה לפנות מיד). תזמן לכתיבה לכל היותר n פעמים (כדי לא להציף בכתיבות).
- אם $age <= \tau$, אזי הדף בקבוצת עבודה אבל מבצעים מעקב אחר הדף ה"יותיק" ביותר (עדיף שיהיה גם בעל סיבית שינוי "כבויה").

לאחר מעבר מעגלי אחד, כאשר עדיין לא פונה דף, ייתכנו שני מצבים:

- אם תוזמנה לפחות כתיבה אחת קודם, המשך לקדם את המחוג עד אשר יימצא דף עם סיבית שינוי כבויה. הדף שיימצא יפונה.
- אם לא תוזמנה שום כתיבה, אזי כל הדפים הם בקבוצת עבודה. לכן מסמנים את הדף ה"יותיק" ביותר בעל סיבית כבויה שעקבנו אחריו. אם אין כזה, מוצאים דף עם סיבית שינוי דלוקה, כותבים אותו לדיסק ומפנים את מסגרתו.

3.4.10 רשימה של האלגוריתמים להחלפת דפים

אלגוריתם	סיכום, יתרונות, חסרונות
אופטימלי	כשמו כן הוא, אך הוא אינו ישים מכיוון שלא ניתן לצפות מראש את הסדר של התייחסויות לדפים. עם זאת, הוא שימושי להערכה השוואתית של אלגוריתמים אחרים.
Not Recently Used – NRU	ביצועים בלתי סבירים. חלוקה גסה למחלקות עדיפויות ולכן בחירה של דפים להחלפה בצורה אקראית גורמת למספר גבוה של פסיקות דף.
FIFO	ביצועים בלתי סבירים. האלגוריתם מתייחס רק לוותק של דפים ולא למידת נחיצותם. לכן דפים חשובים יכולים להיות מפונים, דבר שגורם למספר גבוה של פסיקות דף.
אלגוריתם הזדמנות שנייה ואלגוריתם השעון הבסיסי	ביצועים סבירים (במונחים של פסיקות דף). מתייחס לא רק לוותק אלא גם למידת הנחיצות של הדפים. אלגוריתם השעון הוא מימוש יעיל של אלגוריתם הזדמנות שנייה (מבחינה חישובית).
אלגוריתם שעון משופר	אלגוריתם שעון מאחת השאלות הקודמות שעושה שימוש גם בסיבית ההתייחסות וגם בסיבית השינוי. יכול לספק ביצועים (במונחים של פסיקות דף) קרובים לאלו של LRU. יעיל מבחינה חישובית.
Least Recently Used – LRU	ביצועים דומים מאוד לאלה של האלגוריתם האופטימלי. מסתמך על היסטוריה של פניות לדפים. לא יעיל מבחינה חישובית בצורתו הטהורה. יעיל, אך יקר למימוש בחומרה.
Not Frequently Used – NFU	ביצועים לא סבירים כי אינו שוכח את העבר הרחוק. מהווה קירוב גס ל-LRU.
אלגוריתם ההזדקנות	שיפור של NFU ששוכח את העבר. מספק ביצועים דומים לאלו של LRU (במונחים של פסיקות דף). יעיל יחסית ל-LRU מבחינה חישובית.

קבוצת העבודה	אלגוריתם המספק ביצועים טובים (במונחים של פסיקות דף). קיימת גרסה יעילה יותר מבחינה חישובית.
WSClock	מימוש יעיל של אלגוריתם קבוצת העבודה.

מבחינה מעשית, האלגוריתמים השימושיים המיושמים בפועל בתוכנה הם אלגוריתם ההזדקנות (aging) ו-WSClock.

3.5	**שיקולים בתכנון של מערכות דפדוף בזיכרון**
3.5.1	**מדיניות החלפה מקומית לעומת מדיניות החלפה גלובלית**
3.5.2	**קביעת רמת ריבוי של מקביליות**
3.5.3	**שדים מדפדפים ו-cleaning policy**
3.5.4	**שיקולים בקביעת גודלו של דף**
3.5.5	**מרחבי כתובות נפרדים להוראות ול-data**
3.5.6	**דפים לשימוש משותף**
3.5.7	**ספריות לשימוש משותף**
3.5.8	**מיפוי קבצים לזיכרון**

3.5 שיקולים בתכנון של מערכות דפדוף בזיכרון

בסעיף 3.4 ראינו אלגוריתמים שונים המטפלים בבעיה של החלפת דפים מתוך קבוצת דפים נתונה. בסעיף זה אנו עוברים משיקולים תיאורטיים לשיקולים הנדסיים, והסוגיות שנדון בהן ינסו לתת מענה לשאלות האלה:

- כמה מסגרות מוקצות לכל תהליך ומהם התהליכים שלהם מוקצות מסגרות זיכרון?
- מהן ההשלכות של שינוי מספר המסגרות המוקצה לתהליך ומה קורה אם כמות המסגרות המוקצית היא סטטית?
- האם החלפת דפים מתבצעת על בסיס גלובלי (כאשר אלגוריתם החלפה מביא בחשבון את כל המסגרות במערכת) או על בסיס לוקאלי (כאשר אלגוריתם להחלפת דפים מתבצע על הדפים שמוקצים לתהליך ספציפי)?

האוניברסיטה הפתוחה THE OPEN UNIVERSITY OF ISRAEL الجامعة المفتوحة
Yair Kurtzman

File #0006182 belongs to Yair Kurtzman- do not distribute

3.5.1 מדיניות החלפה מקומית לעומת מדיניות החלפה גלובלית

מדיניות לוקאלית (local page replacement policy)

במדיניות זו, פינוי דף מתבצע רק מתוך קבוצת הדפים של התהליך הדורש דף נוסף. התבוננו באיור 3-22b. כאשר תהליך A גורם לפסיקת דף, אחד מדפיו מפונה והדף החדש תופס את מקומו.

קיימות שתי אפשרויות להקצאה של דפים:

- להקצות כמות קבועה של מסגרות לכל תהליך.
- להקצות כמות מסגרות שיכולה להשתנות עם הזמן.

שימוש במדיניות לוקאלית והקצאת כמות מסגרות קבועה יכולים להביא לסחרור, אם קבוצת העבודה של תהליך גדלה. לחלופין, אם קבוצת העבודה של תהליך מצטמצמת, חלק מהמסגרות שהוקצו יכולות להיות מיותרות, ולכן ההקצאה הקבועה תגרום לבזבוז.

לכן, בחירה טובה יותר היא הקצאה של מספר מסגרות המשתנה בהתמדה. מימוש של שיטת הקצאה לוקאלית עם כמות מסגרות משתנה דורש משאבים חישוביים. מלבד האלגוריתם להחלפת דפים, שפועל על הקצאת המסגרות הנוכחית, דרוש גם רובד חישובי נוסף אשר מבצע הערכה תקופתית של מספר המסגרות הדרושות לתהליכים.

מדיניות גלובלית (global page replacement policy)

מדיניות גלובלית מפנה דפים ללא כל קשר לזהות התהליך הדורש דף חדש. ראו איור 3-22c.

במקרה זה, מתוך האפשרויות:

- להקצות מספר קבוע של מסגרות לכל תהליך
- או להקצות כמות מסגרות שיכולה להשתנות עם הזמן

רק האפשרות האחרונה היא בת-יישום, כי החלפת דפים גלובלית – כאשר לכל תהליך ישנה כמות מסגרות קבועה – היא חסרת היגיון.

שיטה גלובלית עם הקצאת מסגרות משתנה היא קלה ליישום. היא גם תיתן ביצועים טובים אם במערכת קיימות מסגרות פנויות, כי בעת פסיקת דף יהיה קל למצוא מסגרת פנויה מתוך סך המסגרות הפנויות של המערכת. כאשר קבוצת העבודה של תהליך מצטמצמת, מסגרות מוחזרות למאגר הכללי של המסגרות הפנויות ומועברות לשימושם של תהליכים אחרים. החיסרון של שיטה זו מתגלה כאשר צריך להחליט איזה דף להוציא במצב שבו אין עוד מסגרות פנויות. במצב זה, ההחלפה עלולה לפגוע בקבוצת העבודה של תהליך, שלא הוא אשר גרם לפסיקת דף.

ניהול כמות הקצאת המסגרות. אלגוריתם PFF (Page Fault Frequency)

גם במקרה של השיטה הגלובלית וגם במקרה של השיטה הלוקאלית, קיים צורך לעקוב אחר כמות המסגרות המוקצית לתהליכים.

הימצאותה של קבוצת העבודה, בשלמותה, בזיכרון היא תנאי הכרחי למניעת סחרור. מאידך גיסא, הקצאת מסגרות מעבר לגודלה של קבוצת העבודה גורמת בזבוז. לכן, אם יודעים אילו תהליכים גורמים בצורה מתמדת למספר רב של פסיקות דף, ניתן להסיק שמדיניות ההחלפה פגעה בקבוצת העבודה שלהם. עם זאת, אם התהליך מתבצע כמעט בלי פסיקות דף, הדבר מלמד (אולי) שיש לו עודף של מסגרות וייתכן שניתן לצמצם את גודלה של קבוצת העבודה. אבחנות אלו משמשות את האלגוריתם PFF (Page Fault Frequency) להקצאה דינאמית של כמות המסגרות.

תיאור האלגוריתם

המנגנון המוצע הוא סטטיסטי: מודדים את תדירות פסיקות הדף של כל תהליך, ובמידת הצורך מוסיפים לו מסגרות. באותה צורה ניתן לבדוק עד כמה תשפיע הקטנת קבוצת העבודה של תהליך על תדירות שגיאות הדף שלו. אם השפעה זו איננה ניכרת, זהו סימן לכך שקבוצת העבודה התכווצה. לשם כך משמשים שני מדדים של רמות סף: A ו-B (ראו איור 3-23). אם קצב פסיקות הדף בעבור תהליך מסוים גבוה מערך הסף A, נסיק כי לתהליך זה חסרות מסגרות. בהתאמה, אם קצב פסיקות הדף של התהליך יורד מתחת לערך הסף B, אפשר כנראה לדרוש חלק ממסגרותיו.

שימו לב: PFF אינו מתייחס לאלגוריתם להחלפת דפים אלא רק לכמות המסגרות. לכן ניתן להשתמש בו בתור בורר הן במקרה של שיטת ההחלפה הגלובלית והן במקרה של שיטת ההחלפה הלוקאלית.

שאלה 17

לאיזו שיטה (גלובלית או לוקאלית) שייכים האלגוריתמים האלה להחלפת דפים שלמדנו בסעיף 3.4:

- FIFO
- שעון
- LRU
- אלגוריתם ההזדקנות
- WSClock

האם ניתן להשתמש בהם בשתי השיטות?

האוניברסיטה הפתוחה THE OPEN UNIVERSITY OF ISRAEL الجامعة المفتوحة
Yair Kurtzman

File #0006182 belongs to Yair Kurtzman- do not distribute

3.5.2 קביעת רמת ריבוי של מקביליות

אם הגודל הכולל של קבוצות העבודה של התהליכים הנמצאים במערכת עולה על גודל הזיכרון שניתן להקצות, יש צורך גם בהחלטות תזמון לטווח ארוך. תוצאה אפשרית של החלטות תזמון כאלה יכולה לגרום להקטנה של רמת המקביליות של תהליכים. הוצאת תהליך לדיסק מפנה מיד מסגרות שלו לשימושם של תהליכים אחרים.
בהחלטת תזמון לטווח ארוך, קיימות אפשרויות אחדות לבחירת התהליך המוצא לדיסק:

- ניתן להוציא את התהליך בעל העדיפות הנמוכה ביותר.
- ניתן להוציא את התהליך שגורם לכמות הגדולה ביותר של פסיקות דף.
- ניתן להוציא את התהליך שהוקצה לו מספר המסגרות הקטן ביותר (ובכך לצמצם את עלות התעבורה).
- ניתן להוציא את התהליך שהוקצה לו מספר המסגרות הגדול ביותר (ובכך לפנות מקום גדול ככל האפשר).

שאלה 18

כיצד ניתן לזהות מצבים שנדרש בהם צמצום של רמת מקביליות של תהליכים בעזרת שיטת PFF (Page Fault Frequency)?

שאלה 19

לכל אפשרות של הוצאת תהליך לדיסק, ודאו כי אתם מבינים מהו הרציונל העומד מאחוריה.

3.5.3 שדים מדפדפים ו-cleaning policy

ראינו כי אלגוריתמים להחלפת דפים יכולים לגזול זמן רב. בעת התרגום של כתובת וירטואלית, פסיקת דף כרוכה בעיכוב התהליך, אולם אם גם צריך לבצע החלפת דפים, פירוש הדבר הוא השהיית התהליך לתקופה ארוכה יותר וירידה בביצועיו. לכן קיים במערכות הפעלה תהליך רקע המכונה "שד מדפדף", המייעל את מערכת הדפדוף על ידי כך שהוא דואג להחזיק מאגר מספיק גדול של מסגרות פנויות לשימוש. מדי זמן קצוב הוא מתעורר ובודק את מצב המסגרות הפנויות. אם אחוז המסגרות הפנויות במערכת קטן מדי, אזי השד מתחיל לבחור דפים לפינוי, בהתאם למדיניות פינוי הדפים של המערכת (cleaning policy). לעתים פינוי המסגרות יכול להתבצע ללא כתיבה של המסגרות המפונות בחזרה לדיסק, רק על ידי החלפה של סיבית valid בטבלת דפים. השד המדפדף מנהל רשימה של מסגרות עם דפים כאלה, ואם יש התייחסות לדף כזה, הרי שהוא כבר קיים ואין צורך להביאו מן הזיכרון המשני (אם עדיין לא הייתה כתיבה לתוך המסגרת שהוא נמצא בה).

טכניקה שימושית אחרת לפינוי דפים מיושמת באמצעות ***שעון בעל שני מחוגים*** (***two-handed clock***). מחוג אחד עובר על דפים ומסמן אותם לפינוי, מחוג שני נמצא בזווית קבועה מן המחוג הראשון ובכך מוגדר חלון הזדמנויות. אם עד להגעה של המחוג השני התרחשה התייחסות לדף, הסימון של פינוי מתבטל; אחרת הדף מפונה.

3.5.4 שיקולים בקביעת גודלו של דף

בסעיף זה נראה ניתוח מתמטי פשוט שמדגים כיצד מפשרים בין שיקולים שונים לקביעת גודל דף במערכת זיכרון וירטואלי. כפי שהזכרנו קודם, קיימות מערכות שמשתמשות בגדלים שונים של דפים, אך הניתוח שנראה כאן מעניין מעצם פשטותו.

שיקולים בעד דפים קטנים

- דפים קטנים גורמים לפחות בזבוז בדף האחרון של כל סגמנט של התהליך. הריסוק הפנימי בדף האחרון הוא בממוצע כמחצית מגודל הדף.
- דפים קטנים מחלקים את המרחב הווירטואלי של תהליך לכמות גדולה יותר של חלקים. עקב ***הגרעיניות העדינה*** יותר של החלוקה, ייתכן שהגודל הכולל של קבוצת העבודה של תהליך יהיה קטן יותר במונחים של כמות הזיכרון (ולא במונחים של כמות הדפים).

שיקולים בעד דפים גדולים

- שימוש בדפים קטנים מגדיל את כמות הדפים שתוכניות יזדקקו להם. לכן טבלת הדפים תגדל. דפים גדולים מקטינים את גודלה של טבלת הדפים.
- העברת דפים מן הזיכרון הראשי לדיסק או להפך יכולה להיות יעילה כאשר הדפים גדולים, מכיוון שלהעברה של כל דף קיימת תקורה של חיפוש מיקום בדיסק.

האוניברסיטה הפתוחה
THE OPEN UNIVERSITY OF ISRAEL
الجامعة المفتوحة
Yair
Kurtzman

File #0006182 belongs to Yair Kurtzman- do not distribute

ניתוח מתמטי

יהי

- s – הגודל של תהליך ממוצע
- p – גודל הדף
- e – גודל של שורה בטבלת הדפים של תהליך

אזי

- המספר המשוער של דפים שהתהליך זקוק להם יהיה s/p.
- הגודל של טבלת דפים של תהליך יהיה se/p.
- הבזבוז עקב הריסוק הפנימי בדף האחרון של התהליך יהיה p/2.

לכן, התקורה הכוללת של השימוש בזיכרון הווירטואלי תהיה:

$$Overhead(p) = se/p + p/2$$

נרצה למזער את התקורה כפונקציה של p. נגזור אותה ונקבל כי מינימום של תקורה מושג כאשר:

$$-se/p^2 + 1/2 = 0$$

ומכאן שגודל הדף הוא:

$$sqrt(2se) = p$$

מכאן שהגדלה של s פי ארבעה מכפילה את גודל הדף האופטימלי (תחת ההנחות המגבילות של המודל).

3.5.5 מרחבי כתובות נפרדים להוראות ול-data

זהו סעיף רשות. הוא מתאר דרך להתגברות על מרחבים וירטואליים קטנים במחשב PDP-11.

3.5.6 דפים לשימוש משותף

תוצאה מעניינת של מנגנון הדפדוף היא האפשרות להשתמש בו-זמנית בכמה דפים בעבור קבוצת תהליכים. כך תוכניות הנמצאות בשימוש רב, כמו מעבדי תמלילים, יכולות להופיע בזיכרון רק פעם אחת, בעת שתהליכים רבים משתמשים בהן.

כמובן, הסכנה היא שפעולת כתיבה של תהליך אחד תשפיע מיד על מצבם של תהליכים אחרים. זאת ניתן למנוע למשל על ידי נקיטת מדיניות שלפיה רק דפים לקריאה בלבד (כלומר, דפים האסורים לכתיבה) ניתנים לשימוש משותף. טכניקה המכונה copy-on-write יכולה לשמש במקרה של שיתוף דפים שמסומנים גם לכתיבה. הדפים יכולים להיות משותפים עד לניסיון הכתיבה הראשון. כאשר מנגנון תרגום המענים מזהה ניסיון כתיבה לדף כזה, הוא יכול לגרום לפסיקה שבעקבותיה מערכת ההפעלה תשכפל את הדף, וכך יתאפשר לתהליך הכותב לבצע שינויים בעותק שלו.

בעיה נוספת קשורה לעובדה שדפים משותפים אינם יכולים להתפנות מהזיכרון כאשר אחד התהליכים שהשתמש בהם מסתיים. ניתן לפנות דפים אלו רק כאשר התהליך

האחרון גמר את השימוש בדפים המשותפים. לעתים, דרישה זו מנוגדת למדיניות הנקבעת על ידי אלגוריתם החלפת הדפים ודורשת גם מנגנון מיוחד למעקב אחר השימוש בדפים המשותפים. כמו במקרים רבים אחרים, השאיפה לחסוך זיכרון באמצעות שיתוף דפים יוצרת סיבוך נוסף במערכת הדפדוף כולה.

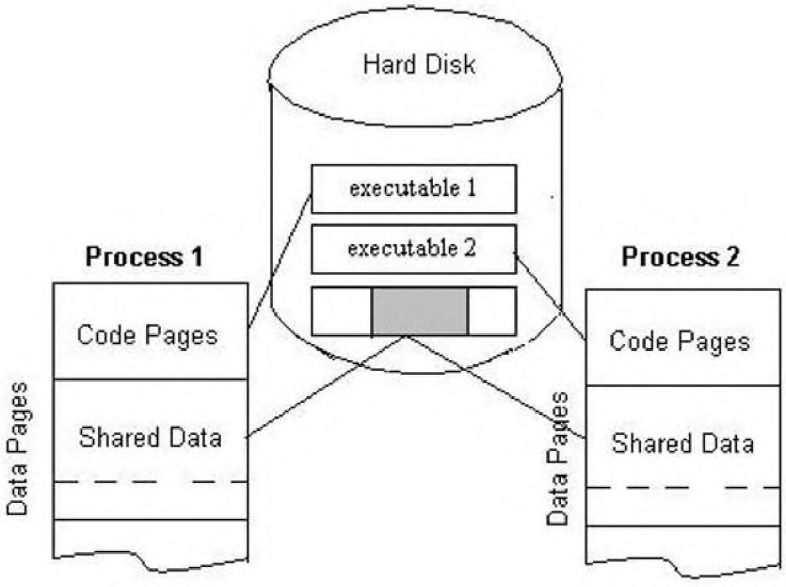
3.5.7 ספריות לשימוש משותף

זהו סעיף רשות.

3.5.8 מיפוי קבצים לזיכרון

בפרק 4 נסקור פעולות טיפוסיות המבוצעות על קבצים. כל שיטה של מיפוי קבצים לזיכרון מספקת אפשרויות עבודה שונות עם קבצים. מיפוי של קבצים לזיכרון מדומה של תהליך מאפשר להתייחס לפעולות קריאה וכתיבה לקובץ כאל פעולות עדכון או קריאה של אזורי זיכרון. הדבר מצריך את ביצועה של פעולת מיפוי הקובץ תחילה. לאחר קריאה או עדכון של אזורי הזיכרון שאליהם מופה הקובץ, יש לבצע פעולה של הסרת מיפוי, דבר שיגרום לעדכון של הקובץ בדיסק אם בוצעו פעולות של עדכון על אזורים ממופים.

המיפוי יכול להיות לא רק של הקובץ כולו אלא גם של חלקים ממנו. יתרה מזו, תהליכים אחדים יכולים למפות אזורים (שונים או זהים) של קובץ אל מרחב הזיכרון המדומה שלהם. באיור שלפניכם, למשל, קובץ הרצה ממופה לסגמנט ה-code של המרחב המדומה של תהליך, ושני תהליכים משתפים גישה לאזור בקובץ כלשהו.



איור 17

החיסרון של מיפוי קבצים לזיכרון מדומה מדופדף הוא שקשה לעקוב אחר הגודל האמיתי של הקבצים לאחר העדכון, מכיוון שהמיפוי לזיכרון מתבצע ביחידות של דפים, ולכן מערכת ההפעלה יכולה לספק רק גודל בכפולות של דפים.

האוניברסיטה הפתוחה
THE OPEN UNIVERSITY OF ISRAEL
الجامعة المفتوحة
Yair
Kurtzman

File #0006182 belongs to Yair Kurtzman- do not distribute

3.6 שיקולי יישום (מימוש)
3.6.1 מעורבותה של מערכת ההפעלה בתהליך הדפדוף
3.6.2 סיכום דרך הטיפול בשגיאות דף
3.6.3 שחזור הוראות מכונה לאחר פסיקת דף
3.6.4 נעילת דפים בזיכרון
3.6.5 אחסון בדיסק
3.6.6 הפרדה בין מדיניות למנגנונים

3.6 שיקולי יישום (מימוש)

מימוש של זיכרון וירטואלי קשור למספר גדול מאוד של בעיות מעשיות. בפרק זה נראה מקצת ההיבטים המעשיים הדורשים פתרון.

3.6.1 מעורבותה של מערכת ההפעלה בתהליך הדפדוף

ראינו שמערכת ההפעלה צריכה לעבוד בשיתוף פעולה הדוק עם החומרה כדי להשיג ביצועים נאותים בעת תרגום כתובות וירטואליות, ובחנו נושאים הקשורים למדיניות החלפת דפים. כעת נסכם את המקרים שדורשים התערבות של מערכת ההפעלה במשימות הקשורות לזיכרון וירטואלי:

1. יצירת תהליך

- מערכת ההפעלה קובעת לפי קובץ ההרצה של תהליך מה יהיה הגודל של טבלת דפים, יוצרת ומאתחלת אותה.
- מערכת ההפעלה מקצה מקום לתהליך באזור מיוחד בדיסק (מקום זה מכונה swapping area) שאליו יועתקו דפים מפונים, ומאתחלת אותו עם הקוד וה-data של התהליך. חלק מהמערכות אינן מאתחלות את ה-swapping area שלהן בקוד, כי הוא נמצא בלאו הכי בקובץ ההרצה.
- מערכת ההפעלה מעדכנת את טבלת התהליכים של המערכת.

2. בזמן החלפת תוכן של תהליכים

- מערכת ההפעלה צריכה לדאוג שה-MMU יכיל מידע רלוונטי לתהליך החדש.
- אם הדבר נמצא בשליטתה, מערכת ההפעלה צריכה לדאוג לכך שהפנייה לסוגים שונים של זיכרונות מטמון לא תגרום לפנייה למידע של תהליך ישן. בחלק מהמקרים זהו תפקיד החומרה אשר למערכת ההפעלה אין שליטה עליו.
- מערכת ההפעלה דואגת לבצע prepaging.

3. בזמן פסיקת דף

- מערכת ההפעלה מזהה איזו כתובת וירטואלית גרמה לפסיקת דף.

- לפי הכתובת הווירטואלית מחושבת כתובת בדיסק של הדף שנמצא ב- swapping area.
- מערכת ההפעלה צריכה למצוא מסגרת פנויה בזיכרון להבאת הדף הנדרש, ואם יש צורך, להפעיל אלגוריתם פינוי דפים.
- לאחר מציאת מסגרת פנויה, יש להביא את הדף לזיכרון הראשי.
- יש לשנות את התוכן של המעבד בחזרה לתוכן של התהליך שגרם לפסיקת דף.
- יש לתת למעבד להמשיך את הביצוע מההוראה שגרמה לפסיקת דף.

4. בסיום תהליך

- מערכת ההפעלה צריכה לשחרר את המשאבים: מסגרות זיכרון, טבלת דפים, אזור ב-swapping area. שחרור משאבים צריך, כמובן, לפסוח על דפים שמשותפים עם תהליכים אחרים, אם היו דפים כאלה.

3.6.2 סיכום דרך הטיפול בשגיאות דף

נציג ביתר פירוט את רצף הפעולות שמתרחשות עם פסיקת דף:

1. בעקבות פסיקת דף מתבצע trap לגרעין המערכת. המונה program counter נשמר על מחסנית.
2. שגרת קוד (שכתובה באסמבלר בשביל מהירות) שומרת רגיסטרים כלליים ומפעילה את שגרת הטיפול בפסיקות של מערכת ההפעלה.
3. בעקבות הפעלתה של השגרה לטיפול בפסיקות, מערכת ההפעלה מתחילה את הטיפול בפסיקת דף. היא שולפת הוראה שגרמה לפסיקת דף כדי לברר באיזה דף מדובר.
4. מערכת ההפעלה מבצעת בדיקת תקינות של הכתובת ובוחנת את ההרשאות של הדף. אם הבדיקות נכשלות, מערכת ההפעלה שולחת אות לתהליך או "הורגת" אותו, ומתבצעת החלפת תוכן לתהליך אחר.
5. אם הבדיקות מצליחות, מערכת ההפעלה בודקת אם קיימת מסגרת זיכרון פנויה. אם אין מסגרת זיכרון פנויה, מופעל אלגוריתם לפינוי דפים, ובמקרה הצורך – מתבצע תזמון כתיבה של דף מפונה. לאחר מכן התהליך שגרם לפסיקת דף מושהה עד לסיום הכתיבה[12] ומתבצעת החלפת תוכן לתהליך אחר. מסגרת זיכרון שנמצאת על ידי אלגוריתם לפינוי מסומנת כנעולה, דבר שלא מאפשר שימוש בה על ידי תהליכים אחרים.
6. אם מסגרת פנויה הייתה קיימת בשלב 5 או אם נמצאה מסגרת פנויה על ידי אלגוריתם להחלפת דפים, מתבצע חישוב של כתובת הדיסק של הדף שגרם לפסיקת דף, ולאחר מכן מתוזמנת בקשת קריאה של הדף מ-swapping area. התהליך מושהה עד לסיום הקריאה ומתבצעת החלפת תוכן לתהליך אחר.

[12] למעשה, התהליך יושהה ועם סיום הכתיבה תתרחש פסיקת חומרה. בעקבות פסיקת החומרה, מערכת ההפעלה תצטרך לבצע החלטת תזמון: אם לתזמן את התהליך שביקש את הכתיבה או לתזמן תהליך אחר.

האוניברסיטה הפתוחה THE OPEN UNIVERSITY OF ISRAEL الجامعة المفتوحة
Yair Kurtzman

File #0006182 belongs to Yair Kurtzman- do not distribute

7. עם סיום קריאת הדף, מתבצע עדכון של טבלת דפים ושחרור נעילה של מסגרת זיכרון.
8. ההוראה שגרמה לפסיקת דף משוחזרת.
9. מערכת ההפעלה מתזמנת את התהליך שגרם לפסיקה ומחזירה שליטה לשגרת האסמבלי המתוזמנת שקראה לה.
10. שגרת האסמבלי מבצעת שחזור אוגרים וחוזרת למצב משתמש להמשך ביצוע תהליכים.

3.6.3 שחזור הוראות מכונה לאחר פסיקת דף

מה קורה כאשר לא ניתן להשלים הוראת מכונה בגלל היעדר דף בזיכרון? ברור כי יש צורך לגלגל את הפעולה לאחור (חזרה מדויקת למצב שלפני ההוראה שנפסקה) ולהתחיל את הפעולה מחדש לאחר שהדף החסר כבר הובא לזיכרון. כדי לגלגל פעולה לאחור, מערכת ההפעלה משתמשת במידע שנשמר באוגרים מיוחדים של מעבד, שתפקידם לספק כמות נתונים מספקת לשחזור מצב המעבד שהיה לפני תחילת ביצועה של ההוראה. עם התרחשותה של פסיקת דף, מערכת ההפעלה שומרת את המידע שהיה באוגרים ומשתמשת בו כאשר הדף שהיעדרותו גרמה לפסיקה הובא לזיכרון.

3.6.4 נעילת דפים בזיכרון

הצורך בנעילה של דפים במסגרות זיכרון מתעורר כאשר התקן קלט/פלט (למשל הדיסק) מעביר נתונים לזיכרון או קורא נתונים מהזיכרון. הנעילה משמשת לצורך אינדיקציה למנגנון של קלט/פלט או אלגוריתמים להחלפת דפים, כי מסגרת הזיכרון לא נועדה לשימוש על ידי גורמים אחרים. בנעילת דפים בזיכרון משתמשים גם לדפים של גרעין מערכת ההפעלה.

פתרון אפשרי נוסף לשמירת תוכן דפים עד לגמר ההעברה להתקני קלט/פלט הוא שכל ההעברות ייעשו לחוצצים הנמצאים בדפים קבועים של מערכת ההפעלה, ורק לאחר מכן יועתקו הנתונים להתקני קלט/פלט.

שאלה 20

בשיטה האחרונה, בעיית סילוק הדף הנכתב איננה מתעוררת. הכיצד?

הבעיה איננה קיימת מפני ש:

א. החוצצים נמצאים על דפים שאינם מוחלפים, כי הם שייכים לדפי גרעין אשר נעולים בזיכרון.
ב. כאשר מועברים נתונים ממערכת ההפעלה אל שטחם של תהליכים אחרים, המערכת נמצאת ב***שליטה של הגרעין*** (***Kernel Mode***), ולפיכך יש לה בקרה מלאה על הרגע שבו יוחלף התהליך. כמו כן, הדף שאליו מועתקים הנתונים נמצא בדרישה מתמדת ולכן אין סכנה שיוחלף.

3.6.5 אחסון בדיסק

הסעיף "אחסון בדיסק" עוסק בנושא של ניהול שטחי דיסק שמוחזקים בהם דפים.

שיטת swapping area הבסיסית

בשיטה זו להחזקה של דפים מיועד להחזקתם אזור בדיסק המכונה swapping area. שטח האזור מנוהל באמצעות רשימה משורשרת של שטחים פנויים, בדומה לניהול של שטחים פנויים בשיטת מחיצות ניידות. לכל תהליך מוקצים שטחים רציפים כגודל של תהליך. לכן כדי להתייחס לדף מסוים, צריך רק לדעת את מיקומו של השטח המוקצה לתהליך בתוך ה-swapping area (ראו איור 3-28 (a)).

מודיפיקציה אחת של השיטה מנהלת אזורים נפרדים ל-code, data ו-stack של תהליכים כדי להתמודד טוב יותר עם שינויים בגודל של תהליכים.

השיטה הדינאמית

בשיטה זו, שטחי זיכרון בדיסק אינם מוקצים מראש. לעומת זאת, בכל פעם שצריך לאחסן דף בזיכרון משני מתבצעת הקצאה של שטח בעבור הדף. עם שיבוצם של דפים לזיכרון הראשי, מתבצע פינוי של השטח שהוקצה לדף בדיסק. בשיטה דינאמית ניהול שטחים הופך להיות משימה קלה, אך בתמורה צריך לשמור לכל דף כתובת מיפוי משלו (ראו איור 3-28(b)).

שאלה 21

מדוע במקרה של השיטה הדינאמית ניהול הופך למשימה קלה?

התשובה היא, שהשיטה הדינאמית הופכת את הניהול למשימה קלה מכיוון שמערכת ההפעלה יכולה להשתמש בבלוקים של מערכת קבצים לאחסון של דפים ולנהל את הבלוקים הללו ברשימה נפרדת.

3.6.6 הפרדה בין מדיניות למנגנונים

הרעיון של הפרדה בין מדיניות למנגנונים איננו חדש וכבר הכרנו אותו בסעיף 2.4.5. אמנם הפרדה כזאת, במקרה של מנגנון ניהול זיכרון, יכולה להיות תובענית בגלל הצורך בתמיכה מאסיבית של חומרה. לכן הפרדה בין מנגנון שחייב להיות בגרעין לבין מדיניות שאמורה להיות ממומשת ברמת משתמש תצריך רמה גבוהה של תיאום ביניהם וכתוצאה מכך תגרום לתקורה. עם זאת, הפרדה בין מנגנון למדיניות יוצרת גמישות רבה.

האוניברסיטה הפתוחה
THE OPEN UNIVERSITY OF ISRAEL
الجامعة المفتوحة
Yair Kurtzman

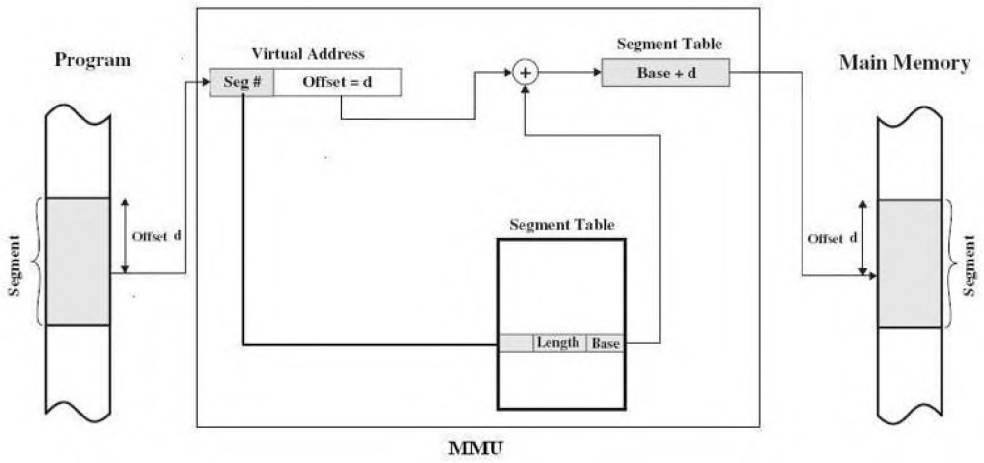
File #0006182 belongs to Yair Kurtzman- do not distribute

3.7 חלוקת הזיכרון לקטעים
3.7.1 יישום חלוקת הזיכרון לקטעים

3.7 חלוקת הזיכרון לקטעים

בסעיף זה נכיר שיטה נוספת לניהול הזיכרון, המבוססת על העיקרון של אחסון חלקי של תהליך בזיכרון המכונה שיטת חלוקת הזיכרון לקטעים (segmentation). כמו שיטת הדפדוף, שאותה למדנו בסעיף 3.3, גם שיטת חלוקת הזיכרון לקטעים מתאפיינת בכך שרק חלק התהליך שהוא חיוני לביצוע ברגע נתון חייב להימצא בזיכרון הראשי, וחלקים אחרים יכולים להיות מאוחסנים בדיסק. וכך גם השיטה הזאת מאפשרת הרצת תהליכים שגודלם עולה על גודל הזיכרון הפיזי במערכת.

בשיטה זו לניהול זיכרון, מרחב הזיכרון המדומה מחולק לקטעים. הקטעים יכולים להיות בגדלים שונים. הכתובת המדומה מורכבת משני חלקים: מספר הסגמנט וההיסט של יחידת ההתייחסות בסגמנט. מיפוי חלקי המרחב המדומה של תהליך לקטעים מוחזק במבנה נתונים המכונה טבלת קטעים (segment table). באיור שלפניכם ניתן לראות כיצד מתבצע תרגום של כתובת מדומה לכתובת פיזית בעזרת טבלת קטעים:



איור 18

בדומה לשיטות לחלוקת זיכרון באמצעות מחיצות משתנות, כל סגמנט של תהליך מאופיין בכתובת base, שהיא הכתובת הפיזית של תחילת הסגמנט בזיכרון הראשי, ובאורך של הסגמנט. לכן תרגום כתובת מדומה מתבצע על ידי חיבור של כתובת הבסיס להיסט.

שאלה 22

מה מתרחש כאשר בעת תרגום כתובת מדומה לכתובת פיזית, מתגלה כי ההיסט גדול מאורך הסגמנט?

יחד עם זאת, בניגוד לשיטת המחיצות המשתנות, הסגמנטים המרכיבים את התהליך אינם חייבים לשכון יחד בזיכרון הראשי. אם מתבצעת פנייה לכתובת זיכרון וירטואלית שהסגמנט המכיל אותה נמצא בזיכרון המשני, פנייה זו תגרום לפסיקת מעבד שבעקבותיה פונקציית הטיפול בפסיקות של מערכת ההפעלה תתזמן בקשה להביא את הסגמנט החסר לזיכרון הראשי. עם הגעת הסגמנט, מערכת ההפעלה תעדכן את טבלת הסגמנטים והתוכנית תמשיך לבצע את ההוראה שגרמה לפסיקה.

להבדיל משיטת הדפדוף, שיטת החלוקה לקטעים איננה שקופה בהכרח למתכנת, והיא נועדה לאפשר למתכנת להגדיר סגמנטים לפי צורכי האפליקציה. למשל, המהדרים למעבדים ממשפחת x86 מחלקים את התוכנית לסגמנטים, כמו שראינו באיור 5 בפרק זה, והחלוקה הזאת היא שקופה למתכנת.[13]

השוו בין התכונות של שיטת הדפדוף ושל שיטת חלוקת הזיכרון לקטעים, המופיעות בטבלה 3.32 בספר הלימוד.

3.7.1 יישום חלוקת הזיכרון לקטעים

החלוקה לקטעים שונים בגודלם יוצרת בעיה של שטחים מבוזבזים עקב הריסוק החיצוני. כבר נתקלנו בבעיה דומה בעת שתיארנו את חלוקת הזיכרון למחיצות משתנות. גם ראינו בסעיף 3.2 שניתן להתמודד עם בעיית הריסוק החיצוני באמצעות ציפוף זיכרון (memory compaction), אך העברת כמויות גדולות של נתונים בזיכרון היא פעולה יקרה.
בסעיפים 3.7.2 ו-3.7.3 בספר הלימוד מופעים תיאורים של מימוש השיטה המעורבת של דפדוף עם חלוקה לקטעים, שגם היא משמשת פתרון לבעיית הריסוק החיצוני.
סעיפים 3.7.2 ו-3.7.3 מתארים היבטים אחדים של מימוש השיטה המעורבת במערכת ההפעלה MULTICS ובמערכות הפעלה שרצות על מעבדי Pentium. החומר שבסעיפים הללו הוא בגדר רשות. במדריך הלמידה נתאר את הרעיון הכללי של השיטה המעורבת בלי להיכנס לפרטים של מערכת הפעלה כזו או אחרת או של מעבד כזה או אחר.

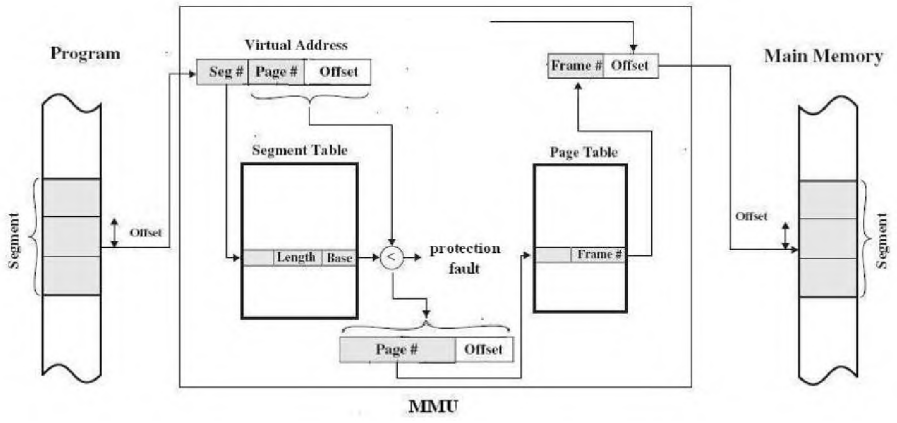
השיטה המעורבת: חלוקה לקטעים עם דפדוף

בשיטה המעורבת, מרחב הזיכרון המדומה של תהליך מחולק לקטעים לפי שיקוליו של המתכנת. כל הסגמנטים מחולקים לדפים בגודל זהה. גם הזיכרון הראשי מחולק למסגרות זיכרון בגודל זהה לגודל הדפים. הכתובת המדומה מחולקת לשני שדות, אשר הראשון מביניהם הוא מספר הסגמנט בטבלת הסגמנטים. המיפוי של חלקי המרחב המדומה של תהליך לסגמנטים מוחזק בטבלת הסגמנטים. בנוסף, לכל סגמנט יש טבלת דפים המכילה את המיפוי של הדפים אשר מרכיבים את הסגמנט אל מסגרות הזיכרון הראשי. הכתובת המדומה מתורגמת בעזרת טבלת הסגמנטים לכתובת המדומה של הזיכרון המדופדף, והכתובת המדומה של הזיכרון המדופדף מתורגמת לכתובת הפיזית.
באיור שלפניכם ניתן לראות כיצד מתבצע תרגום של כתובת מדומה לכתובת פיזית:

[13] למעשה, ברוב התוכניות המהדר ימפה את כל הסגמנטים הלוגיים לסגמנט פיזי יחיד.

האוניברסיטה הפתוחה THE OPEN UNIVERSITY OF ISRAEL الجامعة المفتوحة
Yair Kurtzman

File #0006182 belongs to Yair Kurtzman- do not distribute



איור 19

שימו לב כי כל סגמנט של תהליך מאופיין בכתובת base, שהיא הכתובת של הסגמנט במרחב הזיכרון המדומה. השוו זאת לתרגום כתובות במקרה של שיטה טהורה של חלוקה לקטעים (ללא דפדוף).

השיטה המעורבת מתאפיינת בהיעדר ריסוק חיצוני והיא גם מאפשרת שליטה של המתכנת על חלוקת מרחב הזיכרון של תהליך לקטעים, דבר שלא היה אפשרי בשיטות הדפדוף.

שאלה 23

האם ניתן לבצע חלוקה של מרחב הזיכרון של תהליך לקטעים בשפה עילית כמו C?

סעיפים שאינם מופיעים בספר הלימוד	
3.8	סיכום
3.9	בעיות פתורות
3.10	שאלות לחזרה
3.11	תרגילי תכנות לתרגול עצמי
3.12	רשימת מושגים

3.8 סיכום

בפרק זה למדנו את המבנה של מערכות שונות לניהול הזיכרון. נמנה אותן שוב:

- ניהול זיכרון ללא החלפה עם תהליך יחיד. לדוגמה, OS/360 בשלביה הראשונים (בשנת 1965).
- ניהול זיכרון באמצעות מחיצות קבועות או ניידות ומרחב זיכרון מדומה. מנגנון ההחלפה בשילוב עם מחיצות ניידות מאפשר טיפול במספר משתנה של תהליכים בו-זמנית. הגדלה של דרגת ריבוי התוכניות משפרת לעתים קרובות את נצילות המעבד. לדוגמה, מערכת הפעלה OS/360 בשלבים מתקדמים יותר או מערכות משובצות של ימינו.

- הזיכרון הווירטואלי פורץ את גבולות הזיכרון הפיזי.
- מנגנון הדפדוף דורש קביעת מדיניות להחלפת דפים. ראינו יתרונות וחסרונות של מדיניות הקצאה גלובלית ומדיניות הקצאה לוקאלית. סקרנו כמה אלגוריתמים להחלפת דפים, ביניהם האלגוריתם האופטימלי, LRU, NRU, אלגוריתם השעון ומנגנון ההזדקנות.
- ראינו את חשיבותה המכרעת של תמיכת חומרה בתהליך של תרגום מענים וירטואליים והצבענו על ההיררכיה של הזיכרון.
- הכרנו את שיטת חלוקת הזיכרון לקטעים עצמאיים. הזיכרון המקוטע מאפשר מודולריות רבה בבניית תוכניות, בשיתוף דינאמי של ספריות ובהגנה סלקטיבית על קטעי זיכרון שונים.

3.9 בעיות פתורות

שאלה 1

הראו כי בשיטת ניהול הזיכרון באמצעות מחיצות משתנות, מספר שטחי הזיכרון הרציפים התפוסים הוא, בממוצע, גבוה פי שניים ממספר השטחים הפנויים הרציפים.

תשובה 1

נתבונן בשטח זיכרון תפוס. ההסתברות לכך שמעבר לכתובת העליונה של השטח הזה ישנו שטח פנוי היא 1/2. לכן אם כמות השטחים התפוסים היא n, אזי כמות השטחים הפנויים תהיה n/2.

שאלה 2

נתון כי במערכת מחשב

- זמן הגישה לזיכרון מטמון הוא 20 ns (ננו-שנייה היא 10^-9 שניות).
- זמן הגישה לזיכרון הראשי הוא 60 ns.
- זמן הגישה לזיכרון המשני הוא 12 ms (מילי-שנייה היא 10^-3 שניות).
- יחס ההצלחה של הפנייה לזיכרון מטמון הוא 90%.
- יחס ההצלחה של הפנייה לזיכרון הראשי הוא 60%.

מהו זמן הגישה הממוצע?

תשובה 2

- בהסתברות של 90%, זמן הגישה יהיה זמן גישה לזיכרון מטמון שהוא 20 ns.
- בהסתברות של 6 % (0.6 * 0.1), זמן הגישה יהיה זמן הגישה לזיכרון המטמון בתוספת של זמן הגישה לזיכרון הראשי שהוא 80 ns (20 ns + 60 ns).

האוניברסיטה הפתוחה THE OPEN UNIVERSITY OF ISRAEL الجامعة المفتوحة
Yair Kurtzman

File #0006182 belongs to Yair Kurtzman- do not distribute

- בהסתברות של 4 % (0.1 * 0.6), זמן הגישה יהיה זמן הגישה לזיכרון מטמון בתוספת זמן הגישה לזיכרון הראשי ובתוספת זמן הגישה לזיכרון המשני, שהוא 12000080 ns (12 ms + 20 ns + 60 ns).

מכאן שזמן הגישה המשוקלל יהיה:

T = 0.9 * 20 ns + 0.06 * 80 ns + 0.04 * 12000080 ns = 480026 ns

---

שאלה 3

נתון כי גודל הדף במערכת ניהול זיכרון מדופדף הוא 1 kbyte ויחידת ההתייחסות הקטנה ביותר היא בית אחד (1 byte). מהי הכתובת הפיזית המתאימה לכתובת המדומה 1097 (בייצוג עשרוני) כאשר נתון כי הדף מספר 1 ממופה למסגרת מספר 6?

תשובה 3

בייצוג בינארי הכתובת 1097 = 2^10+73 תהיה 10001001001, כאשר עשר הסיביות הימניות ביותר מייצגות את ההיסט בתוך הדף. לכן הכתובת הפיזית תהיה 1100001001001. שימו לב שעשר הסיביות הימניות ביותר הן זהות (ראו איור 8 בפרק). בייצוג עשרוני הכתובת הפיזית תהיה: 73 + 2^11 + 2^12 = 6217.

שאלה 4

נתונה מחרוזת התייחסויות של תהליך מסוים (משמאל לימין):

1 0 2 2 1 7 6 7 0 1 2 0 3 0 4 5 1 5 2 4 5 6 7 6 7 2 4 2 7 3 3 2 3

הראו כי האלגוריתמים LRU ו-FIFO שקולים מבחינת מספר פסיקות דף על מחרוזת ההתייחסויות הספציפית. הניחו כי מספר המסגרות בזיכרון הפיזי הוא 4 וכי המסגרות ריקות.

תשובה 4

ביצוע של LRU גורם ל-17 פסיקות דף:

```
1 0 2 2 1 7 6 7 0 1 2 0 3 0 4 5 1 5 2 4 5 6 7 6 7 2 4 2 7 3 3 2 3

1 1 1 1 1 1 1 1 1 1 1 1 1 1 4 4 4 4 4 4 4 4 4 4 4 2 2 2 2 2 2 2 2
- 0 0 0 0 0 6 6 6 6 2 2 2 2 2 5 5 5 5 5 5 5 5 5 5 4 4 4 4 4 4 4
- - 2 2 2 2 2 0 0 0 0 0 0 0 0 0 0 2 2 2 2 7 7 7 7 7 7 7 7 7 7 7
- - - - - 7 7 7 7 7 7 3 3 3 3 1 1 1 1 1 6 6 6 6 6 6 6 3 3 3 3

F F F     F F   F   F   F   F F F   F     F F     F F     F
```

וגם ביצוע של FIFO גורם ל-17 פסיקות דף:

```
 1 0 2 2 1 7 6 7 0 1 2 0 3 0 4 5 1 5 2 4 5 6 7 6 7 2 4 2 7 3 3 2 3

 1 1 1 1 1 1 6 6 6 6 6 6 6 6 4 4 4 4 4 4 4 6 6 6 6 6 6 6 6 6 6 2 2
 - 0 0 0 0 0 0 0 0 1 1 1 1 1 5 5 5 5 5 5 7 7 7 7 7 7 7 7 7 7 7 7
 - - 2 2 2 2 2 2 2 2 2 0 0 0 0 0 1 1 1 1 1 1 1 1 1 4 4 4 4 4 4 4
 - - - - - 7 7 7 7 7 7 3 3 3 3 3 3 2 2 2 2 2 2 2 2 2 3 3 3 3
 F F F     F F     F   F F   F F F   F     F F       F     F   F
```

שאלה 5

נתון כי במערכת ניהול זיכרון מעורבת המשתמשת בדפדוף ובחלוקה לקטעים:
מרחב זיכרון מדומה של תהליך מחולק ל-4 סגמנטים בגודל זהה.

- הסגמנטים אינם חופפים.
- הכתובת המדומה היא בעלת 32 סיביות.
- גודל דף הוא 4 kbyte.
- אורך מילת זיכרון (יחידת התייחסות מינימלית) הוא 1 byte.

מהי כמות הדפים שאליה מתחלק כל סגמנט?

תשובה 5

גודל מרחב הזיכרון המדומה הוא 1 Byte * 2^32 = 4 Gbyte.
מכיוון שהסגמנטים אינם חופפים, הגודל של כל סגמנט הוא 1 Gbyte.
לכן מספר הדפים בכל סגמנט הוא:
1Gbyte/4kbyte = 2^20 kbytes/ 2^2 kbytes = 2^18 pages

שאלה 6

בנתונים של השאלה הקודמת (שאלה 5), מהו מספר שכבות הדפדוף הדרושות כדי שטבלת הדפים בכל רמה תיכנס לדף אחד, בהינתן שגודל הכניסה של טבלת דפים (בכל רמה) הוא 4 bytes?

תשובה 6

בהינתן שמספר הדפים במרחב הזיכרון הווירטואלי הוא:
2^32 * 1 byte / 4 Kb = 2^32 bytes / 2^ 12 bytes = 2^ 20 pages
כמות הזיכרון הדרושה לכניסות של טבלאות דפים ברמה האחרונה היא:
2^20 * 4 bytes = 2 ^22 bytes
מכאן שמספר הדפים שעליהם תתפרס טבלת הדפים יהיה:
2^22 bytes / 4 kbytes = 2^22 bytes/2^12bytes = 2^10 pages
ולכן נדרשות 10 סיביות לבחירה של הדף שבו נמצא חלק של טבלת הדפים – החלק של טבלת הדפים המכיל 4 Kb / 4 bytes = 2 ^ 10 משורותיה. כך נוכל להצביע בדיוק ל:
2 ^ 20 = 2^10 * 2^ 10
דפים של מרחב הזיכרון הווירטואלי כולו. ומכיוון שהסגמנטים אינם חופפים, מספר שכבות הדפדוף הנדרש הוא 2.

3.10 שאלות לחזרה

אילו מבין הטענות שלהלן הן טענות נכונות?

1. ניידות של תהליכים דרושה רק בשיטת המחיצות הקבועות, ולא בשיטת המחיצות הניידות.

האוניברסיטה הפתוחה THE OPEN UNIVERSITY OF ISRAEL الجامعة المفتوحة
Yair Kurtzman

File #0006182 belongs to Yair Kurtzman- do not distribute

2. מנגנון לתרגום מענים מספק הגנה על שטחי הזיכרון של התהליכים ושל מערכת ההפעלה.
3. שיטת הקצאת הזיכרון באמצעות מחיצות קבועות גורמת לריסוק פנימי.
4. שיטת הקצאת הזיכרון במחיצות ניידות שמשתמשת במפת סיביות איננה גורמת לריסוק חיצוני.
5. למימוש של שיטת הקצאת הזיכרון באמצעות מחיצות ניידות נדרשת תמיכה של חומרה לביצוע של תרגום מענים.
6. TLB הוא סוג של זיכרון מטמון ששומר בתוכו חלק משורותיה של טבלת דפים או טבלאות דפים.
7. טבלת דפים מהופכת מכילה מיפוי של מסגרות הזיכרון הפיזי לדפים של תהליכי מערכת.
8. אלגוריתם FIFO להחלפת דפים הוא גרסה פרטית של אלגוריתם הזדמנות שנייה.
9. מדיניות לוקאלית להקצאת מסגרות זיכרון גורמת לסחרור.
10. למחרוזת מרחק של תהליך יש משמעות רק אם אלגוריתם להחלפת דפים הוא מסוג אלגוריתם מחסנית.
11. לפי מחרוזת התייחסויות של תהליך ניתן לחזות כיצד תשפיע הגדלת ההקצאה של מסגרות פיזיות בעבור התהליך על מספר פסיקות דף.
12. ניהול באמצעות שיטת סגמנטציה טהורה דורש את מודעותו של המתכנת לשיטת ניהול הזיכרון של מערכת ההפעלה.

3.11 תרגילי תכנות לתרגול עצמי

1. כתבו תוכנית אשר תדגים את השפעת ה-cache על מהירות הריצה בכפל של מטריצות. תחילה ממשו כפל מטריצות לפי הגדרה. בשלב שני חשבו כיצד לבצע כפל מטריצות המנצל את הלוקאליות ב-cache והשוו בין שני המימושים.

2. כתבו תוכנית שמנהלת שטח זיכרון שחלקים רציפים ממנו מוקצים בצורה דינאמית. שיטת הניהול תהיה באמצעות מפת סיביות. ממשו שגרה לחיפוש שטח רציף במפת סיביות.

3.12 רשימת מושגים

להלן רשימת המושגים המופיעים בפרק זה (על פי סדר הופעתם בפרק):

ריבוי תוכניות להרצה	multiprogramming	שיטה המאפשרת הימצאות בו-זמנית של יותר מתהליך אחד בזיכרון. מימושים שונים של השיטה מאפשרים לייעל את ניצול משאבי המערכת ו/או לספק שירות בזמן סביר לכמה משתמשים אינטראקטיביים על ידי כך שכאשר תהליך אחד ממתין לפעולת קלט/פלט, תהליך אחר יכול לרוץ במעבד.
מחיצות קבועות	fixed partitions	מימוש של multiprogramming שבו הזיכרון מחולק באופן קבוע לכמה מחיצות, כל אחת מהן בגודל קבוע (לאו דווקא כולן באותו גודל). בכל מחיצה ניתן להחזיק תהליך אחד, שגודלו אינו עולה על גודל המחיצה.
מחיצות ניידות	variable partitions	מימוש של multiprogramming שבו הזיכרון מחולק דינאמית לכמה מחיצות. מספרן, מיקומן וגודלן של המחיצות משתנים בלי הרף במהלך עבודת המערכת. כאשר תהליך חדש נכנס למערכת או תהליך מוחזר מהדיסק אל הזיכרון, מערכת ההפעלה מוצאת שטח פנוי בזיכרון שיכול להכיל את התהליך, ומקצה לו מחיצה בהתאם לגודלו (כלומר, חלק מהשטח הפנוי מוקצה לטובת המחיצה, והחלק הנותר נשאר פנוי).
הזזה בזיכרון	relocation	אפשרות שתהליך ימוקם באזורים שונים בזיכרון בהרצות שונות. האפשרות הזאת יוצרת בעיה כיוון שהוראות קוד התוכנית מתייחסות לכתובות אבסולוטיות (בעיית ה-relocation). כאשר הוראת קוד מתייחסת לכתובת כלשהי בזיכרון, צריך לחשב את הכתובת האמיתית שאליה יש לפנות, בהתאם למיקומו של התהליך בזיכרון בהרצה הנוכחית.
החלפה בזיכרון	swapping	פינוי זמני של תהליך מהזיכרון אל הדיסק והבאת תהליך מהדיסק בחזרה אל הזיכרון (השמירה נעשית בשטח ייעודי של הדיסק, שמוקצה לצורך זה).
ריסוק פנימי	internal fragmentation	תופעת בזבוז של זיכרון הנגרמת כתוצאה מהקצאת זיכרון עודפת לתהליך (כלומר, תהליך מקבל מחיצה גדולה מזו שהוא באמת צריך). תופעה זו קיימת כאשר גודל המחיצות קבוע, או כאשר גודל המחיצה חייב להיות כפולה של יחידת הקצאה מינימלית (למשל, 1k). באופן דומה, תופעת הריסוק הפנימי קיימת גם בהקשר של ניצול שטח הדיסק.
ריסוק חיצוני	external fragmentation OR checkerboarding	תופעת בזבוז של זיכרון שמהותה היווצרות שטחי זיכרון פנויים בין השטחים שמוקצים לתהליכים, כאשר השטחים הפנויים קטנים מכדי להכיל תהליך כלשהו ולפיכך אינם מנוצלים.
ציפוף של הזיכרון	memory compaction	העתקה של תהליכים שמפוזרים בשטחים שונים של הזיכרון כך שהם ימוקמו בצורה קומפקטית, בזה אחר זה. פעולה זו מאפשרת ניצול טוב יותר של

האוניברסיטה הפתוחה THE OPEN UNIVERSITY OF ISRAEL الجامعة المفتوحة
Yair Kurtzman

File #0006182 belongs to Yair Kurtzman- do not distribute

		הזיכרון, כיוון שבשיטת המחיצות הניידות נוצרים בזיכרון הרבה חורים (שטחים פנויים בין מחיצות התפוסות על ידי תהליכים); זה גורם לריסוק חיצוני ומצריך חיפוש מורכב בכל הקצאה של זיכרון. מאידך גיסא, פעולת ציפוף הזיכרון היא עצמה פעולה יקרה, ולכן על פי רוב לא משתמשים בה.
מפת סיביות לניהול זיכרון	bits map	שיטת ייצוג זיכרון שבה כל יחידת זיכרון (יחידת הקצאה מינימלית) מיוצגת על ידי סיבית שמסמנת אם היחידה פנויה או תפוסה על ידי תהליך כלשהו. כאשר תהליך מסתיים, הסיביות המתאימות ליחידות הזיכרון שהתהליך תפס מתעדכנות בהתאם. כאשר צריך להקצות זיכרון לתהליך חדש שגודלו N יחידות, מבצעים חיפוש במפת הסיביות עד שמוצאים N יחידות פנויות רציפות.
רשימות מקושרות לניהול זיכרון	linked lists	שיטת ייצוג זיכרון שבה כל קטע זיכרון (תפוס או פנוי) מיוצג על ידי איבר ברשימה מקושרת. כל איבר ברשימה מסומן P (תהליך) או H (חור). הרשימה ממוינת לפי כתובת תחילת הקטע. כאשר תהליך מסתיים, האיבר המתאים ברשימה הופך מ-P ל-H, ואם אחד או שניים משכניו של האיבר גם הם מסוג H, האיברים האלה מתאחדים (כלומר, אם יש ברשימה שניים או שלושה חורים סמוכים, הם הופכים לחור גדול אחד). כאשר צריך להקצות זיכרון לתהליך חדש, מוצאים את החור המתאים לפי אחד האלגוריתמים להקצאת שטחי זיכרון. אם החור גדול מהנדרש, האיבר המתאים ברשימה מתפצל לשני איברים – P שיכיל את התהליך ו-H שיכיל את שארית המקום הפנוי. הערה: קיימים מימושים שונים של שיטת ניהול זיכרון זו (לדוגמה, מוחזקות שתי רשימות נפרדות: האחת – בעבור שטחי זיכרון פנויים, השנייה – בעבור שטחי זיכרון התפוסים על ידי תהליכים).
אלגוריתמים להקצאה של שטחי זיכרון	First Fit, Next Fit, Best Fit, Worst Fit, Quick Fit	אלו הם חמשת האלגוריתמים להקצאת שטחי זיכרון שמשתמשים בהם בשיטת הרשימות המקושרות: First Fit – מוצאים את החור הראשון ברשימה שהוא מספיק גדול כדי להכיל את התהליך. Next Fit – כמו First Fit, אך החיפוש מתחיל לא מתחילת הרשימה, אלא מהמקום שבו נעצר בפעם הקודמת. Best Fit – מוצאים את החור הקטן ביותר ברשימה שהוא מספיק גדול כדי להכיל את התהליך. Worst Fit – מוצאים את החור הגדול ביותר ברשימה. הערה: בארבע השיטות האלו ניתן להחזיק רשימות נפרדות לתהליכים ולחורים (החורים ממוינים לפי הגודל ולא לפי הכתובת), ובכך לייעל את הקצאת הזיכרון במחיר של סיבוך נוסף בשחרור הזיכרון (כי איתור החורים הסמוכים הופך לפעולה יקרה).

		מחזיקים רשימות נפרדות בעבור Quick Fit – חורים בגדלים נפוצים שונים (למשל, רשימה של חורים בגודל 4k, 16k וכן הלאה). בשיטה זו ההקצאה היא מהירה ופשוטה, אך איחוד החורים ואחזקת מבני הנתונים מסובכים יותר.
שיטת הזוגות לניהול זיכרון	Buddy System	בשיטה זו, הקצאת הזיכרון נעשית רק ביחידות בגודל חזקה של 2 (1, 2, 4, 8, ...). לתהליך מקצים שטח זיכרון בגודל חזקה של 2, שהוא הזיכרון הקטן ביותר שיכול להכיל את התהליך. ישנן רשימות נפרדות ליחידות בכל גודל. אם צריכים להקצות יחידה בגודל X (X – חזקה של 2) וברשימה של שטחים בגודל X אין יחידה פנויה, לוקחים יחידה בגודל 2X, מפצלים אותה לשניים, מקצים את היחידה הראשונה שהתקבלה לתהליך, והיחידה השנייה בגודל X נשארת פנויה (כאשר שתי היחידות יהיו מעתה ברשימה של שטחים בגודל X). אם אין יחידה פנויה בגודל 2X, מחכים. כאשר תהליך שתפס יחידה בגודל X מפנה את הזיכרון, אם ישנה יחידה פנויה סמוכה אליה בגודל X, מאחדים את שתי היחידות אם ורק אם היחידה הראשונה מביניהן מתחילה בכתובת שהיא כפולה של 2X. היחידות שמתקבלות מאיחוד או מפיצול של יחידות אחרות ממוקמות תמיד ברשימות שמתאימות לגודלן הנוכחי. בשיטה זו ניהול הזיכרון הוא מהיר, אך ניצול הזיכרון אינו אופטימלי בשל ריסוק פנימי.
זיכרון מדומה	virtual memory	שיטת ניהול זיכרון המבוססת על העיקרון של אחסון חלקי של תהליך בזיכרון, כאשר רק החלק שחיוני לביצוע ברגע נתון חייב להימצא בזיכרון הראשי, וחלקים אחרים של התהליך יכולים להיות מאוחסנים בדיסק. שיטה זו מאפשרת הרצת תהליכים שגודלם עולה על גודל הזיכרון הפיזי במערכת.
שיטת הדפדוף	paging	שיטה נפוצה למימוש הזיכרון המדומה. בשיטה זו כל תהליך עובד במרחב זיכרון מדומה בגודל 2^N (N – אורך הכתובת במערכת, למערכות שונות יכול להיות אורך כתובות שונה). מרחב הזיכרון המדומה מחולק לדפים בגודל K (K יכול להיות שונה במערכות שונות). כל דף של הזיכרון המדומה יכול להימצא בזיכרון הראשי או בדיסק. אם התהליך פונה לכתובת אשר שייכת לדף שאינו בזיכרון, הדף מובא מהדיסק אל הזיכרון. דף שנמצא בזיכרון אך אינו בשימוש כרגע יכול להיות מועבר על ידי המערכת לדיסק.
מסגרת בזיכרון	page frame	יחידה של הזיכרון הפיזי שמשמשת לאחסון דף זיכרון אחד במערכת מבוססת דפדוף. גודל המסגרת זהה תמיד לגודל הדף בזיכרון המדומה.
יחידת ניהול הזיכרון (מנגנון תרגום הכתובות)	MMU, Memory Management Unit	רכיב חומרה אשר אחראי על תרגום כתובות וירטואליות לכתובות פיזיות במערכות מבוססות זיכרון וירטואלי. במערכות מסוג זה, תהליך שפונה לזיכרון פונה תמיד לכתובת וירטואלית. פנייה זו

האוניברסיטה הפתוחה
THE OPEN UNIVERSITY OF ISRAEL
الجامعة المفتوحة
Yair
Kurtzman

File #0006182 belongs to Yair Kurtzman- do not distribute

		עוברת דרך MMU, אשר מתרגם את הכתובת הווירטואלית לכתובת הפיזית המתאימה. אם הדף הווירטואלי הדרוש אינו נמצא בזיכרון הפיזי ברגע הפנייה, MMU מוציא פסיקה הנקראת פסיקת דף ובכך מודיע ל-OS על הצורך להביא את הדף.
טבלת הדפים	page table	טבלה שמשמשת למיפוי של דפי הזיכרון הווירטואלי למסגרות פיזיות. מספר הכניסות בטבלה הוא כמספר הדפים בזיכרון הווירטואלי, כניסה מספר N בטבלה מתאימה לדף מספר N. כל כניסה בטבלת הדפים מכילה את השדות האלה: מספר המסגרת (page frame) שבה מאוחסן הדף הווירטואלי המתאים, סיבית הנוכחות, סיבית ההתייחסות, סיבית השינוי, שדה ההגנה וסיבית הסימון לזיכרון המטמון. המבנה המדויק של כניסה בטבלת הדפים הוא תלוי מערכת ויכול להכיל שדות נוספים מעבר לאלה שנמנו לעיל. במערכות מבוססות דפדוף לכל תהליך יש טבלת דפים משלו, וכאשר התהליך הפעיל מתחלף, ה-MMU עובר לעבוד עם הטבלה של התהליך הנוכחי. המימוש של טבלת הדפים בחומרה/תוכנה שונה ממערכת למערכת.
טבלת דפים שכבתית	multilevel page table	שיטת ארגון של טבלת הדפים המאפשרת לא להחזיק בזיכרון באופן קבוע את כל הטבלה. בשיטה זו ישנה טבלה מרמה גבוהה שכל כניסה שלה מצביעה לטבלה מרמה נמוכה יותר, וכניסה של טבלה מהרמה הנמוכה ביותר מכילה מיפוי של הדף המבוקש למסגרת בזיכרון הפיזי.
סיבית הנוכחות	present/absent bit	אחד השדות של כניסה בטבלת דפים. הסיבית מסמנת אם הדף שמיוצג על ידי הכניסה נמצא בזיכרון הפיזי.
סיבית ההתייחסות	reference bit	אחד השדות של כניסה בטבלת דפים. הסיבית מסמנת אם הייתה פנייה לדף לצורך כתיבה או קריאה.
סיבית השינוי	modified bit	אחד השדות של כניסה בטבלת דפים. הסיבית מסמנת אם הייתה פנייה לדף לצורך כתיבה (אם כן – תוכן הדף צריך להישמר על גבי הדיסק, לפני שהדף יפונה מהזיכרון).
סיבית הסימון לזיכרון מטמון	caching disabled/enabled	אחד השדות של כניסה בטבלת דפים. הסיבית מאפשרת למנוע את שמירת הדף בזיכרון המטמון.
שדה ההגנה	protection field	אחד השדות של כניסה בטבלת דפים. השדה אומר מהם סוגי הגישה המורשים לדף (כתיבה, קריאה, ביצוע).
זיכרון אסוציאטיבי	associative memory, translation lookaside buffer	רכיב חומרה מהיר (בדרך כלל בתוך ה-MMU), שמכיל אוסף מצומצם של עותקים של כניסות מטבלת הדפים. בזיכרון האסוציאטיבי יישמרו עותקים של הכניסות המתאימות לדפים שהם בשימוש אינטנסיבי ברגע נתון (כלומר, אלה שהוראות הקוד האחרונות התייחסו אליהם).

		הרכיב מאפשר גישה מהירה לדפים אלה, כי המיפוי שלהם נעשה ללא צורך בפנייה לטבלת הדפים.
יחס ההצלחה	hit ratio	אחוז הפניות לזיכרון שניתן לבצע ללא שימוש בטבלת הדפים אלא באמצעות הזיכרון האסוציאטיבי.
טבלת דפים מהופכת	inverted page table	טבלה הממפה מסגרות זיכרון לדפים וירטואליים במרחבי זיכרון של תהליכים שונים. להבדיל מטבלאות דפים רגילות, שהן ייחודיות לכל תהליך, הטבלה המהופכת היא יחידה במערכת ומשותפת לכל התהליכים. מספר הכניסות בטבלה הוא כמספר מסגרות הדף בזיכרון הפיזי, וכל כניסה מכילה מספר דף וירטואלי ואת מספר התהליך שאליו שייך הדף. השימוש בטבלה מהופכת משולב תמיד בשימוש בזיכרון אסוציאטיבי.
פסיקת דף חסר בזיכרון	page fault	פסיקה שה-MMU מוציאה כאשר מתבצעת פנייה לדף וירטואלי שאינו נמצא בזיכרון הפיזי. כתוצאה מהפסיקה, מופעלת פרוצדורה של מערכת ההפעלה שמביאה את הדף החסר מהדיסק ואז מחזירה את התהליך שגרם לפסיקה לרוץ מהמקום שבו הוא הופסק.
האלגוריתם האופטימלי להחלפת דפים	the optimal page replacement algorithm	אלגוריתם אשר בוחר לפנות מהזיכרון את הדף אשר לא תהיה שום פנייה אליו במשך הזמן הרב ביותר. אלגוריתם זה אינו ניתן למימוש, מאחר שהוא מסתמך על מידע עתידי שאינו זמין ברגע קבלת ההחלטה.
אלגוריתם להחלפת דפים NRU	NRU, Not Recently Used	אלגוריתם זה מחלק את כל הדפים בזיכרון הפיזי לארבע מחלקות, לפי הערכים של סיביות השינוי (M) וההתייחסות (R). דף אקראי מהמחלקה הלא ריקה בעלת המספר הסידורי הקטן ביותר מפונה מהזיכרון. המחלקות הן: R=0, M=0 : 0 R=0, M=1 : 1 R=1, M=0 : 2 R=1, M=1 : 3 אלגוריתם זה הוא אלגוריתם מבוסס מחסנית.
אלגוריתם להחלפת דפים FIFO	FIFO – First In First Out	אלגוריתם זה בוחר לפנות את הדף שנמצא בזיכרון הכי הרבה זמן. לצורך זה מערכת ההפעלה מחזיקה תור (FIFO) של כל הדפים שנמצאים בזיכרון. אלגוריתם זה אינו אלגוריתם מבוסס מחסנית והוא סובל מאנומליה של Belady.
אלגוריתם להחלפת דפים Second Chance	Second Chance algorithm	אלגוריתם זה דומה לאלגוריתם FIFO. ההבדל הוא בכך שהדף שבראש התור אינו מפונה אם ערך הסיבית R שלו הוא 1, אלא הסיבית מתאפסת, הדף מועבר לסוף הרשימה והדף הבא בתור נבדק. אלגוריתם זה אינו אלגוריתם מבוסס מחסנית והוא סובל מאנומליה של Belady.
אלגוריתם להחלפת דפים Clock	Clock algorithm	אלגוריתם זה שונה מהאלגוריתם Second Chance רק במימוש: הדפים נמצאים ברשימה מעגלית, וכאשר דף חדש מובא לזיכרון, הוא תופס

האוניברסיטה הפתוחה THE OPEN UNIVERSITY OF ISRAEL الجامعة المفتوحة
Yair Kurtzman

File #0006182 belongs to Yair Kurtzman- do not distribute

		את מקומו ברשימה של הדף שהוצא מהזיכרון, והמצביע לראש התור מתקדם לדף הבא ברשימה. אלגוריתם זה אינו אלגוריתם מבוסס מחסנית והוא סובל מאנומליה של Belady.
אלגוריתם להחלפת דפים LRU	LRU, Least Recently Used	אלגוריתם זה מבוסס על ההנחה כי הדפים שהיו בשימוש אינטנסיבי לאחרונה יהיו גם בשימוש של הוראות הקוד הבאות, ולפיכך אין לפנות אותם מהזיכרון. מימושו של האלגוריתם דורש חומרה מיוחדת שמספקת מדדים לתדירות השימוש בכל דף (דוגמאות של חומרה כזו מובאות בספר הלימוד).
אלגוריתם להחלפת דפים NFU	NFU, Not Frequently Used	אלגוריתם זה הוא קירוב של LRU בתוכנה. האלגוריתם מבוסס על מונה מוצמד לכל דף. בכל פעימת שעון, הערך של סיבית ההתייחסות מוסף למונה, והסיבית מתאפסת.
אלגוריתם להחלפת דפים Aging NFU	Aging NFU algorithm	אלגוריתם זה שונה מ-NFU בכך שבכל פעימת שעון, המונה של כל דף מוזז סיבית אחת ימינה, וערך סיבית ההתייחסות מוסף לסיבית השמאלית ביותר של המונה.
אלגוריתמים מבוססי מחסנית	stack algorithms	משפחה של אלגוריתמים להחלפת דפים שמקיימים את התנאי המופיע בעמ' 117 בספר הלימוד. אלגוריתמים מסוג זה אינם סובלים מאנומליה של Belady המבוססים מיון מחסנית.
מחרוזת ההתייחסות	reference string	סדרה שמייצגת את רצף הפניות לזיכרון של תהליך. כל פנייה מיוצגת על ידי מספר של דף וירטואלי שאליו מתייחסת הפנייה.
מחרוזת המרחק	distance string	סדרה שמייצגת את רצף הפניות לזיכרון של תהליך. כל פנייה מיוצגת על ידי מרחקו מראש המערך של הדף הווירטואלי שאליו מתייחסת הפנייה (המערך מייצג את מצב הזיכרון במודל).
קבוצת העבודה	working set	קבוצת דפי הזיכרון הנחוצים לריצת התהליך בשלב הנוכחי של הביצוע. קבוצה זו בדרך כלל משתנה במהלך ריצת התהליך.
עקרון המקומיות	locality of reference	עיקרון זה אומר כי בכל שלב של ביצוע תהליך, התהליך פונה לקבוצה מצומצמת של דפי הזיכרון, והפניות אינן מפוזרות על פני כל מרחב הזיכרון של התהליך.
סחרור	thrashing	תופעה במערכות זיכרון וירטואלי, שבה תהליך גורם לפסיקות דף אחרי ביצוע כל כמה הוראות קוד. במצב זה התהליך כמעט אינו מתקדם והמעבד עסוק רוב זמנו בהעברת דפים מהזיכרון ואליו (כיוון שזמן הטיפול בפסיקת דף גדול בהרבה מזמן הביצוע של הוראה שאינה גורמת לפסיקת דף).
מדיניות החלפה מקומית	local allocation policy	כאשר מדיניות זו ממומשת, כל תהליך מקבל הקצאה קבועה של מסגרות דף פיזיות (הקצאה לתהליכים שונים יכולה להיות שונה). כאשר יש צורך להביא לזיכרון הפיזי דף בעבור תהליך מסוים, הדף שיפנה לו את מקומו ייבחר רק מבין הדפים השייכים לאותו התהליך.

מדיניות החלפה גלובלית	global allocation policy	מדיניות זו קובעת כי כאשר יש צורך להביא לזיכרון דף בעבור תהליך מסוים, הדף שיפנה לו את מקומו ייבחר מבין כל הדפים בזיכרון, ללא קשר לשייכות של הדף לתהליך זה או אחר.
גיבוי פקודת מעבד אחרונה	instruction backup	שמירת מצב המעבד וסביבתו לפני תחילת ביצוע ההוראה הנוכחית. הצורך בכך מתעורר במערכות מבוססות דפדוף, שבהן הוראה שגורמת לפסיקת דף מורצת מחדש אחרי שהדף המבוקש מובא לזיכרון: הוראה תופסת בדרך כלל כמה בתים ולא בית בודד; ביצוע ההוראה מתקיים בכמה שלבים שבכל אחד מהם יכולה להתרחש פסיקה; לכן מתחילת ביצוע ההוראה ועד לפסיקת הדף, הסביבה של המעבד (ערכי הרגיסטרים) עלולה להשתנות. הגיבוי מאפשר להריץ מחדש את הפקודה הכושלת בדיוק מהמצב שבו הייתה המערכת לפני הניסיון הראשון. צורת הגיבוי הנדרשת תלויה בארכיטקטורת החומרה, ולכן מימוש הגיבוי שונה ממערכת למערכת.
נעילת דפים בזיכרון	locking pages in memory	נעילת דף מונעת את הוצאתו מהזיכרון הפיזי. מערכת ההפעלה משתמשת באפשרות הנעילה כדי לשמור בזיכרון דפים שאליהם מועברים נתונים על ידי DMA, כדי למנוע פינוי דף ששייך לקבוצת העבודה של תהליך או כדי למנוע פינוי דפים מסוימים ששייכים למערכת ההפעלה עצמה.
דפים משותפים	shared pages	שימוש משותף בדפי זיכרון (לקריאה בלבד) על ידי כמה תהליכים. שיתוף דפים נעשה על ידי מערכת ההפעלה לצורך אופטימיזציה, והוא שקוף לתהליכים. לדוגמה, הקוד של עורך טקסט שמשתמשים אחדים עובדים איתו בו-זמנית יכול להיות משותף כי אין צורך להביא לזיכרון את אותו הקוד בכמה עותקים. הערה: מונח זה שונה מהמונח "זיכרון משותף" (shared memory), אשר מתייחס לשיתוף אזורים בזיכרון על ידי כמה תהליכים כדי לשתף ולהחליף מידע (במקרה זה תהליכים מודעים לעצם השיתוף וניגשים לנתונים המשותפים לפי פרוטוקול מסוים).

האוניברסיטה הפתוחה
THE OPEN UNIVERSITY OF ISRAEL
الجامعة المفتوحة
Yair Kurtzman

File #0006182 belongs to Yair Kurtzman- do not distribute

חלוקת הזיכרון לקטעים	segmentation	סכימה של זיכרון וירטואלי, שבה מרחב זיכרון של תהליך מורכב מכמה סגמנטים. כל סגמנט הוא מרחב זיכרון ליניארי בפני עצמו ומכיל כתובות מ-0 עד למספר כלשהו. סגמנטים שונים יכולים להיות בעלי גדלים שונים, ולפעמים סגמנט משנה את גודלו תוך כדי הביצוע. סכימה זו מאפשרת למתכנת לחלק את תוכניתו לסגמנטים כרצונו, ולהגדיר לכל סגמנט הגנה שונה, בהתאם לתוכן הסגמנט. מימוש החלוקה לקטעים יכול להיעשות בשתי צורות: – pure segmentation – כל סגמנט יכול להימצא בזיכרון או להיות מאוחסן בדיסק רק בשלמותו. – paged segmentation – כל סגמנט הוא מרחב כתובות וירטואלי בפני עצמו שמנוהל בשיטת הדפדוף. במקרה זה, לכל סגמנט תהיה טבלת דפים נפרדת שתשמש למיפוי דפים בתוך אותו סגמנט.

פרק 4

מערכת קבצים

האוניברסיטה הפתוחה
THE OPEN UNIVERSITY OF ISRAEL
الجامعة المفتوحة
Yair
Kurtzman

File #0006182 belongs to Yair Kurtzman- do not distribute

4.1 קבצים
4.1.1 שמות לקבצים
4.1.2 המבנה הפנימי של קובץ
4.1.3 טיפוסי קבצים
4.1.4 גישה לקבצים
4.1.5 מאפיינים של קבצים
4.1.6 פעולות על קבצים
4.1.7 דוגמה לשימוש בקריאות מערכת לעבודה עם מערכת קבצים

כל מערכת הפעלה מספקת למשתמשים מנגנון לניהול קבצים. תכונות המערכת נקבעות על פי:

- אופן מימושם של קבצים
- המבנה הפנימי של קבצים
- אופן זיהוים של קבצים
- מנגנוני הגנה של קבצים
- אופן השימוש בקבצים

בסעיפים 4.1 ו-4.2 נעסוק במערכת הקבצים מנקודת המבט של המשתמשים ונכיר את הממשקים שמערכת הקבצים מספקת. בסעיף 4.3 נעסוק בדרכי המימוש של מערכת הקבצים. בסעיף 4.4 נדון בניהול שטח בדיסק ובביצועים של מערכת קבצים. הסעיף 4.5 בספר הלימוד סוקר את מערכת הקבצים FAT על גרסאותיה השונות (חומר רשות) ומציג דוגמה של ארגון בלוקים של קובץ באמצעות טבלת i-node במערכת ההפעלה UNIX.

הדיון במערכות קבצים מסתיים בהצצה חטופה אל סוגיות הקשורות לאפשרויות ההגנה על קבצים המופיעות בסעיף 9.3 בספר הלימוד.

4.1 קבצים

התכונות המאפיינות קבצים:

- קבצים מיועדים לשמירה ולאחסון של נתונים בזיכרון המשני, כלומר הנתונים שצריכים להישאר זמינים לאחר סיום התהליכים, כיבוי מסודר של המחשב, יציאת המשתמשים מן המערכת הממוחשבת (log-off) וכן הלאה.
- קבצים יכולים לשמש לארגון כמויות עצומות של נתונים.
- קבצים יכולים להיות משותפים לכמה מתהליכי מערכת ההפעלה.

האוניברסיטה הפתוחה
THE OPEN UNIVERSITY OF ISRAEL
الجامعة المفتوحة
Yair
Kurtzman

File #0006182 belongs to Yair Kurtzman- do not distribute

4.1.1 שמות לקבצים

מערכת קבצים מאפשרת לכנות קבצים בשמות לצורך זיהוים. שמו של קובץ יכול להיות מורחב על ידי סיומת, ובחלק ממערכות ההפעלה הסיומת גם קובעת את עצם קישור הקובץ לאפליקציה מסוימת. מערכת קבצים עשויה להגדיר מוסכמות למתן שמות ולסיומות תקניות. לדוגמה:

- במערכות הפעלה של Windows הסיומות com. או exe. מגדירות כי הקבצים הנושאים אותן הם קובצי הרצה בינאריים של מערכת ההפעלה.
- לעומת זאת, במערכות הפעלה שהן גרסאות של Unix או של Linux קובצי הרצה אינם צריכים לשאת סיומת מיוחדת.

4.1.2 המבנה הפנימי של קובץ

מערכת הקבצים משליטה מבנה פנימי על התוכן הפנימי של קובץ. קיימים מבנים רבים מאוד אשר מומשו במערכות הקבצים השונות. ספר הלימוד דן רק בשלוש האפשרויות הנפוצות ביותר למימוש המבנה של קבצים:

- **קובץ המורכב מרצף של בתים (bytes)** – פעולות כתיבה וקריאה של קובץ מתבצעות ביחידות של בתים, וחיפוש בקובץ דורש מעבר סדרתי על רצף הבתים. מערכת ההפעלה, או ליתר דיוק, מערכת הקבצים, איננה מניחה דבר לגבי המבנה הפנימי של הקובץ. האחריות למבנה הקובץ מוטלת על התהליך או על המשתמש עצמו. לדוגמה, קבצים שנוצרים על ידי יישומים כגון מעבד תמלילים, רשמקול, תוכנות לעריכה גרפית וכדומה יכולים להוות רצף של בתים מבחינתה של מערכת הקבצים, והאפליקציות הן אלה שמחוללות מרצף הבתים המרכיבים את הקבצים מבנה, ובדרך כלל רק היישום שיצר אותו יכול גם לפענחו. דוגמאות נוספות לכך הן קבצים המכילים מידע פנימי של מערכת ההפעלה, מדריכים (directories) שנדון בהם בסעיף 4.2, קובצי הרצה (executable files). מערכות הקבצים של UNIX ושל Windows מגדירות את הקבצים כרצף של בתים (ראו איור 4-2a בספר הלימוד).
- **קובץ המורכב מרצף של רשומות (records)** – פעולות כתיבה וקריאה של קובץ באופן לוגי, כלומר מבחינתו של המשתמש, מבוצעות על רשומות. חיפוש בקובץ דורש מעבר סדרתי על רצף של רשומות וחיפוש בתוכן. הרשומות הן בעלות אורך קבוע. גם במקרה של מבנה קובץ המורכב מרשומות, הארגון הלוגי הפנימי של הקבצים הוא באחריות התהליכים. היתרון בגישה הזאת הוא בייעול של פעולות הקריאה והכתיבה של הקבצים (ראו איור 4-2b בספר הלימוד).
- **קובץ המורכב מרשומות המאורגנות במבנה לחיפוש מהיר** – פעולות כתיבה וקריאה של קובץ באופן לוגי, כלומר מבחינתו של המשתמש, מבוצעות על רשומות. פעולת חיפוש בקובץ איננה דורשת מעבר סדרתי על הרשומות. הרשומות יכולות להיות בעלות אורך קבוע או אורך משתנה. באיור 4-2c מוצגת דוגמה שבה לכל רשומה מצורף מפתח חיפוש, והרשומות מאורגנות לפי מפתחות בעץ חיפוש. לאפליקציה יש שליטה על תוכן הרשומות, אך מיקומן נשלט על ידי מערכת הקבצים.

לסיכום, המנגנון של מבנה הקבצים מסופק על ידי מערכת הקבצים. לעומת זאת, התוכן, כלומר הארגון הלוגי הפנימי של הקבצים, הוא באחריות התהליכים. זהו פן נוסף של הרעיון, כי מערכת ההפעלה מספקת **מנגנון** אך אינה מכתיבה **מדיניות**.

4.1.3 טיפוסי קבצים

מבחינת התכנים, ניתן לסווג את הקבצים ***לקבצים רגילים*** (*regular files*) ו***לקבצים פנימיים של מערכת הקבצים המשמשים לארגונה*** (*special files*). קבצים רגילים יכולים להכיל מידע כלשהו לשימוש המשתמש. מערכות קבצים שונות מגדירות סוגים שונים של קבצים המשמשים לארגון פנימי של מערכת הקבצים, אך כמעט בכל מערכת קבצים, ***הספריות*** (*directories*) הן קבצים מיוחדים המשמשים לארגון של קבצים רגילים. דוגמה נוספת לקבצים פנימיים היא קבצים מיוחדים המשויכים להתקנים במערכות הפעלה UNIX, למשל:

- **block special files** – קבצים המשויכים להתקני קלט/פלט הפועלים ביחידות של בלוקים. למשל, פניות לדיסק קשיח יכולות להתבצע דרך ממשק רגיל של כתיבה או קריאה של קבצים, אך למעשה הפעולות מבוצעות על קובץ פנימי של מערכת קבצים ומתורגמות לפעולות הקלט/פלט של הדיסק הפיזי המשויך לקובץ הפנימי. בנוסף, מערכת הקבצים מבצעת הטמנה של בלוקים בזיכרון מטמון המכונה buffer cache או block cache, שנלמד עליו בהמשך.
- **character special files** – קבצים המשויכים להתקני קלט/פלט הפועלים ביחידות של תווים כגון מסופים, מדפסות, כרטיסי תקשורת, מקלדות, עכברים. אך להבדיל מ-block special files, מערכת הקבצים על פי רוב איננה מבצעת הטמנה בעבור character special files. פעולות קלט/פלט על התקן פיזי מבוצעות לאלתר.
- **pseudo-devices** – קבצים המשויכים להתקנים וירטואליים. להלן שלושת הסוגים השימושיים ביותר של הקבצים הללו:
  - **/dev/null** – התקן שאיננו מייצר פלט ומשליך את כל הנתונים שהוא מקבל כקלט. התקן זה הוא מעין "חור שחור" שבו אובדים כל הנתונים המועברים אליו.
  - **/dev/random** – התקן מדומה שמייצר מחרוזת פסידו-אקראית של תווים.
  - **/dev/zero** – התקן מדומה שמייצר מחרוזת של תווי '\0'.

במערכות דמויות UNIX קיימים סוגים נוספים של קבצים לא רגילים:

- ***קובצי FIFO*** – קבצים המשמשים לתקשורת בין תהליכים, המכונים לעתים named pipes. קובצי FIFO נראים כקבצים רגילים במערכת הקבצים, אך מסתירים מנגנון להעברת מידע בין תהליכים המספק גם סנכרון בין תהליכים. ניתן לדמיין את קובץ FIFO כצינור שאליו מוזרם פלט של תהליך (או של כמה תהליכים). הקריאה מצידו השני של הצינור גורמת להיעלמות המידע מתוך הצינור בצורת First In First Out. כמו כן, מנגנון FIFO משהה תהליכים שגורמים להצפה של קיבולת הצינור או לחלופין משהה תהליכים שמנסים לשאוב מצינור ריק.

האוניברסיטה הפתוחה THE OPEN UNIVERSITY OF ISRAEL الجامعة المفتوحة
Yair Kurtzman

File #0006182 belongs to Yair Kurtzman- do not distribute

- ***sockets*** – קבצים שמשמשים לתקשורת (network communication) בין תהליכים אשר אינם נמצאים בהכרח באותו מחשב.
- ***קישורים סימבוליים*** (***symbolic links***) – סוג קובץ המשמש כהפניה לקובץ אחר. פתיחה של קובץ קישור סימבולי גורמת לפתיחתו של הקובץ שאליו מצביע הקישור (אם קיים כזה).

בסעיף זה נעסוק בעיקר בקבצים רגילים. ניתן לסווג את הקבצים הרגילים לשתי קבוצות:

- **קובצי ASCII**[14] או, ליתר דיוק, קובצי הטקסט הפשוט.

ו-

- **קבצים בינאריים** או, במילים אחרות, כל הקבצים שאינם קובצי טקסט פשוט. דוגמאות לקבצים בינאריים הן קובצי הרצה של מערכת ההפעלה, קובצי object המיוצרים על ידי מהדרים, ספריות דינאמיות (dll-ים) וספריות סטטיות, קבצים שהתוכן הפנימי שלהם נשלט על ידי אפליקציה ספציפית.

עיינו באיור 4-3 בספר הלימוד כדי להבין את המבנה של קובץ הרצה גנרי (איור 4-3a) ושל קובץ ארכיון (איור 4-3b).

4.1.4 גישה לקבצים

מבחינים בין אפשרויות שונות לגישה אל קבצים:

- ***גישה סדרתית*** (***sequential access***) – בגישה זו ניתן לגשת לתוכן הקובץ (בתים או רשומות) לפי הסדר, החל מתחילת הקובץ. קבצים אשר נשמרים על גבי מדיה מגנטית למטרת גיבוי מתאפיינים בגישה כזאת. גישה לקבצים בטייפ מגנטי מתאפיינת בזמני גישה ארוכים עקב הצורך בהחזרה לאחור (rewinding) של הסרט המגנטי. החזרה לאחור נדרשת כדי לקרוא או לעדכן מידע על מיקום הקבצים או הבלוקים של הקובץ שנמצא בתחילת מערכת קבצים. כדי למנוע צורך בחזרות תכופות לאחור, כתיבה או קריאה של קבצים בסרט מתבצעות בצורה סדרתית, בלוק אחרי בלוק, ולכן המיקום של בלוקים יכול להיות צפוי מראש בטרם יירשמו הבלוקים עצמם.
- ***גישה אקראית*** (***random access***) – בגישה זו ניתן לגשת לתוכן הקובץ (בתים או רשומות) לאו דווקא על פי סדר הופעתם בקובץ. קבצים הנשמרים בדיסקים מגנטיים מתאפיינים בגישה כזאת. בסעיף 5.4.1 של מדריך הלמידה נראה כיצד מתבצעת גישה לבלוק של דיסק מגנטי. בקשה לקריאה או לעדכון של בית (או רשומה) של קובץ מתורגמת על ידי מערכת ההפעלה לעדכון או לקריאה של בלוק (או בלוקים) בדיסק. ניתן לגשת לכל בלוק בדיסק לפי סדר שרירותי. בנוגע

[14] ASCII (American Standard Code for Information Interchange) הוא קוד לייצוג תווים שמשתמש ב-7 או ב-8 סיביות לקידוד כל תו. את מקומו של קידוד ASCII תופסות בהדרגה צורות שונות של Unicode, שהוא קידוד המגדיר מערכת תווים אשר מקיפה את מערכות הכתב הנמצאות כיום בשימוש בעולם וכוללות את כל שפות תבל ומערכות סימנים המשמשות בתחומים מדעיים וטכניים.

לאופטימיזציה של תנועות זרוע הדיסק, נראה אלגוריתמים לתזמון בסעיף 5.4.3 של מדריך הלמידה או של ספר הלימוד.

4.1.5 מאפיינים של קבצים

כל מערכת קבצים מצמידה לקובץ ***מאפיינים*** (***file attributes***). מאפייני הקובץ משתנים ממערכת קבצים אחת למערכת קבצים אחרת. להלן רשימה של מאפיינים טיפוסיים של קובץ במערכת הפעלה דמוית UNIX:

- ***הרשאות הקובץ*** (***protection***) – מאפיין זה מתאר את אופני הגישה המותרים לקובץ ואת זהות בעלי הרשאות הגישה. במערכות דמויות UNIX מוגדרים שלושה אופנים של גישה: לקריאה, לכתיבה ולביצוע. קובצי הרצה או קבצים המכילים פקודות למערכת ההפעלה (קובצי script) חייבים להיות מסומנים להרצה כדי שמערכת ההפעלה תזהה אותם ככאלה. זהות בעלי הגישה מתוארת באמצעות הסיווג של משתמשי מערכת ההפעלה לשלוש קטגוריות. הקטגוריה הראשונה מכילה רק את בעל הקובץ, הקטגוריה השנייה מכילה את כל המשתמשים הנמנים עם הקבוצה שאליה שייך בעל הקובץ והקטגוריה השלישית מכילה את שאר משתמשי המערכת. לכל אחת משלוש הקטגוריות מוגדר אופן גישה לקובץ. החלוקה של משתמשי המערכת לקבוצות מבוצעת על ידי מנהל המערכת (האנושי: system administrator).
- ***בעל הקובץ*** (***owner***) – מאפיין זה מציין את זהות המשתמש שהוא הבעלים של הקובץ.
- ***קבוצת בעלי הקובץ*** (***group***) – מאפיין זה מציין את הקבוצה שאליה משתייכים הבעלים של הקובץ.
- ***זמן הגישה האחרונה*** (***last access time***) – מאפיין זה מציין את הזמן שבו התבצעה הגישה האחרונה לקובץ.
- ***זמן העדכון האחרון*** (***last modification time***) – מאפיין זה מציין את הזמן של שינוי התוכן האחרון של הקובץ.
- ***זמן עדכון המאפיינים האחרון*** (***last file status change***) – מאפיין זה מציין את הזמן האחרון של עדכון המאפיינים של הקובץ.
- ***גודל הקובץ*** (***file size***) – מאפיין זה מציין את גודל הקבצים הרגילים בבתים.
- ***מספר ההצבעות לקובץ*** (***number of links***) – מאפיין זה מציין את מספר הספריות ו/או התהליכים שמשתמשים בקובץ. במערכות הפעלה דמויות UNIX ניתן לקשר קובץ אחד לשתי ספריות שונות (או יותר) בקישור המכונה ***hard link***. כל יצירת קישור כזה מגדילה את המונה של מספר ההצבעות. גם הפעולה של פתיחת קובץ (למשל על ידי הפונקציה open) מגדילה את המונה. רישום של מספר ההצבעות מאפשר למערכת הקבצים לא לשחרר (לא למחוק) את הבלוקים אשר מרכיבים את הקובץ כל עוד ערך המונה איננו 0, כלומר כל עוד קיימת ספרייה או תהליך שמשתמשים בקובץ. קישור זה שונה מן הקישור הסימבולי המתואר בסעיף 4.1.3 של מדריך למידה זה בכך שיצירת קישור סימבולי איננה מגדילה את המונה של

האוניברסיטה הפתוחה THE OPEN UNIVERSITY OF ISRAEL الجامعة المفتوحة
Yair Kurtzman

File #0006182 belongs to Yair Kurtzman- do not distribute

ההצבעות. יתר על כן, להבדיל מיצירה של hard link לקובץ, יצירת קישור סימבולי איננה מצריכה את קיומו של הקובץ שמצביעים עליו (כך ניתן ליצור הפניה סימבולית לקבצים שייווצרו מאוחר יותר מזמן יצירת הקישור).

- ***מזהה התקן*** **(*device number*)** – מאפיין זה מכיל את מזהה ההתקן הפיזי שעליו יושב קובץ אם הוא קובץ רגיל.
- ***מזהה התקן לקובץ*** **(*device number for special file*)** – מאפיין זה מכיל את מזהה ההתקן כאשר הקובץ הוא קובץ פנימי המשויך להתקן תווי או להתקן בלוקים.
- ***מספר i-node*** **(*i-node number*)** – מאפיין זה מתאר את האינדקס של הטבלה המכילה מידע על הבלוקים שמרכיבים את הקובץ. בהמשך הפרק נלמד ביתר פירוט מהי טבלת ה-i-node.

ראו איור 4-4 בספר הלימוד, המתאר את המאפיינים העיקריים של קבצים.

4.1.6 פעולות על קבצים

להלן רשימה של פעולות טיפוסיות על קבצים שמאפשרת כל מערכת קבצים:

- יצירת קובץ – הפעולה יוצרת קובץ ריק הכולל את מאפייני ברירת המחדל.
- מחיקת קובץ – הפעולה מוחקת קובץ מספרייה ואם לא קיימים קישורי hard link נוספים, מתבצעת הגדלה של השטח הפנוי של הדיסק.
- פתיחת קובץ – הפעולה מפענחת את הנתיב אל הקובץ, בודקת הרשאות גישה ושולפת את כתובות הבלוקים של הקובץ לזיכרון הראשי. בעקבות פתיחת הקובץ, מוחזר למשתמש מזהה של הקובץ שבאמצעותו ניתן לבצע ביעילות פעולות נוספות. כלומר, אם פעולת פתיחת הקובץ מצליחה ומוחזר מזהה, כל פנייה נוספת לקובץ לא תחייב עוד את פענוח הנתיב ואת בדיקת ההרשאות בכל קריאה או עדכון של קובץ.
- סגירת קובץ – בעקבות הפעולה מתבצע שחרור של האזור בזיכרון הראשי שבו אוחסן מידע על הקובץ הנשלף בעת פתיחתו, כגון הכתובות של הבלוקים של הקובץ. סגירת הקובץ גורמת גם לריקון בלוקים השייכים לקובץ הנסגר, אשר היו טמונים בזיכרון המטמון של מערכת הקבצים (buffer cache).
- קריאה מתוך קובץ – פעולה זו מבצעת קריאה של נתונים מתוך קובץ למאגר במרחב הכתובות של התהליך הקורא. המאגר (לדוגמה, מערך של תווים) חייב להיות מסוגל להכיל את כמות הנתונים שנדרשת לקריאה.
- כתיבה אל הקובץ – פעולה זו מבצעת כתיבה של נתונים ממאגר במרחב הכתובות של תהליך כותב אל הקובץ.
- כתיבת צירוף – פעולה זו מבצעת כתיבה בהמשך הנתונים, החל מהבית האחרון של הקובץ.
- הזזה – פעולת ההזזה משנה את המיקום בקובץ אשר ממנו יתחילו פעולות הכתיבה או הקריאה הבאות. לדוגמה, הפעולה של כתיבת צירוף איננה אלא הזזת מיקום

לסוף הקובץ, כלומר הזזה במספר תווים כגודלו של הקובץ בבתים, וכתיבת נתונים החל מן המקום החדש.

- קריאה של מאפייני קובץ – פעולה זו מאפשרת להציג את מאפייניו השונים של הקובץ לעיונו של המשתמש.
- עדכון מאפייני קובץ – פעולה זו מאפשרת שינוי של מאפייניו השונים של הקובץ. חלק מהמאפיינים ניתנים לעדכון על ידי המשתמש, למשל הרשאות הקובץ. אבל קיימים מאפיינים שהמשתמש אינו רשאי לשנותם במישרין, לדוגמה, גודל הקבצים.
- שינוי שם הקובץ – פעולה זו, כמו פעולת הכתיבה של צירוף, יכולה להיות מוחלפת בפעולות אחרות: יצירת קובץ עם השם החדש, הקריאה של נתונים מהקובץ הישן וכתיבתם לקובץ החדש, ולבסוף – מחיקת הקובץ הישן.

4.1.7 דוגמה לשימוש בקריאות מערכת לעבודה עם מערכת קבצים

סעיף זה בספר הלימוד מכיל דוגמה של תוכנית להעתקת קובץ. לאחר קורס מבוא העוסק בתכנות בשפת C, הדוגמה צריכה להיות מובנת לכם. אתם מוזמנים לעיין בה בספר.

4.2 מדריכים (directories)
4.2.1-4.2.2 ספריות
4.2.3 שמות נתיבים
4.2.4 פעולות על מדריכים

4.2 מדריכים (directories)

ברוב מערכות הקבצים, המדריכים – גם הם עצמם קבצים. בסעיפים הבאים נסקור פעולות הקשורות למדריכים.

4.2.1-4.2.2 ספריות

בסעיפים הללו בספר הלימוד מתוארות שיטות לארגון ספריות, החל מן השיטה הפשוטה של ארגון ברמה אחת וכלה בארגון היררכי של ספריות – שיטה שבה משתמשות רוב מערכות הקבצים במערכות הפעלה רב-ייעודיות. ראו איור 4-8 בספר הלימוד, המדגימים את השיטות בהתאמה.

4.2.3 שמות נתיבים

רוב החומר בסעיפים אלו הוא תיאורי, ובוודאי כבר נתקלתם בו בעבר.

האוניברסיטה הפתוחה THE OPEN UNIVERSITY OF ISRAEL الجامعة المفتوحة
Yair Kurtzman

File #0006182 belongs to Yair Kurtzman- do not distribute

במערכת קבצים המאורגנת במבנה היררכי של ספריות ניתן להתייחס לקובץ באחד משני האופנים האלה:

- ***נתיב מלא*** **(*absolute path*)** – נתיב מלא מציין את הנתיב משורת עץ הספריות אל הקובץ.
- ***נתיב יחסי*** **(*relative path*)** – נתיב מלא מציין את הנתיב לקובץ החל מ***ספריית העבודה*** **(*working directory*)** הנוכחית, כלומר הספרייה שבה נמצאים התהליך או המשתמש ברגע הפנייה לקובץ.

שאלה 1

א. מהן ההשלכות האפשריות של שינוי ספריית עבודה נוכחית על ידי תהליך משתמש על תהליכים אחרים?

ב. מהן ההשלכות האפשריות של שינוי ספריית עבודה נוכחית על ידי ספריית תוכנה שעושים בה שימוש מתוך תהליך?

עץ ספריות הוא גרף מכוון ללא מעגלים.[15] לכן קיים רק נתיב אחד לכל היותר משורש העץ אל קובץ או ספרייה כלשהם במערכת הקבצים. עם זאת, לכל ספרייה, למעט ספריית השורש, ניתן לתאר את ספריית-האב שלה על ידי '..' או על ידי השם של ספריית האב. דבר זה גורם לכך שכל נתיב לקובץ שהוא נתיב יחיד ניתן לתיאור על ידי כמות אינסופית של מחרוזות נתיב.

שאלה 2

האם הטענה האחרונה לגבי מספר המחרוזות המתארות נתיבים אפשריים נכונה רק בעבור נתיבים מלאים או גם בעבור נתיבים יחסיים?

4.2.4 פעולות על מדריכים

להלן רשימה של פעולות טיפוסיות על ספריות במערכת הפעלה דמוית UNIX:

- יצירת ספרייה – יוצרת ספרייה חדשה.
- מחיקת ספרייה – משחררת משאבים (בלוקים של מערכת קבצים) שהיו מוקצים לספרייה.
- פתיחה של ספרייה – מבצעת פעולות דומות לאלה שמתבצעות בפתיחת קובץ.
- סגירה של ספרייה – מבצעת פעולות דומות לאלה שמתבצעות בסגירת קובץ.
- קריאת תוכן הספרייה – מבצעת קריאה של רשומות קבצים שהספרייה מכילה.
- שינוי שם הספרייה – משנה את שם הספרייה על ידי עדכון של רשומה מתאימה בספריית-האב.

---

[15] במערכת ההפעלה UNIX, משתמש root (מנהל מערכת) יכול ליצור קישור סימבולי לספרייה כדי להביא לסגירת מעגל. בגלל הסכנה של סגירת מעגל, משתמשים רגילים אינם מורשים לבצע פעולה של יצירת קישור סימבולי לספרייה.

- יצירת קישור סימבולי – אפשרות יצירת קישור סימבולי לספרייה ניתנת רק למנהל המערכת, מכיוון ששימוש בלתי זהיר בפעולה זו יכול לגרום ליצירה של מסלולים מעגליים בעץ הספריות.

שאלה 3

האם הפעולות המפורטות לעיל מהוות קבוצה בלתי תלויה של פעולות או שמא ניתן לממש פעולה אחרת בקבוצה באמצעות פעולות אחדות?

4.3 יישום מערכת הקבצים
4.3.1 תסדיר מערכת הקבצים
4.3.2 יישום קבצים
4.3.3 יישום מדריכים
4.3.4 שיתוף קבצים בין משתמשים שונים
4.3.5 מערכת קבצים בצורת יומן (log-structured)
4.3.6 מערכת קבצים מבוססת פלאש

4.3 יישום מערכת הקבצים

בסעיפים הבאים נעסוק במבנה מערכת הקבצים מנקודת מבטו של המתכנן. את הממשק של המערכת כבר הגדרנו, וכעת נראה כיצד ניתן לממש את הממשקים כך שיעבדו ביעילות ובאופן מהימן.

4.3.1 תסדיר מערכת הקבצים

עיינו באיור 4-10 בספר הלימוד, המציג את התסדיר (layout) הטיפוסי של דיסק:

- ***MBR*** **(*Master Boot Record*)** – מסייעת באתחול של מערכת ההפעלה הנמצאת במחיצה פעילה, כלומר המחיצה המסומנת כמחיצה המכילה מערכת הפעלה שצריכה לעלות כברירת מחדל.
- ***טבלת מחיצות*** **(*partition table*)** – טבלה המכילה מידע על חלוקת דיסק למחיצות. בטבלה קיימת אינדיקציה איזו מהמחיצות היא המחיצה הפעילה.
- ***מחיצות*** **(*partitions*)** – מחיצות הן חלוקה של דיסק לכמה דיסקים מדומים (לוגיים). כל מחיצה יכולה להיות מנוהלת על ידי מערכת קבצים שונה.

להלן הפירוט של מרכיבי התסדיר הטיפוסי של מחיצה במערכת קבצים טיפוסית של UNIX (איור 4-10):

- ***בלוק אתחול*** **(*boot block*)** – בלוק זה מכיל מידע המשמש לאתחול של מערכת ההפעלה אם היא קיימת במחיצה.

האוניברסיטה הפתוחה THE OPEN UNIVERSITY OF ISRAEL الجامعة المفتوحة
Yair Kurtzman

File #0006182 belongs to Yair Kurtzman- do not distribute

- ***סופר-בלוק*** **(*super block*)** – בלוק זה מכיל מידע של מערכת הקבצים השוכנת במחיצה. עבודה מול מערכת קבצים מצריכה שהסופר-בלוק יהיה טעון בזיכרון הראשי, הואיל ובו נמצא המידע על ארגון מערכת הקבצים הממוקמת במחיצה. פגיעה בסופר-בלוק יכולה להביא לאובדן של מערכת הקבצים (וכל המידע הרשום בהם). לכן מערכות קבצים דואגות להחזיק עותקים של הסופר-בלוק.
- ***בלוקים של דיסק המכילים מבני נתונים המשמשים לניהול שטחים פנויים במחיצה*** – בלוקים אלה מכילים מבני נתונים המשקפים את מצב הבלוקים הפנויים של המחיצה.
- ***בלוקים של דיסק המכילים טבלאות I-node*** – בלוקים אלה מכילים טבלאות שמשקפות את מצב הקצאת הבלוקים של דיסק לטובת קבצים וספריות. כל אחד מהבלוקים המיועדים לi-nodes מכיל מספר קבוע של טבלאות I-node.
- ***בלוקים של ספריית השורש*** – בלוקים של ספריית השורש מכילים רשומות המציינות את הקבצים ואת הספריות שספריית השורש מכילה.
- ***בלוקים שיכולים לשמש להקצאה לקבצים ולספריות*** – הבלוקים הללו יכולים להיות בלוקים תפוסים על ידי קבצים או בלוקים פנויים. מצבם של הבלוקים התפוסים על ידי קבצים וספריות מצוין על ידי טבלאות ה-I-node. מצבם של הבלוקים הפנויים רשום במבני נתונים המשמשים לניהול בלוקים פנויים.

חלופה מודרנית ל MBR:
מטעמי יעילות והרחבת יכולות, במערכות הפעלה מודרניות משתמשים ב- **Unified Extensible Firmware Interface (UEFI)** במקום ב-MBR. ראו איור 4-11 והסבר בספר הלימוד.

למעשה, המידע המתואר מפשט את מצב הדברים האמיתי. מבחינת השיקולים הקשורים ליעילות הגישה לנתונים, לא תמיד נכון לשכן מידע ניהולי הרחק מהבלוקים המנוהלים. לכן במחיצות גדולות, הבלוקים המכילים לדוגמה טבלאות I-nodes פזורים בין הבלוקים של המחיצה, ולאו דווקא ממוקמים ברצף בתחילתה. הדבר תקף גם לגבי הבלוקים המכילים מידע על מצבם של הבלוקים הפנויים. בהמשך נדון גם בשיקולים של יעילות, אך כדי להמחיש את האמור לעיל נקדים קצת את המאוחר ונעיין באיור 4-29.

4.3.2 יישום קבצים

כדי לאפשר יישום של קובץ, אנו זקוקים לפונקציה הממפה את תוכנו של הקובץ אל בלוקים בדיסק שבהם רשום המידע המהווה את תוכן הקובץ. קיימות שיטות אחדות לכך:

הקצאה רצופה

עיינו באיור 4-12. בפרק הדן בניהול זיכרון כבר נתקלנו בשיטת ניהול דומה.

שאלה 4

מנו את היתרונות ואת החסרונות של שיטת ההקצאה הרציפה לניהול קבצים.

רשימה משורשרת

עיינו באיור 4-13 הממחיש את השיטה.

שאלה 5

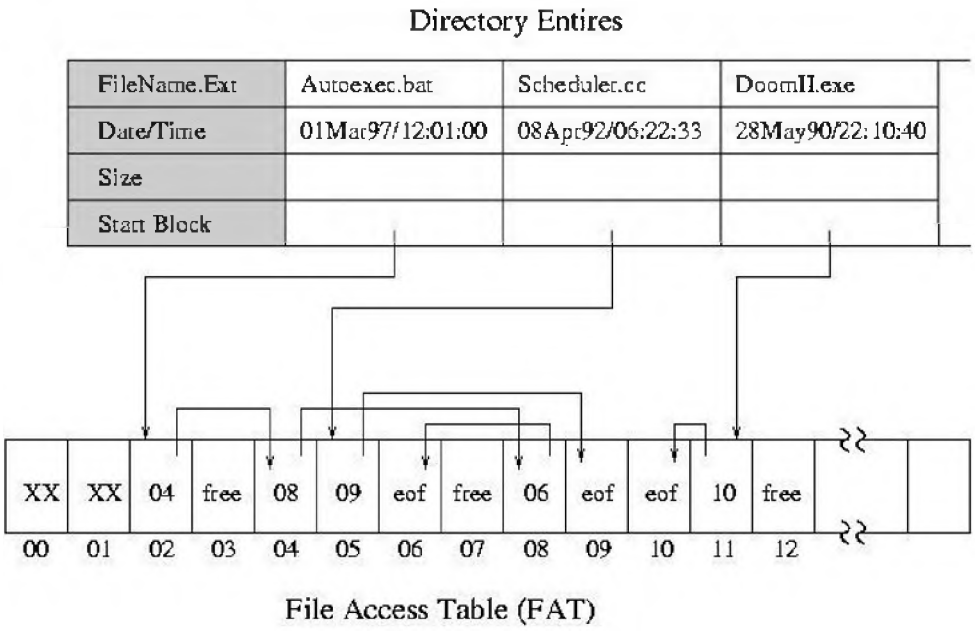
מדוע היעילות של השיטה במקרה של קריאה סדרתית גבוהה יותר מאשר במקרה של קריאה אקראית?

שאלה 6

מנו את היתרונות ואת החסרונות של שיטת רשימה משורשרת לניהול קבצים.

רשימה משורשרת בעזרת טבלת אינדקס בזיכרון (חומר רשות)

עיינו באיור 4-14. מתוארת בו שיטה שמערכות קבצים FAT מבוססות עליה. להבדיל מהשיטה הקודמת, המידע על קישור בין הבלוקים של הקובץ נשמר בטבלה הכוללת מידע על קישוריות הבלוקים של כל קובצי המערכת. האיור שלהלן מציג את הקשר בין המידע המצוי בספריות לבין טבלת האינדקסים.



איור 1

רשומות של ספרייה מצביעות רק לבלוק ההתחלתי של הקובץ בטבלת FAT, שהיא טבלת אינדקס. משם ניתן לבצע סקירה של הרשומה עד אשר מגיעים ל-EOF המציין את סוף הקובץ.

האוניברסיטה הפתוחה
THE OPEN UNIVERSITY OF ISRAEL
الجامعة المفتوحة
Yair
Kurtzman

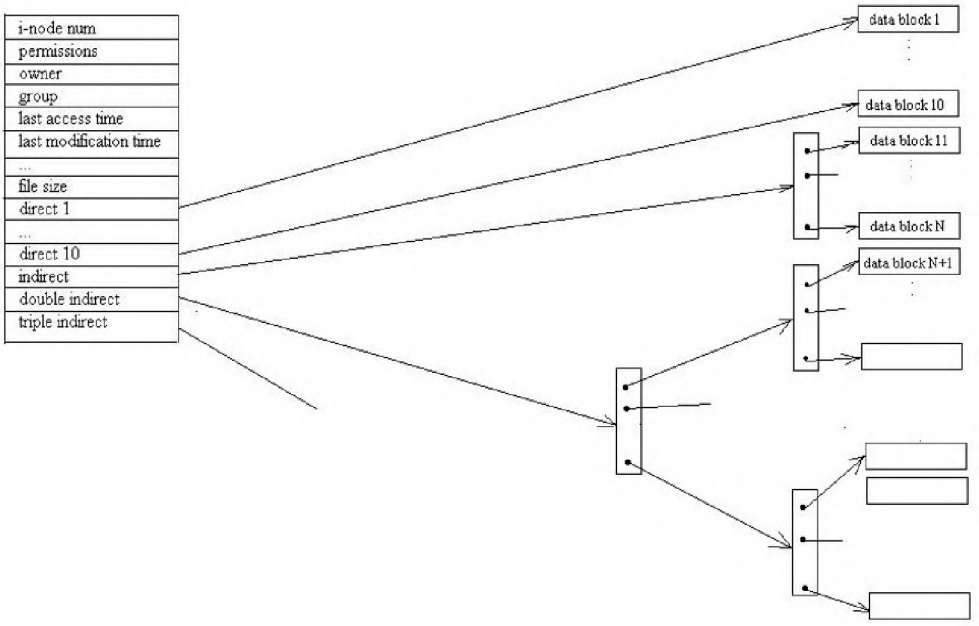
File #0006182 belongs to Yair Kurtzman- do not distribute

שאלה 7

מנו את היתרונות ואת החסרונות של שיטת רשימה משורשרת בעזרת טבלת אינדקס בזיכרון ראשי לניהול קבצים.

שיטת ה-I-NODES

זוהי השיטה המשמשת במערכות הפעלה דמויות UNIX. ספריות של מערכת קבצים מכילות רשומות המורכבות משם של קובץ וממספר I-node. לפי מספר I-node ניתן להגיע לטבלת I-node עצמה, המאגדת מידע על הקובץ ועל הקצאת הבלוקים של דיסק לקובץ. התבוננו באיור שלהלן:



איור 2

טבלת I-node מכילה שלושה סוגי שדות המיועדים לרישום מצב הקראת הבלוקים של הקובץ. שדות direct מכילים ישירות את כתובות הבלוק. שדה indirect מכיל כתובת של בלוק פנימי של מערכת קבצים המשמש לאחסון הכתובות של בלוקים נוספים של הקובץ. נניח שגודל כל בלוק במערכת קבצים הוא 2 kbytes. אזי באמצעות 10 שדות direct ניתן לפנות לקבצים בגודל של עד 20 kbytes. אם נניח כי גודל כתובת הבלוק במערכת קבצים הוא 4 bytes, אזי הבלוק שאליו מצביע שדה indirect יכול להכיל 2kbyte/4bytes = 2^9 = 512 כתובות של בלוקים אשר כל אחד מהם בגודל 2 kbytes. כך, באמצעות שדה indirect ניתן להוסיף עוד 2 kbytes * 512 = 1024 kbytes. לכן, באמצעות שני שדות, direct ו-indirect, ניתן לנהל הקצאה של קבצים עד לסדרגודלשל 1044 kbytes.

השדה double indirect מכיל כתובת של בלוק פנימי, שמכיל כתובות של בלוקים פנימיים אשר מכילים את הכתובות של בלוקים של data. השדה triple indirect משמש לניהול קבצים ענקיים מאחר שבאמצעות השדה double indirect ניתן לנהל קבצים שכמות הבלוקים בהם קטנה יותר.

שאלה 8

א. מהו גודל הקבצים שניתן לנהל באמצעות השדות direct, indirect, double indirect בהינתן הנתונים בפסקה לעיל?

ב. מהו גודל הקבצים שניתן לנהל באמצעות השדות direct, indirect, double and triple indirect בהינתן הנתונים דלעיל?

כמו שהוזכר בסעיף 4.3.1, טבלאות I-node נמצאות בבלוקים ייעודיים של מערכת הקבצים. הגודל הטיפוסי של טבלת I-node הוא 64 או 128 בתים.

סוגיה חשובה הקשורה לשימוש בטבלאות I-node היא שגישה לקבצים גדולים דורשת כמות גדולה של פניות לדיסק, מכיוון שצריך להביא לזיכרון בלוקים פנימיים שמכילים כתובות של בלוקים אחרים. מכאן שלהתפלגות הבלוקים של מערכת קבצים יש השפעה על הביצועים. כדי לחסוך בכמות הגישות לדיסק, משתמשים גם בהטמנה. נדון בנושא זה בסעיף 4.4.4.

4.3.3 יישום מדריכים

הגישה לקובץ דורשת מיפוי בין השם הלוגי של הקובץ (מסלולו בעץ) לבין מיקום הבלוקים שלו במערכת הקבצים. בכך דנו בסעיף 4.3.2. אבל היכן נשמרים השמות של קובצי המידע המצביע על אופן הקצאת הבלוקים? במערכות קבצים רבות מידע זה נמצא ברשומות שמרכיבות את הקבצים הפנימיים שלהן, המכונים מדריכים או ספריות.

באיור 4-14 מוצגות שתי שיטות לארגון מידע במדריכים:

- **מדריכים שהם אוסף רשומות שמכילות את כל המידע הרלוונטי** – ראו חלק א של האיור. שיטה זו ננקטת במערכות קבצים מסוג FAT.
- **מדריכים שכל רשומה שלהם מפנה** ל-I-node – ראו החלק השני של האיור. שיטה זו ננקטת על ידי מערכת קבצים טיפוסית ב-LINUX.

שאלה 9

מהם היתרונות והחסרונות של שתי השיטות הללו?

באיור 4-17 מוצגות שתי שיטות לתמיכה בשמות קבצים ארוכים על ידי המדריכים:

- בשיטה הראשונה רשומות של מדריך הן רשומות באורך משתנה והן מכילות את שם הקובץ (ראו חלק א של האיור).
- בשיטה השנייה רשומות הספרייה הן באורך קבוע והן מכילות מצביע לשם הקובץ. שמות קבצים מנוהלים כערימה שגודלה יכול להשתנות.

האוניברסיטה הפתוחה
THE OPEN UNIVERSITY OF ISRAEL
الجامعة المفتوحة
Yair
Kurtzman

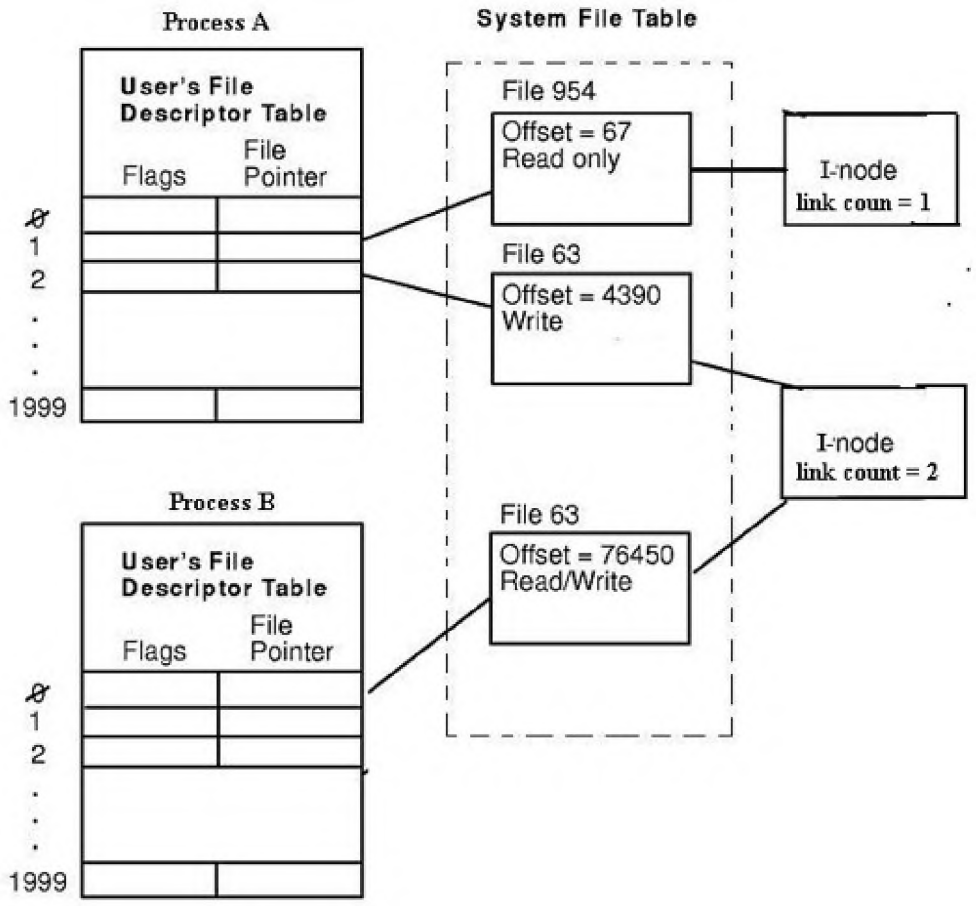
File #0006182 belongs to Yair Kurtzman- do not distribute

שאלה 10

השוו בין היתרונות והחסרונות של שתי השיטות הללו.

4.3.4 שיתוף קבצים בין משתמשים שונים

עיינו באיור שלהלן, המתאר מבנים המשמשים לשיתוף קובץ במערכת הפעלה דמוית UNIX:



איור 3

לכל תהליך קיימת טבלת קבצים פתוחים אשר מכילה מצביע לטבלת הקבצים הפתוחים של מערכת ההפעלה. טבלת הקבצים הפתוחים המערכתית מכילה רשומות לכל קובץ פתוח במערכת. אם שני תהליכים פתחו את אותו הקובץ, יופיעו בטבלת הקבצים הפתוחה של מערכת ההפעלה שתי רשומות. ראו הרשומות המתייחסות לקובץ 63 שפתוח על ידי שני התהליכים A ו-B. בכל רשומה מוחזק המיקום של כל אחד מהתהליכים בתוך הקובץ (שדה ה-offset). שימו לב שהמספר של ה-hard links של הקובץ המשותף הוא 2, בעת שערך מונה הקישורים בקובץ הלא משותף הוא 1 בלבד.

4.3.5 מערכת קבצים בצורת יומן (log-structured)

שימוש בזיכרון מטמון, הבאת בלוקים מראש ופיזור חכם של נתוני שירות של מערכת קבצים יכולים להביא לחיסכון במספר הפניות לדיסק בעבודה עם נתונים גדולים, אך שיטות אלה אינן מועילות מאוד כאשר צריך לבצע עדכונים תכופים של כמויות קטנות מאוד של נתונים (מה שנדרש לדוגמה לשמירה על עקביות של מערכת קבצים).

ניתן להתמודד עם הבעיה על ידי ביצוע של עדכונים בצורת יומן ולא לפי ההתייחסות לנתוני המקור שאליהם מתייחסים העדכונים. במקרה זה, מערכת הקבצים פשוט רושמת את כל העדכונים בצורה של זרם לדיסק. ברקע רץ תהליך שקורא את log השינויים שנוצר ומבצע עדכונים של הנתונים. חלק זה של היומן נמחק לאחר ביצוע העדכון של הנתונים הרלוונטיים וכך מתבצע פינוי של מקום לרישום של עדכונים חדשים הממשיכים להגיע.

4.3.6 מערכת קבצים מבוססת פלאש

כונני SSD (Solid State Drive) משתמשים בזיכרון פלאש ופועלים בצורה שונה למדי מכונני דיסק קשיח.

באחסון הבזק אין חלקים נעים והבעיות של זמני חיפוש ועיכובי סיבוב שהזכרנו קודם לא קיימות יותר. המשמעות היא שזמן הגישה טוב הרבה יותר בסדר גודל של כמה עשרות מיקרו-שניות במקום אלפיות השנייה. לכן בכונני SSDאין פער ניכר בביצועים בין קריאה אקראית לרציפה .כמו שנראה בהמשך, כתיבות אקראיות עדיין יקרות הרבה יותר, במיוחד אם הן קטנות.

ואכן ,בניגוד לדיסקים מגנטיים, לטכנולוגיית הפלאש יש ביצועי קריאה וכתיבה אסימטריים : הקריאה מהירה הרבה יותר מהכתיבה. לדוגמה, בעוד קריאה נמשכת כמה עשרות מיקרו-שניות, כתיבה יכולה להימשך מאות מיקרו-שניות. ראשית, הכתיבה איטית בגלל האופן שבו מתוכנתים תאי הפלאש שמיישמים את הביטים. זו תופעה פיזיקלית ולא נדון בה. שנית, מה שמשפיע יותר על הביצועים הוא שאפשר לכתוב יחידת נתונים רק לאחר מחיקת אזור מתאים בהתקן. למעשה, זיכרון פלאש מבחין בין *יחידת קלט/פלט* (בדרך כלל KB4) ויחידת *מחיקה* (בדרך כלל 64– 256יחידות קלט/פלט ועד כמה מגה בית). למרבה הצער, בתעשייה באופן מטעה מתייחסים ליחידה של I/O כדף וליחידת מחיקה כבלוק.

כדי לכתוב יחידת נתונים, ה- SSDחייב למחוק קודם כול יחידת מחיקה – פעולה יקרה שנמשכת מאות מיקרו-שניות .למרבה המזל, לאחר שהוא מחק את יחידת מחיקה, יש הרבה יחידות קלט/פלט בחלל הזה וה-SSD יכול לכתוב את יחידות ק/פ בבלוק שהתפנה, לפי הסדר. במילים אחרות, הוא כותב תחילה את דף 0, אחריו את דף 1, ואז את 2, לפי הסדר. הוא לא יכול לכתוב את דף 0, אחריו את דף 2 ואחריו את 1. חוץ מזה, SSD לא יכול באמת לשכתב יחידת ק/פ שכבר נכתבה, אלא הוא צריך קודם למחוק את כל יחידת המחיקה (לא רק את יחידת ק/פ הרלוונטית). בשביל לשכתב יחידת ק/פ אחת, צריך להעתיק את כל היחידות למקום חדש, למחוק את כל יחידת המחיקה ולעתיק בחזרה

האוניברסיטה הפתוחה
THE OPEN UNIVERSITY OF ISRAEL
الجامعة المفتوحة
Yair
Kurtzman

File #0006182 belongs to Yair Kurtzman- do not distribute

את כל היחידות ק/פ זו אחר זו, לפי הסדר. לכן למעשה במקרה של שכתוב, SSD פשוט שומר את תוכנה של כל יחידת המחיקה במקום חדש ואת הישנה מסמן כ"לא בשימוש". המחיקה תיעשה אם לא יישאר מקום פנוי. זה מסביר למה סדרת כתיבות אקראיות קצרות יכולה להיות איטית.

אורך החיים של תאי פלאש טיפוסיים (מחזורי כתיבה/מחיקה) הוא בין אלפים לכמה מאות אלפים של מחזורים. לכן חשוב לפזר את הפעולות בשטח SSD.

בבקרי SSD קיים רובד חומרה ותוכנה בשם **FTL** (**Flash Translation Layer**) שבין היתר אחראי גם לזה. בפועל הנתונים נשמרים בחבילות פלאש (FP - flash packages). כל חבילה מורכבת מ"פיסות" (dies), כל פיסה מורכבת ממחיצות (planes) וכל מחיצה מורכבת מיחידות מחיקה, שכל אחת מהן יכולה להכיל כמה יחידות ק/פ (ראו איור 4-20 בספר הלימוד).

עקב כך, הכתובת ב-SSD חייבת להיות היררכית. מערכות הקבצים משתמשות בכתובת ליניארית, ולכן נדרש מנגנון תרגום הכתובת שדומה לתרגום הכתובת בזיכרון הראשי בעזרת טבלאות דפים.

ה-**FTL** מטפל גם ב-**garbage collection**, לדוגמה ביחידת מחיקה מסוימת קיימות יחידות ק/פ שסומנו כלא בשימוש וקיימות גם יחידות בשימוש. כדי למחוק (לפנות) את כל יחידת המחיקה, צריך להעתיק את כל היחידות ק/פ בשימוש שנמצאות בה למקום פנוי חדש, ורק אחרי כך יהיה ניתן למחוק את כל יחידת המחיקה. בנוסף, garbage collector צריך להביא בחשבון שיקולים הדומים לשיקולי דפדוף בניהול זיכרון ראשי. במערכת קיימת פקודת TRIM המיידעת את מערכת ניהול SSD שיחידות ק/פ מסוימות מסומנות כלא בשימוש, כדי שיופעל garbage collector, אחרת זה יתגלה רק בניסיון כתיבה ויגרום עיכוב.

מהאמור לעיל אפשר להבין שמערכות הקבצים הקיימות לא מתאימות ל-SSD. נציין שעיקר הבעיה היא בכתיבות כשצריך להשיג שתי מטרות:

ניתן להשיג זאת בעזרת מערכת קבצים מבוססת

1. להימנע מכתיבות אקראיות (בייחוד קצרות).
2. לפזר את הכתיבות בשטח הזיכרון כדי לצמצם את השחיקה.

רישום, log-structured file system, בזכות השימוש ברישום מוקדם וכתיבות רציפות בהמשך. מובן שרישום זה לא יפתור את כל הבעיות, אבל הוא יכול למזער אותן במידה ניכרת.

אם ניקח כדוגמה מערכת קבצים מבוססת i-node, במקרה של כתיבה נדרש עדכון לבלוקים של מצביעים לא ישירים ברמות שונות, שיכול לגרום בעיית עדכון רקורסיבי הכרוך במספר רב של כתיבות. כדי למזער את הבעיה, במקום לשמור את הכתובת האמיתית של i-node או בלוק מצביעים, ניתן לשמור את מספר המזהה ולנהל טבלת מיפוי גלובלית של המספרים האלה לבלוקים של הדיסק. היתרון הוא שבמקרה של

עדכון הקובץ, יהיה צורך לעדכן (חוץ מאת הנתונים עצמם) רק את המיפוי הגלובלי ואת הבלוק המתאים של מצביעים לא ישירים.

רעיון זה אומץ במערכת קבצים מותאמת פלאש Flash-Friendly File System (F2FS) הנתמכת ב-Linux.

4.4 ניהול ואופטימיזציה של מערכת הקבצים
4.4.1 ניהול שטח הדיסק
4.4.2 גיבוי מערכת הקבצים
4.4.3 עקביות מערכת הקבצים (consistency)
4.4.4 ביצועים של מערכת הקבצים

4.4 ניהול ואופטימיזציה של מערכת הקבצים

4.4.1 ניהול שטח הדיסק

שטח הדיסק מחולק לבלוקים. מערכת הקבצים יכולה להשתמש בגודל בלוק של דיסק או לחלופין לממש חלוקה בגודל שונה מעל החלוקה של הדיסק הפיזי.[16]

גודל הבלוק

לגודל הבלוק של מערכת קבצים יש השפעה על ביצועי המערכת כולה. ראו גרף באיור 4-23 המדגים כי ככל שהבלוקים גדלים, כך גדל קצב העברת נתונים. במונחים של ניצולת הדיסק, התמונה היא הפוכה: בלוקים גדולים יכולים לגרום לריסוק פנימי גדול. לפיכך גם במקרה זה, כמו תמיד, צריך למצוא פשרה בין כמה שיקולים מנוגדים.

מעקב אחר השטח החופשי בדיסק

בסעיף 4.3.2 ראינו שיטות לניהול בלוקים תפוסים, כלומר בלוקים ששייכים לקבצים או בלוקים פנימיים של מערכת הקבצים. כעת נראה שתי שיטות לניהול הבלוקים הפנויים שמערכת קבצים יכולה להקצותם לקבצים, לספריות או לצרכים פנימיים של ניהול.

- ניהול בלוקים פנויים באמצעות מפת סיביות (ראו איור 4-24b).
- ניהול בלוקים פנויים באמצעות רשימה מקושרת של בלוקים המכילים את רשימת הבלוקים הפנויים (ראו איור 4-24b).

[16] קיימות מערכות קבצים שמציעות אף חלוקה בכמה גדלים שונים.

האוניברסיטה הפתוחה THE OPEN UNIVERSITY OF ISRAEL الجامعة المفتوحة
Yair Kurtzman

File #0006182 belongs to Yair Kurtzman- do not distribute

שאלה 11

באיזו משתי השיטות קל יותר למצוא מקום פנוי? מה קורה בכל שיטה כאשר בלוק פנוי הופך לתפוס?

הגבלת השימוש בדיסק

זהו ללא ספק צורך חיוני במערכות בעלות כמה משתמשים, כי כל מרחב פנוי מתמלא במהירות, במיוחד כאשר האחסון ניתן חינם אין כסף. המנגנון המגביל את השימוש בשטח הדיסק מתבסס על רישום של כמות הבלוקים הנמצאים בשימוש על ידי המשתמשים ורישום של כל הקבצים שהם מחזיקים. מנהל מערכת יכול לקבוע מכסה של הבלוקים ושל הקבצים בעבור כל משתמש ולאכוף את משטר השימוש בשטחי הדיסק (ראו איור 4-26 בספר הלימוד).

4.4.2 גיבוי מערכת הקבצים

יש חשיבות רבה לביצוע יעיל של גיבויים. חלקים ניכרים של המידע בדיסק אינם משתנים מיום ליום, לכן ניתן לחסוך בכמות הנתונים המגובים על ידי ביצוע של גיבוי אינקרמנטלי. בגישה זו המידע שיגובה הוא רק המידע שהשתנה מאז הגיבוי האחרון. מובן שבפעם הראשונה חייבים לגבות את כל המידע.

מכיוון שהשחזור מגיבוי אינקרמנטלי דורש ביצוע שחזורים של הנתונים החל מהגיבוי הראשון, מדי פרק זמן כלשהו מבצעים שוב גיבוי מלא, נפטרים מכל הגיבויים הקודמים וחוזרים לבצע גיבוי אינקרמנטלי.

ניתן להבחין בין שתי שיטות של גיבויים:

- גיבוי ברמה הפיזית של הדיסק (physical dump)

או

- גיבוי ברמה הלוגית של המחיצות (logical dump).

הגיבוי ברמה הפיזית עובר על הבלוקים של הדיסק ומבצע גיבוי של הנתונים שהשתנו ללא כל התייחסות למבנה של הנתונים, כלומר ההתייחסות היא לרצף בתים גולמי. לעומת זאת, הגיבוי הלוגי מתבצע בהתייחס לשינוים שהתרחשו במערכת הקבצים: כך מתבצע מעבר על עץ הקבצים וכל שינוי יחסית לגיבוי הקודם נרשם (ראו איור 4-27 בספר הלימוד).

שאלה 12

תארו את אלגוריתם ארבעת השלבים המבצע את הגיבוי הלוגי.

בגיבוי לוגי לא נשמרים הקבצים הפנימיים של מערכת הקבצים כגון ה-FIFOs, sockets. עקרונית, אין גם צורך לשמור קבצים פנימיים הקשורים להתקנים. רשימת הבלוקים הפנויים גם היא לא נשמרת. לאחר השחזור של הנתונים, ניתן לשייך את כל הבלוקים הנותרים ללא שימוש לרשימת הבלוקים הפנויים.

שאלה 13

כיצד ניתן להתגבר על בעיות גיבוי הקשורות:

- לקישורים?
- לחורים בקבצים?

4.4.3 עקביות מערכת הקבצים (consistency)

הקבצים במערכת נחלקים לשתי קבוצות: הקבוצה הראשונה כוללת את כל הקבצים המשמשים לאחזקת מערכת הקבצים עצמה, והקבוצה השנייה – את כל שאר הקבצים. מטעמי נוחות, הקבצים השייכים לקבוצה השנייה מכונים **קובצי נתונים** (data files) אף שהם יכולים להיות קבצים מכל סוג שהוא. תפקידם של הקבצים השייכים לקבוצה הראשונה הוא לתאר נכונה את מצב מערכת הקבצים בדיסק. מצב של חוסר התאמה בין תיאור זה לבין המציאות – מצב בלתי עקבי – עלול להיות מצב חמור למדי.

כל מערכת קבצים כוללת תוכניות שירות (utilities) שמטרתן לבדוק את עקביות המערכת. במקרה של נזק, הן מחזירות את מערכת הקבצים למצב עקבי. החזרה למצב עקבי אינה קשורה בהכרח לשחזור מלא של הנתונים שהיו יכולים להיפגם בתרחיש שגרם להבאת מערכת הקבצים למצב בלתי עקבי.

נמנה את המקרים שבהם קיים חוסר עקביות של מערכת קבצים טיפוסית של UNIX.

- ***בלוק חסר*** – בלוק שאינו מופיע ברשימת הבלוקים הפנויים אך אינו מוקצה לשום קובץ (ראו איור 4-29b).
- ***בלוק חופשי מיותר*** – בלוק המופיע ברשימת הבלוקים החופשיים יותר מפעם אחת (ראו איור 4-29c).
- ***בלוק עם הקצאה עודפת*** – בלוק המוקצה ליותר מקובץ אחד (ראו איור 4-29d).
- ***בלוק חופשי ומוקצה*** – בלוק שמופיע הן ברשימת הבלוקים הפנויים והן ברשימת הבלוקים המוקצים.

שאלה 14

מהן הפעולות שמחזירות את מערכת הקבצים למצב עקבי בכל אחת מהאפשרויות דלעיל?

שאלה 15

מהם הסימנים הנוספים שיכולים להצביע על מערכת קבצים בלתי עקבית?

האוניברסיטה הפתוחה THE OPEN UNIVERSITY OF ISRAEL الجامعة المفتوحة
Yair Kurtzman

File #0006182 belongs to Yair Kurtzman- do not distribute

4.4.4 ביצועים של מערכת הקבצים

זיכרון מטמון (buffer cache)

בשל הפער העצום בין מהירות הגישה לדיסק לבין מהירות עיבוד הנתונים בזיכרון הראשי, הגישה לדיסק יכולה להפוך בנקל לצוואר הבקבוק של המערכת כולה. הדיון כאן מתמקד בדרכים להתגבר על מכשלה זו, וברור כי הדבר הראשון שיש לעשות הוא לצמצם ככל האפשר את מספר הגישות לדיסק. כאן בא לעזרתנו מטמון המכונה buffer cache (או block cache) שהזכרנו אותו פעמים אחדות בהזדמנויות שונות.

מערכת ההפעלה מקצה בזיכרון הראשי מסגרות המשמשות את מערכת הקבצים לצורך הטמנה של בלוקים של דיסק. מבנה הנתונים שמערכת הקבצים משתמשת בו לצורך הניהול של זיכרון המטמון (buffer cache) מוצג באיור 4-30. כל בלוק בדיסק ממופה על ידי פונקציית ערבול לרשימת בלוקים המנוהלים על ידי אלגוריתם LRU, שעליו למדנו בפרק על ניהול זיכרון. בניגוד לניהול זיכרון במערכת קבצים, ניתן לממש כאן אלגוריתם LRU מכיוון שבעבודה עם דיסק אין מרכיב קריטי כל כך של זמן גישה לנתונים. במילים אחרות, יחסית לזמן הבאת בלוק, זמן החישוב של LRU אינו מהווה צוואר בקבוק.

עם זאת, מערכת הקבצים אינה משתמשת ב-LRU בצורתו הטהורה, וזאת אף שאלגוריתם LRU מביא לתוצאות קרובות מאוד לאלגוריתם אופטימלי במונחים של חיסכון בפניות לדיסק. בין השיקולים של תכנון מערכת הקבצים קיים מקום מרכזי למהימנות, ואילו הצמצום של מספר הפניות לדיסק יכול לפגוע במטרה המרכזית הזאת. לדוגמה, קבצים פתוחים הנמצאים בשימוש יכולים לעדכן את ה-I-nodes שלהם בצורה מתמדת וכתוצאה משימוש באלגוריתם LRU יכול לעבור פרק זמן ניכר בין השמירות של טבלאות I-node בדיסק. דבר זה יכול לסכן את עקביותה של מערכת הקבצים.

שאלה 16

מהם השינויים שיש לבצע באלגוריתם LRU כדי שיהיה מותאם לניהול זיכרון מטמון של מערכת קבצים?

רמז – הביאו בחשבון את השיקולים האלה:

- מזעור של פניות הדיסק על ידי חיזוי של הבלוקים שיהיו בשימוש.
- ביצוע עדכונים בדיסק של הנתונים הדרושים לקונסיסטנטיות של מערכת הקבצים.

מלבד התאמת LRU לדרישות של מערכת הקבצים, קיימת אפשרות אחרת שחלק ממערכות הקבצים משתמשות בה. בזיכרון מטמון מסוג write-through מתבצעת כתיבה של כל העדכונים, הן של נתוני המשתמש והן של הנתונים הפנימיים של מערכת הקבצים בדיסק, ללא השהיה. מערכות הקבצים של UNIX, לעומת זאת, משתמשות בשיטת non-write through ומבצעות שמירה של עדכונים בדיסק רק אחת לפרק זמן כלשהו. פרק זמן זה קובע את היחס בין יעילות העדכון לבין מהימנותו.

הבאת בלוקים מראש

כמו במקרה של prepaging, שדנו בו בפרק 3, גם כאן ניתן להביא בלוקים של דיסק מראש.

שאלה 17

מהם היתרונות והחסרונות של שיטת הבאת הבלוקים מראש?
מצאו תחומים נוספים שבהם ניתן לעשות שימוש בשיטה זו. האם היה אפשר להשתמש בה בזיכרון מדופדף?

צמצום התנועות של זרוע הדיסק על ידי פיזור של נתוני שירות של מערכת קבצים

רצוי למקם בלוקים פנימיים של מערכת הקבצים, כגון טבלאות I-node, במקום קרוב ככל האפשר לקבצים שהם מייצגים. הרעיון כאן הוא שלא ניתן לגשת לקובץ לפני איתורו בטבלת הקבצים, ולכן צפוי כי צורת מיקום חכמה תחסוך מראש הדיסק לעבור פעמים רבות בין המסלולים המכילים נתוני שירות ונתוני משתמש (ראו איור 4-31).

4.5 מערכות קבצים (case studies)
4.5.1 מערכת קבצים ב-MS-DOS
4.5.2 מערכת קבצים ב-UNIX V7

הסעיפים 4.5.1 ו-4.5.2 מכילים חומר טכני ולכן אין טעם לחזור עליו כאן. מומלץ לקרוא ישירות מספר הלימוד.

סעיפים שאינם מופיעים בספר הלימוד
4.6 סיכום
4.7 בעיות פתורות
4.8 שאלות לחזרה
4.9 רשימת מושגים

4.6 סיכום

בפרק זה עסקנו במבנה הלוגי ובמימושה הפיזי של מערכת הקבצים. נזכיר כמה מהנושאים הנדונים:

- המבנה הלוגי של מערכת הקבצים, כפי שהוא נראה לתהליכי המשתמשים.
- המימוש של מערכת הקבצים.
- ניהול השטחים על ההתקנים.

האוניברסיטה הפתוחה THE OPEN UNIVERSITY OF ISRAEL الجامعة المفتوحة
Yair Kurtzman

File #0006182 belongs to Yair Kurtzman- do not distribute

- דרכים להבטחת האמינות והעקביות במערכת הקבצים.
- סוגיות יעילות הביצועים של מערכת הקבצים.

4.7 בעיות פתורות

שאלה 1

מהו מספר פעולות הקריאה של בלוק מהדיסק שצריך לבצע בהרצת השורה הזאת:

fd = open("/a/b/c");

במערכת ההפעלה UNIX, כאשר:

- ניתן להניח כי קובץ c כבר קיים על הדיסק.
- גודלו של כל קובץ ספרייה הוא בלוק אחד.
- הבלוקים (למעט ה-super block והבלוק המכיל את ה-i-node של ספריית השורש) אינם נמצאים בזיכרון מראש.
- קריאות המערכת אינן נכשלות.

ה-i-nodes נמצאים בבלוקים שונים.

תשובה

- במערכת ההפעלה UNIX, ספריית השורש "/" נמצאת ב-i-node שמספרו 2. לפי ההנחה, הבלוק שמכיל i-node זה כבר נמצא בזיכרון. לכן הבאת הבלוק של ספריית השורש תצריך רק פעולת קריאה אחת.
- בבלוק של ספריית השורש בודקים מהו מספר ה-i-node של הספרייה a ומביאים את הבלוק המכיל אותה, ולאחר מכן – את הבלוק המכיל את קובץ הספרייה a עצמו. מספר הקריאות מהדיסק הוא 2.
- בבלוק של ספרייה a בודקים מהו מספר ה-i-node של הספרייה b ומביאים את הבלוק המכיל אותה, ולאחר מכן – את הבלוק המכיל את קובץ הספרייה b עצמו. מספר הקריאות מהדיסק הוא 2.
- בבלוק של ספרייה b בודקים מהו מספר ה-i-node של הקובץ c ומביאים את הבלוק המכיל אותה (כלומר, את הבלוק שמכיל את ה-i-node של c). מספר הקריאות מהדיסק הוא 1.

כלומר, פענוח של נתיב על ידי open דורש שש גישות לדיסק. לאחר העדכון של טבלאות הקבצים הפתוחים, יהיה אפשר לפנות לקובץ c, ללא צורך בפענוח הנתיב מחדש.

כלומר, פענוח של נתיב על ידי open דורש שש גישות לדיסק. לאחר העדכון של טבלאות הקבצים הפתוחים, יהיה אפשר לפנות לקובץ c, ללא צורך בפענוח הנתיב מחדש.

שאלה 2

במערכת קבצים נתונה משתמשים בשיטת i-nodes. נתון כי

- גודל הבלוק הוא 1 kbyte
- גודל המצביע לבלוק הוא 8 בתים (8 bytes)
- כל i-node מכיל עשרה מצביעים ישירים ושלושה מצביעים לא ישירים: single-indirect, double-indirect ו-triple-indirect (מצביע אחד מכל סוג).

ידוע כי קובץ מסוים משתמש ב-21 בלוקים (כולל בלוקים של נתונים ובלוקים של מצביעים.

אין מביאים בחשבון את הבלוק המכיל את ה-i-node עצמו).

מהו גודלו של הקובץ הזה?

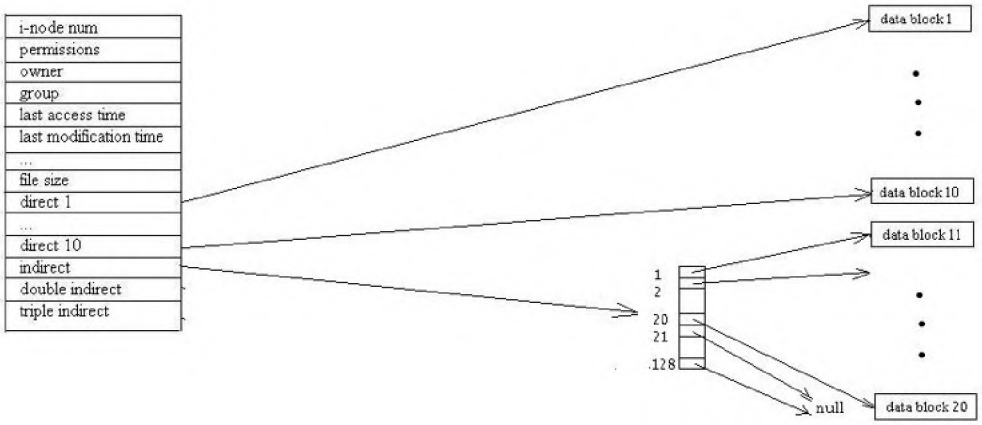
תשובה

10 מצביעים ישירים מצביעים ישירות על 10 בלוקים של נתונים (מ-1 עד 10). נותרו לנו עוד 11 בלוקים, שאחד מהם חייב להיות בלוק פנימי של מערכת הקבצים.

בלוק פנימי של מערכת הקבצים יכול להכיל

1kbyte/8Bytes = 2^7

מצביעים לבלוקים אחרים. במקרה שלנו, אלה בלוקים של נתונים מ-11 עד 20. ראו האיור שלפניכם:



לכן גודל הקובץ יהיה 20 blocks * 1 kbyte = 20 kbyte.

שאלה 3

להלן תוצאות הבדיקה של עקביות מערכת הקבצים:

1 2 3 4 5 6 7 8 9 10 11 12 13 14 15

Blocks in use

1 0 0 1 0 0 0 0 1 1 1 1 1 1 0

Free blocks

0 0 1 0 1 1 1 0 0 1 0 0 0 0 1

מה אפשר להסיק מן הנתונים האלה?

האוניברסיטה הפתוחה THE OPEN UNIVERSITY OF ISRAEL الجامعة المفتوحة
Yair Kurtzman

File #0006182 belongs to Yair Kurtzman- do not distribute

תשובה

- בלוק מס' 2 חסר. כדי להחזיר את מערכת הקבצים למצב עקבי, יש להוסיפו לרשימת הבלוקים הפנויים.
- בלוק מס' 10 הוא בלוק חופשי ומוקצה. כדי להחזיר את מערכת הקבצים למצב עקבי, יש להוציא את הבלוק מרשימת הבלוקים הפנויים.

4.8 שאלות לחזרה

אילו מבין הטענות שלהלן הן טענות נכונות? (כל התשובות נמצאות בחומר הלימוד, בפרק עצמו או במדריך זה.)

1. זמן החיפוש בקובץ, שהוא בעל מבנה פנימי המורכב מרשומות, הוא יחסי לגודל הקובץ.
2. גישה אקראית לקבצים על גבי מדיה סדרתית (כגון טייפ מגנטי) ניתנת למימוש אם כי איננה יעילה.
3. כדי שקבצים הממופים לזיכרון לא יהיו בעלי גודל שהוא כפולה של גודל הדף של הזיכרון המדומה, נדרש מנגנון מעקב מיוחד אחר גודל הקבצים הללו.
4. אם לא משתמשים ב-buffer cache, שיטת i-nodes יכולה לגרום למספר גבוה של גישות לדיסק בעת עבודה עם קבצים גדולים יחסית ל-FAT.
5. FAT משתמשת ב-write-through buffer cache.
6. מערכת קבצים log-structured נועדה לייעל את הביצוע של עדכונים תכופים של כמויות קטנות מאוד של נתונים.
7. מערכת ההפעלה UNIX משתמשת ברשימת היכולות להגנה על העצמים במערכת הקבצים שלה.

4.9 רשימת מושגים

להלן רשימת המושגים המופיעים בפרק זה (על פי סדר הופעתם בפרק):

קובץ	file	יחידת מידע לוגית לאחסון במצע חיצוני (דיסק, תקליטור, קלטת וכדומה). מכיוון שקבצים הם מבנים לוגיים, מימושם נעשה בתוכנה ולא בחומרה. מערכת ניהול הקבצים (שהיא חלק ממערכת ההפעלה) אחראית למימוש הקבצים.
ספרייה/ מדריך	directory	קובץ מערכת (קובץ מיוחד שמבנהו נקבע על ידי מערכת ההפעלה) שמשמש לארגון היררכי של מערכת הקבצים. מימוש ספציפי של ספריות שונה ממערכת למערכת, אך בכל מימוש, ספרייה היא קובץ שמצביע על קבצים וספריות ששייכים לאותה ספרייה.
קובצי מערכת עם גישה ברמת תווים	character special files	קובץ מערכת מיוחד שמדמה התקן קלט/פלט תווי (character I/O device). ב-UNIX קבצים מסוג זה נמצאים בספרייה /dev/.

קובצי מערכת עם גישה ברמת בלוקים	block special file	קובץ מערכת מיוחד שמדמה התקן קלט/פלט עם גישה ברמה של בלוק (block I/O device). ב-UNIX קבצים מסוג זה נמצאים בספרייה /dev/.
גישה אקראית	random access	צורת גישה שבה ניתן לפנות בכל רגע לכל כתובת בתוך קובץ, ללא תלות בסדר הפניות.
גישה סדרתית	sequential access	צורת גישה (שאפיינה בעיקר מערכות ישנות שהשתמשו בסרט מגנטי כמצע אחסון חיצוני) שבה ניתן לפנות לבתים או לרשומות בתוך קובץ אך ורק לפי סדר הופעתם בקובץ.
תכונות של קובץ	file attributes	מידע על הקובץ אשר חיוני מבחינת מערכת ההפעלה ו/או המשתמש (לדוגמה: גודל הקובץ, זמן הגישה האחרונה, הרשאות גישה וכדומה). אוסף התכונות של קובץ שונה ממערכת למערכת.
כותרת של קובץ	file header	בלוק של מידע בתחילת הקובץ אשר מכיל את תכונות הקובץ. לשמירת התכונות כחלק מתוכן הקובץ עצמו יש חסרונות רבים, ולכן בדרך כלל מערכת הקבצים תשמור את תכונות הקבצים בדרך אחרת. גם כאשר התכונות אכן ממומשות כתוספת לקובץ, משאירים את הטיפול בשמירת התכונות למערכת ההפעלה, ולכן מימוש פיזי זה הוא שקוף מבחינת המשתמש.
קובץ ממופה לזיכרון	memory-mapped file	קובץ שחלקו ממוקם (זמנית) בזיכרון הראשי, דבר המאפשר לגשת אל תוכנו באמצעות גישות זיכרון ולא באמצעות פונקציות גישה לקובץ. במערכות אשר תומכות במיפוי קבצים לזיכרון, מוגדרות קריאות מערכת מתאימות למיפוי וביטול המיפוי (לדוגמה, mmap ו-munmap במערכת ההפעלה UNIX).
מימוש קבצים באמצעות הקצאה רציפה	contiguous allocation	צורת מימוש קבצים שבה לכל קובץ מוקצה שטח רצוף על גבי מצע האחסון (לדוגמה, בלוקים עוקבים בדיסק).
מימוש קבצים באמצעות הקצאה בשיטת רשימה מקושרת	linked list allocation	צורת מימוש קבצים שבה כל קובץ נשמר כרשימה מקושרת של בלוקים בדיסק. כל בלוק ששייך לקובץ מכיל, מעבר לתוכן הקובץ, גם מצביע לבלוק הבא השייך לאותו הקובץ. גרסה משופרת של השיטה היא רשימה מקושרת באמצעות אינדקס (linked list allocation using an index). בשיטה זו המצביעים אינם שמורים בתוך הבלוקים, אלא בטבלה (אינדקס) הנמצאת בזיכרון באופן קבוע. מספר הכניסות בטבלה הוא כמספר הבלוקים בדיסק, ובכל כניסה שמור מצביע לבלוק הבא השייך לאותו קובץ, או 0 (אם הכניסה מתאימה לבלוק האחרון בקובץ, או ערך שאומר שהבלוק פנוי).
רכיב בשיטת ארגון קבצים	I-Node	הרכיב הבסיסי במימוש של מערכת הקבצים במערכות שונות (לדוגמה, UNIX). רכיב זה הוא מבנה נתונים שמכיל את תכונות הקובץ ואת המצביעים לבלוקים המכילים את תוכן הקובץ.
בלוק המצביעים	indirect block	מונח זה מופיע בהקשר של מימוש הקבצים באמצעות I-Nodes, ומתייחס לבלוק שמכיל מצביעים לבלוקים אחרים (כאשר הבלוקים שמצביעים עליהם הם בלוקים של נתונים או בלוקים של מצביעים מרמה נמוכה יותר). בלוקים מסוג זה מאפשרים גמישות רבה בארגון הפנימי של הקבצים.
קישור לקובץ	link	קישור "קשיח" (hard link) – יצירת שם לקובץ. ישנן מערכות (לדוגמה, UNIX) שמאפשרות לפנות לאותו קובץ בנתיבים ובשמות שונים (כלומר, נתיבים /home/usr1/file1 ו-/home/usr2/proj/file2 יכולים להתייחס לאותו קובץ). אם קיים יותר מקישור אחד לקובץ, רק מחיקה של כל הקישורים גורמת למחיקת הנתונים מהדיסק.

האוניברסיטה הפתוחה
THE OPEN UNIVERSITY OF ISRAEL
الجامعة المفتوحة
Yair Kurtzman

File #0006182 belongs to Yair Kurtzman- do not distribute

		קישור סמלי (symbolic/soft link) – קובץ מיוחד אשר מכיל הפניה (נתיב מלא בדיסק או ברשת) לקובץ אחר. קישור סמלי אינו מונע מחיקה של קובץ היעד. אם קובץ היעד נמחק, ניסיון לפנות אליו באמצעות קישור סמלי גורם הודעת שגיאה.
מכסות לשימוש מוגבל בשטח של הדיסק	disk quotas	ישנן מערכות רבות-משתמשים שמאפשרות למנהל המערכת להטיל מגבלות על מספר הקבצים ועל מספר הבלוקים שמשתמש כלשהו מורשה לנצל.
אמינות מערכת הקבצים	file system reliability	עמידות של המערכת בפני איבוד או שיבוש מידע היכולים להיגרם מסיבות שונות.
עקביות מערכת הקבצים	file system consistency	תאימות בין מצב מערכת הקבצים כפי שהוא משתקף ממבני הנתונים הפנימיים שלה ובין מצבה האמיתי.
מימושי תוכנה של זיכרון המטמון	buffer cache, block cache	שיטה להורדת מספר הפניות לדיסק: אוסף מצומצם של בלוקים מהדיסק נמצאים בזיכרון הראשי, וגישות לבלוקים אלה מתבצעות ללא צורך בפנייה לדיסק בפועל. ישנן שתי צורות מימוש של זיכרון המטמון לבלוקים: write-through cache – מימוש שבו כל בלוק ששונה בזיכרון נכתב מיד אל הדיסק. non-write-through cache – מימוש שבו הבלוקים אינם נשמרים בדיסק מיד אחרי השינוי, אלא באחד מהמקרים האלה: כאשר בלוק מפונה מזיכרון המטמון, כאשר תהליך מיוחד שרץ ברקע שומר את כל הבלוקים ששונו בזיכרון המטמון, או כאשר המשתמש מבקש זאת במפורש. חשוב לא לבלבל בין המונח block cache לבין המונח cache, אשר בדרך כלל מתייחס לרכיב חומרה מיוחד שהגישה אליו מהירה מאוד.

פרק 5

קלט/פלט

האוניברסיטה הפתוחה
THE OPEN UNIVERSITY OF ISRAEL
الجامعة المفتوحة
Yair
Kurtzman

File #0006182 belongs to Yair Kurtzman- do not distribute

5.1 חומרה של התקני קלט/פלט
5.1.1 התקני קלט/פלט
5.1.2 בקרים (device controllers)
5.1.3 שיטות מיעון של התקני קלט/פלט
5.1.4 גישה ישירה לזיכרון
5.1.5 חזרה נוספת על פסיקות חומרה

אחד התפקידים החשובים ביותר של מערכת ההפעלה הוא ניהול של התקני קלט/פלט. בגלל הריבוי של התקני קלט/פלט והשוני הגדול במהירות העברת הנתונים ובממשקים שלהם, קיימת חשיבות רבה ליצירת פתרון המספק גישה עקבית ואחידה לספקטרום רחב מאוד של התקנים.

מערכת ההפעלה מסתירה את המורכבות של בקרת התקני קלט/פלט, פענוח שגיאות והטיפול בהן. היא מבצעת טיפול בפסיקות באופן המתייחס להבדלים בזמני העברת נתונים של התקני קלט/פלט שונים. מערכת ההפעלה מספקת ממשק אחיד להתקני קלט/פלט הנבדלים זה מזה בגרעיניות של חלוקת נתונים ליחידות. היא דואגת למדיניות של תזמון בקשות לנתונים בהתאם לאופיין של הבקשות ובהתחשב בקדימותם של התהליכים שיזמו את הבקשות.

בסעיף 5.1 נכיר היבטים אחדים הקשורים להתקני קלט/פלט. בסעיף 5.2 נעיין בסוגיות הקשורות לתוכנת ניהול קלט/פלט. סעיף 5.3 מציג את ההיררכיה השכבתית של תוכנת קלט/פלט. סעיף 5.4 עוסק בדיסקים. החומר שנמצא בספר הלימוד החל מעמ' 390 ועד סוף הפרק הוא חומר רשות.

5.1 חומרה של התקני קלט/פלט

נסקור כעת כמה מהתכונות הפיזיות של התקני קלט/פלט. להיכרות עם התכונות הללו יש השלכות הן על תוכנת ניהול הקלט/פלט של מערכת ההפעלה שאותה נכיר בסעיף 5.3 והן על טכניקות התכנות ברמת המתכנת.

5.1.1 התקני קלט/פלט

באופן כללי, ניתן לחלק את התקני הקלט/פלט לשתי קטגוריות:

- ***התקן בלוקים*** **(*block devices*)** – התקנים המאחסנים מידע בבלוקים בגודל קבוע. לכל בלוק ניתן להתייחס לפי כתובת הבלוק, ולכן ההתייחסות לבלוק מסוים יכולה להתבצע באופן בלתי תלוי בהתייחסות לשאר הבלוקים. לדוגמה, דיסק קשיח ו-CD-ROM הם התקני בלוקים.

האוניברסיטה הפתוחה THE OPEN UNIVERSITY OF ISRAEL الجامعة المفتوحة
Yair Kurtzman

File #0006182 belongs to Yair Kurtzman- do not distribute

- ***התקנים תוויים*** **(*character devices*)** – פועלים עם רצפים של תווים. לא ניתן לדרוש מהתקן תווי לקבל תו מסוים מכיוון שאין לתווים כתובות, ולכן לא ניתן לבצע פעולת הזזה כמו שניתן לבצע על בלוק בהתקן בלוקים. מדפסות, עכבר, כרטיסי תקשורת הם דוגמאות להתקנים תוויים.

5.1.2 בקרים (device controllers)

למעשה, מערכת ההפעלה מפעילה כל התקן באמצעות כרטיס אלקטרוני מיוחד הנקרא **בקר** (***controller***). הבקר מציג בפני מערכת ההפעלה ממשק לניהול כל פעולות הקלט/פלט האפשריות בהתקן זה. אוסף הפקודות של הבקר יוצר ממשק שבאמצעותו יש לפנות אליו, וברוב המקרים ממשקים אלה כפופים לסטנדרטים.

כך, למשל, אחד הסטנדרטים הוא סטנדרט SCSI. בקרים עם ממשק SCSI מספקים אפשרות לחיבור של עד 16 דיסקים קשיחים. מערכת ההפעלה יכולה להפעיל את הדיסקים הללו באמצעות פקודות שהיא נותנת לבקר.

5.1.3 שיטות מיעון של התקני קלט/פלט

מתן פקודות לבקרי התקני קלט/פלט מתבצע על ידי כתיבה לאוגרים של בקר ו/או לחוצצים מיוחדים שמסוגלים להכיל נתונים. המיעון של אוגרים וחוצצים אלו יכול להתבצע בשלוש שיטות (ראו איור 2-5 בספר הלימוד):

1. לבקרי התקנים יש מרחב כתובות נפרד ממרחב הכתובות של הזיכרון הראשי.
2. בקרי התקנים משתמשים במרחב כתובות המשותף עם הזיכרון הראשי. לשיטה זו קוראים ***memory mapped I/O***.
3. השיטה המעורבת, שבה למרחב של הזיכרון הראשי ממופים החוצצים ואילו כתובות של אוגרי הבקרה ממופות למרחב נפרד.

היתרונות של memory-mapped I/O

1. כאשר אוגרי הבקרה והחוצצים ממופים לכתובות של מרחב הזיכרון הראשי, אפשר להתייחס אליהם כאל משתנים בשפת תכנות גבוהה ולא בשפת סף.
2. סוגיית ההגנה על התקני קלט/פלט מפני תהליכים של משתמשים אחרים נפתרת במסגרת מנגנון ההגנה הקיים. לדוגמה, אם כתובות של התקנים שונים ממופות למסגרות שונות של זיכרון ראשי, הקצאת מסגרות שבהן אוגרי בקרה של התקן מסוים ממופים לתהליך יכולה למנוע מתהליכים אחדים לבצע פעולות בהתקן.
3. בהוראה אחת של שפת סף ניתן להתייחס הן לכתובות בזיכרון הראשי והן לנתונים של התקני הקלט/פלט. דבר זה יכול לחסוך העתקה של נתוני הקלט/פלט מהזיכרון הראשי לחוצצים של ההתקן ולהפך.

החסרונות של memory-mapped I/O:

1. החומרה צריכה לאפשר ביטול caching של אזורים בזיכרון הראשי כדי למנוע שימוש בתוכן ישן. בפרק 3 ראינו שאפשרות כזאת יכולה להינתן ברמה של דפים.

2. ראו איור 5-3a בספר הלימוד. כל הוראה של CPU המכילה פנייה לכתובות הזיכרון עוברת באפיק חומרה ומצריכה תגובה של כל ההתקנים המחוברים לאותו האפיק, מכיוון שההתקנים צריכים לזהות אם הפנייה מתייחסת אליהם.

5.1.4 גישה ישירה לזיכרון

ללא קשר לשיטת מיפוי הכתובות של חוצצי נתונים של התקני הקלט/פלט שהזכרנו בסעיף הקודם, נדרשת העברה של הנתונים מהחוצצים הללו אל הזיכרון הראשי כדי שתהליכי המערכת יוכלו להשתמש בהם. העתקות בכיוון ההפוך נדרשות כדי לבצע את פעולת הפלט. העתקות אלו דורשות זמן CPU, אלא אם כן הבקר של התקן הקלט/פלט תומך באפשרות של גישה ישירה לזיכרון (DMA – Direct Memory Access).

בקר שתומך ב-DMA מקבל הוראה לבצע פעולת קלט/פלט, כאשר הוראה כוללת את הכתובת בזיכרון הראשי שבה יש לאחסן נתונים או שממנה מתבצעת העתקת נתונים. לאחר קבלת ההוראה, הבקר מבצע פעולת קלט/פלט ואינו נזקק ל-CPU. כלומר, העתקה של נתונים מזיכרון ראשי להתקן מבוצעת על ידי בקר DMA, ובמקרה של הבאת נתונים מהתקן קלט/פלט לזיכרון הראשי, הבקר דואג לשכנם בזיכרון לפי הכתובת שניתנה לו.

בקרי DMA מסוגלים לבצע העברה של נתונים באחת משתי השיטות האלה:

- ***work-at-a-time mode*** – בשיטה זו בקר פועל עם מילות זיכרון שאותן הוא מעביר בזו אחר זו. לכל העברה, הבקר מנצל (גונב) מחזור אחד של CPU, ולכן השיטה נקראת ***cycle stealing***. מכיוון שההעברות מפוזרות על ציר הזמן, תיגרם רק האטה קלה במהירות ה-CPU. אם צריך להעביר כמות גדולה של מילים, התקורה של זמן הנעילה של אפיק הזיכרון יכולה להיות גבוהה למדי.
- ***block mode*** – בשיטה זו בקר DMA מעביר בלוק שמורכב מכמה מילות זיכרון. העברה בשיטה כזאת יעילה יותר מכיוון שנעילת האפיק מתבצעת רק פעם אחת, אך היא יכולה לגרום לכך שה-CPU יצטרך להמתין זמן רב יותר לנתונים מהזיכרון (משום שהאפיק תפוס).

שאלה 1

אם בכל מקרה ה-CPU חייב להמתין לשחרור של אפיק זיכרון, מהו ההיגיון בשימוש בשיטה של גישה ישירה לעומת השיטה שבה ה-CPU מבצע העתקת נתונים מן הזיכרון הראשי להתקני הקלט/פלט בעצמו?

5.1.5 חזרה נוספת על פסיקות חומרה

בסעיף 1.3.1 של מדריך הלמידה עסקנו בטיפול בפסיקות חומרה. סעיף 5.1.5 בספר הלימוד חוזר לדון בפסיקות חומרה. לא נחזור על הדברים כאן. עם זאת, מומלץ לקרוא את סעיף 5.1.5 בספר הלימוד.

האוניברסיטה הפתוחה THE OPEN UNIVERSITY OF ISRAEL الجامعة المفتوحة
Yair Kurtzman

File #0006182 belongs to Yair Kurtzman- do not distribute

5.2 עקרונות למימוש קלט/פלט בתוכנה
5.2.1 מטרות תוכנת הקלט/פלט
5.2.2 קלט/פלט באמצעות המעבד הראשי
5.2.3 קלט/פלט מונחה אירועים
5.2.4 קלט/פלט באמצעות DMA

5.2 עקרונות למימוש קלט/פלט בתוכנה

בסעיף 5.3 מתוארות שכבות של תוכנת קלט/פלט של מערכת ההפעלה. בסעיף 5.2.1 נציג את העקרונות שתוכנת הקלט/פלט של מערכת ההפעלה מיישמת אותם. לאחר מכן, בסעיף 5.2.2, נבחן היבטים של השימוש בעקרונות קלט/פלט.

5.2.1 מטרות תוכנת הקלט/פלט

נמנה את היעדים:

- ***אי-תלות בהתקנים*** (*device independence*) – מערכת ההפעלה צריכה לספק ממשק אחיד לעבודה עם התקנים שונים ולהסתיר את ההבדלים הנובעים מייחודיותם של התקנים שונים.
- ***כינוי אחד*** (*uniform naming*) – מערכת ההפעלה צריכה לספק כינוי אחיד להתקנים שונים. לדוגמה, במערכת ההפעלה Unix משתמש יכול לפנות לכל ההתקנים לפי שם הנתיב במערכת הקבצים.
- ***טיפול בשגיאות*** (*error handling*) – טיפול בשגיאות הן בחומרת הקלט/פלט והן בכל רמות התוכנה. למשל, בקר הדיסקים בודק את תקינות הנתונים שקיבל מהדיסק, ובמקרה של בעיה – מקצת הבקרים יכולים לתזמן קריאה חוזרת. אם לא ניתן לפתור את הבעיה ברמה של החומרה, היא מגולגלת לפתחה של שכבת תוכנת הקלט/פלט הנמוכה ביותר, שעליה נלמד בסעיף 5.3; זו היא תוכנת מתאם התקן. במקרים מסוימים, כאשר שום רמה של תוכנת קלט/פלט במערכת ההפעלה איננה מסוגלת להתגבר על הבעיה, מתקבלת הודעת שגיאה בתוכנית המשתמש.
- העברת נתונים

א) ***בצורה סינכרונית*** (*synchronous transfer*) – המעבד נותן הוראה לבקר לבצע העברת נתונים וממתין לסיום ההעברה.

ב) ***בצורה א-סינכרונית*** (*asynchronous transfer*) – המעבד נותן פקודה להעברה של נתונים, ועד אשר יגיעו הנתונים – הוא עובר לבצע משימות אחרות. עם סיום העברת הנתונים, המעבד מקבל פסיקה. על מערכת ההפעלה לספק דרך שבה העברת הנתונים בצורה א-סינכרונית תיראה כהעברת נתונים בצורה סינכרונית בלי לפגוע בניצולת של המעבד. כלומר, מבחינת התהליך, הפעולות יהיו סינכרוניות, גם כאשר מבחינה פיזית המצב שונה לחלוטין. מערכת ההפעלה מעוניינת כי פעולות אלו יהיו א-סינכרוניות וכך יתפנה המעבד לעבודה במקביל

על משימות אחרות. לעומת זאת, תהליך רגיל מעוניין על פי רוב בפעולות קלט/פלט סינכרוניות, כלומר, תהליך המעוניין להמתין לביצוע בקשות הקלט/פלט ורק לאחר מכן להמשיך, יועבר למצב חסום עד לתום פעולות הקלט/פלט ויתעורר מיד עם השלמתן. מבחינת התהליכים, הפעולות יבוצעו אפוא באופן סינכרוני.

- ***הטמנה*** **(*buffering*)** – הביצועים של פעולות הקלט/פלט יכולים להיות מושפעים מהטמנה של נתונים בזיכרון הראשי כדי לחסוך זמן גישה לדיסק להעברה של נתונים בעתיד.
- מערכת הפעלה צריכה לאפשר ***שיתוף*** **(*sharing*)** של התקני קלט/פלט אם ניתן לעשות זאת. לחלופין, יש לתת גם אפשרות של **שימוש בלעדי** בהתקנים בצורה שתמזער את הסיכויים להיווצרות מצבי קיפאון.

שאלה 2

מדוע רוב התוכניות מתבססות על ההנחה שפעולות הקלט והפלט מתבצעות באופן סינכרוני?

ברוב המקרים המשך פעולת התוכנית תלוי בקלט ולעתים גם בפלט. עם זאת, לעתים קיים צורך בפעולות א-סינכרוניות.

שאלה 3

תארו לפחות מקרה אחד שבו קיים צורך בפעולה א-סינכרונית של קלט/פלט בתהליך.

5.2.2 קלט/פלט באמצעות המעבד הראשי

קיימות שלוש דרכים לביצוע של פעולות קלט/פלט. שיטת קלט/פלט באמצעות המעבד היא שיטה מיושנת שלא משתמשים בה כיום כמעט. ראו איור 5-8 בספר הלימוד, שבו מוצגת תוכנית שמדפיסה את התוכן של החוצץ buffer למדפסת. בדוגמה המוצגת, כדי לבצע הדפסה צריך להעתיק את התוכן של החוצץ ממרחב הזיכרון של תהליך משתמש לגרעין של מערכת ההפעלה. ראו איורים 5-7(a) ו-5-7(b) המדגימים זאת. לאחר מכן, המעבד רץ בלולאה בניסיון להעביר רצף של תווים למדפסת. ייתכן שבדיקה של מוכנות המדפסת לקבלת תו תגרום להמתנה פעילה (busy waiting).

ברור שהחיסרון המובהק של השיטה הוא העסקת ה-CPU עד סיום פעולת הקלט/פלט, ולכן השיטה איננה בשימוש במערכות רב-ייעודיות.

5.2.3 קלט/פלט מונחה אירועים

בשיטה זו המעבד איננו נמצא בלולאת המתנה פעילה לאחר העתקה של חוצץ נתונים לגרעין. לעומת זאת, מערכת ההפעלה מעבירה את התהליך שביקש קלט/פלט למצב חסום, המעבד מבקש מבקר לבצע פעולת קלט/פלט ומתפנה לעיסוקיו האחרים. בתום

האוניברסיטה הפתוחה THE OPEN UNIVERSITY OF ISRAEL الجامعة المفتوحة
Yair Kurtzman

File #0006182 belongs to Yair Kurtzman- do not distribute

פעולת הקלט/פלט, הבקר שולח אל המעבד פסיקה, והמעבד יכול להמשיך בפעולת קלט/פלט אחרת או לשחרר את התהליך ולהעבירו למצב מוכן לריצה אם כל פעולות הקלט/פלט הנחוצות, נכון לרגע זה, הסתיימו.

איור 5-9 בספר הלימוד מציג את ההדפסה של מחרוזת למדפסת. בחלק a של האיור לולאת while גורמת להמתנה פעילה. בכל איטרציה של הלולאה, עד לסיום של מחרוזת ההדפסה, בקר המדפסת מקבל הוראה להדפסת תו. בינתיים המעבד מתפנה לפעולות אחרות. עם סיום הדפסת תו, הבקר שולח למעבד פסיקה, ומבחינת המשתמש מתחילה האיטרציה השנייה של לולאת while שמדפיסה תו נוסף.

שיטה זו איננה בשימוש עם התקנים מהירים, מכיוון שיש לה תקורה גבוהה מאוד של טיפול בפסיקות.

5.2.4 קלט/פלט באמצעות DMA

שיטת הגישה הישירה לזיכרון מתגברת על הקשיים שכרוכים בשימוש בשתי השיטות הקודמות. המעבד נותן פקודה לבקר, מציין מהיכן ולאן לבצע את ההעברה ומתפנה לעיסוקיו. בתום העברת כל הנתונים, הבקר שולח למעבד פסיקה, ומערכת ההפעלה מטפלת בה ומבצעת תזמון תהליכים לאחר תום הטיפול בפסיקה.

5.3 תוכנת קלט/פלט – חלוקה לשכבות
5.3.1 מנהל הפסיקות (interrupt handler)
5.3.2 מתאמי התקנים (device drivers)
5.3.3 תוכנת קלט/פלט המשוחררת מאילוצי חומרה
5.3.4 תוכנת קלט/פלט ברמת תהליכי המשתמשים

5.3 תוכנת קלט/פלט – חלוקה לשכבות

ניתן להתייחס אל תוכנת הקלט/פלט של מערכת ההפעלה כאילו היא מאורגנת בארבע שכבות. אין חלוקת תפקידים סטנדרטית בין שכבה לשכבה ואת מקצת התפקידים אפשר לממש ביותר משכבה אחת. אנחנו נסקור את המכלול הטיפוסי של הפעולות המבוצעות בכל אחת מן השכבות.

5.3.1 מנהל הפסיקות (interrupt handler)

שכבה זו עוסקת בעיקר בהסתרה של פסיקות חומרה. הסיבות לכך הן:

- הפסיקות הן תלויות חומרה באופן מובהק, ולכן תפקידה של מערכת ההפעלה כמכונה מורחבת הוא לספק מבנים ופעולות שאינם תלויי חומרה. תהליכים

מסתמכים על פסיקות ואף עשויים לבקש במפורש פעולות הדורשות פסיקות, כגון הרדמה או הערה. אולם פעולות אלו הופכות לבלתי תלויות בחומרה, למשל סמפור.

- הסיבה השנייה נובעת מהצורך לספק הגנה על החומרה מכיוון שלא ייתכן שלמשתמש תהיה גישה כלשהי לפסיקות החומרה של המערכת. גישה כזו עשויה לתת לו שליטה מלאה על משאבי המערכת.

5.3.2 מתאמי התקנים (device drivers)

השכבה השנייה של תוכנת קלט/פלט מכילה מתאמים להתקני קלט/פלט שונים. מתאם ההתקן הוא תוכנית שמספקת ממשק להפעלת התקן פיזי ביחס לשכבה השלישית של תוכנת הקלט/פלט, שהיא השכבה הבלתי תלויה בהתקן. מתאמי התקנים מכילים קוד שמסתיר מהשכבה של התוכנה הבלתי תלויה בהתקנים את מורכבותה של בקרת ההתקנים הפיזיים.

הפעלה של התקן מצריכה גישה לאוגרים ולחוצצים של בקר התקן, ולכן מתאמי התקנים הם חלק מגרעין מערכת ההפעלה. ממשקים של מתאמי התקנים המשרתים את השכבה הבלתי תלויה בהתקנים מוגדרים על ידי מערכת ההפעלה, ויצרני התקנים פיזיים מספקים יחד עם ההתקן גם תוכנה מתאמת.

5.3.3 תוכנת קלט/פלט המשוחררת מאילוצי חומרה

התפקידים של רובד זה הם:

- **קביעת ממשקים אחידים לשכבה של מתאמי התקנים** – כפי שניתן להבין מאיור 5-14a, היעדר ממשק אחיד יכול לדרוש מאמץ תכנותי לא מבוטל בפיתוח מערכת הפעלה להפעלה של התקנים שונים, מכיוון שלכל מתאם התקן צריך ליצור מודול נפרד המתממשק איתו. לעומת זאת, באיור 5-14b מוצג המצב שבו הממשק לסוגים שונים של מתאמי התקנים הוא אחיד.
- **הטמנה של נתוני קלט/פלט בחוצץ (או חוצצי) ביניים** – כבר נוכחנו ביעילותה של טכניקת ההטמנה. גם במקרה של פעולות קלט/פלט, הטמנה של נתונים מאפשרת גישה יעילה יותר לנתונים אלה הן לצורך העברה של הנתונים ממרחב המשתמש של תהליך להתקן הקלט/פלט והן לצורך ההעברה בכיוון ההפוך.
- **טיפול בשגיאות** – פונקציה זו קיימת בכל השכבות של תוכנת הקלט/פלט, בשכבה שאיננה תלויה באילוצי חומרה. עיקר תפקידו של הטיפול בשגיאות הוא טיפול בשגיאות תכנות כגון ניסיונות להעברה של פרמטרים שונים של קלט/פלט או טיפול בשגיאות המדווחות על ידי מתאמי התקנים כגון כישלון של העברת נתונים מהתקן לחוצץ של מערכת ההפעלה.
- **הקצאה ושחרור של התקנים שאינם ניתנים לשיתוף** – הקצאה ושחרור משאבים בלעדיים (כגון CD-ROM) צריכים להתבצע בצורה שתקטין ככל האפשר את הסיכוי להיווצרות מצבי קיפאון. קיימים שני מישורים שבהם ניתן לטפל בסוגיות אלו:

האוניברסיטה הפתוחה THE OPEN UNIVERSITY OF ISRAEL الجامعة المفتوحة
Yair Kurtzman

File #0006182 belongs to Yair Kurtzman- do not distribute

- במישור התכנותי הבעיה מגולגלת לפתחו של המתכנת. אם המתכנת רוצה לקבל משאב בלעדי שמצא כי הוא תפוס, הוא יקבל הודעת שגיאה. באחריות המתכנת גם למנוע מצב קיפאון.
- מערכת ההפעלה יכולה לשים בהמתנה תהליך שביקש גישה למשאב בלעדי ולהנהיג מדיניות של הימנעות כלשהי שתקטין את הסיכוי להיווצרות מצבי קיפאון או תמנע אותם לחלוטין.

- ***גודל בלוק אחיד*** – להתקני קלט/פלט שונים ייתכנו גדלים שונים של יחידות העברת נתונים (בלוקים). אחד התפקידים של שכבת קלט/פלט הוא לספק למערכת ההפעלה גודל בלוק אחיד.

5.3.4 תוכנת קלט/פלט ברמת תהליכי המשתמשים

אף שרוב תוכנות ה-I/O הן חלק ממערכת ההפעלה, קיימים שני סוגים של תוכנת קלט/פלט שנמצאים ברמה של המשתמש והם מהווים את השכבה האחרונה של תוכנת הקלט/פלט:

- ספריות סטנדרטיות של קלט/פלט

ו-

- שדים האחראים לריכוז של פעולות קלט/פלט

דוגמה לסוג הראשון היא ספריית stdlib שמכילה את הפונקציה fprintf. הפונקציה fprintf מאפשרת שינוי פורמט של נתונים המועברים אליה והטמנה של הנתונים בחוצץ פנימי של הספרייה.

דוגמה לשדים היא ה-spooler של מדפסת, שבו נדון בפרק 6. מדפסת מוקצית לשד באופן בלעדי והשד מבצע בקשות של תהליכים שרוצים לשלוח נתונים להדפסה.

5.4 דיסקים
5.4.1 חומרת הדיסק ותזמון זרוע הדיסק
5.4.2 כונני מצב מוצק (Solid State Drive - SSD)
5.4.3 מערך דיסקים (RAID)

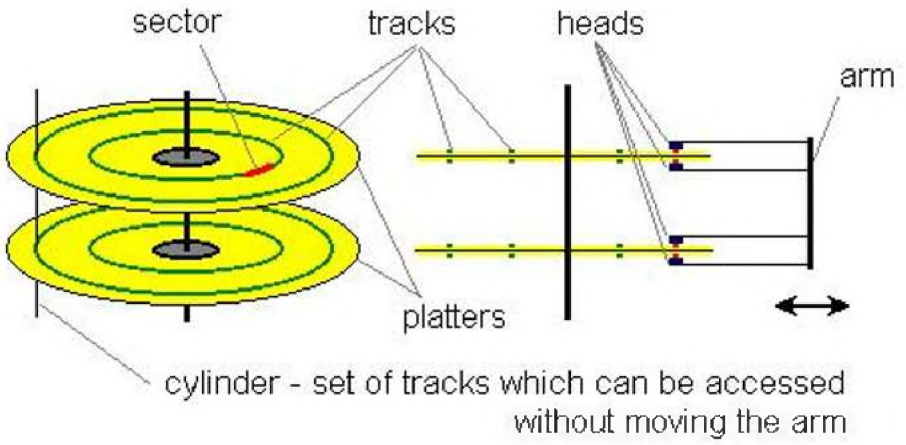
5.4 דיסקים

בסעיף זה נראה מבנה של דיסקים מגנטיים ונציג שיטה של אחסון נתונים באמצעות מערך דיסקים המכונה RAID. לאחר מכן נסביר כיצד מתבצע פרמוט של דיסק ונראה אלגוריתם לתזמון זרוע של דיסק.

5.4.1 חומרת הדיסק ותזמון זרוע הדיסק

דיסקים מגנטיים

מתקן דיסק קשיח מורכב מכמה דיסקים נפרדים המסתובבים על ציר משותף בחלל ריק. כל דיסק בודד מחולק למסלולים (tracks) שמחולקים ***לגזרות*** (***sectors***). אוסף המסלולים בכל התקליטים שנמצאים במרחק זהה מציר הסיבוב נקרא גליל או צילינדר (cylinder). המידע נשמר על שני צידי התקליטורים של כל דיסק. כתיבה או קריאה של נתונים מתבצעות על ידי ראשים קוראים/כותבים (reading/writing heads). ראו האיור שלהלן:



איור 1

זמן העברת הנתונים מושפע מן הפרמטרים האלה:

זמן חיפוש המסלול (***seek time***) – זהו הזמן שבו הראש הקורא מתמקם על המסלול הדרוש.

זמן הסיבוב (***rotation time***) – זהו הזמן שבו דיסק מבצע סיבוב שלם מסביב לציר. זמן זה משפיע על מהירות החיפוש של הגזרה הדרושה. בממוצע, חיפוש הגזרה נמשך מחצית הזמן שנמשך הסיבוב השלם.

זמן העברת הנתונים (***transfer time***) – זמן ההעברה של הנתונים תלוי בשלושה פרמטרים: זמן הסיבוב, כמות הנתונים הדרושים וכמות הנתונים במסלול.

נסמן ב-S את זמן חיפוש המסלול.
נסמן ב-R את זמן הסיבוב המלא.
נסמן ב-b את כמות הנתונים הדרושה ב-bytes.
נסמן ב-N את כמות ה-bytes במסלול ונקבל כי זמן ההעברה של הנתונים הוא:

$$T = S + 0.5 * R + R * b / N$$

שאלה 4

אילו הנחות סמויות באות לידי ביטוי בנוסחה המחשבת את זמן העברת הנתונים?

האוניברסיטה הפתוחה THE OPEN UNIVERSITY OF ISRAEL الجامعة المفتوحة
Yair Kurtzman

File #0006182 belongs to Yair Kurtzman- do not distribute

אלגוריתמים לתזמון זרוע הדיסק

בקשות לדיסק מגיעות לבקר כדרישות לקריאה או לכתיבה של בלוקים מסוימים. בסעיף 5.4.1 ראינו שהפעולות הנדרשות להבאת בלוק מדיסק הן חיפוש מסלול, המתנה עד שגזרת דיסק נחוצה תחלוף על פני ראש קריאה/כתיבה, והעברה של נתונים. בהינתן דיסק בעל מהירות סיבוב נתונה, הפעולה שיש לה השלכה על היעילות היא פעולת חיפוש של מסלול (seek time). לכן יש חשיבות רבה לתזמון של זרוע הדיסק שעליו יושב ראש דיסק הקריאה/כתיבה.

אנחנו נראה אלגוריתמים אחדים לתזמון זרוע הדיסק, המבוססים על מסלולים של דיסק. כל בקשה לקריאה או לכתיבה של בלוק מתורגמת למספר המסלול שעליו הבלוק נמצא. האלגוריתמים שנלמד מתזמנים את תזוזות זרוע הדיסק בהסתמך על מספרי המסלולים הללו.

- **אלגוריתם התור FCFS (First Came First Served)** – זהו האלגוריתם הפשוט ביותר, והוא פועל על המסלולים לפי סדר הופעת הבקשות. אלגוריתם FIFO הוא האלגוריתם ההוגן ביותר, הואיל והבקשות מטופלות בהתאם לסדר הגעתן.
- **אלגוריתם זמן החיפוש הקצר, SSF (Shortest Seek First)** – אלגוריתם זה אינו מתייחס לסדר המסלולים לפי זמן ההגעה של הבקשות, אלא בוחר את המסלול הקרוב ביותר לזרוע הדיסק. במקרה של קיום בקשות בשני מסלולים קרובים במידה שווה, אפשר לבחור כיוון תנועה באופן אקראי. אלגוריתם SSF, עקב מדיניותו החמדנית, מקטין את מספר התזוזות של זרוע הדיסק, אולם הוא יכול לגרום להרעבה של בקשות שנמצאות במסלולים מרוחקים כאשר במשך זמן רב יגיעו בקשות הנמצאות על מסלולים קרובים יותר.
- **אלגוריתם המעלית (Elevator) או אלגוריתם סריקה (SCAN)** – האלגוריתם עובר על מסלולים שיש בקשות אליהם, בדומה למעלית. תחילה הוא מטפל בבקשות שכיוונן מהמסלולים הפנימיים למסלולים החיצוניים, ולאחר מכן הוא משנה את הכיוון ומטפל בבקשות שהגיעו מאז המעבר האחרון שלו, אך בכיוון ההפוך, ממסלולים חיצוניים למסלולים פנימיים. הכיוון של הטיפול בבקשות מתחלף בכל פעם שמגיעים למסלול אשר מעבר אליו, בכיוון התנועה הנוכחי, אין עוד בקשות ממתינות. אלגוריתם הסריקה מעדיף מסלולים אמצעיים, מכיוון שעל פניהם הוא חולף תמיד בשני כיווני תנועתו. לכן הבקשות שנמצאות על מסלולים אמצעיים ייהנו משירות עדיף, בעוד שהבקשות ממסלולים בפריפריה (מסלולים פנימיים ביותר וחיצוניים ביותר) יקופחו.
- **אלגוריתם הסריקה החוזרת (Circular SCAN)** – זוהי וריאציה של אלגוריתם המעלית עם כיוון סריקה אחד בלבד. עם סיום הסריקה, זרוע הדיסק חוזרת במהירות להתחלה וממשיכה לסרוק באותו כיוון. במילים אחרות, אלגוריתם הסריקה דומה מאוד לאלגוריתם המעלית, אלא שבמקום לעלות ולרדת בהתאם לדרישות הנוסעים, המעלית עולה בהתמדה תוך מילוי הבקשות בכיוון זה. כאשר המעלית מגיעה לקומה העליונה, היא חוזרת מיד (ריקה) לקומה הנמוכה ביותר שבה מצפים לה. כך, האלגוריתם אינו עובר על מסלולים אמצעיים פעמיים, אך ייתכן שעד שהוא מגיע לטפל במסלול, כבר ממתינה לו רשימה ארוכה של בקשות

לאותו מסלול. במקרים כאלה ניתן פשוט להגביל את אורך הרשימה ובכל סריקה לטפל רק במספר קטן של בקשות למסלול.

מכיוון שלאחר מציאת המסלול, ראש הדיסק ממתין בממוצע מחצית מזמן הסיבוב של הדיסק עד שהגזרה הרצויה מגיעה אליו, הרי שגם באלגוריתם FIFO יהיה זה הגיוני מאוד לטפל במספר קטן של בקשות לאותו מסלול בזמן שהמסלול חולף על פני ראש דיסק הקריאה/כתיבה. באמצעות שיפור כזה, מדיניות FCFS יכולה להקטין באופן משמעותי את מספר התזוזות של זרוע הדיסק, ובד בבד גם להיות הוגנת.

כדי לאפשר טיפול בגזרות החולפות מתחת לראש דיסק הקריאה/כתיבה, הדיסקים מצוידים בזיכרון מטמון. קיימות טכניקות רבות לניהול של זיכרון מטמון זה. דוגמה פשוטה מאוד היא זו שבה מחצית מזיכרון המטמון יכולה להיות מיועדת לשמירה של נתונים שחולפים על פני הראש וכך לחסוך חיפוש מסלולים לעתיד לבוא, ומחציתו יכולה להיות מיועדת לבלוקים שצריך לכתוב במסלול החולף.

יש לציין כי זיכרון המטמון של הדיסק מנוהל על ידי בקר הדיסק, ולמערכת ההפעלה אין שליטה עליו. בהמשך נראה כיצד מערכת ההפעלה עושה הטמנה של בלוקים שמגיעים אליה מהדיסק באזור של הזיכרון הראשי המיועד לצורך זה, והמכונה buffer cache.

שאלה 6

איזה מבין האלגוריתמים שסקרנו הוא הבטוח ביותר למניעת הרעבה?

התשובה היא אלגוריתם התור ו-Circular SCAN המשופר. באלגוריתמים הללו מספר הבקשות המטופלות למסלול הוא קצוב, ולכן מובטח כי בתוך זמן סביר יתבצע מעבר לטיפול במסלול אחר. שני האלגוריתמים האחרים משנים בהתמדה את העדיפויות למילוי בקשות בהתאם למרחקו של המסלול ממקומה הנוכחי של זרוע הדיסק (SSF) או לפי מספר הבקשות שממתינות במסלול (SCAN). לכן ייתכן שבקשה מסוימת תידחה לזמן רב, והתהליך המבקש יורעב. עם זאת, בהשוואה ל-SSF, סכנת ההרעבה פחותה בהרבה כאשר משתמשים באלגוריתם המעלית.

5.4.2 כונני מצב מוצק (Solid State Drive SSD)

כפי שראינו בסעיף 4.3.7, כונני SSD הם מהירים ,בעלי ביצועי קריאה וכתיבה אסימטריים ואינם מכילים חלקים נעים. הם יכולים לבוא בצורות שונות: יש כאלה שעומדים בתקן SATA עבור התקני אחסון המשמשים גם לדיסקים מגנטיים, אבל מכיוון ש-SATA תוכנן עבור דיסקים מכניים שהם איטיים לעומת טכנולוגיית פלאש, יותר ויותר כונני SSD מתממשקים למערכת באמצעות **זיכרון אקספרס לא נדיף (Non-Volatile Memory Express – NVMe)**, שהוא תקן לניצול טוב יותר של מהירות חיבור ה- PCI Express בין ה- SSD למערכת, אשר מאפשר גם ניצול המקביליות הזמינה ב- SSD עצמו.

האוניברסיטה הפתוחה THE OPEN UNIVERSITY OF ISRAEL الجامعة المفتوحة
Yair Kurtzman

File #0006182 belongs to Yair Kurtzman- do not distribute

מכיוון שלמחשבים מודרניים יש כמה ליבות וה-SSD מורכבים מכמה דפים, בלוקים וגם שבבים, משתפר הביצוע של בקשות במקביליות. כדי לאפשר זאת, NVMe תומך בכמה תורים.

ל-NVMe כמה יתרונות: ראשית, בעוד ממשק SATA מציע רק תור יחיד עם מספר קטן של כניסות, NVMeמאפשר תורים רבים וארוכים יותר (יותר מ- 64Kתורים עם יותר מ-64K פקודות I/O בכל אחד). כל תור מטופל במקביל, ובכך מאפשר לבקר לדחוף יותר פקודות לשבבי הפלאש ולהאיץ את קלט/פלט האחסון במידה ניכרת. שנית, כיוון שמערכת המחשב צריכה כעת פחות מכשירים כדי לתמוך באותו מספר פעולותI/O , צריכת החשמל ודרישות הקירור פוחתות.

5.4.3 מערך דיסקים (RAID)

מערך הדיסקים (RAID) הוא שיטה לאחסון נתונים בכמה דיסקים במקביל, המיועדת להגביר את שרידותם ושלמותם של הנתונים ולספק קצבים גבוהים של העברת נתונים ושל טיפול בבקשות, כאשר מבחינתה של מערכת ההפעלה מערך הדיסקים נראה כדיסק לוגי יחיד. שרידות של נתונים מושגת על ידי הוספה של דיסק עודף, ואילו קצבים גבוהים של העברת נתונים ושל טיפול בבקשות מתקבלים באמצעות העברת נתונים במקביל. סטנדרט תעשייתי מגדיר שבע רמות שונות של מערכי דיסקים. קיימות גם סכמות אחרות שפותחו על ידי חברות שונות אשר לא התקבלו כסטנדרט מוסכם על כולם.

החלוקה לרמות (עד סוף הסעיף) היא חומר רשות.

RAID level 0

בשיטה זו נתונים פזורים על פני דיסקים של מערך בפיסות (strips) בגודל קבוע (ראו איור 5-26a). כלומר הנתונים של הדיסק הלוגי שמערכת ההפעלה "רואה" למעשה מחולקים לפיסות ומפוזרים על דיסקים שונים של המערך.

היתרון של פיזור כזה הוא שגישה לנתונים בדיסק הלוגי מתורגמת לכמה גישות מקביליות לדיסקים של המערך, שבהם שוכנות הפיסות שמרכיבות את הנתונים המבוקשים. היעילות של הגישה תלויה בצורת הפיזור של הפיסות על גבי הדיסקים של המערך ובתבניות של בקשות לנתונים שמערך הדיסקים מקבל.

אם תבנית הבקשות הטיפוסית היא לבקש כמות נתונים רציפה מבחינה לוגית שהיא גדולה יחסית לגודלה של פיסה, אזי הדבר מביא לגישה מקבילית לדיסקים, וכתוצאה מכך להגדלה של קצב העברת הנתונים. כך שככל שגודל הפיסות קטן, כך גדל קצב העברת הנתונים.

אם תבנית הבקשות היא כמות גדולה של בקשות לנפחים קטנים של נתונים, אזי ישנה חשיבות לשאלה כיצד הבקשות יהיו פזורות על פני הדיסקים של המערך. באופן עקרוני, פיסות קטנות יותר יגרמו לפיזור טוב יותר, וגישה מקבילית לדיסקים תביא להגדלת קצב שירות הבקשות.

RAID level 1

השיטה הקודמת (level 0) איננה תורמת לשרידותם של דיסקים, ולכן רבים גם אינם מחשיבים אותה כ-RAID. למעשה, היא אף מקטינה את השרידות. אמנם בימינו דיסקים קשיחים הם אמינים למדי, אך הסיכוי שאחד מ-N דיסקים של מערך ייפגם גדול יותר באופן תיאורטי מהסיכוי שדיסק אחד ייפגם. והרי כשל של אחד הדיסקים של RAID level 0 מביא לאיבוד שלמותם של הנתונים.

שיטת level 1 מכפילה את מספר הדיסקים שהיו ב-RAID level 0 ומשתמשת בהם לגיבוי של דיסקים שהיו (ראו איור 5-26b). כל היתרונות של RAID level 0 נשמרים אפוא גם בשיטה החדשה. בנוסף, level 1 מגדיל כמעט פי שניים את היעילות של הגישה לנתונים שאפיינה את level 0, מכיוון שאפשר לספק בקשות במקביל גם מדיסקים של גיבוי. לגבי העדכון, היעילות אינה משתנה מכיוון שבכל כתיבה צריך לעדכן שני דיסקים, והעדכון מתבצע לפי האיטי מבין שני הדיסקים (כלומר, לפי הדיסק שהראש הקורא שלו רחוק יותר מיעד הכתיבה).

השרידות של RAID level 1 היא גבוהה מאוד. בעת כשל של אחד הדיסקים, ניתן להמשיך לעבוד מה-mirror שלו, ולאחר החלפה של הדיסק הפגום – להעביר נתונים מהדיסק ששרד לדיסק חדש. הבעיה עם שיטת level 1 היא העלות של שכפול דיסקים.

RAID level 2

שיטה זו עושה שימוש בגישה מקבילית לדיסקים של המערך, כך שכל הדיסקים של המערך משתתפים בכל בקשה של נתונים. בדרך כלל, הדיסקים במערך מתואמים כך שהראשים של כל הדיסקים נמצאים על אותם מסלולים וגזרות בעת ובעונה אחת. הנתונים מפוזרים על פני הדיסקים בפיסות קטנות מאוד. בדוגמה שבאיור 2-26c, אלה היחידות של הסיביות.

כל ארבע סיביות של נתונים מפוזרות על פני ארבעת הדיסקים הראשונים שבאיור (מובן שמספר הדיסקים אינו חייב להיות דווקא 4). שלושת הדיסקים הנותרים משמשים לשמירה של קוד לתיקון שגיאות. כך שלכל ארבע סיביות נוספות שלוש סיביות של קוד לתיקון שגיאות, ושביעיית הסיביות נשמרת על גבי שבעה דיסקים. בדרך כלל, הקוד לתיקון שגיאות הוא Hamming code שמסוגל לתקן סיבית אחת ולגלות שגיאה בשתי סיביות. בזמן קריאה לבקר של מערך דיסקים מועברת שביעיית סיביות, הבקר בודק ואם יש טעות בסיבית אחת הוא מתקן אותה בצורה מקוונת (on-line) ללא האטה בביצועים. כל כתיבה של נתונים דורשת חישוב של קוד לתיקון שגיאות וגישה לכל שבעת הדיסקים של המערך.

קצב העברת הנתונים ב-level 2 הוא גבוה מאוד מכיוון שפיסות הנתונים הן קטנות ומפוזרות על פני דיסקים שונים וזה מגביר את המקביליות. קצב הטיפול בבקשות עדיין דומה לדיסק רגיל מכיוון שרק בקשה אחת מטופלת בזמן נתון.

האוניברסיטה הפתוחה
THE OPEN UNIVERSITY OF ISRAEL
الجامعة المفتوحة
Yair
Kurtzman

File #0006182 belongs to Yair Kurtzman- do not distribute

כמות הדיסקים שדרושים ל-RAID level 2 היא N + logN כאשר N הוא מספר הדיסקים לשמירה של נתונים ו-log N הוא מספר הדיסקים לשמירה של מידע עודף לתיקון שגיאות. הערה: הנוסחה נותנת רק סדר גודל. אין נוסחה מדויקת לחישוב מספר הסיביות הכללי של קוד המינג על פי מספר סיביות הנתונים. בניגוד ל-RAID level 1 שיטת level 2 דורשת פחות דיסקים, אך היא עדיין בזבזנית למדי במונחים של חומרה.

RAID level 3

בשיטה זו הנתונים מאורגנים בצורה דומה לצורת ארגון הנתונים ב-RAID level 2. ההבדל הוא שלמידע עודף משתמשים בדיסק אחד בלבד (ראו איור 5-26d). המידע העודף הוא ה-parity bit של סיביות מדיסקים של נתונים.

בשיטה של RAID level 2 התיקון התאפשר בכל מקרה שהייתה טעות בסיבית אחת של שביעייה, ללא קשר לדיסק. במקרה של RAID level 3, השימוש ב-parity bit לצורך תיקון מצריך ידיעה באיזה דיסק הייתה הבעיה. לכן במקרה של כשל באחד מהדיסקים של המערך ניתן לשחזר את המידע שהיה שמור עליו בעזרת שאר הדיסקים של המערך. בהשוואה לכמות השגיאות שיכולות לקרות בדיסק, השיטה של RAID level 3 מאפשרת שרידות מספקת למספר לא קטן של צרכים.

הדברים שנאמרו על RAID level 2 לגבי יעילות העברת הנתונים וקצב הטיפול בבקשות תקפים גם לגבי level 3.

RAID level 4

בשיטה זו הדיסקים פועלים בצורה עצמאית, להבדיל מן השיטות level 2 ו-level 3 שבהן היו הדיסקים של המערך מסונכרנים. בעצם, השיטה level 4 היא שילוב של השיטות level 0 ו-level 3. כמו ב-level 0, הנתונים מחולקים לפיסות מידע וכמו ב-level 3 קיים דיסק של parity bits.

level 4 מאפשרת רמת שרידות זהה לזו של level 3. מבחינת קצב הטיפול בבקשות, בגלל עצם העובדה שהדיסקים הם עצמאיים, מערך הדיסקים מסוגל לטפל בכמה בקשות בו-זמנית. עם זאת, בהשוואה לקצב העברת הנתונים שאפשר להשיג ב-level 3 כאשר משתמשים במערך דיסקים מסונכרנים, קצב ההעברה במקרה של level 4 יהיה ככל הנראה נמוך יותר. הסיבות לכך הן שפיסות המידע יהיו גדולות מפיסות בגודל של סיבית, וגישות מקביליות לפיסות מידע בדיסקים שונים ידרשו פרקי זמן שונים.

מבחינת עדכון נתונים, לשיטת level 4 תהיה תקורה של עדכון parity bits של פיסה, אף אם העדכון נוגע רק לסיבית אחת מתוך הפיסה. כל עדכון כזה מצריך גישה לפיסות בשאר הדיסקים, חישוב של parity ועדכון של דיסק עם מידע עודף. לכן זהו שיקול נוסף לטובת פיסות קטנות. לתופעה הזאת קוראים write penalty.

שאלה 5

האם גם בשיטות levels 1-3 קיים write penalty?

RAID level 5

בשיטה RAID level 4, הדיסק שמחזיק parity strips יכול להפוך לצוואר בקבוק אם כמות העדכונים גבוהה. כדי להתגבר על התופעה ולבזר עומסים בשיטת level 5, ה-parity strips מפוזרים על פני כל הדיסקים של המערך (למשל בשיטת ה-round-robin). ראו איור 5-26f.

RAID level 6

שיטה זו דומה למדי ל-level 5. RAID level 6 מצריך שני דיסקים עודפים במקום דיסק עודף אחד, כפי שנדרש ב-level 5. בשיטה level 6 ה-parity strips מחושבות פעמיים ותוצאות שני החישובים מוחזקות בדיסקים במטרה להגדיל עוד יותר את השרידות. התוצאות של שני חישובי ה-parity אינן תלויות אלה באלה ומפוזרות על פני כל הדיסקים במערך. מובן שה-write penalty של השיטה גבוה יותר בהשוואה ל-level 5, כי כל עדכון דורש גם עדכון של שני parity strips (ראו איור 5-26g).

סעיפים שאינם מופיעים בספר הלימוד
5.5 סיכום
5.6 בעיות פתורות
5.7 שאלות לחזרה
5.8 רשימת מושגים

5.5 סיכום

חלק גדול מחייהם של התהליכים עובר עליהם ב"שינה" (השהיה זמנית). חלק אחר עובר עליהם במחשבה (זמן עיבוד). בשאר הזמן (לעתים זהו רוב הזמן) הם עסוקים ביחסים עם העולם החיצון: הם מבצעים פעולות (פלט) וקולטים מסרים (קלט). פרק זה עוסק במרכיב השלישי – התקשורת עם העולם שמחוץ למחשב. הבעיה הקשה ביותר בתחום זה היא התלות הרבה בחומרה שמוכתבת על ידי ההתקנים השונים. התקני הקלט/פלט הם מסוגים שונים, ועם כל אחד מהם יש לדבר בשפתו הוא.

המאמץ העיקרי בתכנון תהליכי הקלט/פלט של מערכת ההפעלה מיועד להסתיר ככל האפשר את החומרה. ההסתרה נעשית על ידי חלוקת הטיפול בהתקנים לארבע רמות:

א. מערכת הטיפול בפסיקות שיוצרים ההתקנים.
ב. תוכנת התיאום בין ההתקן לבין מערכת ההפעלה.
ג. תוכנת הקלט/פלט המשוחררת מאילוצי החומרה.
ד. שדים לארגון הפלט ופונקציות ספרייה שאינן תלויות חומרה.

האוניברסיטה הפתוחה THE OPEN UNIVERSITY OF ISRAEL الجامعة المفتوحة
Yair Kurtzman

File #0006182 belongs to Yair Kurtzman- do not distribute

5.6 בעיות פתורות

שאלה 1

זרוע דיסק ממוקמת מעל המסלול ה-100. להלן סדרת בקשות למסלולים משמאל לימין:
27, 129, 110, 186, 147, 41, 10, 64, 120

מהו מספר המעברים שמבצעת זרוע הדיסק המשתמשת לצורך התזמון באלגוריתם:

- FIFO?
- SSF?
- SCAN (ההנחה היא שכיוון התנועה הנוכחי הוא כלפי המסלולים שהמספר הסידורי שלהם נמוך)?
- Circular SCAN (ההנחה היא שכיוון התנועה הנוכחי הוא כלפי המסלולים שהמספר הסידורי שלהם נמוך)?

תשובה

להלן רשימת המסלולים לפי סדר הטיפול בהם על ידי כל אלגוריתם. בסוף כל רשימה מופיע סיכום של מספר המעברים שכל אלגוריתם גרם לו עקב סדרת בקשות נתונה:

```
FIFO   27  129 110 186 147 41  10  64  120  Total 556 moves
SSF    110 120 129 147 186 64  41  27  10   Total 262 moves
SCAN   64  41  27  10  110 120 129 147 186  Total 266 moves
C-SCAN 64  41  27  10  186 147 129 120 110  Total 342 moves
```

שאלה 2

מהו קצב העברת הנתונים של דיסק שמהירות הסיבוב שלו היא X rpm, אם בכל מסלול משוכנים Y bytes של נתונים?

תשובה

קצב העברת הנתונים הוא X*Y bytes/min.

5.7 שאלות לחזרה

אילו מבין הטענות שלהלן הן טענות נכונות? (כל התשובות נמצאות בחומר הלימוד, בפרק עצמו או במדריך זה).

1. ביצוע פעולות קלט/פלט באמצעות מעבד ראשי יעיל יותר מביצוען בשיטת DMA.
2. השימוש ב-memory mapped I/O יכול לגרום לתקורה ניכרת בעת הביצוע של כתיבות לדיסק.
3. בדיסקים עכשוויים זמן החיפוש במסלולים גבוה יותר ממחצית זמן הסיבוב של הדיסק.

4. שיטת RAID 5 חסינה מפני היווצרות של עומסים בעת ביצוע כמות גדולה של עדכונים.

5.8 רשימת מושגים

להלן רשימת המושגים המופיעים בפרק זה (על פי סדר הופעתם בפרק):

התקן קלט/פלט	input/output device	רכיב חומרה אשר נועד לאחסון ו/או להעברת מידע. דוגמאות להתקני קלט/פלט: דיסק קשיח, כונן דיסקטים, כרטיס רשת. ישנם התקנים המשמשים כהתקני קלט בלבד (עכבר, מקלדת), או כהתקני פלט בלבד (מדפסת, מסך).
התקן בלוקים	device block I/O	התקן קלט/פלט השומר ומעביר מידע בבלוקים בגודל קבוע. לכל בלוק יש כתובת מוגדרת בהתקן. לבלוקים ניתן לגשת (לכתיבה או לקריאה) בסדר אקראי (דוגמה: דיסק קשיח).
התקן תווים	character I/O device	התקן קלט/פלט המעביר או המקבל מידע כזרם תווים נטול מבנה שאינו מאפשר חיפוש או פנייה על פי כתובת (דוגמה: כרטיס רשת, מקלדת).
בקר I/O	device controller adapter	רכיב אלקטרוני (כרטיס) של התקן קלט/פלט אשר מנהל רכיב מכני או כמה רכיבים דומים. בדרך כלל, הבקר מסופק יחד עם ההתקן המכני, אך לפעמים הוא נרכש בנפרד. הבקר מספק את הממשק להעברת מידע בין ה-CPU לבין ההתקן.
גישה ישירה לזיכרון	DMA – Direct Memory Access	רכיב של בקר שמאפשר העברת נתונים ישירה בין חוצץ (buffer) פנימי של הבקר לבין הזיכרון הראשי, ללא מעורבות המעבד הראשי.
כתיבה לסירוגין בדיסק	interleaving	מספור לא רציף של בלוקים בדיסק, המביא בחשבון את העובדה שכאשר בלוק מועבר לזיכרון על ידי הבקר, רוב הבקרים אינם מסוגלים לקרוא באותו הזמן את הבלוק הבא שמעליו עובר הראש הקורא, ולכן הדיסק צריך לעשות סיבוב שלם עד שניתן יהיה לקרוא את הבלוק הבא. הכתיבה לסירוגין מאפשרת לצמצם את מספר סיבובי הדיסק הדרושים כדי לקרוא בלוקים עוקבים.
אי-תלות בחומרה	device-independence	עקרון ההסתרה של פרטי מימוש ההתקנים על ידי מערכת ההפעלה והצגת ממשק אחיד לשימוש בהתקנים בעלי תפקידים דומים, המאפשר לתוכניות משתמש לעבוד עם התקנים שונים ללא צורך בשינוי הקוד.
אי-תלות בשמות	uniform naming	העיקרון של מתן שמות לקבצים ולהתקני I/O בצורה אחידה, ללא תלות בסוג ההתקן.
מנהל פסיקות	interrupt handler	שגרה של מערכת ההפעלה המתבצעת מיד ברגע שהמעבד מקבל פסיקה מסוג מסוים (לכל סוג של פסיקה יש שגרה שונה) ועושה את הפעולות הנדרשות בעקבות הפסיקה (למשל, הערת תהליך שהיה "רדום" עד לסיום פעולת הקלט/פלט).

האוניברסיטה הפתוחה
THE OPEN UNIVERSITY OF ISRAEL
الجامعة المفتوحة
Yair Kurtzman

File #0006182 belongs to Yair Kurtzman- do not distribute

מנהל התקן	device driver	תוכנה האחראית להפעלת ההתקן ברמה נמוכה, תלוית המימוש הפיזי של ההתקן. מנהל ההתקן מרכז את כל הקוד שהוא תלוי-התקן ונותן ממשק סטנדרטי לשכבות תוכנה גבוהות יותר, ובכך מקל את ההוספה של התקנים חדשים למערכת: מספיק להתקין מנהל התקן מתאים כדי לאפשר למערכת ההפעלה לעבוד עם ההתקן החדש.
אלגוריתמים לתזמון זרוע הדיסק	FCFS, SSF, SCAN, Circular SCAN	4 האלגוריתמים המגדירים את סדר הטיפול בבקשות הגישה לבלוקים: FCFS – בקשות מתמלאות בדיוק בסדר הגעתן. SSF – תמיד מתבצעת הבקשה הקרובה ביותר למיקום הנוכחי של זרוע הדיסק. SCAN (אלגוריתם המעלית) – לזרוע הדיסק מוגדרים שני כיווני תנועה: מעלה ומטה. בכל רגע נתון יש כיוון תנועה נוכחי. כל עוד יש בקשות בכיוון הנוכחי, זרוע הדיסק נעה בכיוון הזה וכל הבקשות שבדרך מתמלאות. כאשר אין עוד בקשות בכיוון הנוכחי, הכיוון מתחלף. Circular SCAN – כמו אלגוריתם המעלית, בשינוי יחיד: יש רק כיוון תנועה אחד. ברגע שאין עוד בקשות בכיוון זה, זרוע הדיסק חוזרת לנקודת המוצא בלי למלא בקשות בדרך, והתהליך חוזר על עצמו.
מערך דיסקים	RAID	מערך של דיסקים שהכתיבה/הקריאה בהם מתבצעת במקביל. קיימות קונפיגורציות שונות של מערכי דיסקים, שברובן דיסק אחד או יותר משמשים להתאוששות במקרה של תקלה באחד הדיסקים במערך.
זיכרון מטמון פנימי	track-at-a-time cache	זיכרון השומר את כל הבלוקים מהמסלול האחרון שעליו עבר הראש הקורא. טכניקה זו מאפשרת למלא בקשות לבלוקים שנמצאים ב- cache ללא פנייה נוספת לדיסק. זיכרון מטמון פנימי יכול להיות ממומש בחומרה (בקר) או בתוכנה (driver).
שעון	clock, timer	שעון מתוכנת – רכיב חומרה שיוצר פסיקות מחזוריות בזמן מחזור שהוגדר, או פסיקה בודדת אחרי פרק זמן שהוגדר לשעון.

פרק 6

קיפאון

האוניברסיטה הפתוחה
THE OPEN UNIVERSITY OF ISRAEL
الجامعة المفتوحة
Yair
Kurtzman

File #0006182 belongs to Yair Kurtzman- do not distribute

6.1 משאבים
6.1.1 משאבים שאינם ניתנים לחילוץ לעומת משאבים הניתנים לחילוץ
6.1.2 הקצאת משאבים

6.1 משאבים

משאבים רבים במחשב אינם דורשים גישה בלעדית. הם נקראים ***משאבים הניתנים לחלוקה***. הנה דוגמאות אחדות למשאבים שאינם דורשים גישה בלעדית:

- מסך. באמצעות כמה חלונות, ניתן לבצע חלוקה של שטח המסך. החלונות יכולים להיות חופפים באופן חלקי, ולכן קיימת גם חלוקה בזמן.
- זיכרון. בעזרת חלוקת שטחי זיכרון, כמה תהליכים יכולים להימצא בזיכרון הראשי בו-זמנית. בפרק 3 ראינו שקיימות שיטות ניהול זיכרון שאינן מצריכות את הימצאותה של כל מפת הזיכרון של תהליך בזיכרון הראשי, ולכן מתקיימת גם חלוקה בזמן.
- מקרה מעניין הוא המעבד עצמו. לכאורה זהו משאב שאיננו ניתן לחלוקה, אך הואיל ובאמצעות חלוקת זמן מהירה מאוד אפשר ליצור אשליה של מקביליות, הרי שמבחינת התהליכים המעבד איננו מסווג כמשאב הדורש גישה בלעדית.

לעומת המשאבים שאינם מחייבים גישה בלעדית, במערכות מחשב קיימים משאבים רבים שאינם ניתנים לחלוקה בין כמה תהליכים, וכדי להשתמש במשאב כזה, תהליך צריך להבטיח גישה רצופה ובלעדית אליו. משאבים כאלה נקראים ***משאבים בלעדיים***.

הנה דוגמאות אחדות למשאבים אשר מחייבים גישה בלעדית:

- מדפסת. הדפסה בו-זמנית על ידי שני תהליכים (או יותר) יכולה להביא לכך שהפלטים ישתלבו זה בזה.
- רשומות של טבלאות פנימיות של מערכת הפעלה, כגון טבלאות מידע לגבי תהליכים. גישה בו- זמנית על ידי שני תהליכי המערכת אל אותה רשומה יכולה להביא לחוסר עקביות בתוכן הרשומה. כל המשאבים הלוגיים אשר שימוש בהם מצריך מניעה הדדית הם משאבים בלעדיים.
- הודעות. משיכה של הודעה אחת על ידי שני תהליכים יכולה להרוס את שלמות ההודעה או להביא לתוצאות בלתי צפויות אחרות.

תופעת הקיפאון, שבה עוסק פרק זה, קשורה לשימוש במקביל במשאבים בלעדיים על ידי תהליכים שונים. כפי שניתן לראות בדוגמאות שהזכרנו, כשאנו מדברים על משאבים, אין אלו בהכרח ***משאבים פיזיים***. גם ***משאבים לוגיים***, כמו משתנים של תוכנית או רשומה בקובץ, נחשבים משאבים, וככאלה, גם הם לעתים קרובות בלעדיים. תופעת

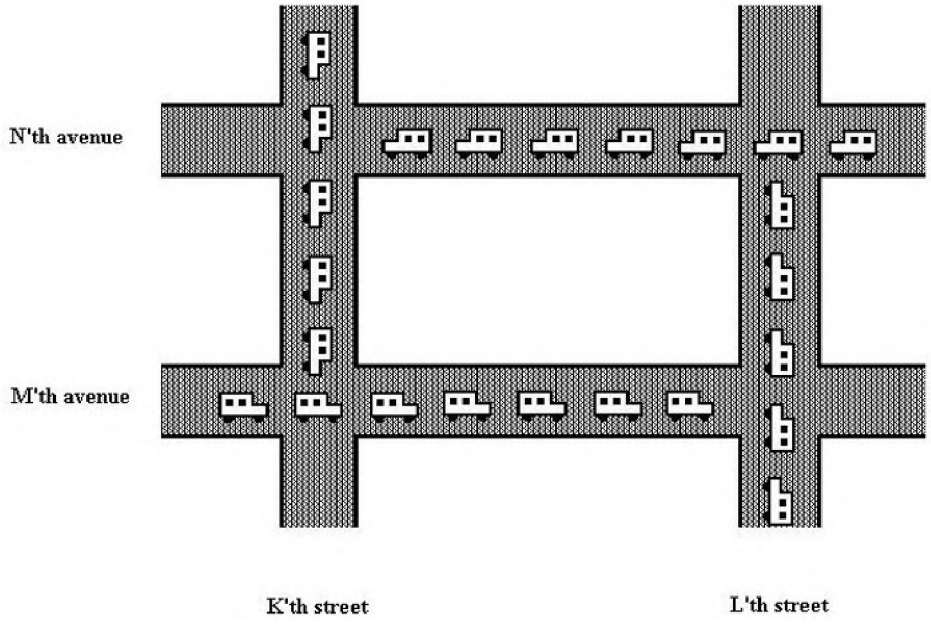
האוניברסיטה הפתוחה THE OPEN UNIVERSITY OF ISRAEL الجامعة المفتوحة
Yair Kurtzman

File #0006182 belongs to Yair Kurtzman- do not distribute

הקיפאון בצורתה הפשוטה ביותר מתרחשת כאשר קיימים שני תהליכים, שכל אחד מהם מחזיק ברשותו משאב בלעדי הנחוץ לתהליך האחר. במצב זה, אף אחד משני התהליכים אינו יכול להתקדם, והם נמצאים בקיפאון.

נראה עוד דוגמה לקיפאון עם משאב לוגי. תהליך עשוי למשל לחכות לסיומו של תהליך אחר. המשאב הוא סיום התהליך. נניח כי תהליך B יכול להתחיל לפעול רק לאחר שתהליך A הסתיים, אבל כדי לסיים, תהליך A זקוק למשאב המוחזק על ידי תהליך B. שני התהליכים נמצאים במצב של קיפאון.

לבסוף, נציג דוגמה הממחישה היטב מהו מצב הקיפאון – קיפאון תחבורתי:



איור 1

זהו מצב קיפאון. ההנחה היא שמסיבות מעשיות, שום שיירה של מכוניות איננה יכולה לנסוע אחורנית. האיור גם רומז לכמה פתרונות למניעת מצב הקיפאון (למשל, הצבת רמזור) או לניסיונות להיחלץ ממנו. זהו נושא הדיון המרכזי שלנו בפרק זה.

6.1.1 משאבים שאינם ניתנים לחילוץ לעומת משאבים הניתנים לחילוץ

לצורך הדיון בבעיית הקיפאון, נניח בפרק זה כי כאשר אנו מדברים על משאבים, הכוונה היא למשאבים בלעדיים, אלא אם נציין במפורש אחרת. כאמור, נבחין בין ***משאבים הניתנים לחילוץ*** (***preemtable***) לבין משאבים שחילוצם הכפוי מידיו של תהליך יגרום נזק משמעותי (ולעתים בלתי הפיך) לתהליך זה. בדרך כלל, רק דרישה למשאבים מהסוג השני עלולה לגרום לקיפאון. משאבים מהסוג הראשון ניתנים לחילוץ ללא קושי, ובדרך זו גם קל להיחלץ ממצב שאחרת היה מוביל בוודאות לקיפאון.

תהליך הדורש משאב הנחוץ לו עשוי לקבלו (ואז להמשיך), או להמתין לו (ולהישלח לישון עד שיקבלו). מכל מקום, ההנחה היא כי תהליך איננו יכול להימשך כל עוד לא קיבל משאב שביקש. לאחר השימוש, המשאבים משוחררים.

6.1.2 הקצאת משאבים

בפרק 2, בפתרון של בעיית היצרן-הצרכן, ראינו כיצד בנסיבות מסוימות שגיאת תכנות יכולה להביא למצב קיפאון. בספר הלימוד, באיור 6-2 (b), מוצגת דוגמה דומה. הנה דוגמה נוספת הנובעת מהצורך בהעברת הודעות:

תהליך 1	תהליך 2
… ReceiveMessageFrom(P2); … SendMessageTo(P1, Msg1); …	… ReceiveMessageFrom(P1); … SendMessageTo(P2, Msg2); …

אם קבלת הודעה גורמת להרדמתו של התהליך עד לקבלת הודעה נוספת, שני התהליכים יכולים להירדם תוך שכל אחד מהם מצפה שהתהליך האחר ישלח לו הודעה. אם כך יהיה, שני התהליכים עלולים לישון לנצח. שגיאות כאלה קשה מאוד לגלות. קוד יכול להיות מסועף מאוד. ההבדל בין המתנה רגילה לקיפאון איננו מובן מאליו בעבור מערכת הפעלה. לעצם גילוי הקיפאון נדרש מנגנון מיוחד.

בגלל חשיבותה הקריטית של בעיה זו בעולם של שיתוף מתמיד במשאבים, זכתה בעיית הקיפאון לעיסוק מחקרי מקיף. בחלק מהתוצאות נדון כאן. ראשית, נגדיר מהו מצב קיפאון.

6.2 קיפאון
6.2.1 תנאים לקיפאון
6.2.2 מודל לקיפאון

6.2 קיפאון

הגדרה: קבוצת תהליכים נתונה במצב קיפאון כאשר כל תהליך בקבוצה מצפה לאירוע הנמצא בשליטתו הבלעדית של תהליך אחר בקבוצה. שליטה בלעדית פירושה כי תהליך המצפה לאירוע אינו יכול להשפיע בדרך כלשהי על התרחשותו.

הגדרה זו של קיפאון תשמש אותנו לצורך הדיון המופשט, כי היא מניחה שתהליך רדום המצפה לאירוע מתהליך אחר אינו יכול להתעורר מאירוע שונה. במציאות הדבר איננו תמיד נכון. במערכת הפעלה UNIX, תהליך שממתין במצב רדום עקב קריאה לפונקציית מערכת יכול "להתעורר" בעקבות קבלת סיגנל. אם בעקבות קבלת הסיגנל התהליך

האוניברסיטה הפתוחה THE OPEN UNIVERSITY OF ISRAEL الجامعة المفتوحة
Yair Kurtzman

File #0006182 belongs to Yair Kurtzman- do not distribute

ישחרר חלק ממשאביו, הדבר יכול להביא לסיומו של מצב הקיפאון. אך מכיוון שאנחנו דנים בפתרונות מערכתיים שאינם נמצאים בשליטת המתכנת, נניח שתהליכים הממתינים לאירוע יכולים להמשיך רק בעקבות קבלת אותו אירוע שלו הם ממתינים.

בהגדרה זו, המושג "משאב" מוצג בצורה המופשטת ביותר: לא רק התקנים פיזיים או משתנים לוגיים הם משאבים לצורך קיום הקיפאון. גם משאבים שעדיין אינם קיימים יכולים להיות בין הגורמים לקיפאון: הודעה שטרם נשלחה (ואולי אף לא נכתבה עדיין) היא דוגמה למשאב כזה. כפי שראינו בסעיף הקודם, עצם העובדה שתהליך מסוים מצפה לקבלתה מתהליך אחר בקבוצה עשויה להיות בין גורמי הקיפאון.

ברגע שקיבלנו הגדרה זו, איננו מתעניינים עוד בטיבם של המשאבים בהקשר של קיפאון. אנו מסתפקים בידיעה שהם נמצאים בשליטה בלעדית ואינם ניתנים לחילוץ.

6.2.1 תנאים לקיפאון

מצב קיפאון מתאפיין בקיום ארבעת התנאים האלה:

1. **מניעה הדדית (mutual exclusion)**
   כל משאב יכול להיות מוקצה לתהליך אחד לכל היותר או להיות פנוי.
2. **החזקה והמתנה**
   תהליך המחזיק משאב יכול לדרוש משאב אחר בלי לשחרר את המשאב שהוא מחזיק.
3. **אין חילוץ כפוי**
   רק התהליך המחזיק במשאב יכול לשחררו.
4. **מעגל ההמתנה**
   כדי שיתקיים קיפאון בקבוצת תהליכים, חייב להתקיים מעגל שבו כל תהליך מחזיק במשאב הנדרש על ידי התהליך הבא במעגל.

תנאים 1-3 הם תכונות הכרחיות של מערכת שיכול להתקיים בה מצב קיפאון. תרחישים שמביאים ליצירת מעגל במערכות המקיימות את שלושת התנאים הראשונים גורמים למצבי קיפאון. לפיכך התנאים 1-3 הם תנאים הכרחיים לקיום הקיפאון, והתנאי הרביעי הוא תנאי מספיק.

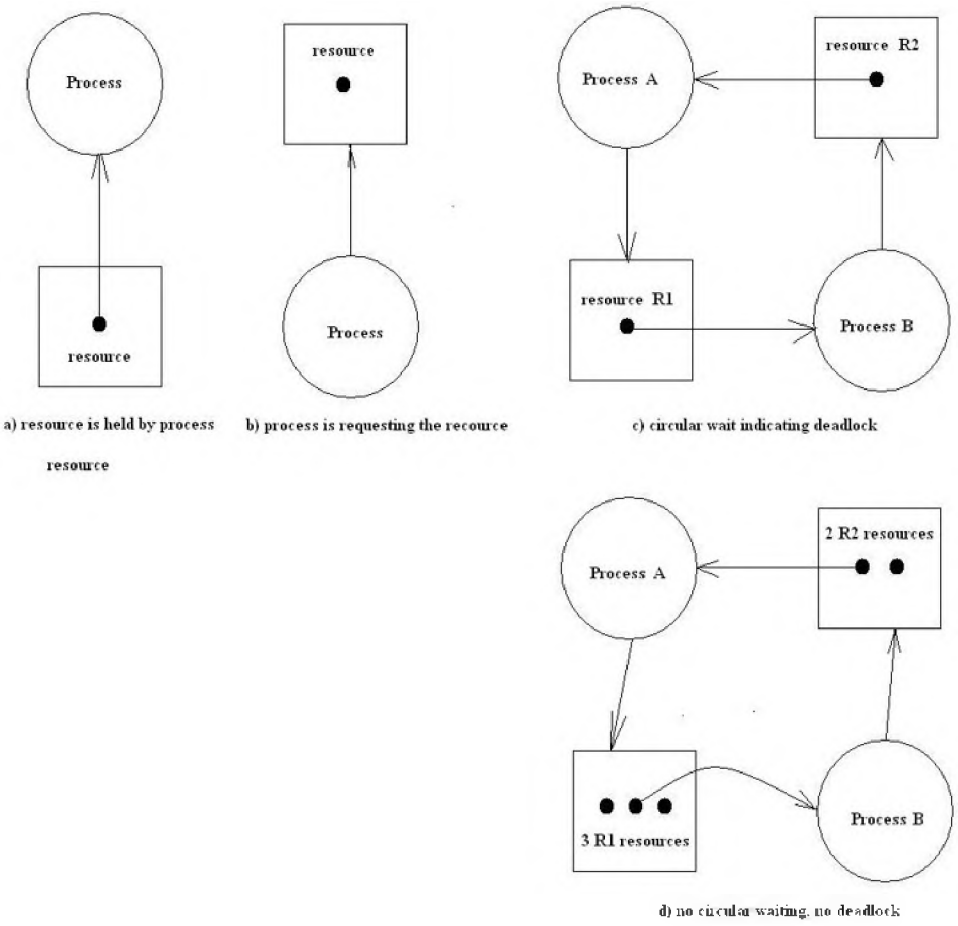
6.2.2 מודל לקיפאון

ספר הלימוד מתאר מודל מוגבל לקיפאון, המניח שקיים רק משאב אחד מכל סוג. במדריך למידה זה נראה כיצד אפשר להרחיב בקלות את המודל לכמה משאבים מאותו סוג.

באמצעות גרף מכוון, ניתן לייצג מצב של הקצאת משאבים במערכת שמקיימת את שלושת התנאים הראשונים ולזהות באמצעותו אם מתקיים התנאי הרביעי – התנאי שגורם ישירות לקיפאון. בגרף זה יש שני סוגים של צמתים:

- צמתים המייצגים משאבים מאותו סוג. משאבים מאותו סוג הם המשאבים שיכולים להוות תחליף זה לזה. נצייר צמתים כאלה כריבועים המכילים נקודות כמספר המשאבים מהסוג הנתון.
- צמתים המייצגים את התהליכים. נצייר צמתים כאלה כעיגולים.

הגרף בנוי כך שכל קשת מצומת תהליך אל צומת משאב מייצגת בקשה של התהליך לקבל את אחד המשאבים מהסוג המיוצג על ידי הצומת. קשת מנקודה בצומת משאב אל תהליך מציינת שהמשאב נמצא בחזקתו של התהליך. אנו מניחים כי תהליך המבקש משאב עובר למצב של המתנה עד קבלתו.



איור 2

כלומר, באמצעות הגרף נוכל לדעת בכל זמן נתון מהו מצב הקצאת המשאבים. באיור 6-4 בספר הלימוד מוצגת דוגמה לשינוי סדר ההקצאות שיכול לעזור להימנע ממצב קיפאון. הגרף להקצאת המשאבים מאפשר לפתח אלגוריתמים שונים למניעת קיפאון או לטיפול בו.

אנו נסקור ארבע אסטרטגיות להתמודדות עם הבעיה:

האוניברסיטה הפתוחה
THE OPEN UNIVERSITY OF ISRAEL
الجامعة المفتوحة
Yair
Kurtzman

File #0006182 belongs to Yair Kurtzman- do not distribute

- **התעלמות** – משיקולים שונים (למשל, נדירות) נניח שהבעיה איננה קיימת. כפי שנראה בהמשך, טיפול בקיפאון דורש מאמץ חישובי רב. במערכות שבהן הסיכוי להתרחשות הקיפאון הוא אפסי ובעל סיכון מועט, יש היגיון בבחירה בהתעלמות.
- **גילוי והיחלצות** – בגישה זו מבצעים מעת לעת ניסיונות לגילוי מצבי קיפאון, ואם הם מתגלים, מתבצעות פעולות להיחלצות ולהשבת הסדר על כנו.
- **התחמקות** – הקצאה דינאמית של משאבים בצורה מבוקרת ומחושבת שאינה מאפשרת יצירת מצב קיפאון.
- **מניעה** – גישה זו מונעת קיום מצבי קיפאון באמצעות שינוי מדיניות הקצאת המשאבים של המערכת, כך שלפחות אחד מארבעת התנאים ההכרחיים לקיום הקיפאון לא יוכל להתממש.

6.3 שיטת בת היענה

6.3 שיטת בת היענה

גישה זו, שנראית משונה במבט ראשון, מקורה במחיר הגבוה שגובים הפתרונות למניעת קיפאון והיא מבוססת על העובדה הידועה, כי יש שהתרופה למחלה מזיקה יותר מן המחלה עצמה. כפי שראינו בדוגמה המוצגת באיור 5-6 בספר הלימוד, הקצאה בלתי מבוקרת של משאבים עלולה לגרום לקיפאון, והדרך היחידה למניעת הקיפאון היא פיקוח על החלוקה ועל דרך השימוש במשאבים. אך המחיר עלול להיות כבד מדי. פיקוח הדוק הופך בקלות למכשיר דיכוי. אם נזכור כי כל פעולה של תהליך דורשת משאבים, הרי שפיקוח מעין זה מצריך זמן חישוב ניכר. חמור מזה: ההגבלות על חופש הפעולה של התהליכים עלולות להוות נטל כבד מאוד על המשתמשים.

6.4 גילוי קיפאון והיחלצות ממנו
6.4.1 גילוי קיפאון כאשר יש רק משאב יחיד מכל סוג
6.4.2 גילוי קיפאון כאשר המשאבים מחולקים למחלקות
6.4.3 היחלצות מקיפאון

6.4 גילוי קיפאון והיחלצות ממנו

בגישה זו מבצעים מעת לעת ניסיונות לגילוי מצבי קיפאון, ואם הם מתגלים, מתבצעות פעולות להיחלצות ולהשבת הסדר על כנו. נציג תחילה שיטות לגילוי הקיפאון ונבדוק את מחירן, ולאחר מכן נדון בטכניקות להיחלצות ממנו.

6.4.1 גילוי קיפאון כאשר יש רק משאב יחיד מכל סוג

כדי לגלות מצבי קיפאון בגרף הקצאת המשאבים, ניתן לבצע חיפוש מעגלים בגרף בעזרת אלגוריתם החיפוש לעומק, DFS.

```
DFS(vertex){
        List = empty;
        for every edge of the graph
                edge = unmarked
        v = vertex;
        List.addLast(v); // takes O(1) time

        while(List.NotEmpty() == TRUE){
                if (List.ContainsVertexTwice(v) == TRUE){ // takes O(|V|) time
                        cycle including node vertex was found;
                        return;
                }
                if (any unmarked edges originate at vertex v){
                        edge = unmarked edge originating at v pick upped
                        randomly;
                        edge = marked;
                        v = another vertex of edge;
                        List.addLast(v);
                }
                else{
                        List.removeLast();
                        v = List.lastElement();
                        if (v == vertex){
                                no cycle including node vertex was not found;
                                return;
                        }
                }
        }
}
```

מבצעים את האלגוריתם DFS(vertex) בעבור כל צומת בגרף הקצאת המשאבים, ואם בסוף לא נמצא מעגל, משמעות הדבר שהתהליכים המשתתפים בגרף להקצאת המשאבים אינם כלואים במצב קיפאון.

שאלה 1

בצעו שינוי של האלגוריתם שידפיס את כל המעגלים. דאגו להדפיס כל מעגל רק פעם אחת. תארו בפסידוקוד את מבני הנתונים בצורה מפורטת.

האוניברסיטה הפתוחה THE OPEN UNIVERSITY OF ISRAEL الجامعة المفتوحة
Yair Kurtzman

File #0006182 belongs to Yair Kurtzman- do not distribute

עתה נחשב את זמן הריצה של האלגוריתם לחיפוש מעגלים בגרף שתיארנו. ה-if הראשון צורך זמן בסדר גודל של O(|V|), כאשר V היא קבוצת הצמתים בגרף. הבדיקה של ה-if הראשון תבוצע לכל היותר O(|E|) פעמים, כאשר E היא קבוצת כל הקשתות בגרף. לכן, כל הפעלה של DFS(vertex) דורשת לכל היותר O(|V||E|) יחידות זמן. את הפרוצדורה של DFS מריצים לכל היותר O(|V|) פעמים, ולכן הזמן הכולל הוא $O(|E||V^2|)$. אפשר לייעל את האלגוריתם כך שיצריך סדר גודל של $O(|V|^3)$ יחידות זמן, אך עדיין, כפי שאפשר לראות, מחירו של גילוי הקיפאון הוא גבוה.

שאלה 2

הציעו אלגוריתם לחיפוש מעגלים בגרף הקצאת משאבים עם כמה משאבים מאותו סוג. נתחו את זמן הריצה שלו.

6.4.2 גילוי קיפאון כאשר המשאבים מחולקים למחלקות

נראה אלגוריתם אחר לגילוי של מצבי קיפאון, אשר אינו משתמש בגרף להקצאת משאבים:

א. סימונים
1. ישנם n תהליכים, והם מסומנים P1 עד Pn.
2. ישנן m מחלקות משאבים, המסומנות R1 עד Rm.

ב. מבני נתונים
1. E הוא מערך חד-ממדי בגודל m. בכל תא Ej מאוחסן מספר המשאבים הכולל השייך למחלקת המשאבים j (1=<j<=m). כל המשאבים הנמצאים באותה מחלקה שקולים זה לזה מבחינת התהליכים הדורשים אותם.
2. A הוא מערך חד-ממדי בגודל m. בכל תא Aj מאוחסן מספר המשאבים החופשיים (שלא הוקצו עדיין) מהמחלקה j.
3. C הוא מערך דו-ממדי בגודל n*m. בתא Cij מאוחסן מספר המשאבים מהמחלקה j שכבר הוקצו לתהליך i.
4. R הוא מערך דו-ממדי בגודל n*m. בתא Rij מאוחסן מספר המשאבים מהמחלקה j הנדרשים עדיין על ידי התהליך i.

ג. מכיוון שכל משאב יכול להיות חופשי או מוקצה, הרי שהסכום של מספר המשאבים מהמחלקה j שכבר הוקצו לאחד התהליכים ומספר המשאבים ממחלקה זו שעדיין חופשיים הוא תמיד Ej.

ד. אנו מגדירים יחס => בין שני וקטורים A ו-B שווים באורכם על ידי: A<=B כאשר לכל i מתקיים Ai<=Bi.

ה. אלגוריתם לגילוי מצב הקיפאון

```
DetectDeadlock(){
        P = {P1, P2, …, Pn};

        while (P is not empty){
                runable_process_exists = false;
                for each p in P{
                        if (Rp <= A){
                                A = A + Cp;
                                P = P \ {Pp};
                                runnable_process_exists = true; // at least
                                one process exists
                        }
                }
                if (runnable_process_exixts == false)
                        report processes in P as deadlocked;
        }

        report no deadlock;
}
```

1. מחפשים תהליך p שנמצא במצב של המתנה למשאבים ואשר הווקטור Rp המייצג את בקשותיו (וקטור השורה p במערך R) קטן או שווה לווקטור A המייצג את המשאבים הזמינים (Rp<=A). במילים אחרות, אנו מחפשים תהליך הממתין למשאבים, שניתן להיענות לכל בקשותיו. תהליך כזה יכול להמשיך ולסיים את פעולתו ללא חשש מקיפאון.
2. אם מצאנו לפחות תהליך אחד כזה, אנו מחברים חיבור וקטורי ומציבים: A=A+Cp. כמו כן אנו מעדכנים את הקבוצה P כדי שלא תכיל את התהליך Pp. ניתן לתאר במילים את מה שנעשה בשלבים אלה: התהליך Pp יכול להמשיך לרוץ כי הבקשות שלו מסופקות, ובסיום הוא יחזיר את כל המשאבים שהחזיק בהם לשימושם של תהליכים אחרים.
3. כאשר לא ניתן עוד למצוא תהליך Pp שניתן לספק את מבוקשו, ויחד עם זאת הקבוצה P איננה ריקה, הדבר מצביע על קיום מצב הקיפאון. התהליכים שנותרו בקבוצה P נמצאים במצב קיפאון.
4. כאשר האלגוריתם מסתיים והקבוצה P ריקה, ברור שאין מצב קיפאון.

ראו בספר הלימוד דוגמה לשימוש באלגוריתם זה באיור 6-10.

שאלה 3

אם זמן החישוב של פעולת חיבור וקטורי הוא O(m), מהו זמן חישוב האלגוריתם שתיארנו כעת – אלגוריתם המבוסס על מטריצות – אם נתון כי מספר התהליכים הוא n?

האוניברסיטה הפתוחה THE OPEN UNIVERSITY OF ISRAEL الجامعة المفتوحة
Yair Kurtzman

File #0006182 belongs to Yair Kurtzman- do not distribute

6.4.3 היחלצות מקיפאון

עתה, כשאנו מצוידים בכלים (אמנם יקרים במונחים של זמן חישוב) לגילוי מצב הקיפאון, ננסה לראות מה ניתן לעשות כדי להיחלץ ממצב ביש זה. לרוע המזל, אין פתרונות קלים. בסעיף זה נתאר שלוש דרכים להיחלצות מקיפאון. שימו לב למחיר הכבד של כל דרך.

החלמה על ידי חילוץ כפוי

קיפאון נוצר כתוצאה מהקצאת משאבים, ולכן חילוץ מבוקר של משאבים עשוי לחלץ גם מקיפאון. החילוץ הוא כפוי, שכן הוא נעשה בהוראה חיצונית ולא ביוזמתו של התהליך. חילוץ כפוי הוא התערבות שרירותית, ולכן הוא עלול לגרום נזק חמור למהלך התקין של התהליך שאולץ לוותר על משאב. למשל, חילוץ של מדפסת יגרום לצורך בהתערבות אנושית כדי לזהות באיזה דף הופסקה ההדפסה ולאחר מכן להפעיל מחדש את פעולת ההדפסה מהעמוד שבו הופסקה. דוגמאות נוספות לנזק שעלול להיגרם על ידי חילוץ כפוי הן:

- השארת מסד נתונים במצב בלתי עקבי.
- ניתוק קו תקשורת במהלך קבלת הודעות.
- איבוד תוצאות של חישוב ארוך בגלל אי-היכולת לשמור אותן בקובץ.

שאלה 4

כיצד ניתן לצמצם את ממדי הנזק שגורם חילוץ כפוי?

הבחירה של המשאב לחילוץ כפוי אינה משימה פשוטה לביצוע אוטומטי. למשל, אם ידוע שאפשר להיחלץ ממצב הקיפאון על ידי חילוץ כפוי של שני משאבים הנמצאים בידי שני תהליכים שונים, או לחלופין על ידי חילוץ של שני משאבים זהים הנמצאים בחזקתו של תהליך אחד, הבחירה איננה פשוטה. שיקולים כמו חשיבות התהליך ואורך הזמן שהמשאבים יכולים להיות תפוסים יכולים להשפיע על ההחלטה הסופית.

מספר הגורמים והשיקולים האפשריים הוא עצום, הנתונים לגבי העתיד אינם ברורים תמיד (למשל, האם יש עדיין טעם להמתין להודעה בערוץ תקשורת?) והתוצאות של חילוץ כפוי, למרות משמעותן הקריטית לעתים, אינן מובטחות.

היחלצות על ידי גלגול לאחור (rollback)

שיטה זו אינה ייחודית להיחלצות מקיפאון. הבעיה של שחזור המצב אחרי תקלה פתאומית (במקרה זה קיפאון; במקרה אחר, הפסקת חשמל, למשל) היא בעיה מוכרת וידועה. פתרון מקובל הוא שמירה תמידית על קבוצה של ***מצבים עדכניים (checkpoints)***. כאשר מתגלה מצב קיפאון, אפשר לגלגל את התהליך שגרם לסגירת מעגל הקיפאון למצב העדכני האחרון הידוע, וכך להיחלץ מהקיפאון. היתרון בשיטה זו הוא שהתהליך מתחיל ממצב עדכני ידוע, וכך מתאפשר שחזור אוטומטי.

כדי להשתמש בשיטת שמירת המצבים העדכניים להיחלצות ממצב קיפאון, צריך לבחון את ההיבטים האלה:

- שמירת מצבים עדכניים לתהליכים עלולה לדרוש שמירה של כמות גדולה מאוד של מידע.
- ככל שמבצעים את השמירה לעתים רחוקות יותר, כך גדל הסיכוי לאבד את המידע על מה שהתרחש בין העדכונים העוקבים.
- אם מבצעים את השמירה לעתים תכופות, אזי מלבד ההחלטה בדבר כמות הנתונים שיש לשמור בכל פעם, נדרשת החלטה בקשר לבחירת התהליכים ולשלבי העדכון שבהם מתבצעת השמירה כדי למזער את הנזק. מועד זה עלול להיות שונה לגבי כל תהליך.

כפי שניתן לראות, הסוגיות אינן פשוטות כלל וכלל. קיימים מצבים שבהם שיטת הגלגול לאחור יכולה להביא יותר נזק מאשר תועלת. למשל, במקרה של תהליכים המשמשים להעברת הודעות, גלגול לאחור יכול בקלות להביא לאובדן הודעות, ומכאן לבעיית קיפאון חמורה בתהליכים שתוכנתו נכון או לתופעות בלתי צפויות אחרות.

שאלה 5

בעבור בעיית יצרן-צרכן המאורגנת באמצעות העברת הודעות, תארו תרחיש שגלגול לאחור יכול להביא לאובדן הודעה, וכתוצאה מכך – למצב קיפאון.

היחלצות על ידי הריגת תהליכים

זוהי ללא ספק פעולה אלימה. למעשה, זוהי גרסה נוספת של חילוץ כפוי. יתרונה טמון בכך שהריגת תהליך גורמת לשחרור מוחלט של משאבים, וזאת להבדיל משיטת החילוץ הכפוי, אשר משחררת משאבים רק לתקופה קצרה. הריגת תהליכים שימושית כאשר לאחר חילוץ כפוי או גלגול לאחור, מצב הקיפאון חוזר שוב לאחר תקופה קצרה מאוד. התופעה ידועה כתופעת צפיפות (congestion) של המשאבים. מותו של התהליך איננו מעניין אותנו כשלעצמו, אבל הידיעה כי עם מותו ישתחררו המשאבים שהוא מחזיק בהם, מעודדת את ההריגה. הקורבן אינו חייב להיות דווקא תהליך קפוא. העיקר הוא שכך ישתחרר משאב החיוני לשבירת הקיפאון.

האוניברסיטה הפתוחה
THE OPEN UNIVERSITY OF ISRAEL
الجامعة المفتوحة
Yair
Kurtzman

File #0006182 belongs to Yair Kurtzman- do not distribute

6.5 התחמקות מקיפאון
6.5.1 מסלולים להקצאה בטוחה של משאבים (resource trajectories)
6.5.2 מצבים בטוחים לעומת מצבים מסוכנים
6.5.3 אלגוריתם הבנקאים בעבור משאב יחיד
6.5.4 אלגוריתם הבנקאים בעבור מחלקות של משאבים

6.5 התחמקות מקיפאון

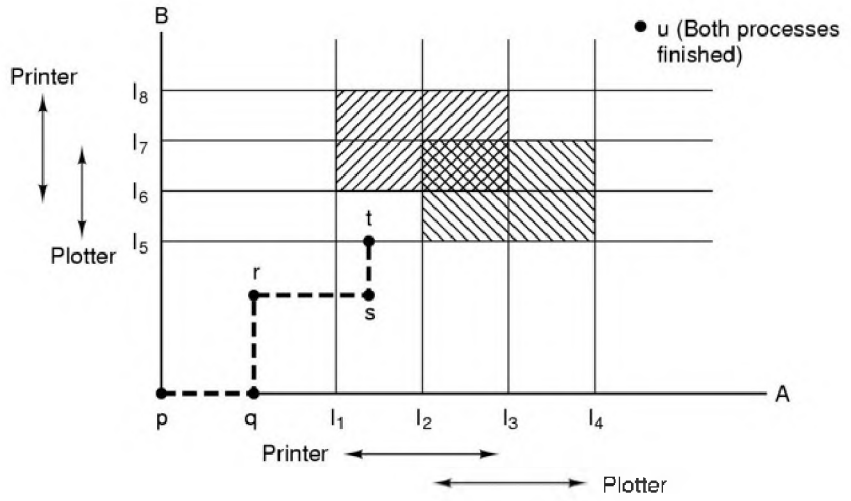
הואיל ומחירה של היחלצות מקיפאון הוא כבד, ייתכן שראוי לנסות להקצות משאבים בדרך שתבטיח מראש כי המערכת לא תיכנס לקיפאון. מערכת ההפעלה היא זו שמקצה את רוב המשאבים במחשב, לכן יש מקום להניח כי מדיניות נבונה יכולה לשמור על התהליכים מפני כניסה לקיפאון. אם נמצא מדיניות הקצאה בטוחה מסוג זה, נוכל ליישם אותה בכל מקרה שקיימת בו סכנת קיפאון, גם כאשר המשאבים הם משאבים לוגיים בלבד ואינם מוקצים על ידי מערכת ההפעלה.

אפשרות מתבקשת בהקשר זה היא שכל קבוצה של תהליכים המשתפים ביניהם משאבים לוגיים תמנה לעצמה מנהל שיקצה את המשאבים בהתאם למדיניות שנקבעה. מובן שבעיות קיפאון יכולות להתעורר עדיין עקב חוסר תיאום דו-צדדי בין המנהל לבין מערכת ההפעלה שמנהלת משאבים, אך בכל מקרה, בהנחה שאכן קיימת מדיניות הקצאה המבטיחה התחמקות מקיפאון, הדבר יקטין משמעותית את הסכנה.

6.5.1 מסלולים להקצאה בטוחה של משאבים (resource trajectories)

כל הצירופים של צעדי הקצאות ושחרור משאבים בעבור קבוצת תהליכים יוצרים מרחב מצבים. אם קיים מסלול במרחב זה העובר ממצב ההתחלה של התהליכים אל המצב שבו כולם מסיימים, בלי לחצות מצב שבו ייתכן קיפאון, הרי שיש בידינו סדרת הקצאות ושחרורים המאפשרת השלמת התהליכים בלי קיפאון. מסלול כזה נקרא **מסלול בטוח**.

במקרה של שני תהליכים, מרחב המצבים הוא בעל שני ממדים:



איור 3

I עם אינדקס מציין בקשה למשאב או לשחרור של משאב. כל אחד מהצירים מציין את ציר הזמן של אחד מהתהליכים. האזור הבעייתי בגרף הוא אזור המשבצת המתוחמת בין הקווים I_5,I_6,I_1,I_2. הקו המקווקו מדגים את סדר הריצות של שני התהליכים ב-CPU. קו זה הוא מסלול אפשרי במרחב מצבים. מסלול בטוח הוא המסלול שאינו נוגע במשבצת המסוכנת, שהיא כאמור המשבצת המתוחמת בין הקווים I_5,I_6,I_1,I_2 – כל מסלול שנוגע במשבצת זו מביא בהכרח למצב קיפאון בגלל סדר הקצאת המשאבים על ידי התהליכים.

בהמשך נכיר אלגוריתם שמונע את התקדמותם של תהליכים במסלולים לא בטוחים, וזאת גם במקרה הכללי של כמה תהליכים וכמה קבוצות משאבים, כאשר בכל קבוצה כמה משאבים מאותו סוג.

6.5.2 מצבים בטוחים לעומת מצבים מסוכנים

האלגוריתם שנלמד משתמש באותם מבני מתונים שבהם משתמש האלגוריתם לגילוי מצבי הקיפאון שלמדנו. בכל נקודת זמן, האלגוריתם דואג להשאיר את המערכת במצב שאין בו סכנה להיקלע לקיפאון. לשם כך נזדקק להגדרה שלהלן:

הגדרה:
מצב נקרא בטוח אם התהליכים שבו אינם קפואים ויש דרך להשלים את כל הבקשות של התהליכים, כך שכולם יסתיימו בוודאות ללא קיפאון.

התבוננו בדוגמאות המוצגות באיורים 6-11 ו-6-12 בספר הלימוד להדגמה של מצב בטוח לעומת מצב שאיננו בטוח. המצב ההתחלתי (שבו לאף אחד מהתהליכים אין שום משאב) הוא תמיד בטוח. המצב עלול להפוך למסוכן רק ברגע שבו שני תהליכים לפחות מתחילים לפעול במקביל. חשוב לשים לב כי גם במצב מסוכן (כמו מצב b באיור 6-12) ייתכן שיש אפשרות שהתהליכים יסתיימו ללא קיפאון, אולם המצב נקרא **מסוכן** מכיוון שאין

האוניברסיטה הפתוחה
THE OPEN UNIVERSITY OF ISRAEL
الجامعة المفتوحة
Yair
Kurtzman

File #0006182 belongs to Yair Kurtzman- do not distribute

ערובה לכך. לעומת זאת, מצב נקרא **בטוח** כאשר קיים בוודאות מסלול בטוח ממנו אל מצב הסיום התקין.

6.5.3 אלגוריתם הבנקאים בעבור משאב יחיד

האלגוריתם המשאיר מערכת במצב בטוח מחקה עיקרון בנקאי מקובל. בבנק יש סכום כסף נתון, אך מכיוון שלא כל לקוחותיו של הבנק זקוקים לכל סכום הכסף הזה בעת ובעונה אחת, הבנק יכול להקצות קווי אשראי הגדולים מסך כל הכסף המופקד בו (ובכך להגדיל את רווחיו). כדי לא להיות חשוף לסכנה של פשיטת רגל אם, למשל, כל הלקוחות של הבנק ידרשו לנצל את מלוא מסגרת האשראי, הבנק מנהל מדיניות של הקצאת אשראי בטוחה. במסגרת המדיניות הזאת, הוא מקצה אשראי כך שתמיד תהיה לו אפשרות לספק את כל בקשות האשראי במרוצת הזמן.

חוקי המשחק הם:

1. לכל לווה (תהליך) יש קו אשראי אישי מוגבל (מספר משאבים מקסימלי שהוא רשאי לדרוש).
2. כאשר לווה ממצה את כל בקשותיו לאשראי (משאבים), הלווה פורע את כל הלוואותיו לבנק (התהליך מסתיים ומחזיר את משאביו למערכת ההפעלה).
3. לבנק מותר לא להיענות לבקשות אשראי באופן זמני, אך הוא חייב להיות תמיד במצב המאפשר למלא את בקשות כל לקוחותיו לפחות בזה אחר זה.

ברור כי אם חוקים 2 ו-3 יישמרו, הבנק לעולם לא יפשוט רגל (והתהליכים לעולם לא יקפאו). חוקים אלה הם תמציתו של אלגוריתם הבנקאים.

עיינו בדוגמה שבאיור 6-14. מצב נחשב לבטוח אם קיימת אפשרות אחת לפחות של **הרצה סדרתית** של התהליכים (הרצת התהליכים בזה אחר זה, אך לא במקביל) כך שכולם יסתיימו. ההרצה הסדרתית היא המוצא האחרון המונע קיפאון ולכן, כל עוד אפשרות זו קיימת, המצב נחשב לבטוח. שימו לב כי אין פירוש הדבר שמעתה והלאה ההרצה תהיה בהכרח סדרתית בלבד, שכן הדבר תלוי בסדר הדרישות של התהליכים, אולם מצב בטוח מבטיח שגם במקרה של דרישות מקסימליות, לא ייווצר קיפאון ולו על ידי הרצה סדרתית.

שימו לב כי סדר בחירת התהליכים, מבין אלו שניתן להריצם, אינו משנה, משום שעם כל הרצה אנו מטפלים רק בכל המשאבים הזמינים.

6.5.4 אלגוריתם הבנקאים בעבור מחלקות של משאבים

הפסידוקוד של אלגוריתם הבנקאים זהה לזה של האלגוריתם לגילוי מצבי קיפאון. ההבדל הוא בעיתוי הרצת הקוד ובאופן השימוש בתוצאות. במסגרת הגישה של גילוי הקיפאון וההיחלצות ממנו, אנו מאפשרים לקיפאון להתרחש ורק מדי פעם אנו מריצים אלגוריתם לזיהוי שבודק את קיום הקיפאון. בגישה של הימנעות ממצבי קיפאון, מריצים את הקוד לפני כל הקצאה של משאבים ובודקים אם היענות לבקשה תשאיר את

המערכת במצב בטוח. במקרה של תשובה חיובית, מאפשרים את הקצאת המשאבים, ואם לאו – מונעים את ההקצאה.

אלגוריתם הבנקאים מתאים למערכת סגורה, שבה כל התהליכים, צורכיהם וכמות המשאבים ידועים מראש. אלא שמערכת ההפעלה אינה מערכת סגורה. במערכת הפעלה אופיינית, מספר התהליכים משתנה בהתמדה. הצרכים של תהליך אינם ידועים מראש בדרך כלל, וגם כמות המשאבים משתנה בהתמדה. לפיכך, אלגוריתם הבנקאים הוא פתרון תיאורטי נאה, אך הוא אינו ניתן ליישום במקרה הכללי של מערכות הפעלה.

6.6 מניעת קיפאון
6.6.1 תקיפת תנאי המניעה ההדדית
6.6.2 תקיפת תנאי ההחזקה וההמתנה
6.6.3 חילוץ כפוי
6.6.4 מניעת המתנה מעגלית

6.6 מניעת קיפאון

אנו רואים כי התחמקות מקיפאון היא עניין מסובך מדי לצרכים מעשיים, שכן היא דורשת את ידיעת העתיד. בתחילת הדיון הזכרנו ארבעה תנאים הכרחיים לקיומו של קיפאון. הניסיונות הבאים למניעת קיפאון מבוססים כולם על תקיפת תנאים אלו.

6.6.1 תקיפת תנאי המניעה ההדדית

ברור כי אם כל משאב ניתן לחלוקה, אזי לא ייתכן קיפאון. מצד אחר ברור גם כי לא כל משאב ניתן לחלוקה, והדוגמה המובהקת לכך היא כאמור המדפסת. אך מתברר כי לעתים אפשר להפוך משאב בלעדי למשאב הניתן לחלוקה. למשל, בעזרתו של תהליך נוסף – שדון מדפיס – ניתן להתגבר על הבלעדיות של המדפסת. לפי הרעיון הזה, התהליך היחיד שמחזיק במדפסת הוא התהליך של השדון המדפיס; שאר התהליכים יכולים לשלוח בקשות הדפסה לשדון, והוא יפנה אותן למדפסת לפי הסדר. היות שהשדון החביב הזה איננו מבקש כל משאב אחר, אנו יכולים להימנע מקיפאון בהקשר של המדפסת.

למרבה הצער, לא תמיד קיימת אפשרות כזאת. דוגמה לכך היא הקטע הקריטי, שהוא משאב בלעדי מעצם הגדרתו. כמו כן, הפתרון שהובא בהקשר של המדפסת מתבסס על העובדה כי ניתן לדחות את ההדפסה, ובינתיים להמשיך בפעולת התהליך. פתרון זה כמובן בלתי אפשרי כאשר המשך התהליך תלוי בתפיסה מיידית של המשאב. מהדיון הזה עולה ההיריסטיקה (כלל שכדאי לנהוג על פיו, אך תועלתו איננה מוכחת בכל תנאי) שלהלן:

- צריך לנסות להפוך משאבים בלעדיים הניתנים לדחייה למשאבים הניתנים לחלוקה.
- אין לדרוש משאב אלא אם כן הוא הכרחי לחלוטין.

האוניברסיטה הפתוחה THE OPEN UNIVERSITY OF ISRAEL الجامعة المفتوحة
Yair Kurtzman

File #0006182 belongs to Yair Kurtzman- do not distribute

חולשתה הגדולה של ההיריסטיקה הזאת נעוצה בעובדה שהיא תלויה ברצונם הטוב של התהליכים (או, ליתר דיוק, במשתמשים שמריצים אותם). אמנם, כפי שכבר הזכרנו, מערכות הפעלה אכן משתמשות בהיריסטיקות דומות. הסיבה לכך היא שבניגוד למצב בין תהליכי המשתמשים, במקרה של מערכת ההפעלה עצמה (תהליכי המערכת) מנסים למנוע קיפאון בתוך קבוצת תהליכים שכולם תוכננו מראש לשתף ביניהם פעולה. כך הדבר גם בקבוצת תהליכים המפעילה מסד נתונים. במקרה כזה, כאשר המטרה היא משותפת ואין ניגודי אינטרסים, יש אפשרות מעשית להקפיד על כללים נאותים שיצמצמו עד מאוד את סכנת הקיפאון. כך גם כאשר מדובר בתהליכונים שהם בניו של תהליך אחד.

6.6.2 תקיפת תנאי ההחזקה וההמתנה

מדיניות זו מנסה למנוע את ההמתנה למשאבים נוספים. כידוע, אם אין המתנה, אין קיפאון. הדבר נראה פשוט אך, כרגיל, החיים אינם פשוטים. אם רוצים למנוע את ההחזקה וההמתנה, יש לספק לתהליך את כל צרכיו מראש. הבעיות שנוצרות לפי המדיניות הזאת הן:

- לא כל תהליך יודע את צרכיו מראש. הצרכים הללו עלולים להשתנות לפי נסיבות שונות.
- הקצאה מראש של כל המשאבים הבלעדיים יוצרת בזבוז משאבים עצום, שכן לעתים התהליכים נאלצים להחזיק במשאבים זמן רב בטרם יזדקקו להם בפועל. ומאחר שמשאבים אלו הם בלעדיים, במשך כל הזמן הזה שום תהליך אחר לא יוכל להשתמש בהם.

שאלה 6

מהי ההשפעה של ההקצאה מראש על דרגת המקבילות במערכת ההפעלה?

דרך נוספת לשבור את תנאי ההמתנה היא לדרוש מתהליך שמבקש משאב לשחרר תחילה את כל המשאבים שהוא מחזיק, ואז לנסות לבקש את כל המשאבים שלהם הוא זקוק בבת אחת. זוכרים את בעיית הפילוסופים הסועדים? מדיניות דומה יכולה, באופן תיאורטי, לגרום לסדרה אינסופית של הקצאות ושחרורים.

6.6.3 חילוץ כפוי

בחסרונות של שיטה אלימה זו כבר עסקנו בהרחבה בסעיף 6.4.3. מחירה כבד מדי לכל יישום כללי.

6.6.4 מניעת המתנה מעגלית

ניתן לתקוף את התנאי בכמה דרכים:

א. הקצאת משאב יחיד

אפשר לקבוע, כמדיניות, שתהליך רשאי להחזיק רק משאב אחד. ברור כי תהליך המחזיק במשאב יחיד איננו יכול, בשום אופן, להשתתף במעגל הקיפאון. אלא שכרגיל, מדיניות זו, אף על פי שהיא בטוחה מבחינה תיאורטית, מעשית אין בה טעם רב.

ב. הקצאת משאבים בסדר קבוע מראש

בשיטה זו מסדרים את המשאבים בסדר מסוים וממספרים אותם. הקצאתם תיעשה מעתה על פי סדר זה בלבד. כלומר, אין להקצות משאב שמספרו נמוך אחרי משאב שמספרו גבוה יותר.

שאלה 7

הוכיחו בדרך השלילה כי הקצאת משאבים מראש היא מדיניות בטוחה למניעת קיפאון.

אף שסידור מספרי של משאבים מונע קיפאון, השיטה הזאת איננה ישימה, מכיוון שקשה למספר את כל המשאבים האפשריים, ואילו הדבר היה אפשרי, היינו נתקלים בבעיית סידור המשאבים לפי מידת חשיבותם.

6.7 נושאים נוספים הקשורים לקיפאון
6.7.1 נעילה בשני שלבים (two-phase locking)
6.7.3-6.7.2 קיפאון שאינו קשור במשאבים מקובלים
6.7.4 הרעבה (starvation)

6.7 נושאים נוספים הקשורים לקיפאון

6.7.1 נעילה בשני שלבים (two-phase locking)

עד כה ראינו כי לשיטות ההתחמקות מקיפאון ולדרכים שהוצעו למניעתו יש מחיר כה כבד עד שיישומן הופך במקרים רבים לבלתי כדאי. אולם במקרים שבהם התהליכים המעורבים משתפים פעולה, ואין חשש כי ינסו לחטוף משאבים, ניתן להציע פרוטוקולים סבירים להימנעות מקיפאון. פרוטוקול הנעילה בשני שלבים הוא אולי הפרוטוקול הידוע ביותר מסוג זה, והוא פותח לצרכים של מסדי נתונים. מצב אופייני במערכות מסדי נתונים הוא שכמה תהליכים מעדכנים רשומות במקביל. רשומה נכתבת היא משאב בלעדי, ולכן המערכת כולה חשופה לסכנה של קיפאון.

האוניברסיטה הפתוחה THE OPEN UNIVERSITY OF ISRAEL الجامعة المفتوحة
Yair Kurtzman

File #0006182 belongs to Yair Kurtzman- do not distribute

אלגוריתם הנעילה בשני שלבים פועל כדלקמן: בשלב הראשון התהליך מנסה לנעול את כל הרשומות שהוא צריך בזו אחר זו. אם כל הנעילות מצליחות, בשלב השני התהליך מתחיל לעדכן ולשחרר את הנעילות. אם במהלך השלב הראשון, התהליך נכשל בנעילתה של רשומה, התהליך משחרר את כל הרשומות שהוא נעל, וחוזר לשלב הראשון. גישה זו דומה לבקשת כל המשאבים מראש.

6.7.3-6.7.2 קיפאון שאינו קשור במשאבים מקובלים

בניגוד לקיפאון התלוי בהקצאת משאבים על ידי מערכת ההפעלה, במצבי קיפאון הנגרמים כתוצאה משימוש במשאבים לוגיים כגון קטעים קריטיים, מערכת ההפעלה אינה מעורבת כלל, ולכן היא איננה יכולה לטפל בגורמים של קיפאון מסוג זה.

ובכל זאת, בחלק מהמקרים אפשר לגרום לכך שמערכת ההפעלה תתמודד עם מצבי קיפאון כאלה. אם לא ניתן לטפל בגורמי המחלה, כל שנותר הוא לטפל בסימפטום, והסימן הברור ביותר לקיומו של קיפאון הוא שהתהליכים אינם מתקדמים. אפשר, למשל, לעקוב אחר התקדמות התהליכים שממתינים בסמפורים ולנסות לאתר תהליכים שנמצאים בקיפאון. מובן שפתרונות מסוג זה גורמים לתקורה גבוהה והם גם קשים ליישום כאשר מדובר בשילוב של סוגים שונים של משאבים, בין שמדובר במשאבים לוגיים, בין שמדובר במשאבים פיזיים ובין שמדובר בשילוב של משאבים משני הסוגים.

6.7.4 הרעבה (starvation)

זהו הנושא האחרון שנטפל בו בפרק זה. הרעבה, במקרה הכללי, היא מצב שבו תהליך איננו יכול להמשיך במהלכו התקין בגלל המתנה ארוכה מאוד (או אינסופית) למשאב אחד לפחות. קיפאון, להבדיל ממצב הרעבה, מתייחד בכך שלתהליך קפוא יש חלק במצב שהביא אותו לשיתוק. כל תהליך קפוא מחזיק לפחות משאב אחד שחילוצו היה מאפשר לצאת מהקיפאון. לעומת זאת, במקרה הכללי של הרעבה, חסימת התהליך אינה קשורה כלל לפעולותיו בעבר. משאב חיוני נמנע ממנו ללא קשר למעשיו בעקבות מדיניות של הסביבה החיצונית.

למשל, הקצאת משאבים בהתאם לעדיפויות קבועות היא גורם אופייני להרעבה. תהליך בעל עדיפות נמוכה צפוי להרעבה. דרך מקובלת להימנע ממלכודת זו היא הוספת מנגנון הזדקנות המבטיח כי תהליך הממתין זמן רב – עדיפותו גדלה. תור FIFO, לעומת זאת, הוא מדיניות הקצאת משאבים החסינה מפני הרעבה. ברגע שהתהליך נכנס לסוף תור, כבר ידוע בבירור כי ככל שיחלוף הזמן הוא יתקדם לכיוון ראש התור.

בתחילת הקורס קבענו כי ניהול המשאבים הוא אחת משתי מטרותיה העיקריות של מערכת ההפעלה. מניעת ההרעבה של תהליכים חיונית למימוש מטרה זו.

פרקים שאינם מופיעים בספר הלימוד
6.8 סיכום
6.9 בעיות פתורות
6.10 שאלות לחזרה
6.11 רשימת מושגים

6.8 סיכום

- בעיית הקיפאון עלולה להתעורר בכל מערכת שקיימת בה הקצאה של משאבים בלעדיים. בפרט הדבר נכון לגבי מערכת ההפעלה. פתחנו את הדיון בהגדרת ארבעת התנאים ההכרחיים והמספיקים לקיומו של קיפאון.
- הצענו שני סוגי אלגוריתמים לגילויו של מצב קיפאון. האחד – בניית גרף הקצאת המשאבים, והאחר – אלגוריתם לגילוי קיפאון כאשר קיימות מחלקות של משאבים שקולים.
- מרגע שנתגלה קיפאון, יש לשקול את הצעדים להיחלץ ממנו. הצענו כמה שיטות לעשות זאת, אך גם הדגשנו את מחירן הכבד.
- לאור מחיר זה, דנו באלגוריתמים המאפשרים, בתנאים מסוימים, להתחמק מקיפאון. אלגוריתם הבנקאים הוא המפורסם שבהם. המחיר אינו מבוטל, וגרוע מזה: כדי להיות יעיל, על האלגוריתם לדעת מראש את ההקצאות העתידיות – משימה קשה לכל הדעות.
- קו ההגנה האחרון שבחרנו בו היה מניעת קיפאון על ידי תקיפת כל אחד מהתנאים ההכרחיים לקיומו. גם כאן הדגשנו את המחיר הכבד של שיטות אלו – דבר המקשה מאוד את יישומן.
- המחיר הכבד הכרוך בשיטות לגילוי קיפאון או למניעתו, והמחיר הניכר הכרוך בהיחלצות ממנו, הם חלק מהתנאים הקשים שמערימים האלגוריתמים הכבדים להתחמקות מקיפאון. קשיים אלו מצביעים על ההיגיון המונח ביסוד שיטת בת היענה, מדיניות התעלמות הפועלת ביעילות במערכות הפעלה רבות מכיוון שהתהליכים שרצים בהן תוכננו לשתף פעולה.
- בניגוד למצטייר מבעיית הקיפאון במקרה הכללי, הרי שבתנאים מוגדרים היטב, כמו במקרה של מסד נתונים, ישנם פרוטוקולים – כגון פרוטוקול הנעילה בשני שלבים – אשר מונעים בהצלחה אפשרות של קיפאון. פרוטוקולים אלו מסתמכים על העקרונות שהתווינו בפרק זה.
- את הדיון חתמנו בבעיית ההרעבה, שהיא בעיה כללית של ניהול משאבים בכל מערכת מרובת-תהליכים.

האוניברסיטה הפתוחה
THE OPEN UNIVERSITY OF ISRAEL
الجامعة المفتوحة
Yair
Kurtzman

File #0006182 belongs to Yair Kurtzman- do not distribute

6.9 בעיות פתורות

שאלה 1

שלושה תהליכים עושים שימוש בארבעה משאבים בלעדיים. כל תהליך דורש, לכל היותר, שני משאבים. הראו כי במערכת זו לא יתקיים קיפאון.

תשובה 1

קיפאון יכול להתרחש רק בשעה שכל המשאבים תפוסים ולפחות תהליך אחד מבין השלושה מנסה לבקש עוד משאב. אם כל ארבעת המשאבים תפוסים, אזי לפחות תהליך אחד משתמש בשני משאבים. לכן כאשר התהליך התופס שני משאבים יסתיים, הוא יוכל לשחרר שני משאבים ולאפשר לתהליכים אחרים להתקדם.

שאלה 2

N תהליכים עושים שימוש ב-M משאבים בלעדיים. כל תהליך יכול לבקש לכל היותר M משאבים, כאשר כל התהליכים גם יחד יכולים להשתמש רק בכמות משאבים פחותה מ-M+N. הראו כי במערכת זו לא יתקיים קיפאון.

תשובה 2

- נסמן ב-c[i] את מספר המשאבים הדרושים לתהליך i.
- נסמן ב-a[i] את מספר המשאבים המוקצים לתהליך i.
- נסמן ב-r[i] את מספר המשאבים שהתהליך ה-i-י עדיין צריך.

אזי, נתון כי sum(c[i]) < M+N, כאשר הסכימה היא על כל ה-I-ים מ-1 עד N.
ברור ש-sum(c[i]) = sum(a[i]) + sum(r[i]).
מצב קיפאון יכול להתקיים רק כאשר כל המשאבים תפוסים, כלומר כאשר מתקייים sum(a[i]) = M .

מכל זה נובע ש- sum(r[i]) < N. מכיוון שהסכימה ב-sum היא על N תהליכים, הרי שבעבור j כלשהו מתקיים r[j] = 0, ובמילים אחרות – תהליך j ביקש את כל המשאבים שרצה לבקש, ובבוא הזמן הוא ישחרר אותם לטובת תהליכים אחרים.

שאלה 3

האלגוריתם שלפניכם מממש ויסות זרימה מעגלית של נתונים. הנתונים צריכים לעבור בצורה סדרתית עיבוד על ידי התהליכים P0, P1, P2, …, Pn-1. תהליך Pi-1 מעביר נתונים המעובדים לתהליך Pi בעבור כל i בין 1 ל-n-1. תהליך Pn-1 מעביר נתונים מעובדים לתהליך P0. הוכיחו כי התהליכים אינם נקלעים לקיפאון.
המערכים המוגדרים מטה משותפים לתהליכים:

```
semaphore buffer[MaxItemNum]; // controls access to the item i from total
                              // of MaxItemNum items
int pendingItemsNum[n];       // indicates the number of pending items for
                              // the process i
semaphore pendingItems[n];    // used to suspend process i when it has no
                              // pending items and to awake him upon the item
                              // availability
```

לפני תחילת הריצה של התהליכים, מתבצע האתחול שלהלן של המבנים המשותפים:

```
for each i in buffer set buffer(i) = 1;
for each j in pendingItemsNum set pendingItemsNum(j) = 0;
pendingItemsNum(0) = MaxItemNum;
for each j in pendingItems set pendingItems(j) = 0;
pendingItems(0) = 1;
```

האלגוריתם הוא:

```
Process(i){
        int j = 0;
        int nextProcess = (i+1) mod n;
        while (true){
              down (mutex);
              if (pendingItemsNum(i) == 0){
                    up (mutex);
                    down (pendingItems(i));
        }
        else
              up (mutex);
        down (buffer(j));
        Process(buffer(j));
        up (buffer(j));
        down (mutex);
        pendingItemsNum(i)--;
        pendingItemsNum(nextProcess)) ++;
        up (pendingItems(nextProcess));
        if (pendingItemsNum(i)==0){
              up (mutex);
              down (pendingItems(i));
        }
        up (mutex);

        j = (j+1) mod MaxItemNum;

        }
}
```

האוניברסיטה הפתוחה THE OPEN UNIVERSITY OF ISRAEL الجامعة المفتوحة
Yair Kurtzman

File #0006182 belongs to Yair Kurtzman- do not distribute

שימו לב, הסמנטיקה של הסמפורים שהשתמשנו בהם היא:

- down – משהה את התהליך הקורא, אם ערך הסמפור היה 0 ואינו משנה את ערכו. אם ערך הסמפור היה גדול מ-0, down מחסירה ממנו 1.
- up – מוסיפה לערך הסמפור 1. אם לפני ההוספה התהליך היה רדום, up מעירה אותו.

תשובה 3

מצב קיפאון יכול לקרות כאשר כל תהליך P_{i-1} ממתין לנתונים מ-P_i בעבור i מ-1 עד n-1 וגם P_0 ממתין לנתונים מ-P_{n-1}. כלומר, הערך של pendingItems(i) הוא 0 בעבור כל i. מצב זה אינו יכול לקרות, מכיוון ש-sum(pendingItems(i)) על גבי i מ-1 עד n-1 חייבת להיות MaxItemNum ולא 0, כפי שקיבלנו.

6.10 שאלות לחזרה

אילו מבין הטענות שלהלן הן טענות נכונות? (כל התשובות נמצאות בחומר הלימוד, בפרק עצמו או במדריך זה).

1. במצבי קיפאון מעורבים רק משאבים שניתנים לחילוץ.
2. גלגול לאחור הוא השיטה הפשוטה ביותר להיחלצות ממצבי קיפאון.
3. שיטת מספור המשאבים שתוקפת את תנאי ההמתנה המעגלית איננה בת-יישום מכיוון שקיים קושי למצוא מספור הולם לכל המשאבים.
4. שיטת הנעילה בשני צעדים מבצעת הקצאת רשומות בסדר מסוים במטרה להימנע ממצבי הקיפאון.
5. שני תהליכים המשתמשים בסמפורים אינם יכולים להיקלע למצב קיפאון.
6. מדיניות FIFO יכולה למנוע הרעבה.
7. החזק והמתן הם תנאי הכרחי לקיום קיפאון.
8. המתנה מעגלית איננה תנאי הכרחי לקיום קיפאון.

6.11 רשימת מושגים

להלן רשימת המושגים המופיעים בפרק זה (על פי סדר הופעתם בפרק):

קיפאון	deadlock	קבוצה של תהליכים נמצאים במצב קיפאון אם כל אחד מהתהליכים אינו יכול להתקדם בטרם יקרה אירוע היכול להיגרם אך ורק על ידי תהליך אחר בקבוצה. להלן ארבעה תנאים הכרחיים להיווצרות קיפאון: מניעה הדדית (mutual exclusion) – כל משאב מוקצה לתהליך אחד לכל היותר בכל נקודת זמן. החזקה והמתנה (hold and wait) – תהליך המחזיק במשאבים כלשהם מורשה לבקש עוד משאבים בלי לשחרר את המשאבים המוחזקים. אין חילוף כפוי (no preemption) – רק תהליך שמחזיק במשאב יכול לשחררו. המתנה מעגלית (circular wait) – קיים מעגל המתנה, שבו כל תהליך ממתין למשאב שמוחזק על ידי התהליך הבא במעגל. הערה: כאשר המשאבים הנדרשים אינם משאבי מערכת, אלא משאבים לוגיים, אזי התנאי היחיד להיווצרות קיפאון הוא המתנה מעגלית (לדוגמה, תהליך A ממתין לאירוע מתהליך B, ו-B ממתין לאירוע מ-A).
הרעבה	starvation	מצב שבו תהליך(ים) שאינו נמצא בקיפאון אינו מקבל שירות (משאב) שהוא ביקש במשך פרק זמן ארוך מדי ביחס לזמן התגובה הנדרש ממנו (תיאורטית – גם זמן אינסופי), משום שהשירות ניתן תמיד לתהליכים אחרים לפניו. חשוב לציין כי כאשר רוצים לדעת אם אלגוריתם מסוים עלול לגרום הרעבה, צריך לבדוק אם קיים תסריט שבו תהליך לא יקבל לעולם את המשאב המבוקש. מצב שבו תהליך אחד נאלץ להמתין לשירות יותר מתהליך אחר, אך מקבל את השירות בסופו של דבר, אינו מצב של הרעבה, אלא (אולי) של חוסר הגינות.
משאב ניתן לחילוץ	preemptable resource	משאב שניתן לקחת אותו מתהליך המחזיק בו ולהחזירו אליו מאוחר יותר, בלי לשבש את תוצאות הריצה של התהליך.
משאב בלעדי	non-preemptable resource	משאב שחילוצו מתהליך שמחזיק בו גורם לתהליך נזק בלתי הפיך.
שיטת בת היענה	Ostrich Algorithm	למרות שמו, זה אינו אלגוריתם, אלא אסטרטגיה של התעלמות מאפשרות של קיפאון במערכת. גישה זו מוצדקת אם הסבירות של קיפאון במערכת היא נמוכה, וטיפול מונע הוא יקר מדי מבחינת המשאבים הנדרשים ומבחינת המגבלות שהוא מטיל על המשתמשים.
גילוי והתאוששות	Detection and Recovery	אסטרטגיה לטיפול בקיפאון, שלפיה לא ננקטים שום אמצעים למניעת קיפאון אלא מופעלים רק אלגוריתמים

האוניברסיטה הפתוחה THE OPEN UNIVERSITY OF ISRAEL الجامعة المفتوحة
Yair Kurtzman

File #0006182 belongs to Yair Kurtzman- do not distribute

		לגילוי קיפאון. אם מתגלה קיפאון במערכת, ננקטים צעדים להתאוששות ממנו.
התאוששות על ידי חילוץ כפוי	Recovery Through Preemption	חילוץ מבוקר של משאב מהתהליך המחזיק בו לצורך שבירת הקיפאון, תוך נקיטת אמצעים (ובכלל זה לעתים התערבות אנושית) כדי למזער את שיבוש תוצאות הריצה של התהליך. היכולת להשתמש בשיטה זו תלויה בסוג המשאב, ולעתים השיטה איננה ישימה.
התאוששות על ידי גלגול לאחור	Recovery Through Rollback	השיטה מבוססת על שמירה תקופתית של המצב הנוכחי של תהליך בנקודות ביקורת מוגדרות מראש על ידי מערכת ההפעלה (checkpoints). הדבר מאפשר להריץ כל תהליך מנקודת הביקורת האחרונה שבה הוא נשמר, כאשר רק העבודה שנעשתה על ידי התהליך מהנקודה הזאת הולכת לאיבוד. התאוששות מקיפאון נעשית על ידי גלגול לאחור של אחד התהליכים, לנקודה שבה התהליך טרם קיבל משאבים שנדרשים על ידי תהליכים אחרים שנמצאים בקיפאון (בכך, משאבים אלה משתחררים והקיפאון נשבר).
התאוששות על ידי הריגת תהליך	Recovery Through Killing a Process	הריגה של תהליך שמעורב בקיפאון, שחרור כל משאביו ושבירת הקיפאון. אפשרות נוספת: הריגה של תהליך שאינו בקיפאון, אך מחזיק במשאבים שנדרשים על ידי התהליכים בקיפאון.
שיטת ההתחמקות מקיפאון	Deadlock Avoidance	אסטרטגיה לטיפול בקיפאון, שלפיה הקצאת משאבים על ידי מערכת ההפעלה תתבצע בצורה מבוקרת, שתבטיח שלא ייתכן מצב של קיפאון.
מסלול הקצאת משאבים	Resource Trajectory	ייצוג מהלכם של תהליכים הרצים במערכת, מבחינת הקבלה והשחרור של משאבים. מסלול זה עובר במרחב מצבים שמוגדרים על ידי כל הצירופים האפשריים של הקצאות משאבים לכל התהליכים בנקודות זמן שונות.
מצב בטוח	Safe State	מצב שבו אין קיפאון וניתן לספק את כל הבקשות הממתינות להקצאת משאבים בסדר כלשהו. כל מצב אחר נקרא מצב לא בטוח (unsafe).
אלגוריתם הבנקאים	The Banker's Algorithm	אלגוריתם הבוחן אם הקצאת משאב מסוים לתהליך מסוים תשאיר את המערכת במצב בטוח.
מניעת קיפאון	Deadlock Prevention	אסטרטגיה לטיפול בקיפאון שלפיה המערכת מתוכננת באופן שישלול את האפשרות לקיום אחד מארבעת התנאים להיווצרות קיפאון. כך יימנע הקיפאון באופן אוטומטי.
נעילה בשני שלבים	Two Phases Lock	שיטה למניעת קיפאון שנהוגה בעיקר במערכות מסדי נתונים. בשיטה זו, תהליך שמבקש לבצע עדכונים שמעורבות בהם כמה רשומות מבקש לנעול את כל הרשומות הדרושות. אם שלב הנעילה מצליח, התהליך מבצע את העדכונים ומשחרר את כל הרשומות. אם נעילה של רשומה אחת לפחות נכשלה, התהליך משחרר את כל הרשומות ומתחיל את שלב הנעילה מחדש.

פרק 7

וירטואליזציה וענן

האוניברסיטה הפתוחה
THE OPEN UNIVERSITY OF ISRAEL
الجامعة المفتوحة
Yair
Kurtzman

File #0006182 belongs to Yair Kurtzman- do not distribute

7 וירטואליזציה וענן
7.1 היסטוריה
7.2 דרישות חומרה לווירטואליזציה
7.3 hypervisor מטיפוס ראשון ושני
7.4 היבטים למימוש וירטואליזציה יעילה
7.5 מיקרו-קרנל של hypervisor
7.6 וירטואליזציה של זיכרון
7.7 וירטואליזציה של I/O
7.8 מכונות וירטואליות על מחשבים מרובי מעבדים
7.9 ענן
7.10 וירטואליזציה ברמת מערכת ההפעלה
7.11 תיאור מקרה: VMware

למונח "וירטואליזציה" יש משמעויות רבות ושונות. אפשר לדון בווירטואליזציה של זיכרון, של מעבד או של דיסק קשיח. את הסוגים האלה של וירטואליזציה הכרנו בפרקים הקודמים. בפרק זה נכיר סוג נוסף של וירטואליזציה: **וירטואליזציה של מערכות מחשב**. נכיר את ההיסטוריה, את היתרונות ואת החסרונות של גישות שונות לווירטואליזציה של מערכות מחשב, נדון בטכנולוגיות השונות המשמשות למטרה זו ונסקור את מאפייני החומרה הנדרשים כדי לתמוך במימוש יעיל של וירטואליזציה מסוג זה.

וירטואליזציה של מערכות מחשב מושגת באמצעות שכבת תוכנה המכונה ***hypervisor*** או ***virtual machine monitor*** (***VMM***). שכבת תוכנה זו מאפשרת יצירה והרצה של כמה ***מכונות וירטואליות*** (***virtual machines, VMs***), אשר משתפות משאבי חומרה של המחשב שעליו מותקן ה-VMM. "מכונה וירטואלית" היא מערכת מחשב וירטואלית שאפשר להתקין עליה מערכת הפעלה. במילים אחרות, VMM מספקת הפשטה של מערכת מחשב, והפשטה זו נקראת מכונה וירטואלית. מכונות וירטואליות ממודרות זו מזו, ואף שהן רצות על מחשב פיזי משותף אחד, נוצרת אשליה שכל מערכת הפעלה פועלת על מכונה ייעודית. זאת בדומה לווירטואליזציה של מעבד, שבה כמה תהליכים רצים על מעבד משותף.

וירטואליזציה של מערכות מחשב אינה רעיון חדש (7) (8): בשנים האחרונות, וירטואליזציית מערכות מחשב הפכה לתחום חם מאוד, וקיים מגוון רחב מאוד של פתרונות וירטואליזציה שונים. מרכזי המידע (datacenters) של ימינו מאחסנים כמויות גדולות של מערכות מחשב, אשר צורכות חשמל רב הן לעבודת עיבוד נתונים והן לצורך קירור המערכות. וירטואליזציה עוזרת להגיע לניצולת רבה יותר של משאבי החומרה של המערכות הללו, ולפיכך לחיסכון ניכר בעלויות התחזוקה.

שרת יקר וחזק המריץ מכונות וירטואליות ממודרות זו מזו יכול במקרים רבים להיות משתלם יותר מכמה שרתים זולים וחלשים, כי כל אחד מהשרתים הללו תופס מקום

האוניברסיטה הפתוחה THE OPEN UNIVERSITY OF ISRAEL الجامعة المفتوحة
Yair Kurtzman

File #0006182 belongs to Yair Kurtzman- do not distribute

במרכז החישוב והשימוש בו כרוך בהוצאות תפעוליות. להלן נמנה שימושים טיפוסיים ומשתלמים כלכלית של וירטואליזיציית מערכות.

איחוד שרתים (server consolidation). במקרים רבים אפשר לאחד שרתים ייעודיים שאינם מנצלים את מלוא יכולות החומרה (hardware capabilities) שלהם. בדרך זו אפשר לצמצם מאוד את עלויות הרכישה והתחזוקה.

ניוד מערכת (system migration). מכונה וירטואלית אפשר לנייד למכונה פיזית אחרת. הצורך בניוד יכול לנבוע מעומס על המכונה הפיזית או מכישלון של אחד מרכיבי החומרה במכונה הפיזית. להבדיל מניוד תהליכים באשכול של מכונות שמנוהל על ידי מערכת הפעלה מבוזרת (6), ניוד של מכונה וירטואלית הוא קל יותר הן בהיבט הטכני, דהיינו מבחינת המימוש, הן קונספטואלית.

פריסה קלה של תוכנות התקן וירטואליות. תוכנות התקן וירטואליות (virtual appliances) הן מכונות וירטואליות מוכנות לשימוש שמותקנות בהן מערכת הפעלה ואפליקציות, וכך נחסך מן המשתמשים לדאוג לתאימות של רכיבי תוכנה שונים ומתאפשרת הפצה מהירה של תוכנית אפליקטיבית מסוימת.

ניפוי שגיאות ובדיקות. מכונה וירטואלית יכולה לשמש לבדיקות של מערכות הפעלה ושל רכיבי תוכנה מגרסאות שונות. אלמלא וירטואליזציית המערכות, היה נדרש להחזיק מכונה פיזית נפרדת לכל צירוף של רכיבי התוכנה הנבדקת.

מידור (isolation) כהגנה נגד פריצה ווירוסים. וירטואליזציה מאפשרת מידור של תוכנות בלתי אמינות במכונה וירטואלית, וכך מגינה על המכונה הפיזית ועל המכונה הווירטואלית זו מזו.

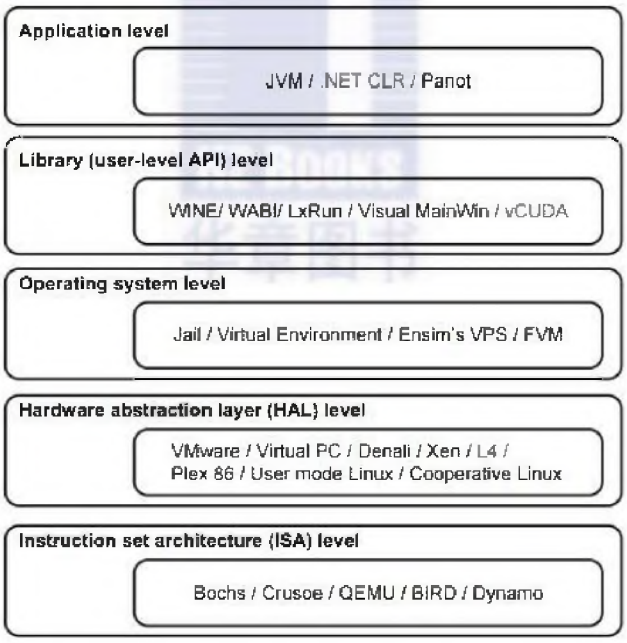
ענן (cloud). זהו ככל הנראה השימוש החשוב ביותר של וירטואליזציה של מערכות. הרעיון שביסוד הענן הוא פשוט: לבצע מיקור חוץ של אחסון המידע ותהליכי החישוב של ארגון למרכז חישובים חיצוני. את מרכז החישובים הזה מנהלת חברה שתחום התמחותה הוא חומרה, מכונות פיזיות, הספקת חשמל 24 שעות 7 ימים בשבוע, קירור ותחזוקה. אמנם, מרכז החישובים שייך לצד ג וחברה המשתמשת בשירותיו צריכה לשלם עליהם, אך העסקה עשויה להיות כדאית מאוד מבחינה כלכלית.

היכן ממוקם ה-VMM?

במובן רחב יותר של המושג וירטואליזציה, יש כמה אפשרויות למיקום שכבת הווירטואליזציה במודל השכבות של מערכת מחשב:

- וירטואליזציה ברמת ה-Instruction Set Architecture – ISA
- וירטואליזציה ברמת ה-Hardware Abstraction Layer – HAL
- וירטואליזציה ברמת מערכת ההפעלה
- וירטואליזציה ברמת הספרייה
- וירטואליזציה ברמת האפליקציה

באיור שלהלן אפשר לראות דוגמאות למוצרי וירטואליזציה שונים, בהתאם למיקומה של שכבת הווירטואליזציה במודל השכבות:

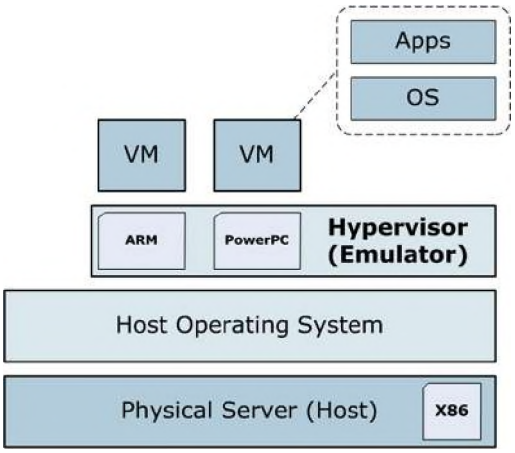


איור 1

בפרק זה נעסוק בעיקר ברמת ה-HAL, אך בטרם נתמקד בה, נתאר בקצרה את כל האפשרויות.

וירטואליזציה ברמה של ISA

וירטואליזציה ברמה של ISA אינה אלא חיקוי (emulation) של כל פקודות ה-ISA של המכונה הווירטואלית שבתוכנה. בשיטה זו, הוראות מכונה מהמערכת האורחת (guest system) מתורגמות לפקודות של המכונה הפיזית (host system). השיטה בצורתה הטהורה מפגינה ביצועים חלשים (הרצה של מכונה וירטואלית איטית מאוד), ולכן אין בה תועלת מעשית לרוב הצרכים, למעט אולי בדיקת התנהגות של רכיבי תוכנה בעבור מערכות מחשב פיזיות בטרם יישומן בפועל. אנו לא נדון בסוג זה של וירטואליזציה.



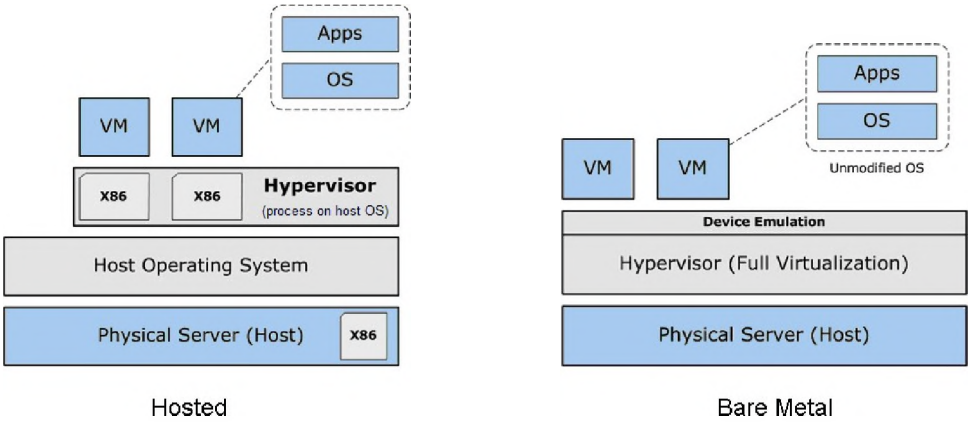
איור 2 וירטואליזציה ברמה של ISA
(על פי Jones 2010)

האוניברסיטה הפתוחה
THE OPEN UNIVERSITY OF ISRAEL
الجامعة المفتوحة
Yair
Kurtzman

File #0006182 belongs to Yair Kurtzman- do not distribute

וירטואליזציה ברמת ה-HAL

וירטואליזציה ברמת ה-HAL משתמשת בדמיון שבין ארכיטקטורות מחשב. מצד אחד הארכיטקטורה שעליה רצה מערכת הפעלה של מכונה וירטואלית, הנקראת מערכת הפעלה אורחת (guest operating system), ומצד אחר הארכיטקטורה של המכונה הפיזית שעליה רצה שכבת הווירטואליזציה. בשל הדמיון בין הארכיטקטורות, רוב ההוראות של המכונה הווירטואלית יכולות להתבצע ישירות על ה-CPU הפיזי. זהו אחד מסוגי הווירטואליזציה שנדון בהם בפרק זה ונראה אפשרויות למימושם.

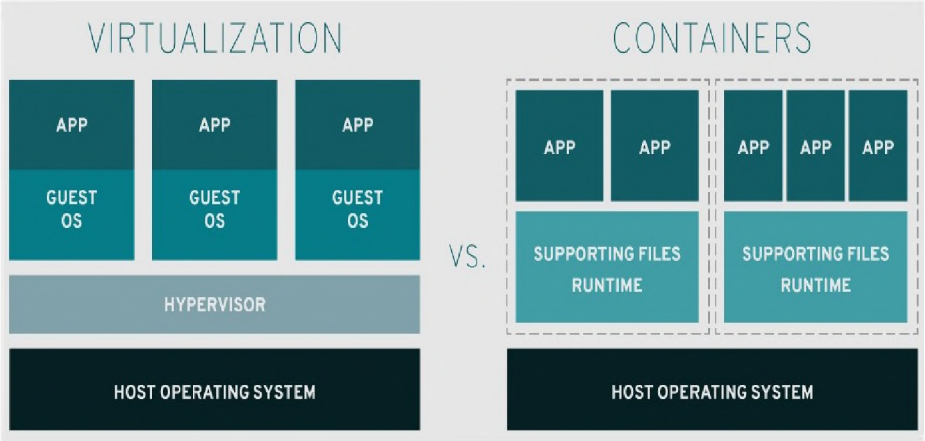


איור 3 וירטואליזציה ברמת ה-HAL

(על פי Jones, 2010)

וירטואליזציה ברמת מערכת ההפעלה

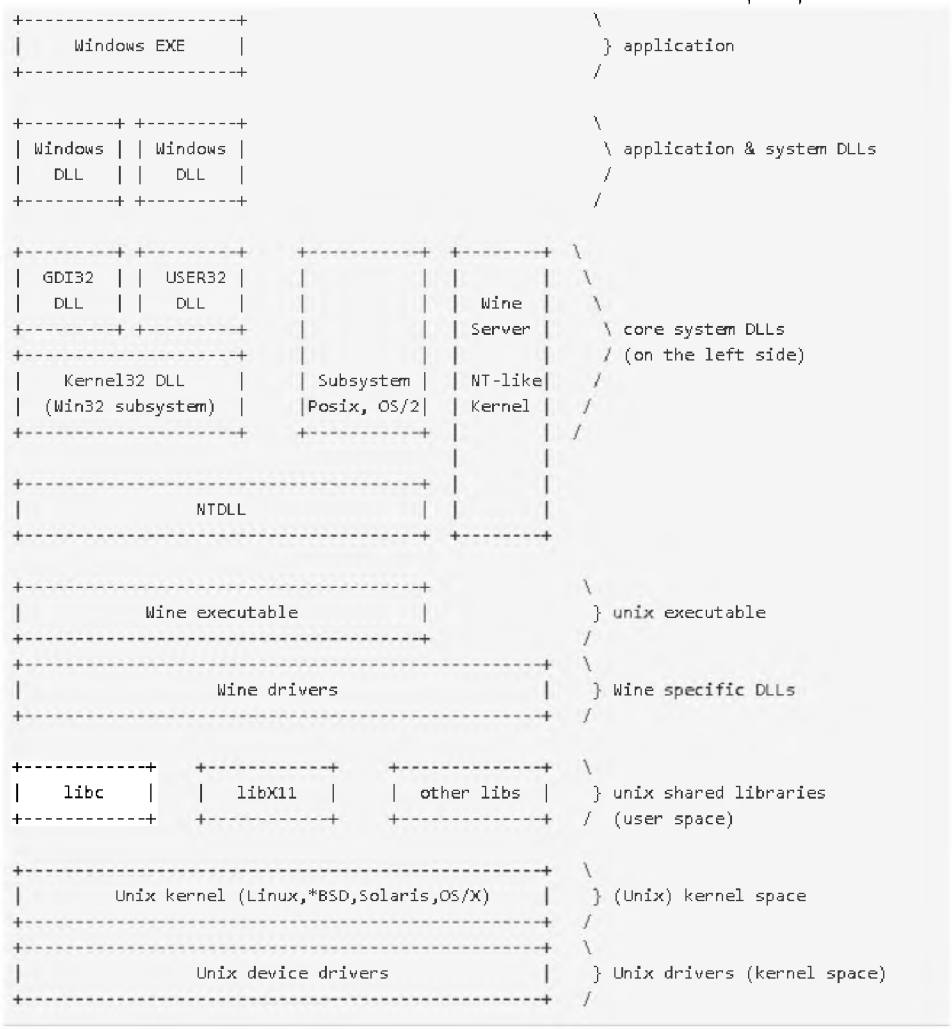
וירטואליזציה ברמת מערכת ההפעלה מאפשרת יצירת קונטיינרים (containers). ניהול משאבי החומרה והתוכנה המוקצים לקונטיינר מתבצע בעזרת מערכת ההפעלה. להבדיל מווירטואליזציה מלאה, במקרה של וירטואליזציה ברמת מערכת ההפעלה, קונטיינרים משתפים מבני נתונים פנימיים של המערכת המארחת. לא נדון כאן בסוג זה של וירטואליזציה.



איור 4 וירטואליזציה מלאה לעומת וירטואליזציה ברמת מערכת ההפעלה

וירטואליזציה ברמת הספרייה

וירטואליזציה ברמת הספרייה מבוצעת באמצעות העברה (תרגום) של קריאות מערכת לקריאת פונקציות של ספריות וירטואליזציה. דוגמה לסוג זה של וירטואליזציה היא תוכנת WINE, המאפשרת הרצת אפליקציות של מערכת חלונות (Windows) על מערכות Unix. לא נדון כאן גם בסוג זה של וירטואליזציה.



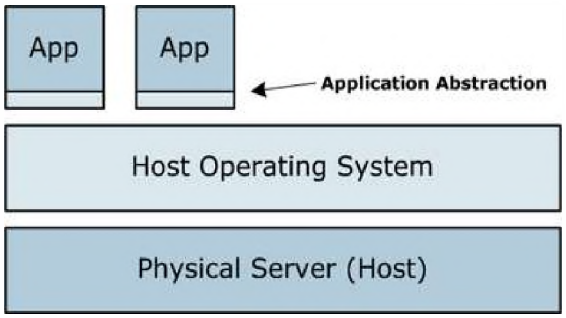
איור 5 וירטואליזציה ברמת הספרייה

וירטואליזציה ברמת האפליקציה

כשמדובר על וירטואליזציה ברמת האפליקציה, הכוונה היא בדרך כלל למימוש של סביבה וירטואלית אשר מסוגלת לבצע אינטרפרטציה של bytecode ב- run-time environment. גם בסוג זה של וירטואליזציה לא נדון כאן.

האוניברסיטה הפתוחה
THE OPEN UNIVERSITY OF ISRAEL
الجامعة المفتوحة
Yair
Kurtzman

File #0006182 belongs to Yair Kurtzman- do not distribute



איור 6 וירטואליזציה ברמת האפליקציה

גורמים המושפעים מרמת הווירטואליזציה

כמה מהגורמים המשפיעים על המימוש של וירטואליזציה ברמות שונות:

- רמת המידור (isolation) שהווירטואליזציה מספקת
- כמות המשאבים הנדרשת מהמכונה המארחת
- תקורה בביצועים
- סקלביליות (scalability) – שיפור של ביצועים עקב הוספת משאבים
- גמישות מבחינת האפשרות לקומבינציות שונות של חומרה

באופן כללי, ככל שווירטואליזציה ממומשת נמוך יותר, כלומר ככל שהמימוש קרוב יותר לחומרה, המכונות הווירטואליות ממודרות טוב יותר ממערכת המכונה המארחת. מאידך גיסא, מימוש קרוב לחומרה דורש יותר משאבים ומתבטא בגמישות מועטה יותר. באיור שלפניכם מתוארים כמה היבטים של רמת הווירטואליזציה לפי רמת ההשפעה על ביצועים, גמישות, מידור ומורכבות מימוש.

Comparison of Implementation Levels of Virtualization

Implementation Level	Performance	Implementation Complexity	Application flexibility	Application Isolation
ISA	Very poor	Medium	Excellent	Medium
HAL	Excellent	High	Medium	Very Good
OS Level	Excellent	Medium	Low	Very Poor
Application Level	Medium	Low	Low	Very Poor
Library Level	Poor	High	Low	Excellent

איור 7

! פרק זה במדריך הלמידה מתמצת את הנאמר בספר הלימוד ואף מוסיף ומרחיב את היריעה – אך איננו מחליף אותו. עליכם לקרוא את הפרק המקביל בספר הלימוד.

7.1 היסטוריה

פרק זה כולל סקירה קצרה וחלקית של התפתחות המוצרים התומכים בווירטואליזציה מלאה. לא נרחיב בסוגיה זו כאן.

7.2 דרישות חומרה לווירטואליזציה

מכונות וירטואליות צריכות לפעול בדיוק כמו המכונות האמיתיות. בפרט, הן חייבות לאפשר אתחול של המחשב (ובכלל זה שינויי קונפיגורציית BIOS) ולספק אפשרות להתקין ולהריץ מערכת הפעלה שרירותית, בדיוק כפי שאפשר לעשות על חומרה אמיתית. משימה זו היא באחריות hypervisor ויש לה שלושה ממדים:

- ניהול משאבים: על hypervisor לשלוט בצורה מלאה במשאבי המחשב, בדומה למערכת הפעלה.
- נאמנות למקור: ההתנהגות של תוכנה צריכה להיות זהה על מכונה וירטואלית ועל חומרה אמיתית.
- יעילות: ברוב המקרים, תוכנה צריכה לרוץ ללא התערבות מצידו של ה-hypervisor.

נסתכל על דרך פשוטה לניהול משאבים. Hypervisor יכול לבצע הוראות מכונה אחת אחת. את חלקן הוא יכול לבצע ישירות. לדוגמה, אין חשד להשתלטות על משאבי המערכת אם הוראת inc תבוצע על החומרה האמיתית. אך הוראות שאינן בטוחות לביצוע ישיר יתורגמו על ידי ה-hypervisor לכמה הוראות בטוחות. דוגמאות להוראות שאינן בטוחות הן ביטול פסיקות או שינוי מיפויים בטבלאות דפים. כאשר ניתנות הוראות כאלה, ה-hypervisor ישמור אינדיקציה לשינויים כדי שמערכת ההפעלה שרצה מעליו "תחשוב" ששינתה את מצב הפסיקות או את טבלת המיפויים. לאינטרפרטציה של הוראות מכונה בזו אחר זו יש חסרון – היעילות. אם מספר הוראות המכונה שידרשו תרגום יהיה גבוה ביחס לשאר ההוראות, המערכת תתקשה לספק ביצועים טובים. בהמשך נראה כיצד משיגים ביצועים טובים; בשלב זה נסתפק בכך שבאופן כזה אפשר לפתור את בעיית ניהול המשאבים.

נבאר כעת את ממד הנאמנות למקור. ארכיטקטורת מחשבי x86 הציבה אתגר לא פשוט לדרישת הנאמנות למקור. הסיבה לכך היא, שישנן הוראות מכונה הפועלות באופן שונה בעת ביצוע במצב מיוחס מאשר בעת ביצוע במצב משתמש. דוגמאות לעניין זה הן הוראות של קלט/פלט, הוראות שינוי הגדרות MMU וכיוצא בהן. הוראות כאלה נקראות ***הוראות רגישות*** (***sensitive instructions***). נוסף על הוראות רגישות, ישנן הוראות הגורמות ל-TRAP אם הן נקראות במצב משתמש. הוראות אלו נקראות ***הוראות מיוחסות*** (***privileged instructions***). אם קבוצת הוראות רגישות בארכיטקטורה מסוימת יוצרת תת-קבוצה של הוראות מיוחסות, אזי ביצוע ההוראות הרגישות יגרום ל-TRAP, ו-hypervisor יוכל לקחת פיקוד ובסופו של דבר להביא לכך שביצוע של הוראה על החומרה הווירטואלית ועל החומרה האמיתית יביאו לתוצאות זהות. אם יש הוראות

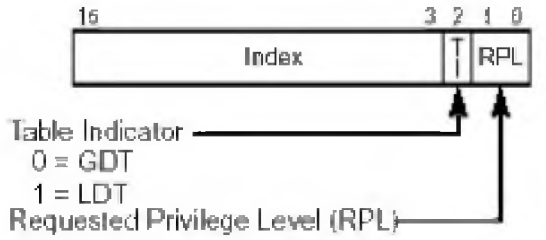
האוניברסיטה הפתוחה
THE OPEN UNIVERSITY OF ISRAEL
الجامعة المفتوحة
Yair
Kurtzman

File #0006182 belongs to Yair Kurtzman- do not distribute

רגישות שאינן גורמות ל-TRAP, ביצוע ההוראות הללו במצב משתמש יהיה שונה מביצוען במצב מיוחס. וכאשר ההוראות הללו מתבצעות על ידי מערכת הפעלה אשר רצה מעל hypervisor במצב מיוחד, "וירטואלי", נצפה לתוצאה שונה מזו שתתקבל כאשר מערכת ההפעלה מבצעת את ההוראה ישירות על החומרה (הלא-וירטואלית). דוגמה להוראה כזו היא הוראת POPF אשר ביצועה יכול להשפיע על ביטול או קבלה של פסיקות. אם הוראה זו מבוצעת כאשר מעבד נמצא במצב משתמש, שינוי מצב הפסיקות פשוט אינו מתרחש. יתרה מזו, הדבר קורה בצורה "שקטה", המבחינה את ההוראות הרגישות מהוראות מיוחסות.

למעשה, המצב מורכב עוד יותר. נוסף על עצם קיומן של הוראות רגישות, חשוב להבחין שקיימים מצבים רגישים. ***מצב רגיש*** (***sensitive state***) הוא מצב שקריאתו בזמן שהמעבד נמצא במצב משתמש אינה גורמת ל TRAP. לדוגמה, כאשר מעבד x86 שולף הוראת מכונה, הכתובת שבהוראה נקבעות על ידי הזוג CS:EIP. אם ערך הרגיסטר EIP% הוא 0x100 המעבד מתייחס לבית ה-256 מתוך סגמנט קוד שערכו מופיע ברגיסטר CS%. הרגיסטר CS% כולל את הסלקטור של סגמנט קוד אשר מורכב משלושה שדות:

- מספר הסגמנט (segment number) (סיביות 3-16)
- סלקטור של טבלה (table selector flag) (סיבית 2)
- מזהה ייחוס ההוראה (request privilege level) (סיביות 0-1)



איור 8 הרגיסטר CS

RPL מגדירה באיזו רמת ייחוס נמצא המעבד. לצורך הדיון שלנו, רמות הייחוס הן מצב משתמש או מצב מיוחס. קריאת ערך ה-RPL על ידי התוכנה שרצה על מכונה וירטואלית תחזיר ערך המתאים למצב משתמש. הדבר תקין כל עוד מדובר על תוכנית שרצה במצב משתמש מעל ה-hypervisor. אבל אם ההוראה לקריאת RPL מתבצעת על ידי מערכת הפעלה אשר רצה מעל ה-hypervisor, היא תקבל אינדיקציה של ריצה במצב משתמש. אי לכך, RPL הוא מצב רגיש שקריאתו צריכה לגרום TRAP ל-hypervisor. במקרה של TRAP, ה-hypervisor יתערב, ואם המעבד יימצא במצב מיוחס וירטואלי, ה-TRAP יביא לכך שהקורא יראה ב-RPL ערך המראה כי המעבד נמצא במצב מיוחס.

כל האמור לעיל מביא אותנו למשפט שהוכיחו Golberg ו-Popek (8): "למערכת מחשב אופיינית מדור שלישי אפשר לבנות VMM יעיל אם קבוצת הוראות רגישות היא תת-קבוצה אמיתית של הוראות מיוחסות". דהיינו, כדי לבנות VMM יעיל מספיק, דרוש שכל ההוראות העלולות להשפיע על פעילות נאמנה למקור של VMM יגרמו ל-TRAP ויעבירו פיקוד ל-VMM, וכי ה-VMM יעשה להן חיקוי (emulation). לעומת זאת,

הפעולות הלא-מיוחסות יכולות להתבצע באופן ישיר וביעילות על המחשב הפיזי. שיטה זו נקראת שיטת ***Trap and Emulate*** או שיטת ***וירטואליזציה בחומרה*** (***hardware virtualization***).

עד שנת 2005, משפחת המעבדים x86 לא יכלו לממש וירטואליזציית חומרה (hardware virtualization) עקב בעיות נאמנות למקור. בעיות אלו נפתרו עם הופעתם של מעבדי אינטל ו-AMD התומכים בטכנולוגיית וירטואליזציה (virtualization technology, VT). הרעיון הבסיסי הוא ש-hypervisor יכול לקבוע בעזרת מפת סיביות חומרתית אילו הוראות מכונה יגרמו ל-TRAP בעת ריצתה של מכונה וירטואלית, ובהמשך לדאוג לביצוע תוכנית בצורה "נאמנה למקור".

וירטואליזציה על מעבדי x86 הייתה קיימת גם לפני שנת 2005. למעשה, כבר בשנת 1998 פיתחה חברת VMWare שיטה לווירטואליזציה על מעבדי x86 ללא טכנולוגיית VT, על ידי hypervisor המשכתב קוד של תוכנה שרצה עליו בעת ריצתו (on the fly) ומחליף הוראות רגישות או הוראות המכילות פנייה למצבים רגישים בסדרת הוראות המכילות TRAP ל-hypervisor. אחרי ה-TRAP, ה-hypervisor בודק את חוקיות הפעולה הרגישה ומבצע את אותה הפעולה כאשר המעבד נמצא במצב מיוחס. שיטה זו נקראת ***תרגום בינארי*** (***binary translation***).[17]

לא כל פעולה רגישה דורשת תרגום בינארי. תוכניות משתמש הרצות במערכת הפעלה שמעל ה-hypervisor יכולות להמשיך לרוץ ללא תרגום כזה, מכיוון שהן רצות ממילא במצב משתמש. התרגום אמור להספיק במקרה של הוראות רגישות של מערכת ההפעלה עצמה, אשר רצה מעל ה-hypervisor.

שאלה 1

כיצד hypervisor אמור לקבל את ה-TRAP שביצעה הוראה שהוא החליף בעת התרגום הבינארי?

התשובה לשאלה זו מתחלקת לשני חלקים. אחד, כיצד הדבר קורה ב-hypervisor הרץ על מערכת הפעלה מארחת דוגמת VMware Player ; ושני, כיצד הוא קורה ב-hypervisor שרץ ישירות על חומרה. את התשובות על שאלות אלו תוכלו למצוא בסעיף 7.4.1.

שיטת התרגום הבינארי לא הייתה השיטה היחידה לווירטואליזציה על x86. (1) Denali משנת 2002 ו-Xen (2) משנת 2002 הם ה-VM monitors המספקים פתרונות של פרא-וירטואליזציה (para-virtualization). להבדיל מפתרונות של וירטואליזציה מלאה (full virtualization), פתרונות פרא-וירטואליזציה מספקים סביבת וירטואליזציה דרך API של ***hypercalls***, המאפשר למערכת הפעלה אורחת להשתמש בשירותי ה-VMM. מובן שלשם כך יש לשנות את מערכת ההפעלה. נרחיב על שיטה זו בהמשך.

---

[17] יש ששיטה זו נקראת גם binary recompilation.

האוניברסיטה הפתוחה THE OPEN UNIVERSITY OF ISRAEL الجامعة المفتوحة
Yair Kurtzman

File #0006182 belongs to Yair Kurtzman- do not distribute

חומר הלימוד שבסעיף זה מתמצת את הנאמר בספר הלימוד ומוסיף עליו, אך אינו מחליף אותו. עליכם לקרוא את הסעיף המקביל בספר הלימוד.

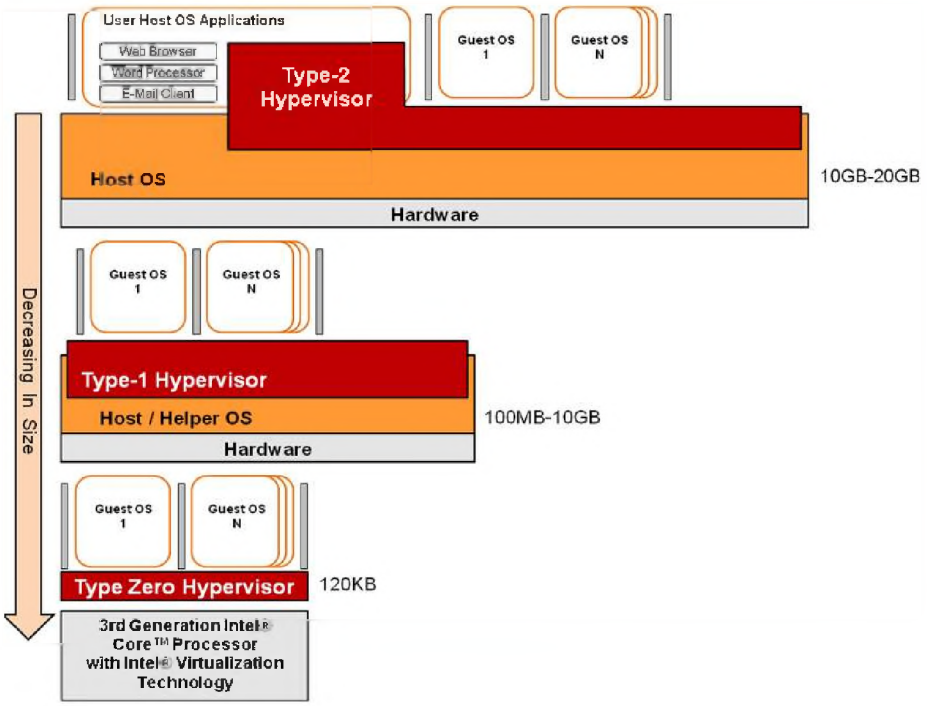
7.3 Hypervisor מטיפוס ראשון ושני

Hypervisor מטיפוס ראשון – מתואר באיור 1-29 (א) בספר הלימוד. למעשה, hypervisor מטיפוס ראשון הוא סוג של מערכת הפעלה, שכן זוהי התוכנית היחידה הפועלת במצב מיוחס (kernel mode). תפקידו של hypervisor מסוג זה הוא להציג כלפי מעלה עותקים של חומרה הנקראים מכונות וירטואליות, בצורה דומה למדי לתהליכים במערכת הפעלה "רגילה".[18] סוג זה של VMM נועד להריץ הן מערכות הפעלה כלליות והן מערכות הפעלה ייעודיות בעלות סט אפליקציות המממש מטרה מסוימת. כתוצאה מכך, הגודל של VM תלוי בנסיבות ובהיקף השירותים: בין עשרות מגה-בתים לעשרות ג'יגה-בתים.

Hypervisor מטיפוס שני – מודגם באיור 1-29 (ב) בספר הלימוד. טיפוס זה הוא תוכנה שרצה על מערכת הפעלה מארחת (host operating system) ומספקת שירותי VMM למערכת הפעלה אורחת (guest operating system) או לכמה מערכות. סוג זה של VMM נועד להריץ תוכנות שנכתבו למערכת הפעלה אחת על גבי מערכת הפעלה אחרת, ומכיוון שמדובר על מערכות הפעלה כלליות, גודל ה-VM יכול להגיע לעשרות ג'יגה-בתים.

בשנים 2010-2013 נעשה ניסיון להגדיר טיפוס נוסף – hypervisor מטיפוס אפס (4). עיקר השימוש ב-VMM מטיפוס זה הוא במערכות משובצות. טיפוס זה הוא שכבת תוכנה דלה מאוד של מנהל משאבים. עיקר שירותי הווירטואליזציה מסופקים על ידי חומרה. הדבר מאפשר להגביל מאוד את היקף הקוד של VMM, לשפר את הביצועים ולהגיע לגודלי VM קטנים מאוד (לדוגמה, LynxSecure's Type Zero hypervisor דורש שטח אחסון בגודל של 120K ו-4Gb של RAM בלבד).

[18] hypervisor מותקן על מחשב פיזי, עולה עם עליית המחשב שעליו הוא מותקן ומספק, בין היתר, שירותי מכונה מורכבת לאפליקציות (מכונות וירטואליות). השירותים הללו כוללים את השירותים המסורתיים כמו תזמון וניהול זיכרון וכן שירותים חדשים כגון ניוד של מכונות וירטואליות.



איור 9 השפעת טיפוס ה-hypervisor על גודלו

חומר הלימוד שבסעיף זה מתמצת את הנאמר בספר הלימוד ומוסיף עליו, אך אינו מחליף אותו. עליכם לקרוא את הסעיף המקביל בספר הלימוד.

7.4 היבטים למימוש וירטואליזציה יעילה

כעת נתבונן באיור 7-3 בספר הלימוד. אנו רואים hypervisor מסוג 1 עם מכונה וירטואלית אחת. כלפי מעלה, ה-hypervisor מספק העתק של חומרה. המכונה הווירטואלית פועלת כתהליך משתמש במצב משתמש, והדבר נכון הן למערכת הפעלה שרצה על המכונה הווירטואלית והן לתהליכי משתמש שרצים על מערכת ההפעלה האורחת.

מכיוון שתהליכי משתמש שרצים על מערכת ההפעלה "חושבים" שהם רצים במצב משתמש, ואכן זה המצב בפועל, אין כל קושי בהרצת תהליכי משתמש. לעומת זאת, מערכת ההפעלה האורחת שרצה על המכונה הווירטואלית "חושבת" שהיא רצה במצב מיוחס, אך למעשה היא רצה במצב משתמש. למצב כזה קוראים לפיכך ***מצב מיוחס וירטואלי***. נבאר כעת מה קורה להוראות מכונה שמערכת ההפעלה מבצעת אותן במצב מיוחס וירטואלי. ביצוע הוראות רגילות והוראות מיוחסות אינו גורר שום קושי. הוראות רגילות יתבצעו ישירות על החומרה, ואילו הוראות מיוחסות (כגון גישה ל-I/O) יגרמו ל-TRAP והפיקוד יעבור ל-hypervisor, אשר יבחן את ההוראה ויבצע הוראות נדרשות כחיקוי. נותר רק לבחון מה קורה כאשר ניתנות הוראות רגישות.

שאלה 2

האוניברסיטה הפתוחה THE OPEN UNIVERSITY OF ISRAEL الجامعة المفتوحة
Yair Kurtzman

File #0006182 belongs to Yair Kurtzman- do not distribute

מה קורה כאשר מערכת הפעלה שרצה על מכונה וירטואלית מבצעת הוראה רגישה?

כאשר מערכת הפעלה שרצה על מכונה וירטואלית מבצעת הוראה רגישה על מעבדים ללא VT, ההוראה תיכשל ויש סיכוי גבוה שמערכת ההפעלה תקרוס. על מעבדים עם VT, הוראה רגישה תגרום ל-TRAP וה-hypervisor יבחן את ההוראה. אם ההוראה בוצעה על ידי מערכת ההפעלה האורחת במכונה הווירטואלית, אזי היא תבוצע על ידי ה-hypervisor במצב מיוחס. לעומת זאת, אם ההוראה בוצעה על ידי תוכנית המשתמש שרצה על מערכת ההפעלה האורחת, ה-hypervisor יבצע חיקוי של מה שהחומרה האמיתית הייתה עושה אילו קיבלה לביצוע הוראה רגישה במצב משתמש.

שאלה 3

כיצד VMware מממשת וירטואליזציה על מעבדי x86 ללא VT?

נבאר כעת כיצד אפשר להשתמש בצורה חכמה בתרגום בינארי ובמצבי ההגנה (protection rings) של מעבדי x86 ללא VT כדי לממש את הווירטואליזציה ל-VMM מטיפוס ראשון.

מעבדי x86 תומכים בארבעה מצבי הגנה. במצב ההגנה השלישי רצים תהליכי משתמש רגילים. במצב הגנה 0 אפשר לבצע כל הוראה. הליבה של מערכת הפעלה מתבצעת במצב הגנה 0, ושני מצבי ההגנה הנותרים אינם בשימוש כל עוד לא מדובר על הרצת מערכת הפעלה במכונה וירטואלית. כפי שאפשר לראות באיור 4-7 בספר הלימוד, הפתרון של VMware לווירטואליזציה על x86 ללא VT היה להריץ את hypervisor במצב הגנה 0, את מערכת ההפעלה שרצה על המכונה הווירטואלית במצב הגנה 1 ואת תוכניות המשתמש במצב הגנה 3.

כתוצאה מכך, הגרעין של מערכת ההפעלה האורחת רץ במצב הגנה גבוה יחסית לתהליכי המשתמש, וכל ניסיון לגשת לזיכרון של הגרעין מתוכנית משתמש יגרום ל- memory violation. הוראות מיוחסות אשר מבוצעות על ידי גרעין מערכת ההפעלה גורמות ל-TRAP ב-hypervisor אשר בוחן את ההוראות ומבצע אותן על החומרה הווירטואלית. נותר רק לבאר מה קורה להוראות רגישות.

באשר להוראות רגישות המבוצעות על ידי גרעין מערכת ההפעלה האורחת, הפתרון הוא, בפשטות, שה-hypervisor יעלים אותן. כדי להעלים הוראות רגישות, ה-hypervisor מזהה בלוקים הנקראים ***בלוקים בסיסיים*** (***basic blocks***). הבלוקים הללו הם רצף של הוראות שאינו כולל הוראות רגישות או הוראות המשנות את ***זרימת התוכנית*** (***control flow***), מלבד הוראה אחת כזאת בסוף הבלוק. כלומר, בלוק בסיסי מכיל הוראות רגילות, ובסוף הבלוק קיימת לכל היותר הוראה אחת שהיא הוראה רגישה או הוראה המשנה את הזרימה (כגון jump, call וכדומה (3)). בטרם יבוצע בלוק בסיסי, ה-hypervisor מוודא שהבלוק איננו מכיל הוראות רגישות. היה ובלוק בסיסי מכיל הוראה רגישה, ה-hypervisor משכתב את הבלוק על ידי החלפת הוראה רגישה בקריאה לשגרת שירות של hypervisor, והפיקוד עובר ל-VMM. התערבותו של ה-hypervisor נדרשת גם כאשר ההוראה האחרונה של הבלוק הבסיסי היא הוראה המשנה את זרימת התוכנית, מכיוון

שהוראת בקרת הזרימה מתייחסת לקוד ישן ללא שינויים שהוכנסו על ידי ה-hypervisor בעת שכתוב ההוראות הרגישות.

לסיכום, נחזור שוב על אופן ההפעלה של התרגום הבינארי בצורתו הבסיסית: ה-hypervisor מבצע חלוקה של התוכנית לבלוקים בסיסיים. הבלוקים הבסיסיים מתבצעים ישירות על CPU. ההוראה האחרונה בבלוק הבסיסי מעבירה שליטה ל-hypervisor, שמחליט כיצד לחקות את הפעולה, ולאחר החיקוי בוחר בלוק בסיסי חדש אשר שוב מתבצע ישירות על "ברזלים".

השיטה שתיארנו זה עתה נקראת שיטת ***התרגום הבינארי*** או ***שיטת התרגום הדינאמי יחד עם חיקוי*** (***dynamic translation and emulation***). במבט ראשון נראה אולי שהשיטה של תרגום דינאמי עם חיקוי טובה רק במקצת משיטת החיקוי הנאיבית, אך למעשה רוב הבלוקים הבסיסיים מכילים הוראות רגילות ללא הוראות רגישות וללא הוראות מיוחסות, ולכן ניתן לבצע את ההוראות שבבלוקים הללו ישירות על החומרה ללא התערבות ה-hypervisor. נוסף על כך, לאחר התרגום הבינארי ניתן להשתמש במטמון (cache) לשמירה של הבלוקים המתורגמים, וכך להשתמש בהם בעתיד בלי צורך לשוב ולתרגם אותם.

נוסף על השימוש במטמון, ישנם סוגים נוספים של אופטימיזציות היכולות להביא לשיפור הביצועים:

- אם בלוק בסיסי מסתיים בקריאה ל-jump, אפשר להחליף את ההוראה האחרונה בבלוק בהוראה "לקפוץ" ישירות לבלוק מתורגם, במקום לקרוא ל-hypervisor שימצא לאיזה בלוק יש לקפוץ.
- ויתור על תרגום של הוראות רגישות או מיוחסות בתוכניות משתמש: אין צורך בכך, כי תוכניות משתמש רצות ממילא במצב משתמש (ring 3).
- לתרגם קוד של מערכת הפעלה אורחת ואף להחליף הוראות מיוחסות בהוראות של קוד מתורגם. בדרך זו, במקום לגרום ל-TRAP ל-hypervisor (פעולה יקרה מאוד), יתבצע ביצוע של הקוד המתורגם (פעולה זולה יותר).

ומה נוכל להגיד על יעילות במקרה של VMM מטיפוס שני? כיצד אפשר להבחין ב-hypervisor בין הקוד השייך למערכת הפעלה אורחת ובין הקוד של התהליכים הרצים במערכת הפעלה אורחת? אמנם, התפיסה ש-VMM מטיפוס ראשון מבוסס עליה שונה מזו ש-VMM מטיפוס שני מבוסס עליה, אך הטכניקות ששני סוגי ה-VMM משתמשים בהן אינן שונות באופן מהותי ורעיון התרגום הבינארי ממומש בשניהם. יחד עם זאת, יש להביא בחשבון שמנקודת מבטה של מערכת הפעלה אורחת, יש לה גישה לכל מרחב הזיכרון הווירטואלי,[19] וזה כמובן אינו המצב בפועל. נוסף על כך, כדי לבצע הוראות מתורגמות, VMM מטיפוס שני צריך גישה ישירה לחומרה, ול-hypervisor מטיפוס שני אין כמובן גישה כזו, משום שהוא רץ כתהליך משתמש. להלן רשימה חלקית של המקרים

[19] במערכת בעלת מיעון של 32 סיביות מתוך 4 GB של הזיכרון, רק 3 GB מותרות לגישה ממרחב המשתמש.

האוניברסיטה הפתוחה THE OPEN UNIVERSITY OF ISRAEL الجامعة المفتوحة
Yair Kurtzman

File #0006182 belongs to Yair Kurtzman- do not distribute

שבהם על ה-VMM להימצא במצב מיוחס לקבלת גישה מתאימה. כדי לאפשר ל-hypervisor מטיפוס 2 גישה מתאימה לביצוע הפעולות האלו בצורה נאמנה למקור, רוב ה-hypervisors משתמשים ברכיב שרץ כחלק של גרעין במערכת ההפעלה המארחת. כך הוראות של הרכיב מקבלות גישה לפי ההרשאות של ring 0.

מקרה ראשון – שינוי ה-segment descriptor

כידוע, segment descriptor משמש לקיבוע של מאפייני אזורי זיכרון במרחב זיכרון של תהליך. אזורי הזיכרון המכונים סגמנטים (segments) נחלקים לשני סוגים: סגמנטים למשתמש (user segments) וסגמנטים למערכת (system segments). סגמנטים למשתמש הם למעשה הסגמנטים של קוד ו-data. הסגמנטים של stack הם סוג מיוחד של סגמנטים של data. סגמנטים למערכת מתחלקים לשני סוגים עיקריים: סוג אחד מתואר על ידי system-segment descriptors וסוג שני על ידי gate descriptors. לא נרחיב כאן על הטקסונומטריה של סגמנטים מסוגים שונים. די לומר כי gate descriptors משמשים לקביעת הרשאות גישה עם רמות שונות של ייחוס (ring 0, …, ring 3) ולכן, כדי לגרום לכך שקוד של מערכת הפעלה אורחת שרצה על VMM מסוג 2 תוכל לשנות את תוכן ה-segment descriptors, ה-VMM צריכה לטפל ב-gate descriptors. קוראים סקרנים מוזמנים לקרוא על סוגי הסגמנטים השונים ב-(4).

מקרה שני – טיפול בפסיקות

אחד התפקידים של VMM הוא טיפול בפסיקות. לדוגמה, כאשר דיסק שולח פסיקה, הפסיקה צריכה לעבור בצורה כלשהי אל מערכת הפעלה אורחת אשר גרמה לקריאה מהדיסק. ב-VMM מסוג שני, פנייה לדיסק מתורגמת על ידי ה-VMM לפנייה לקובץ בפורמט מיוחד המחקה דיסק, ופנייה לקובץ זה גורמת לפנייה לדיסק פיזי, השולח פסיקה למעבד של המחשב האורח. ה-VMM מסוג 2 רץ כתהליך, ומי שמקבל את הפסיקה הוא מערכת ההפעלה המארחת, שצריכה להעביר את הפסיקה ל-VMM. העברת הפסיקות מתבצעת בדרך כלל על ידי מודול של גרעין מערכת ההפעלה המארחת, אשר אמור לפעול במצב מיוחס (ring 0). במילים אחרות, VMM מסוג שני אמנם רץ בתור תהליך משתמש, אך לרוב יש לו חלק אשר רץ כחלק ממערכת ההפעלה המארחת במצב מיוחס.

מקרה שלישי – טיפול ב-TRAP

כיצד מעבירים שליטה ל-VMM ומטפלים ב-TRAP אשר נגרם על ידי תוכנה? אם ה-VMM מטיפוס שני משתמש בטכניקת trap and emulate לביצוע ההוראות המיוחסות, אזי בדומה למקרה של פסיקות, ה-VMM זקוק לתמיכה באמצעות רכיב (module) בגרעין של מערכת ההפעלה המארחת. רכיב זה "מודיע" ל-VMM על קרות ה-TRAP בעקבות ביצוע של הוראות המכונה המיוחסות על ה-VMM.

מקרה רביעי – world switch

תיארנו למעלה דוגמה של טיפול בפסיקות כאשר פסיקת התקן צריכה להיות מועברת ל-VMM. כעת נעיין ביתר תשומת לב במצב המערכת ממש לפני קבלת הפסיקה. ייתכנו 4 מצבים:

א. הקוד שרץ על CPU הוא של VMM מסוג 2 או של מערכת ההפעלה האורחת או של תהליך משתמש אשר רץ על מערכת ההפעלה האורחת, וגם הפסיקה שהתקבלה מתייחסת למערכת הפעלה אורחת.

ב. הקוד שרץ על CPU הוא של VMM מסוג 2 או של מערכת ההפעלה האורחת או של תהליך משתמש אשר רץ על מערכת ההפעלה האורחת, ואילו הפסיקה שהתקבלה מתייחסת למערכת הפעלה מארחת.

ג. הקוד שרץ על CPU הוא של מערכת ההפעלה המארחת או של תהליך משתמש אשר רץ על מערכת ההפעלה המארחת, ואילו הפסיקה שהתקבלה מתייחסת למערכת הפעלה אורחת.

ד. הקוד שרץ על CPU הוא של מערכת ההפעלה המארחת או של תהליך משתמש אשר רץ על מערכת ההפעלה המארחת, וגם הפסיקה שהתקבלה מתייחסת למערכת הפעלה מארחת.

במצבים ג ו-ד מופעל טיפול רגיל בפסיקות. במצב ב מופעל מנגנון הטיפול בפסיקות שתיארנו למעלה (מקרה שני – טיפול בפסיקות). מצב א הוא המסובך מבין כל ארבעת המצבים. אם פסיקת החומרה מתרחשת בעוד ה-CPU מריץ קוד של ה-VMM או של מערכת ההפעלה האורחת, יש לבצע שתי פעולות:

א. לשמור את המצב הנוכחי של המערכת הווירטואלית.

ב. לבצע טעינה של מצב שבו מתאפשרת קבלת פסיקות על ידי מערכת ההפעלה המארחת.

שתי הפעולות הללו נקראות ***החלפת עולם*** ***(world switch)***.

שאלה 4

האם יש צורך לבצע החלפת עולם כאשר פסיקת חומרה המיועדת למערכת ההפעלה המארחת מתרחשת בעת ביצוע של תהליך משתמש אשר רץ על מערכת הפעלה אורחת?

חומר הלימוד שבסעיף זה מתמצת את הנאמר בספר הלימוד ומוסיף עליו, אך אינו מחליף אותו. עליכם לקרוא את הסעיף המקביל בספר הלימוד.

האוניברסיטה הפתוחה
THE OPEN UNIVERSITY OF ISRAEL
الجامعة المفتوحة
Yair
Kurtzman

File #0006182 belongs to Yair Kurtzman- do not distribute

7.5 מיקרו-קרנל של hypervisor

שני סוגי ה-hypervisor (סוג 1 ו-2) מסוגלים להריץ מערכות הפעלה אורחות ללא שינוי. יכולת זו נקראת ***וירטואליזציה מלאה (full virtualization)***. מימוש מטרה זו תוך השגת ביצועים טובים כרוך במורכבות רבה. גישה שונה מווירטואליזציה מלאה היא זו שמתירה ומחייבת שינוי של מערכת ההפעלה. באחד הסעיפים הקודמים למדנו ששיטה זו נקראת פרא-וירטואליזציה. בשיטת הפרא-וירטואליזציה מוחלפות כל הקריאות להוראות רגישות במערכת הפעלה אורחת בקריאות API של ה-hypervisor, המכונות hypercalls. למעשה, הסרה של ההוראות הרגישות והחלפתן ב-hypercalls הופכת את ה-hypervisor הפועל בשיטה זו למיקרו-קרנל, כפי שראינו באיור 1-26 בספר הלימוד. לפיכך עלתה במאמרים שונים הטענה, שלמעשה hypervisor אינו אלא מיקרו-קרנל בנוי היטב. טענה זו היא שנויה במחלוקת, ואחד הטיעונים המרכזיים כנגדה הוא ש-hypervisors אינם בטיחותיים מכמה בחינות, שבהן דן Hohmuch במאמרו (12). מאחר שבסופו של דבר אין הסכמה גורפת לגבי מה נכון ומהי ה"הלכה" לגבי hypervisors, סעיף זה דן במציאת קווי דמיון והבדלים בין hypervisor לבין מיקרו-קרנל וכן בנסיבות שהביאו לקיומן של שתי הגישות.

עליכם לקרוא סעיף זה בספר הלימוד, אך עיקרי הדברים הם:

- נימוק אחד לקיומם של hypervisors התומכים בווירטואליזציה מלאה הוא שלא כל מערכת הפעלה מספקת קוד פתוח. אם בעתיד תוצע סטנדרטיזציה של פניות למיקרו-גרעין ו/או ל-hypervisors, ומערכות הפעלה ישתמשו בפניות סטנדרטיות, יקטן הצורך ב-hypervisors התומכים בווירטואליזציה מלאה.

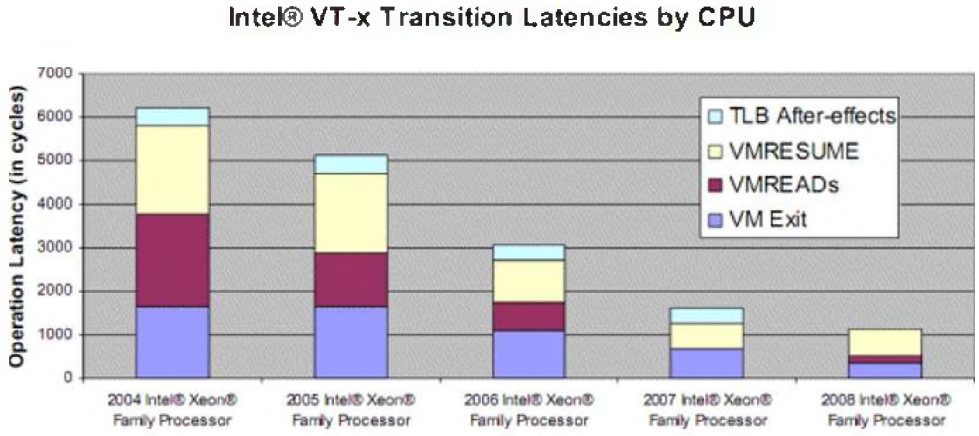
- עיינו באיור 7-5 בספר הלימוד. ההבדל בין hypervisor מסוג 1 לבין מיקרו-קרנל הוא מטושטש למדי. המשותף ביניהם הוא ששניהם מטפלים בהוראות רגישות. אמנם, התפקיד הפונקציונלי של השניים שונה וצורת הטיפול בהוראות רגישות שונה (בשיטה של פרא-וירטואליזציה הרי מסלקים הוראות רגישות עוד בשלב הטרום-ריצה, כהכנה של מערכת הפעלה), אך הן hypervisor והן מיקרו-קרנל מכילים עשרות אלפים שורות קוד בלבד (לעומת מיליוני שורות קוד במערכת הפעלה).

בשנת 2005, קבוצת חוקרים בראשותו של Amsden, שעבדו בחברת Vmware, הציעו את VMI (Virtual Machine Interface) – שגרות שקרנל של מערכת הפעלה קורא להן כאשר צריך לבצע הוראה רגישה (10). הוצע אף שמערכת ההפעלה Linux תעבור שינוי ותשתמש בקבוצת השגרות VMI. הסתכלו באיור 7-6 כדי להבין כיצד VMIL הייתה מסוגלת לרוץ על חומרה אמיתית, VMM של חברת VMware או VMM של XEN.

שינוי דומה ל-VMIL ברמה העקרונית, אם כי שונה ברמת המימוש, נכנס לגרעין של Linux החל מגרסה 2.6.33 וסיפק תמיכה בפרא-וירטואליזציה. בדרך זו, code base של מערכת ההפעלה Linux היה יכול לרוץ גם על חומרה אמיתית וגם על VMM במצב פרא-

וירטואליזציה.[20] השינוי נתמך על ידי חברות בעלות עניין הכוללות את IMB, RedHat, Xen ו-Vmware. קבוצת הפעולות בלינוקס נקראה **paravirt ops**, ואפשר לקנפג את הגרעין לתמוך בפרא-וירטואליזציה על ידי שינוי של פרמטרים בעלי התחילית CONFIG_PARAVIRT.

שיטת VMI אינה נתמכת עוד: בשנת 2009 הכריזה VMWare שכל מוצרי החברה יפסיקו לתמוך ב-VMI החל משנת 2011.[21] הסיבה לכך הייתה הופעתם בשוק של מעבדים בעלי יכולת משופרת של וירטואליזציה. אם נסתכל (איור 10) על ההתפתחות הטכנולוגית של טכנולוגיית ה-VT-x של Intel, נראה תמונה של שיפור מתמיד שנה אחרי שנה, ותופעה דומה אפיינה גם את טכנולוגיית ה-SVM של AMD המתחרה.



איור 10

WMware ראתה בשילוב של שיטת הפרא-וירטואליזציה, ובפרט ב-VMI, אופטימיזציה שבאה להשיג ביצועים נאותים. כאשר המעבדים החדישים הראו ביצועים המשתווים לאלה שפרא-וירטואליזציה עם VMI הציעה, הם הסירו את ה-VMI באופן הדרגתי.

לסיכום דיוננו בשיטת הפרא-וירטואליזציה, נמנה את יתרונותיה ואת חסרונותיה של השיטה. יתרון בולט של שיטת הפרא-וירטואליזציה הוא שה-VMM אינו נדרש לממש סמנטיקה מסובכת למדי של הוראות רגישות. יעיל יותר לקרוא להיפר-קריאה ישירות מתוך מערכת ההפעלה, ואכן, מספר לא קטן של מערכות מסחריות משתמשות בטכניקות של פרא-וירטואליזציה.[22] יחד עם זאת, עד היום לא נוצרה אקו-מערכת (ecosystem) שתהפוך את ה-API של פרא-וירטואליזציה לסטנדרט. יש לכך כמה סיבות, ובהן:

- ישנן מערכות הפעלה בעלות קוד סגור, כמו מערכות Windows

---

[20] מאמר 1003644 של VMware Knowledge Base ממחיש את הצעדים שהיה צורך לבצע כדי להריץ לינוקס עם גרעין בגרסת 2.6.22 על ESX 3.5, שהוא ה-VMM מטיפוס 1.

[21] תודעה בבלוג של חברת VMware: Support for guest OS paravirtualization using VMware VMI to be retired from new products in 2010-2011.

[22] דוגמה נוספת היא ה-VMWare Tools של VM Player. התקנת VMWare Tools משנה את הגרעין של מערכת ההפעלה האורחת. בעקבות השינוי, מקצת הפעולות במערכת האורחת מתורגמות לפרא-קריאות של VM Player, ושאר הפעולות עוברות במסלול הרגיל של חיקוי הוראות.

האוניברסיטה הפתוחה THE OPEN UNIVERSITY OF ISRAEL الجامعة المفتوحة
Yair Kurtzman

File #0006182 belongs to Yair Kurtzman- do not distribute

- ישנן מערכות הפעלה רבות ומגוונות בעלות קוד פתוח, כדוגמת מערכות Linux שונות
- קהילת ה-IT השמרנית אינה נחפזת להחליף את המערכות המסחריות המוטמעות הפועלות בשיטת חיקוי הוראות, לצד טכניקות פרא-וירטואליזציה במקומות קריטיים להשגת ביצועים נאותים
- רצונן של החברות הגדולות למצות את הרווחים מניצול ה- IP (intellectual property) בתחום הווירטואליזציה

חומר הלימוד שבסעיף זה מתמצת את הנאמר בספר הלימוד ומוסיף עליו, אך אינו מחליף אותו. עליכם לקרוא את הסעיף המקביל בספר הלימוד.

7.6 וירטואליזציה של זיכרון (חומר רשות)

עד כה עסקנו בהיבטים של וירטואליזציה הקשורים למעבד. כעת נתבונן בבעיות הדורשות פתרון מצד ניהול הזיכרון. לשם הפשטות, נניח שמדובר על hypervisor מסוג 1 ומעליו 2 מכונות וירטואליות. מערכת ההפעלה של אחת מהן ממפה דפים חיוניים כלשהם למסגרות פיזיות הנושאות את המספרים X, Y, Z. גם מערכת ההפעלה של המכונה הווירטואלית השנייה ממפה דפים כלשהם לאותן מסגרות פיזיות בעלות אותם המספרים X, Y, Z. ברור אפוא שעל ה-hypervisor לספק מיפוי שיקשר בין המסגרות של החומרה הווירטואלית של המכונות הרצות עליו לבין מסגרות פיזיות של חומרה אמיתית. טבלה שמופיע בה מיפוי כזה נקראת ***shadow page table***.

בעיה נוספת נובעת מהצורה שבה מערכת הפעלה אורחת מעדכנת מיפוי של דפים למסגרות. מבחינת מערכת ההפעלה האורחת, שינוי של מיפוי בטבלת דפים אינו אלא כתיבה לזיכרון פיזי. כיצד ידע ה-hypervisor שעליו לעדכן את ה-shadow page table? הרי כתיבה לזיכרון אינה פעולה רגישה. בספר הלימוד מתוארות 2 טכניקות להתמודדות עם הבעיה.

טכניקה אחת היא לשמור את המיקום של טבלת דפים וירטואלית ברמה העליונה בפעם הראשונה שמנסים לטעון אותה. מאחר שזוהי הוראה רגישה, אפשר לזהות אותה ב-hypervisor. לאחר מכן ה-hypervisor יכול למפות את טבלאות הדפים שטבלת הדפים שברמה העליונה מצביעה אליהן לדפים שהם read only. כך כל ניסיון לערוך שינוי בטבלאות דפים יגרום לשינוי שאפשר להבחין בו ב-hypervisor. באשר לזיהוי של הסרת המיפוי, מאחר שההסרה מלווה בקריאה להוראת INVLPG – הוראה רגישה שנועדה לפנות כניסה ב-TLB – הרי שאפשר לזהות מצב זה ב-hypervisor.

הטכניקה השנייה נוקטת גישה אחרת, המאפשרת למכונה מארחת להוסיף מיפויים חדשים. שינוי המיפויים הללו אינו גורם לשינוי ב-shadow page table, מכיוון ש-hypervisor אינו מודע כלל להתרחשות השינוי. כאשר מתרחשת גישה לפי המיפוי

החדש, השליטה חוזרת ל-hypervisor והוא בוחן את טבלת הדפים של המכונה ומעדכן את המיפוי ב-shadow page tables.

אפשר לראות שהטכניקות הללו גורמות למספר לא מבוטל של פסיקות דף, ולכן הן יקרות מבחינת הביצועים. שימו לב: אם בעבר דיברנו על פסיקת דף ברמת מערכת ההפעלה, כעת נצטרך להבחין בין פסיקת דף הנגרמת על ידי תוכנה הרצה על מערכת הפעלה אורחת (***guest induced page faults***) לבין פסיקות דף הנגרמות מן הצורך לעדכן את ה-shadow page tables (***hypervisor induced page faults***).

מצב של העברת השליטה מן המכונה הווירטואלית ל-hypervisor נקרא ***VM exit***. זוהי פעולה מורכבת ויקרה המצריכה עדכון של הסיבה להעברת השליטה, שמירה של כל האוגרים (registers), טעינה של אוגרים של hypervisor, ביצוע הטיפול של ה-hypervisor בפסיקת דף ולבסוף שחזור לאחר ביצוע הפעולה שגרמה לפסיקה. קל לראות שמחיר הטיפול גבוה יותר ממחיר הטיפול בפסיקת דף, כפי שלמדנו בפרק 3, "ניהול זיכרון".

במקרה של פרא-וירטואליזציה, המצב שונה. כאן מערכת ההפעלה שרצה במכונה הווירטואלית "יודעת" שהתרחש שינוי בטבלת הדפים, ומבצעת פרא-קריאה. כך ה-hypervisor מסיים את הטיפול בפסיקת הדף במהלך אותו ה-hypercall – והדבר משפר מאוד את הביצועים.

תמיכה ב-nested page tables

ראינו עד כה שטיפול בפסיקות דף מצריך תחזוקה של רמת מיפוי נוספת. אם עד כה מיפינו בין כתובת וירטואלית לכתובת פיזית (פרק 3, "ניהול זיכרון"), הרי שכעת אנו נדרשים למפות בין **כתובת וירטואלית במכונה וירטואלית** ל**כתובת פיזית של המכונה הווירטואלית** ובינה לבין ה**כתובת הפיזית של המכונה המארחת**. ללא תמיכה של החומרה, המיפוי האחרון התבצע באמצעות ה-shadow page tables והיה יקר. כיום הן מעבדי AMD והן מעבדי x86 מציעים תמיכה כזאת. היא נקראת Extended Page Tables במעבדי Intel או Nested Page Tables במעבדי AMD. הרעיון שביסודה הוא ש-CPU תומך בתרגום בשתי הרמות הנדרשות, ללא התערבות של תוכנה. עיינו באיור 7-7 בספר הלימוד כדי לראות כיצד מתבצע מיפוי בעזרת NPT/EPT.

שחרור זיכרון

תכונה שימושית מאוד בענן מחשוב המריץ עשרות מכונות וירטואליות על מחשבי ענן היא ***התחייבות-יתר בזיכרון*** (***memory overcommitment***). קיים סיכוי שמכונות וירטואליות לא ישתמשו בכמות המקסימלית של הזיכרון ש-hypervisor התחייב לספק. סיבה אחת היא שהשימוש בזיכרון משתנה במהלך חייה של מכונה וירטואלית. סיבה נוספת היא קיומה של טכניקת מניעת כפילויות (deduplication), שנדון בה בהמשך הפרק, המאפשרת לשתף דפים בעלי תוכן זהה. הבעיה המתעוררת במקרה של התחייבות-יתר היא, כיצד hypervisor יודע אילו מסגרות שדפים של מכונות וירטואליות ממופים אליהן אפשר לפנות כדי להעמיד אותן לרשות מכונות אחרות, שנדרש להן זיכרון רב יותר.

האוניברסיטה הפתוחה THE OPEN UNIVERSITY OF ISRAEL الجامعة المفتوحة
Yair Kurtzman

File #0006182 belongs to Yair Kurtzman- do not distribute

קראו את הפרק ועמדו על ההבדל בין הגישה הנאיבית, שבה הדבר נעשה באמצעות רמת מיפוי נוספת של דפים, לבין שיטת ה***התנפחות*** ***(ballooning)***.

חומר הלימוד שבסעיף זה מתמצת את הנאמר בספר הלימוד ומוסיף עליו, אך אינו מחליף אותו. עליכם לקרוא את הסעיף המקביל בספר הלימוד.

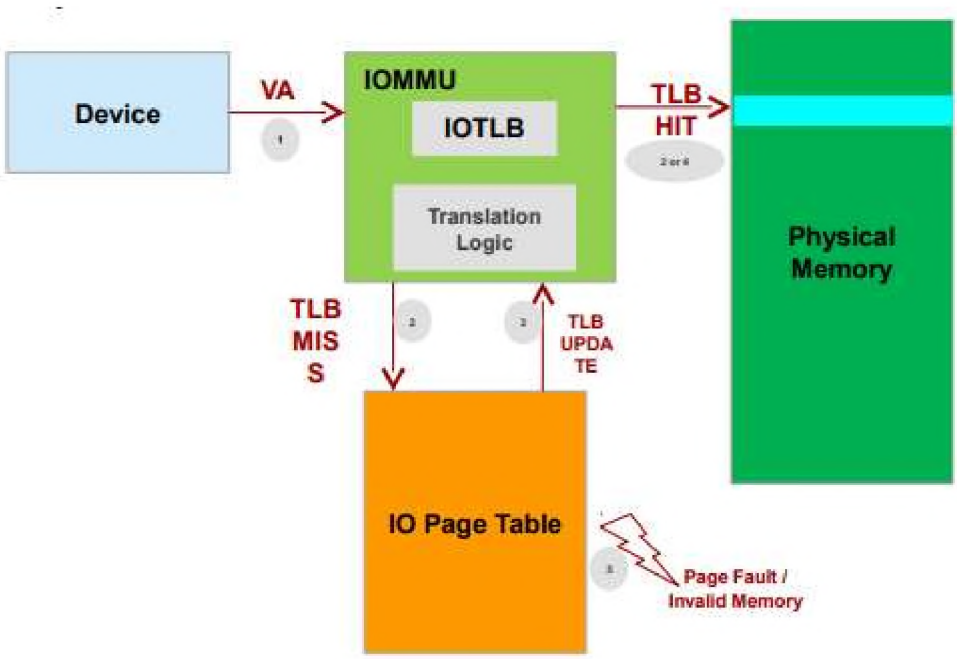
7.7 וירטואליזציה של I/O (חומר רשות)

סעיף זה בספר פותח בהסבר כיצד hypervisor יכול לספק למערכת הפעלה אורחת אשליה של עבודה מול סוג מסוים של התקן קלט/פלט, כאשר בפועל הנתונים מאוחסנים בהתקן קלט/פלט מסוג אחר.

I/O MMU

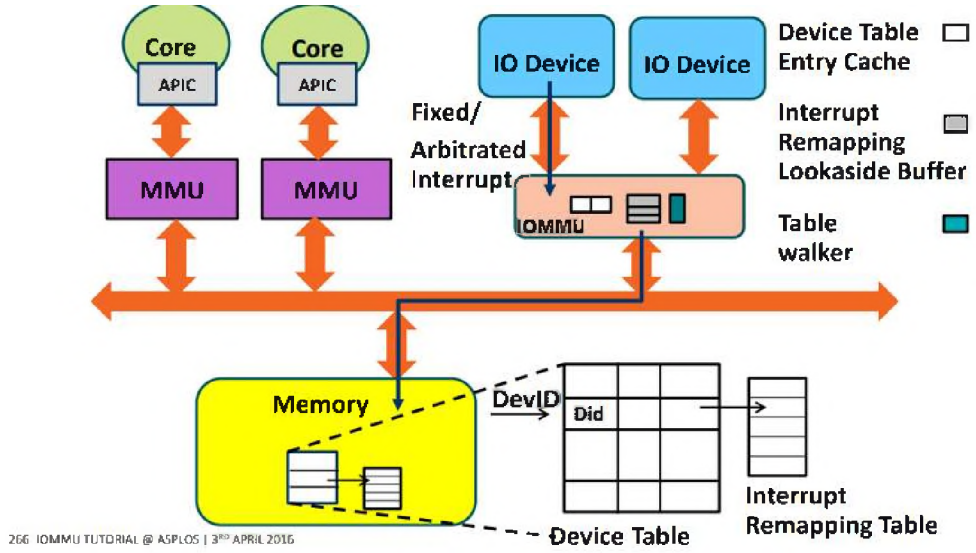
ההמרה המתוארת בתחילת הסעיף, אם היא מבוצעת כולה בתוכנה, תשפיע על ביצועי הקלט/פלט. נתבונן לדוגמה כיצד hypervisor יבצע DMA בצורה תכנותית. התקן קלט/פלט אמור להביא נתונים ישירות לזיכרון הראשי. הזיכרון שאליו ייכתבו הנתונים אמור להיות מוגן מפני כתיבות מהתקני קלט/פלט אחרים, להיות ממופה בטבלאות דפים של hypervisor וגם להיות ממופה למרחב זיכרון של תהליך במכונה וירטואלית. למעשה מדובר כאן על שתי סוגיות. סוגיה אחת היא ***מעבר נתונים ממכונה וירטואלית ישירות להתקן*** ***(device passthrough)*** והסוגיה השנייה היא ***מידור התקן קלט/פלט*** ***(device isolation)*** כדי לאפשר גישה מהתקן קלט/פלט לזיכרון של מכונה וירטואלית נתונה בלבד. I/O MMU נועד לפתור את הבעיות הללו ביעילות.

נתבונן באיור שלהלן כדי להבין כיצד IO-MMU פותר את הבעיות שלעיל. רכיב זה מתרגם כתובות של התקן קלט/פלט לכתובות של הזיכרון הראשי. טבלת מיפוי בין התקן קלט/פלט לזיכרון מוחזרת על ידי hypervisor, וכבר למדנו בסעיף "וירטואליזציה של זיכרון" כיצד ה-hypervisor מנהל את הזיכרון הראשי בעבור מכונות וירטואליות.



איור 11

אך נוסף על מעבר נתונים ישיר ומידור התקן הקלט/פלט, נדרשת גם וירטואליזציה של פסיקות – פסיקת התקן הקלט/פלט אמורה להגיע למכונה הווירטואלית הנכונה. נבאר כיצד IO-MMU תומך בהוספת ***רמת מיפוי פסיקות חדשה להתקני קלט/פלט*** (***interrupt remapping***).



איור 12

האיור דלעיל מראה כי hypervisor מחזיק בזיכרון טבלת התקני קלט/פלט. לכל התקן קלט/פלט מוחזקת טבלת מיפוי פסיקות. רכיב IO-MMU מסוגל לטעון חלקים מטבלת התקני הקלט/פלט, וטבלאות מיפוי פסיקות ל-device table entry cache ול- interrupt

האוניברסיטה הפתוחה THE OPEN UNIVERSITY OF ISRAEL الجامعة المفتوحة
Yair Kurtzman

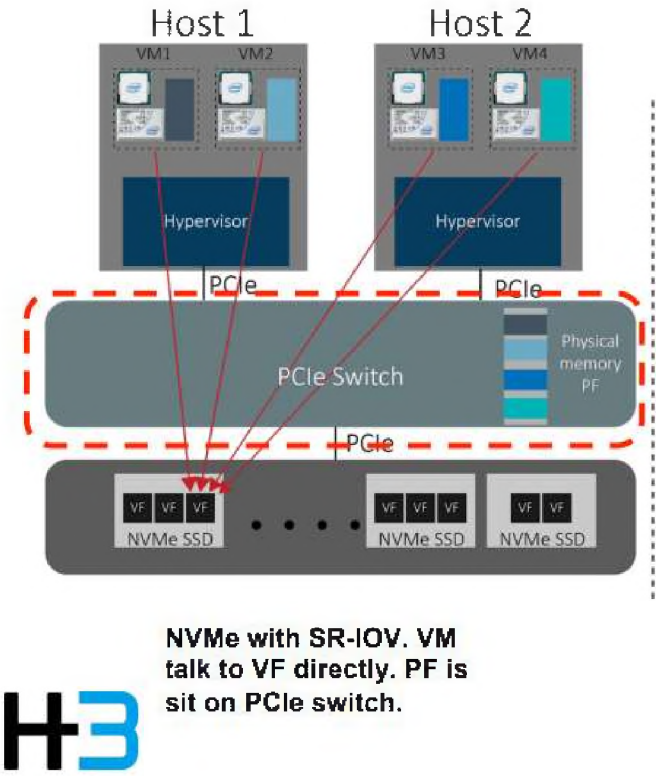
File #0006182 belongs to Yair Kurtzman- do not distribute

remapping lookaside buffer בהתאם. כאשר אחד מהתקני הקלט/פלט מעלה פסיקה, IO-MMU יודע להעביר אותה למכונה וירטואלית מתאימה (לפי הרישום המופיע בטבלת התקני הקלט/פלט).

SR-IOV

שימוש ב-IO-MMU מאפשר הקצאת התקני קלט/פלט למכונות וירטואליות ומוריד נתח גדול של עבודה מה-hypervisor. יחד עם זאת, ה-hypervisor עדיין חייב להיות מעורב בתהליכי הקלט/פלט, ונוסף על כך קיים קשר חד-חד-ערכי בין המכונה הווירטואלית להתקן הקלט/פלט. היה זה טוב, כמובן, אילו התקני הקלט/פלט עצמם היו תומכים בווירטואליזציה. התקן של PCIe מכסה אפשרות כזאת, והתקנים התומכים ב- single root I/O virtualization מציגים את עצמם ככמה התקנים פיזיים שמכונה וירטואלית יכולה לעבוד איתם ישירות ללא התערבותו של hypervisor.

לא ניכנס במסגרת מצומצמת זו לכל הקשור ל-PCIe ול-SR-IOV. נסתפק בסיכום הפרק בעזרת איור 13, הממחיש שמכונה וירטואלית רואה חלק מהדיסק NVMe SSD התומך ב-single root IO virtualization כדיסק ייעודי למכונה.



NVMe with SR-IOV. VM talk to VF directly. PF is sit on PCIe switch.

איור 13

אם עודכם סקרנים וברצונכם להרחיב את ידיעותיכם, אתם מוזמנים לעיין באתר של PCI SIG, הדורש רישום כחברים, או במאמר 11.

חומר הלימוד שבסעיף זה מתמצת את הנאמר בספר הלימוד ומוסיף עליו, אך אינו מחליף אותו. עליכם לקרוא את הסעיף המקביל בספר הלימוד.

7.8 מכונות וירטואליות על מחשבים מרובי מעבדים (חומר רשות)

מכונות וירטואליות מקבלות את מספר הליבות כפרמטר אשר מציינים אותו לפני הרצת המכונה. לדוגמה, שרת עם 4 מעבדי Intel Xeon Platinum 9200 שבכולן יחד 224 ליבות מסוגל להריץ בו-זמנית 54 מכונות וירטואליות בעלות 4 ליבות כל אחת. מפעיל של ענן פרטי (ראו סעיף 7.9) מסוגל ליצור כמה סוגים של מכונות וירטואליות (ראו איור 14). לדוגמה, בענן OpenStack המבוסס על קוד פתוח, יצירת סוג חדש של מכונה מתבצעת כך:

```
# openstack flavor create --ram 512 --disk 1 --vcpus 1 m1.tiny
```

איור 14

אפשר לראות את רשימת כל סוגי המכונות שנוצרו כמתואר באיור 15:

```
# openstack flavor list
+-----+-----------+-------+------+-----------+-------+-----------+
| ID | Name      |   RAM | Disk | Ephemeral | VCPUs | Is_Public |
+-----+-----------+-------+------+-----------+-------+-----------+
| 1   | m1.tiny   |   512 |    1 |         0 |     1 | True      |
| 2   | m1.small  |  2048 |   20 |         0 |     1 | True      |
| 3   | m1.medium |  4096 |   40 |         0 |     2 | True      |
| 4   | m1.large  |  8192 |   80 |         0 |     4 | True      |
| 5   | m1.xlarge | 16384 |  160 |         0 |     8 | True      |
+-----+-----------+-------+------+-----------+-------+-----------+
```

איור 15

ואילו משתמשי הענן יכולים להעלות מכונה וירטואלית מן הסוג הרצוי באופנים שונים, כשבסופו של דבר כל אחד מן האופנים מתורגם לפקודת CLI הקוראת ל-nova. nova היא קומפוננטה של OpenStack שבאמצעותה מנהלים מחזור חיים של מכונות וירטואליות. לדוגמה:

```
# nova boot --flavor m1.small --config-drive true --image myLinuxVMImage
instance-01-myLinux
```

האוניברסיטה הפתוחה THE OPEN UNIVERSITY OF ISRAEL الجامعة المفتوحة
Yair Kurtzman

File #0006182 belongs to Yair Kurtzman- do not distribute

מכונות וירטואליות המריצות אותה מערכת הפעלה יכולות לשתף זיכרון פיזי של מחשב מארח. הדבר מתאפשר באמצעות טכניקה ***למניעת כפילויות*** (***deduplication***). הרעיון שמאחורי מניעת הכפילויות הוא להחזיק hashing של דפי זיכרון ולבדוק דפים בעלי אותו ערך hash. אם גם תוכן הדפים זהה, מפנים לאותו תוכן משני דפים. מובן שהטכניקה של מניעת כפילויות מצריכה גם את הטכניקה של copy on write כדי להתמודד עם שינויים אפשריים. במקרה של hypervisor מטיפוס ראשון, מניעת כפילויות ממומשת ב-hypervisor עצמו. במקרה של hypervisor מטיפוס שני, אפשר להשתמש במנגנון מניעת כפילויות המסופק על ידי מערכת ההפעלה (אם במערכת ההפעלה יש מנגנון כזה). לדוגמה, החל מגרסת קרנל 2.6.23 קיים ב-Linux מנגנון המכונה KSM (Kernel Samepage Merging) אשר סורק אזורי זיכרון שהוגדרו למנגנון ומפנה דפים בעלי תוכן זהה למסגרת אחת מוגנת מפני כתיבות, המשתכפלת אם משנים את אחד הדפים הפונים אליה (9).

חומר הלימוד שבסעיף זה מתמצת את הנאמר בספר הלימוד ומוסיף עליו, אך אינו מחליף אותו. עליכם לקרוא את הסעיף המקביל בספר הלימוד.

7.9 ענן

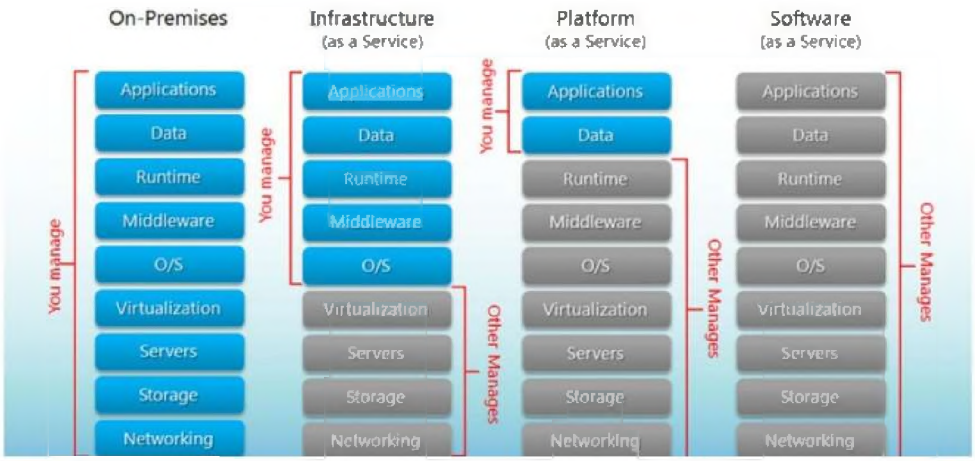
סעיף זה בספר הלימוד הוא בהיר למדי. אך כדי לשרטט תמונה כללית רחבה יותר, נוסיף כאן מעט פרטים בנושא של שירותי ענן. אפשר להבחין בין שלושה סוגים שונים של עננים:

- ענן ציבורי: ספק השירות מספק את שירותי הענן דרך האינטרנט. חישוב ואחסון מתבצעים במרכז מחשוב של ספק השירות. משאבי מרכז החישוב משותפים לכלל המשתמשים בשירותי הענן.
- ענן פרטי: שירותי חישוב ואחסון ניתנים במרכז מחשוב פרטי. משתמשי השירותים שייכים לארגון המספק את שירותי הענן.
- ענן היברידי: ענן פרטי המספק את דרישות החישוב והאחסון של משתמשיו, ומסוגל לספק שירותים מענן ציבורי בזמן עומס.

כל אחד מסוגי הענן האלה יכול לספק במידה זו או אחרת את השירותים שלהלן:

- Infrastructure As A Service – IAAS
- Platform As A Service – PAAS
- Software As A Service – SAAS

האיור שלהלן מבאר אילו שירותים ניתנים על ידי הענן בכל אחד מן הסוגים שלעיל:



איור 16

להלן נמנה, ללא פירוט, עוד כמה סוגים של שירותים שאפשר לספק דרך הענן:

- Analytics As A Serivce – AaaS
- API As A Service – APIaaS
- Backend As A Service – BaaS
- Business Process As A Service – BPAAS
- Desktop As A Service – DAAS
- Data As A Serice – DAAS
- DB As A Service – DBaaS
- Function As A Service – FAAS
- Monitoring as a Service – MaaS
- SECurity As A Service – SECaaS

חומר הלימוד שבסעיף זה מתמצת את הנאמר בספר הלימוד ומוסיף עליו, אך אינו מחליף אותו. עליכם לקרוא את הסעיף המקביל בספר הלימוד.

7.10 וירטואליזציה ברמת מערכת ההפעלה

עד כה למדנו על וירטואליזציה מבוססת hypervisor, אבל בתחילת הפרק הזכרנו גם וירטואליזציה ברמת מערכת ההפעלה. במקום להציג אשליה של מכונה, הרעיון הוא ליצור סביבת מרחב משתמש מבודדת, המכונה גם **מכולה (container)** או **כלא (jail)**.
כל מכולה וכל התהליכים בה מבודדים משאר המערכת. למשל, לכולם יכולה להיות מערכת קבצים משלהם "name space" (מרחב שמות) – תת-עץ שמתחיל משורש מערכת הקבצים.
לתהליך בתוך ה-name space אין אפשרות לגשת למרחבי שמות (תת-עצים) אחרים. עם זאת, זה כשלעצמו לא מספיק לבידוד תקין. קונטיינרים זקוקים גם למרחבי שמות

האוניברסיטה הפתוחה THE OPEN UNIVERSITY OF ISRAEL الجامعة المفتوحة
Yair Kurtzman

File #0006182 belongs to Yair Kurtzman- do not distribute

נפרדים עבור מזהי תהליך, מזהי משתמש, ממשקי רשת (וכתובות ה-IP המשויכות להם), הגבלות על שימוש במעבד, בזיכרון וכו'.

באופן כללי, אנו זקוקים לדרך מעקב אחר שימוש במגוון רחב של משאבים עבור קבוצות של תהליכים לאורך זמן. למערכות הפעלה שונות פתרונות שונים לסוגיה זו. אחד המוכרים הוא תכונת קבוצות הבקרה cgroups במערכת Linux. מנגנון cgroups גמיש מאחר שהוא אינו רושם מראש את המשאבים המדויקים, אלא מאפשר הצמדת בקר משאבים וירטואלי המכונה לעתים "subsystem" ל-cgroup עבור משאב מסוים, והוא מנטר ו/או מגביל את הגישה למשאבים עבור כל התהליכים החברים באותו cgroup.

ניתן להגביל את השימוש בקבוצה אחת של משאבים, כגון זיכרון, מעבד ורוחב הפס קלט/פלט עבור תהליך P_1, ובקבוצה אחרת לדוגמה להגביל רק רוחב פס קלט/פלט לתהליך P_2, ולאחר מכן לחבר את בקר המעבד והזיכרון ל-cgroup הראשון, ואת בקר ה-I/O לשני.

חשוב להבין שבקרת משאבים אפשרית לעתים קרובות ברמות שונות של גרעיניות. לדוגמה, בפירוט עדין, ניתן לחלק את זמן ה-CPU על ליבה בין תהליכים באופן דינמי באמצעות תזמון (פרק 2). בגרעיניות גסה הרבה יותר, ניתן פשוט לחלק את הליבות של המעבד ולהגביל את התהליכים בקבוצה, למשל ל-4 מתוך 16 ליבות. כך, בכל מצב תהליכים בקבוצה לא ירוצו על 12 ליבות נוספות. בפועל, חלוקת הליבה היא פופולרית מאוד.

המנגנון מאפשר להקצות cgroup לקבוצת ליבות מעבד ולחוליות (nodes) זיכרון, למשל ב-NUMA.
באמצעות cgroups, מרחבי שמות, מנגנון וירטואליזציה ברמת מערכת ההפעלה מאפשר יצירת קונטיינרים מבודדים מבלי להזדקק ל-hypervisors או לווירטואליזציה של חומרה. קונטיינרים כאלה קיימים כבר שנים רבות, אבל הם נעשו נפוצים הרבה יותר לאחר הצגתן של פלטפורמות נוחות, כגון Docker, Kubernetes ו-Microsoft Azure Container Registry, שעוזרות למנהלי מערכת לבנות, לפרוס ולנהל קונטיינרים.

לעומת מכונות וירטואליות מבוססות hypervisor, קונטיינרים הם בדרך כלל קלים יותר: מהירים יותר להפעלה ויעילים יותר בצריכת משאבים. יש להם גם יתרונות אחרים. לדוגמה, ניהול המערכת קל יותר אם אנחנו צריכים לתחזק רק מערכת הפעלה אחת ולא מערכת הפעלה נפרדת לכל מכונה וירטואלית.

עם זאת, יש להם גם חסרונות. ראשית, לא ניתן להפעיל כמה מערכות הפעלה על אותו מחשב. אם נידרש להריץ את Windows ו-UNIX בו-זמנית, קונטיינרים לא יספיקו. שנית, אף שהבידוד די טוב, הוא בשום פנים ואופן לא מוחלט, שכן הקונטיינרים השונים עדיין חולקים את אותה מערכת הפעלה ועלולים להפריע זה לזה. אם במערכת ההפעלה יש מגבלות סטטיות על משאבים מסוימים (כגון מספר הקבצים הפתוחים) ומיכל אחד משתמש כמעט בכולם, הקונטיינרים האחרים יהיו בבעיה. באופן דומה, פגיעה אחת

במערכת ההפעלה מסכנת את כל הקונטיינרים. לשם השוואה, הבידוד מיושם באמצעות hypervisors במידה רבה יותר.

7.11 תיאור מקרה: VMware

החומר שבסעיף זה הוא ברמה שמעבר לקורס בסיסי במערכות הפעלה. אנו לא נעסוק בו כאן, ונותירו כחומר רשות לקוראים הסקרנים.

האוניברסיטה הפתוחה
THE OPEN UNIVERSITY OF ISRAEL
الجامعة المفتوحة
Yair
Kurtzman

File #0006182 belongs to Yair Kurtzman- do not distribute

רשימת מקורות

1. Whitaker, A., Shaw, M., Gribble, S.D. Denali: *Lightweight Virtual Machines for Distributed and Networked Applications*. s.l. : University of Washington, 2001.
2. Barham, P.R., Dragovic, B., Fraser, K.A, Hand, S.M., Harris, T.L., Ho, A.C., Kotsovinos, E., Madhavapeddy, A.V.S., Neugebauer, R., Pratt, I.A., Warfield, A.K. *Xen 2002 - Technical Report 553.* 2002.
3. X86_Assembly Control_Flow. *X86_Assembly Control_Flow.* [Online] http://en.wikibooks.org/wiki/X86_Assembly/Control_Flow.
4. Keegan, W. The Rise of the Type Zero Hypervisor. *embeddedinnovator.com.* [Online] http://embeddedinnovator.com/2012/06/the-rise-of-the-type-zero-hypervisor/.
5. Jones, T. [Online] 2010. http://www.datamation.com/netsys/article.php/3884091/Virtualization.htm.
6. The MOSIX Multicomputer Operating System for High Performance Cluster Computing. Barak, A., La'adan, O. 1998, *Journal of Future Generation Computer Systems*.
7. *A Program Simulator by Partial Interpretation.* Fuchi, K., Tanaka , H., Manago, Y., Yuba, T. s.l. : SOSP'69, 1969. Proceedings of the 2nd Symposium on Operating Systems Principles.
8. *Survey of Virtual Machine Research.* Goldberg, R.P. s.l. : IEEE Comput., 1974, IEEE Comput., pp. 34–45.
9. How to Use the Kernel Samepage Merging Feature, [Online] https://www.kernel.org/doc/Documentation/vm/ksm.txt
10. Paravirtualization API Version 2.5 - vmi_spec.txt. [Online] VMware, 2005. http://www.vmware.com/pdf/vmi_specs.pdf
11. Single Root I/O Virtualization Scpecification Rev 1.1 [Online] PCI SIG, 2010. https://composter.com.ua/documents/sr-iov1_1_20Jan10_cb.pdf
12. *Reducing TCB Size by Using Untrusted Components: Small Kernels Versus Virtual Machine Monitors*. Hohmuth, M., Peter, M., Haertig, H., Shapiro, J. Proceedings of the 11th ACM SIGOPS European Workshop, ACM, Art. 22, 2004.

פרק 8
מערכות מרובות מעבדים

האוניברסיטה הפתוחה
THE OPEN UNIVERSITY OF ISRAEL
الجامعة المفتوحة
Yair
Kurtzman

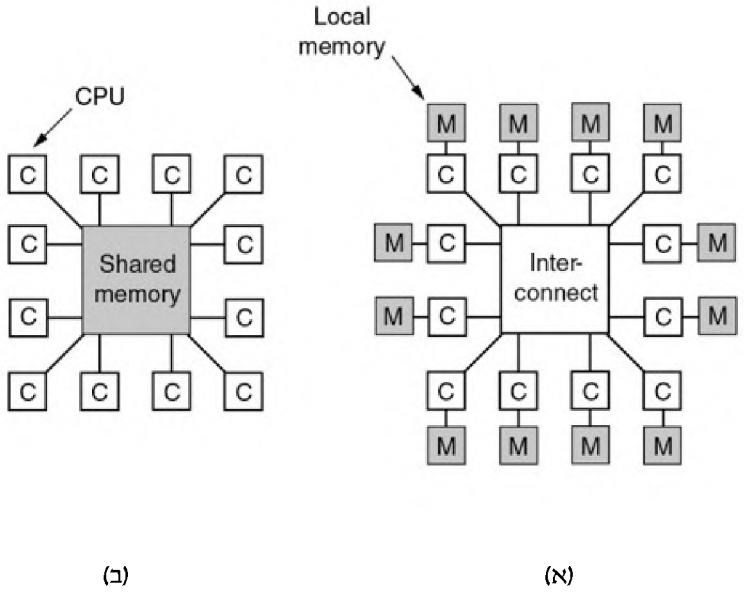
File #0006182 belongs to Yair Kurtzman- do not distribute

8 מערכות מרובות מעבדים
8.1 ריבוי מעבדים
8.5 סיכום
8.6 פרקים שאינם מופיעים בספר הלימוד

אחת הגישות המאפשרת את הגברת התפוקה של המערכת היא שימוש בכמה מעבדים היכולים לבצע פעולות במקביליות אמיתית. כשמעבדים בנויים על אותה פיסת מצע (die) הם נקראים ליבות (cores).

במבנה זה ליבות חולקות זו עם זו את מרחב הזיכרון הראשי, ולפעמים גם מטמון. קיימות שתי אפשרויות:

1. לכל הליבות יש גישה ישירה לכל מרחב הזיכרון הראשי (איור 1-8א).
2. לכל ליבה יש גישה ישירה לחלק ממרחב הזיכרון, ולחלקים אחרים ניתן לגשת בצורה עקיפה (איור 1-8ב) באמצעות שליחת הודעות.



איור 8.1 (א) מעבד רב-זיכרון משותף. (ב) רב-מחשב המעביר הודעות.

לכל אפשרות יש יתרונות וחסרונות: הגישה הראשונה בדרך כלל מאפשרת גישה מהירה יותר לזיכרון הראשי, אך היא מורכבת ויקרה יותר מבחינת מימושה בחומרה. במסגרת הקורס לא ניכנס לצורות מימוש חומרתי המאפשרות זאת. בכל מקרה, לכל ליבה יש גישה לכל מרחב הכתובות, וצורת המימוש החומרתי שקופה מבחינה תכנותית.

8.1 ריבוי מעבדים

עקרונית, מערכת ההפעלה המיועדת לריבוי ליבות היא מערכת רגילה, מלבד כמה עניינים מיוחדים שיידונו בהמשך.

האוניברסיטה הפתוחה THE OPEN UNIVERSITY OF ISRAEL الجامعة المفتوحة
Yair Kurtzman

File #0006182 belongs to Yair Kurtzman- do not distribute

8.1.1 חומרת ריבוי ליבות

אף על פי שלכל המעבדים יש יכולת לגשת לכל הזיכרון, קיימות שתי צורות מימוש שלהם, כאמור לעיל:
הראשונה – **UMA (Uniform Memory Access)** והשנייה – **NUMA (Nonuniform Memory Access)**.
חשוב להדגיש שבמעבדים מרובי ליבות קיימת בעיית עקביות המטמון (**cache-coherence**) – שינוי ערך משתנה משותף שמבוצע באחת הליבות (למשל כאשר תהליכונים שונים משתפי נתונים רצים על ליבות שונות) צריך להיות "נראה" על ידי ליבה אחרת. במעבד יכול להיות מנגנון חומרתי שמבטיח זאת, ויכול להיות שאין מנגנון כזה. אם אין, המתכנת צריך להביא בחשבון שמימושים של אלגוריתמים מסוימים לא יעבדו באופן תקין (פתרון פיטרסון לבעיית קטע קריטי למשל). דוגמה זו ממחישה מדוע מתכנת צריך להתחשב במאפיינים מסוימים של החומרה. למען הפשטות, נצא מנקודת הנחה שמנגנון כזה קיים (אלא אם כן יצוין אחרת).

שבב רב-ליבתי (multicore chip)

בשבבים אלו יש בדרך כלל 4–16 ליבות. מבחינת המתכנת, כמעט שאין הבדל בין ריבוי ליבות לריבוי מעבדים (multicore vs. multiprocessor). כל הליבות יכולות לגשת לכל מרחב הזיכרון (בדרך כלל באופן ישיר). לעתים לכל ליבה יהיה מטמון משלה, ולעתים יש לכולן מטמון משותף.

שבב מרובה ליבות (manycore chip)
בשבבים כאלה יש הרבה מאוד ליבות, בין עשרות למאות, ואפילו אלפים. הם מאפשרים ביצוע מקבילי המקנה מהירות לפעולות חישוביות מורכבות.

חלוקת העבודה בין הליבות
קיימות שתי צורות של ריבוי ליבות:

1. הומוגנית – כל ליבה יכולה למלא כל תפקיד (להריץ כל קוד) באותה יעילות, ואפשר לומר שהן זהות.
2. הטרוגנית – לליבות שונות יש תכונות שונות שבזכותן הן מתאימות יותר לתפקידים מסוימים.

ריבוי תהליכונים (multi-threading או hyper-threading)
מושג זה מתייחס ליכולת של כל ליבה להריץ כמה תהליכונים כמעט במקביל. חלק מהמשאבים הפיזיים קיימים בכמה עותקים (לדוגמה קבוצת רגיסטרים נפרדת לכל תהליכון) או שהם משותפים לכל התהליכונים הרצים על אותה ליבה (כמו מטמונים ויחידת הביצוע עצמה). לכן טכניקה זו לא מוסיפה הספק כליבה נוספת, ובדרך כלל תוספת התפוקה שלה מוגבלת ל-30% לכל היותר.

תכנות עם ריבוי ליבות

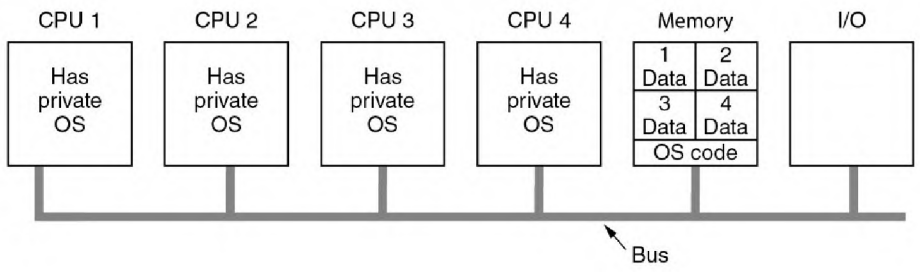
החומרה ומערכת הפעלה לא יכולות לפצל קוד סדרתי למקבילי ולבצע אותו. לכן על מפתח הקוד מוטלת האחריות לכתוב את הקוד בצורה שתאפשר ביצוע מקבילי (שימוש בתהליכים או בתהליכונים). בחלק משפות התכנות קיימות אפשרויות שונות או ספריות שמקילות על יצירת מבנה מקבילי של התוכנית.

8.1.2 סוגי מערכות הפעלה לריבוי מעבדים

נתמקד בשלושה סוגים של מערכות הפעלה מרובות מעבדים. שימו לב שסוגים אלה מתאימים באותה מידה למערכות מרובות ליבות וכן למערכות עם מעבדים נפרדים.

לכל מעבד מערכת הפעלה משלו

הדרך הפשוטה ביותר לארגן מערכת הפעלה מרובת מעבדים היא לחלק את הזיכרון באופן סטטי לכמה מחיצות, כמספר המעבדים, ולתת לכל מעבד זיכרון פרטי ועותק פרטי משלו של מערכת ההפעלה. אופטימיזציה ברורה אחת היא לאפשר לכל המעבדים לשתף את קוד מערכת ההפעלה וליצור עותקים פרטיים של מבני הנתונים של מערכת ההפעלה בלבד, כמוצג באיור 8-7 .



איור 8.7 חלוקת זיכרון מרובה מעבדים בין ארבעה מעבדים, אך שיתוף עותק יחיד של קוד מערכת ההפעלה. תיבות ה-Data הן התפעול, כלומר הנתונים הפרטיים של המערכת עבור כל מעבד

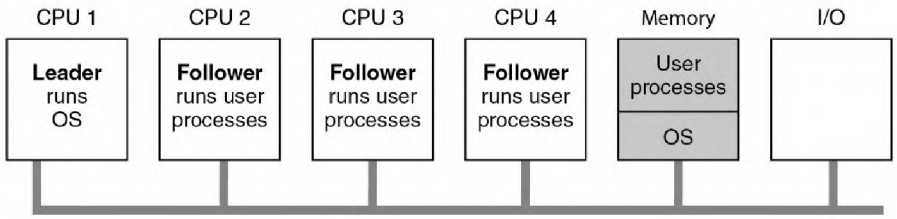
בגלל הכפילות המיותרת גישה זו אינה נפוצה.

מבנה מוביל-נגרר (leader-follower)

הצורה השנייה מוצגת באיור 8-8. כאן, עותק יחיד של מערכת ההפעלה והטבלאות שלה קיים ב-CPU1 ולא באף אחת מהאחרות. כל קריאות המערכת מופנות למעבד 1 לשם עיבוד. CPU1 עשוי להפעיל גם תהליכי משתמש, אם נותר לו זמן פנוי.

האוניברסיטה הפתוחה
THE OPEN UNIVERSITY OF ISRAEL
الجامعة المفتوحة
Yair Kurtzman

File #0006182 belongs to Yair Kurtzman- do not distribute

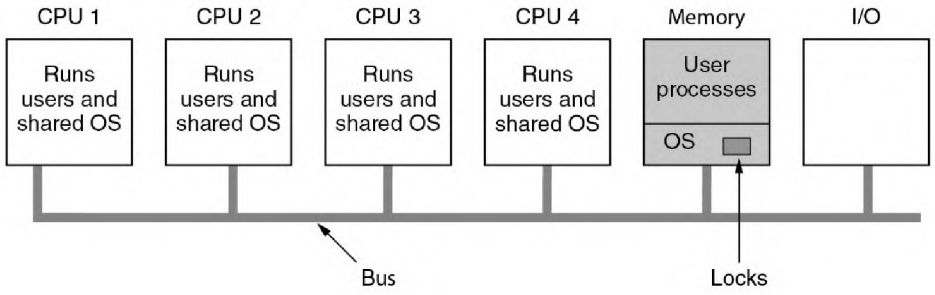


איור 8.8 רב-מעבד מוביל-נגרר

המודל עובד מצוין כשיש מעט מעבדים. אבל כשמספרם גדל, המעבד שמריץ את מערכת הפעלה הופך לצוואר הבקבוק כי הוא אינו מספיק לשרת (לטפל בקריאות מערכת ולנהל) את כל התהליכים הרצים על הרבה מעבדים. כאמור, הוא פשוט ושימושי למספר מועט של מעבדים.

מבנה סימטרי

המודל השלישי, **SMP (symmetric multiprocessor)**, מבטל את האסימטריה הזו. קיים עותק אחד של מערכת הפעלה בזיכרון, אבל כל מעבד (או כמה מעבדים בו-זמנית) יכול להפעיל אותו. כאשר מתבצעת קריאת מערכת, המעבד שעליו הייתה קריאת המערכת עושה trap לקרנל ומעבד את קריאת המערכת. מודל SMP מודגם באיור 8-9.



איור 8.9 רב-מעבד מדגם SMP

מודל זה מאזן תהליכים וזיכרון באופן דינמי, מאחר שיש רק סט אחד של טבלאות מערכת הפעלה. כמו כן, לא נוצר צוואר בקבוק במעבד המוביל. עם זאת, למודל זה חסרונות משלו. בפרט, אם שני מעבדים או יותר מריצים קוד מערכת הפעלה בו-זמנית, עלולה להיגרם תקלה חמורה. תארו לעצמכם ששני מעבדים הפועלים בו-זמנית בוחרים באותו תהליך כמועמד להרצה או מקצים אותו דף זיכרון פנוי. הכי פשוט לעקוף את הבעיות האלה בעזרת שיוך מנעול, mutex, כלומר לנעול את מערכת ההפעלה, מה שהופך את כל המערכת לאזור קריטי אחד גדול. כאשר מעבד רוצה להריץ קוד מערכת הפעלה, הוא צריך קודם כול לתפוס את המנעול. אם הוא כבר נעול, הוא פשוט מחכה. בדרך זו, כל מעבד יכול לבצע את מערכת ההפעלה, אבל רק מעבד אחד יכול לעשות זאת בכל נקודת זמן. גישה זו נקראת לעתים **מנעול קרנל גדול (big kernel lock)**.

המודל הזה עובד, אבל הוא גרוע כמעט כמו המודל **מוביל-נגרר** במקרה שיש הרבה מעבדים. אבל כעת יש בידינו פתרון לכך. רוב חלקי מערכת ההפעלה הם עצמאיים ולא תלויים זה בזה. לדוגמה, אין בעיה כאשר מעבד אחד שמפעיל את המתזמן, בעוד מעבד אחר מטפל בקריאה למערכת קבצים ומעבד שלישי מטפל בתקלת דף.

כדי לממש זאת, נדרש לפצל את מערכת ההפעלה לכמה אזורים קריטיים עצמאיים שאינם נמצאים באינטראקציה זה עם זה. כל אזור קריטי מוגן על ידי מנעול משלו, כך שרק מעבד אחד בכל פעם יכול להפעיל אותו. כך ניתן גם להשיג מקביליות רבה יותר. עם זאת, ייתכן בהחלט שחלק מהטבלאות, כגון טבלת התהליכים, משמשות כמה אזורים קריטיים. לדוגמה, טבלת התהליכים נחוצה עבור התזמון, אבל גם עבור קריאת המערכת fork וגם עבור הטיפול ב-signals. כל טבלה שעשויה להיות בשימוש על ידי קטע קריטי, צריכה מנעול משלה.

כך, כל אזור קריטי יכול להתבצע רק על ידי מעבד אחד בכל פעם, ורק מעבד אחד יכול לגשת לכל טבלה קריטית בכל פעם.

רוב הרב-מעבדים המודרניים משתמשים במודל זה. החלק הקשה בכתיבת קוד מערכת ההפעלה עבור מכונה כזו הוא לפצל אותו לאזורים קריטיים שיכולים להתבצע במקביל על ידי מעבדים שונים, מבלי להפריע זה לזה. בנוסף, כל טבלה הנמצאת בשימוש של יותר מאזור קריטי אחד צריכה להיות מוגנת במנעול נפרד, וכל קוד הניגש לטבלה חייב להשתמש בו. יתר על כן, יש לנקוט זהירות רבה כדי למנוע קיפאון. אם שני אזורים קריטיים זקוקים גם לטבלה א וגם לטבלה ב, ואחד מהם ניגש ל-א תחילה והשני ל-ב, יכול להתרחש קיפאון.

8.1.3 סנכרון עם ריבוי מעבדים

המעבדים במערכות ריבוי מעבדים צריכים לעתים קרובות להסתנכרן זה עם זה. בסעיף זה נלמד כיצד מתרחש סנכרון זה.

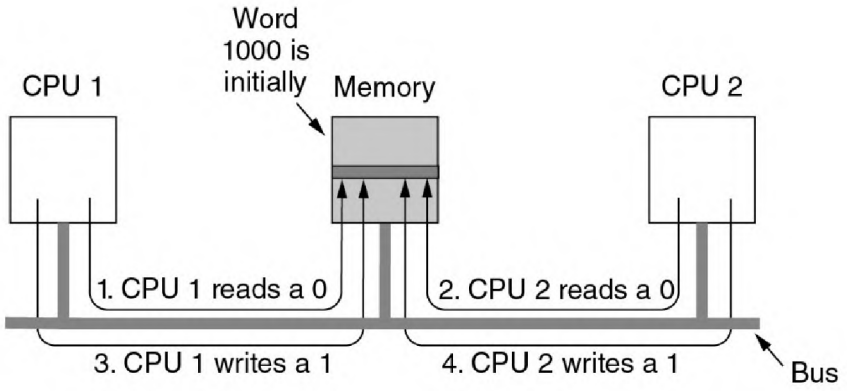
כשבמכונה בעלת מעבד אחד בלבד מתבצעת קריאת מערכת, קוד הליבה יכול פשוט להשבית פסיקות לפני גישה לנתונים הדורשים סנכרון. בריבוי מעבדים, חסימת פסיקות משפיעה רק על המעבד שמבצע את החסימה. מעבדים אחרים ממשיכים לפעול ועדיין יכולים לגשת לנתונים קריטיים. עקב כך, כל המעבדים חייבים להשתמש בפרוטוקול מנעול תקין, כדי להבטיח שהמניעה ההדדית פועלת.

הלב של כל מימוש מנעול הוא הוראה מיוחדת המאפשרת לבחון ולשנות את ערך המשתנה כפעולה אטומית יחידה (לדוגמה פעולת Test and Set Lock TSL – שנידונה בפרק 2).
כעת נחשוב מה יכול לקרות בריבוי מעבדים. באיור 8-10 אנו רואים את התזמון במקרה הגרוע ביותר שבו מילת זיכרון 1000 המשמשת כמנעול, היא תחילה 0. בשלב 1, CPU1 קורא את המילה ומקבל 0. בשלב 2, לפני ש-CPU1 מספיק לשכתב את ערך המילה ל-1, CPU2 קורא גם הוא את ערך המילה 0. בשלב 3, CPU1 כותב 1 לתוך המילה, ובשלב 4

האוניברסיטה הפתוחה
THE OPEN UNIVERSITY OF ISRAEL
الجامعة المفتوحة
Yair
Kurtzman

File #0006182 belongs to Yair Kurtzman- do not distribute

CPU2 כותב גם הוא 1 לתוך המילה. שני המעבדים קיבלו ערך חזרה 0 מהוראת ה-TSL, כך שלשניהם יש כעת גישה לאזור הקריטי והמניעה ההדדית נכשלת.



איור 8.10 ההוראה TSL יכולה להיכשל אם לא ניתן לנעול את האפיק. ארבעת השלבים מציגים רצף של אירועים שבהם הוכח כישלון

כדי למנוע בעיה זו ,הוראת ה- TSLחייבת לנעול תחילה את האפיק (bus) כדי למנוע ממעבדים אחרים לגשת לזיכרון, לאחר מכן לבצע את שתי הגישות לזיכרון, ולאחר מכן לבטל את נעילת האפיק. נעילת האפיק מתבצעת ברמת החומרה.

שיטה זו משתמשת **בהמתנה פעילה (spin lock)** פנימית ברמת החומרה, מכיוון שה- CPU המבקש גישה לאפיק שוהה בלולאה ובודק את נעילת האפיק הכי מהר שהוא יכול. כך, לא רק שמתבזבז הזמן של המעבד המבקש (או המעבדים), אלא גם עלול להיווצר עומס על האפיק, שבעקבותיו יואטו כל המעבדים האחרים שמנסים לעשות את העבודה הרגילה שלהם (לגשת למילות זיכרון אחרות). בזמן הזה מעבדים אחרים יוכלו לעשות שימוש חלקי במטמון שלהם (קוראינו הסקרנים יכולים לקרוא בספר הלימוד מדוע), אבל עדיין תהיה פגיעה בביצועים. קיימות דרכים שונות להקטין את מספר הבדיקות החוזרות ונשנות בזמן ביצוע **spin lock** (שוב, הסקרנים מוזמנים לקרוא על כך בספר הלימוד).

המתנה פעילה לעומת מיתוג

עד כה הנחנו שמעבד שצריך לבצע נעילה, פשוט ממתין עד שיתאפשר הדבר. במקרים מסוימים, במקום להמתין אפשר לעבור בינתיים לביצוע משימה אחרת. לדוגמה, אם תהליכון כלשהו צריך גישה לקובץ ונאלץ להמתין עקב נעילת חוצץ, המעבד יכול להחליט לעבור לתהליכון אחר (מיתוג) במקום להמתין (המתנה פעילה).

קיימות גישות שונות בעניין זה, המאפשרות שיפור ביצועים, בהתאם לאופי הקודים המורצים.

8.1.4 תזמון במערכות ריבוי מעבדים

אם תהליכונים נוצרים על ידי ספריית מרחב משתמש והקרנל לא יודע דבר על התהליכונים, אז התזמון מתרחש על בסיס תהליך.

בתהליכוני גרעין התמונה שונה – הקרנל מודע לכל התהליכונים ויכול לבחור מבין התהליכונים השייכים לתהליך.
במעבד יחיד התזמון הוא חד-ממדי, והשאלה היחידה היא איזה תהליכון הוא הבא בתור לרוץ. בריבוי מעבדים תזמון הוא דו-ממדי, והשאלה היא לא רק איזה תהליכון הוא הבא לרוץ, אלא גם על איזה מעבד להריץ אותו. הממד הנוסף מסבך מאוד את התזמון של ריבוי מעבדים. כשהתהליכונים לא קשורים זה לזה, ניתן לתזמן אותם ללא שיקולים נוספים. אך כאשר הם מגיעים בקבוצות שבהן התהליכונים תלויים זה בזה, למשל בתוך אותה אפליקציה, נכנסים שיקולים נוספים. לדוגמה, כשהתהליכונים מתקשרים בצורה נרחבת, לא רק באותו זמן אלא גם קרובים זה לזה במקום (מאגר משותף) כמו במקרה יצרן-צרכן, הם עשויים להפיק תועלת משיתוף מטמונים, ולכן כדאי לתזמן אותם.

שיתוף זמן
תחילה נעסוק במקרה של תזמון תהליכונים עצמאיים, ולאחר מכן נלמד כיצד לתזמן תהליכונים משתפי פעולה. אלגוריתם התזמון הפשוט ביותר להתמודדות עם תהליכונים לא קשורים הוא להחזיק מבנה נתונים כלל מערכתי אחד עבור תהליכונים במצב מוכן, או קבוצה של רשימות תהליכונים בעדיפויות שונות. המעבד הראשון שמסיים את עבודתו הנוכחית נועל את תורי התזמון ובוחר תהליכון בעל עדיפות עליונה. כל עוד התהליכונים אינם קשורים לחלוטין, ביצוע תזמון בדרך זו הוא בחירה סבירה וקלה מאוד ליישום, המספקת בנוסף איזון עומסים אוטומטי.

גורם נוסף שמשפיע על התזמון הוא העובדה שבעוד כל המעבדים שווים (מודל הומוגני) , חלק מהמעבדים "שווים יותר". בפרט, כאשר תהליכון A רץ במשך זמן רב במעבד, כלומר המטמון שלו מלא בבלוקים של A, אם A יחזור לפעול בקרוב ,ייתכן שהוא יפעל טוב יותר אם ירוץ על אותו מעבד, מכיוון שהמטמון עדיין עשוי להכיל חלק מהבלוקים של A. טעינה מראש של בלוקי מטמון תגביר את קצב הפגיעה במטמון, ובכך את מהירות התהליכון. בנוסף, ה- TLB עשוי להכיל גם את כתובות הדפים הנכונים.

חלק מרבי-מעבדים מביאים בחשבון את ההשפעה הזו ומשתמשים בתזמון זיקה (affinity scheduling). הרעיון הבסיסי של תזמון זיקה הוא להשתדל להפעיל תהליכון על אותו המעבד שהוא רץ עליו בפעם הקודמת .אחת הדרכים ליצור זיקה זו היא להשתמש באלגוריתם תזמון דו-שלבי. כאשר תהליכון נוצר ,הוא מוקצה למעבד, למשל על סמך עומס – הוא יוקצה למעבד הכי פחות עמוס באותו רגע. הקצאה זו של תהליכונים למעבדים היא הרמה העליונה של האלגוריתם, ועקב כך לכל מעבד יש אוסף תהליכונים משלו.

התזמון בפועל של התהליכונים הוא הרמה התחתונה של האלגוריתם. הוא נעשה על ידי כל מעבד בנפרד, תוך כדי שימוש בסדרי עדיפויות או באמצעים אחרים. בעקבות הניסיון

האוניברסיטה הפתוחה THE OPEN UNIVERSITY OF ISRAEL الجامعة المفتوحة
Yair Kurtzman

File #0006182 belongs to Yair Kurtzman- do not distribute

לשמור תהליכון על אותו מעבד במשך כל חייו, זיקת המטמון משתפרת .עם זאת, אם למעבד אין תהליכונים להפעיל, הוא לוקח תהליכון ממעבד אחר במקום להתבטל.

שיתוף מרחבי

ניתן להשתמש בגישה כללית אחרת לתזמון מרובה מעבדים כאשר תהליכונים קשורים זה לזה בדרך כלשהי .

לדוגמה, אם התהליכונים של תהליך מתקשרים הרבה, כדאי שיפעלו בו-זמנית על מעבדים שונים. תזמון בו-זמני של תהליכונים מרובים על פני כמה מעבדים נקרא שיתוף מרחבי (space sharing).

אלגוריתם שיתוף מרחבי הפשוט ביותר עובד כך: נניח שקבוצה שלמה של תהליכונים קשורים נוצרת בבת אחת. בזמן יצירתו, המתזמן בודק אם יש כמות מעבדים פנויים כמספר תהליכונים. אם יש, כל תהליכון מקבל מעבד ייעודי משלו עד שיסתיים. אחרת , אף אחד מהתהליכונים לא יופעל עד שיהיו מספיק מעבדים זמינים. כל תהליכון מחזיק במעבד שלו עד שהוא מסתיים, ואז המעבד חוזר לאוסף (pool) המעבדים הזמינים. אם תהליכון נחסם ב-I/O, הוא ממשיך להחזיק ב- CPU שפשוט נותר לא פעיל עד שהתהליכון מתעורר.

תזמון "כנופיות" (gang scheduling)

אחד היתרונות של שיתוף מרחבי הוא הגבלת ריבוי תוכניות, שמבטל את התקורה של החלפת הקשר. עם זאת, חיסרון ברור הוא הזמן המבוזבז כאשר מעבד חסום נותר בטל עד שתהליכון יהיה מוכן שוב. עקב כך, יש צורך באלגוריתמים שמנסים לתזמן תהליכונים הן בזמן והן במרחב, בייחוד עבור תהליכים שיוצרים תהליכונים מרובים, שבדרך כלל צריכים לתקשר זה עם זה.

כדי להדגים את הבעיה שעלולה להתרחש כאשר התהליכונים של תהליך מתוזמנים באופן עצמאי, נסתכל על מערכת שבה התהליכונים A_0 ו-A_1 שייכים לתהליך A , ו- B_0 ו- B_1 שייכים לתהליך B. התהליכונים A_0 ו- B_0 חולקים זמן על מעבד 0 והתהליכונים A_1 ו- B_1 חולקים זמן על מעבד 1. התהליכונים A_0 ו- A_1 צריכים לתקשר זה עם זה לעתים קרובות, כשכל אחד שולח הודעה לאחר ומחכה לתגובה.

ברור שבמקרה כזה יתבזבז זמן רב, כי תהליכון היעד לא פעיל עד שמתקבלת אצלו הודעה.

הפתרון לבעיה זו הוא תזמון "כנופיות", שהוא פועל יוצא של תזמון משותף, והוא בנוי משלושה חלקים:

קבוצות תהליכונים קשורים מתוזמנת כיחידה אחת – "כנופייה" (gang).
כל חברי הכנופייה ירוצו בבת אחת על מעבדים שונים.
כל חברי הכנופייה יתחילו ויסיימו את יחידות הזמן שלהם יחד.

תזמון כנופיות פועל כהלכה מאחר שכל המעבדים מתוזמנים באופן סינכרוני. משמעות הדבר היא שהזמן מחולק לקוונטות בדידות. רק בתחילת כל קוונטה חדשה מתבצע תזמון חדש של כל המעבדים, ותהליכון חדש מתחיל בכל אחד מהם. בתחילת הקוונטה הבאה, מתרחש תזמון נוסף. אם תהליכון נחסם, המעבד שלו נשאר בטל עד סוף הקוונטה.

הרעיון של תזמון כנופיות הוא להפעיל את כל התהליכונים של תהליך יחד, בו-זמנית, על מעבדים שונים. להסבר נוסף ראו איור 8-12 בספר הלימוד.

8.5 סיכום

ניתן להפוך מערכות מחשב למהירות יותר על ידי שימוש בכמה מעבדים.

רב-מעבד מורכב משני מעבדים או יותר, החולקים זיכרון RAM משותף. לעתים קרובות מעבדים אלה עצמם מורכבים מכמה ליבות. הליבות והמעבדים יכולים להיות מחוברים באמצעות אפיק, מתג צולב או רשת מיתוג רב-שלבית. קיימות אפשרויות הפעלה בתצורות שונות, לרבות מתן מערכת הפעלה לכל אחד מהמעבדים, מערכת הפעלה מובילה אחת, או ריבוי מעבדים סימטרי, שבו יש עותק אחד של מערכת ההפעלה שכל מעבד יכול להריץ. במקרה האחרון, יש צורך במנעולים כדי לספק סנכרון. כאשר מנעול אינו זמין, מעבד יכול לבצע המתנה פעילה או לבצע החלפת הקשר. אלגוריתמים שונים של תזמון אפשריים, לרבות שיתוף זמן, שיתוף מרחב ותזמון כנופיות.

8.6 פרקים שאינם מופיעים בספר הלימוד
8.6.1 סיכום
8.6.2 שאלות לחזרה
8.6.3 רשימת מושגים

8.6.1 סיכום

נמנה כמה מהנושאים המרכזיים שנלמדו בפרק זה:

- מעבד עם ריבוי ליבות – multi/many core
- שיתוף זיכרון ראשי בין הליבות
- אופן חיבור ליבות המעבד – אפיק, מתג, רשת
- צורת הפעלת הליבות – סימטרית או אסימטרית (מוביל-נגרר)
- תזמון מעבד מרובה ליבות, והעדפה וזיקה לליבה

8.6.2 שאלות לחזרה

ענו על השאלות (כל התשובות נמצאות בחומר הלימוד, בפרק עצמו או במדריך זה).

1. מה קורה כשכל ארבע הליבות במעבד מנסות לגשת לאותה כתובת זיכרון, באותו הזמן בדיוק?
2. האם ניתן לממש סנכרון בין כמה ליבות בעזרת הפקודה SWP שמחליפה בצורה אטומית בין תוכן של תא זיכרון לרגיסטר? הסבירו כיצד ניתן או מדוע לא ניתן.
3. תזמון עם זיקה מקטין cache misses. האם הוא גם מקטין TLB misses ו- Page faults?

האוניברסיטה הפתוחה THE OPEN UNIVERSITY OF ISRAEL الجامعة المفتوحة
Yair Kurtzman

File #0006182 belongs to Yair Kurtzman- do not distribute

4. את ריבוי הליבות ניתן לנצל בשביל הרצת אפליקציות במקביליות אמיתית או בשביל הרצת שירותי מערכת כלליים. למה צריך לתת עדיפות?

8.6.3 רשימת מושגים

להלן רשימת המושגים המופיעים בפרק זה (על פי סדר הופעתם בפרק):

ליבה	**core**	יחידת ביצוע נפרדת בתוך המעבד, בעלת רגיסטרים ומטמון משלה.
מעבד מרובה ליבות	**multicore processor (MCP)**	מעבד שבתוכו קיימות כמה ליבות עצמאיות.
שיתוף זיכרון אחיד	**UMA**	לכל ליבה יש גישה ישירה לכל מרחב הזיכרון.
שיתוף זיכרון לא אחיד	**NUMA**	לכל ליבה יש גישה ישירה לאזור מסוים בזיכרון. גישה לאזורים אחרים איטית יותר ונעשית דרך תקשורת עם ליבה אחרת.
מבנה מוביל-נגרר	**leader-follower**	לאחת הליבות יש תפקיד לנהל את חלוקת העבודה בין שאר ליבות.
מבנה סימטרי	**SMP (Symmetric MultiProcessor)**	כל הליבות פועלות בצורה אחידה ובלתי תלויה.
תזמון עם זיקה	**affinity scheduling**	העדפת תזמון תהליכון לליבה מסוימת כדי לנצל זיכרון מהיר יותר.
תזמון שיתוף מרחבי	**space sharing scheduling**	ההליכונים הקשורים והמשתפים פעולה זה עם זה מופעלים כיחידה אחת – או כולם יחד או אף לא אחד.
תזמון "כנופיות"	**gang scheduling**	דומה לתזמון שיתוף מרחבי, רק שבכל יחידת זמן מתקבלת החלטת תזמון חדשה ומופעלת רק קבוצת תהליכונים שמוכנים לתקשר (לא נמצאים בהמתנה כלשהי). מאפשר ניצול ליבות טוב יותר.

נספח

מעט על בטיחות

האוניברסיטה הפתוחה
THE OPEN UNIVERSITY OF ISRAEL
الجامعة المفتوحة
Yair
Kurtzman

File #0006182 belongs to Yair Kurtzman- do not distribute

נספח: מעט על בטיחות
9.2 מנגנוני הגנה
9.2.1 הגנה על ידי קביעת תחומים (protection domains)
9.2.2 רשימות גישה (ACL)
9.2.3 רשימות יכולות (C-lists)

9.2 מנגנוני הגנה

את הדיון במערכות קבצים נסיים בהצצה קצרצרה לסעיף 9.3 בספר הלימוד. סעיף זה דן בסוגיות הקשורות למימושים אפשריים של מנגנון הגנה של מערכת קבצים.

9.2.1 הגנה על ידי קביעת תחומים (protection domains)

הדרך ליצירת מנגנון הגנה עוברת דרך תכנון מופשט. ביטחון אינו דומה לתבלין שאפשר להוסיף לתבשיל בכל שלב בהכנתו; הוא מבוסס על עקרונות יסוד שיש ליישמם כבר בשלב התכנון הראשוני של המערכת. כדי להתוודע לעקרונות אלו ולאופן היישום שלהם, נפשט את בעיית ההגנה ונעסוק במה שמכונה ***תחומים (domains)***. תחומים הם קבוצות של זוגות (עצם, הרשאות), וההרשאות מייצגות את הפעולות המותרות ביחס לעצם בתחום. עיינו באיור 9-3 בספר הלימוד. באיור מיוצגים שלושה תחומים. כל תחום מכיל עצמים והרשאות, המייצגות את הפעולות המותרות על העצמים שבדוגמה: קבצים, מדפסות ותווייניים.

עוד דוגמאות אפשריות לעצמים הן:

- עצמים פיזיים, למשל התקני קלט/פלט.
- עצמים לוגיים, למשל קבצים או קטעים קריטיים. גם התחומים עצמם הם עצמים, מכיוון שעצם הוא כל ישות במערכת שאפשר להגדיר פעולות ביחס אליהן. ברגע שהגדרנו פעולות על תחומים, כגון הוספת עצמים לתחום או מחיקת עצמים, גם תחום הופך לעצם.
- עצמים פעילים, כלומר בעיקר תהליכים. כך למשל, יכולה להיות הרשאה המאפשרת ליצור או להפסיק תהליך אחר במערכת.

בדרך זו אפשר לשייך לכל תהליך תחום. התחום שבו התהליך רץ יגדיר את גבולות אפשרויותיו להשפיע על גורמים אחרים במערכת בכל רגע ורגע, מכיוון שרשימת העצמים וההרשאות המצויה בתחום היא תיאור ממצה של האיסורים המוטלים על התהליך.

על סמך ההבחנות שעשינו לעיל, אפשר להגדיר מטריצה המכונה "מטריצת תחומי הגנה" (protection domain matrix) המגדירה את כל תחומי ההגנה ואת כל העצמים והאפשרויות של מעברים מתחום הגנה אחד לתחום הגנה אחר (ראו איור 9-5 בספר הלימוד). כך, תהליך השייך לתחום מסוים יוכל לבצע פעולות המותרות לפי תחום ההגנה

האוניברסיטה הפתוחה THE OPEN UNIVERSITY OF ISRAEL الجامعة المفتوحة
Yair Kurtzman

File #0006182 belongs to Yair Kurtzman- do not distribute

שאליו הוא שייך. התהליך יוכל גם לעבור מתחום הגנה אחד לתחום אחר, הכול בהתאם להרשאות המעבר מן התחום שאליו הוא שייך.

9.2.2 רשימות גישה (ACL)

שמירת מטריצת תחומי ההגנה כרוכה בבזבוז רב, מכיוון שצריך לשמור את ההרשאות של כל העצמים במערכת בעבור כל התחומים שבה. אפשר לחסוך במקום אם נשמור לכל עצם רק את ההרשאות לכל אחד מן התחומים אשר בהם קיימת לעצם זה הרשאה כלשהי. שיטה זו נקראת "שיטת רשימות גישה" (access control list).

איור 9-6 בספר הלימוד מדגים כיצד אפשר לנהל הרשאות גישה לקבצים באמצעות רשימות גישה. במקרה זה, הקבצים הם העצמים. תהליכים A, B, C משתייכים לתחומים שונים, והרשאות הגישה לכל תחום (תהליך) מפורטות ברשימות הגישה של כל עצם (קובץ).

9.2.3 רשימות יכולות (C-lists)

קיימת גם אפשרות אחרת: במקום לשמור את מטריצת ההגנה כולה, אפשר לשמור בעבור כל תחום את רשימת העצמים שקיימת הרשאה אליהם. כל תהליך בתחום יחזיק מצביע לרשימה כזו, שנכנה בשם ***רשימת היכולות (capability list)***. בכל פעם שבה תהליך מבקש לבצע פעולה על עצם, רשימת היכולות שלו נסרקת, והפעולה מתאפשרת אך ורק אם העצם אכן מופיע ברשימה זו והפעולה הנדרשת מותרת.

כדי להגן על רשימת היכולות של תהליך מפני בעל התהליך עצמו, כדי שלא ינסה למשל להרחיב את סמכויותיו על דעת עצמו, יש שלוש דרכים לשמירה על רשימות היכולות:

- הדרך הראשונה דורשת תמיכה של חומרה. כל מילת זיכרון של הזיכרון הראשי חייבת להיות מצוידת בתו האוסר על שינוי תוכן המילה ממצב משתמש, אף אם המילה נמצאת במרחב הזיכרון של התהליך. כך אפשר לשמור את רשימת היכולות בזיכרון, ולהבטיח שרק מערכת ההפעלה תוכל לשנות את היכולות כאשר המעבד נמצא במצב מיוחס.
- הדרך השנייה היא לשמור את רשימת היכולות במבנים פנימיים של מערכת ההפעלה שנמצאים במרחב זיכרון השייך למערכת ההפעלה. כך, כל שינוי של רשימת היכולות חייב לעבור דרך מערכת ההפעלה.
- הדרך השלישית היא לשמור את רשימת היכולות במרחב הזיכרון של התהליך, אך להצפין אותה כך שרק מערכת ההפעלה תוכל לבצע פענוח, שינוי ועדכון של רשימות. הדרך הזאת מתאימה במיוחד למערכות מבוזרות.

שאלה 1

איזו שיטת הגנה על העצמים של מערכת קבצים ממומשת במערכת ההפעלה UNIX: ACL או רשימת היכולות?

מהדורה פנימית
לא להפצה ולא למכירה
מק"ט 20594-5090

האוניברסיטה הפתוחה
THE OPEN UNIVERSITY OF ISRAEL
الجامعة المفتوحة
Yair
Kurtzman